



Building RAG Agents with LLMs

Introduction



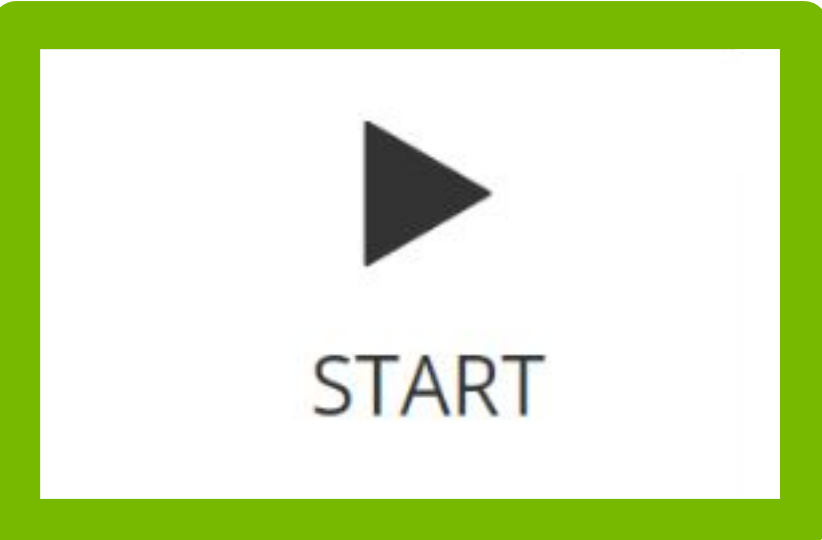
Building RAG Agents with LLMs

Introduction

Course Environment

 EDIT     

 **DEEP
LEARNING
INSTITUTE**


START

Please click the start button to the top-right. The course will take about 5 minutes to load. The course lecture slide decks will appear here when you launch the course. The numbers on the lectures correspond to the numbers of the interactive notebooks in the course.

Large Language Models

Backbones for Language Understanding



Backbones for language tasks, including classification and generation.

Dialog/Retrieval Agents

LLMs with Context and Control

User Asks
Something

Agent
Responds

Dialog/Retrieval Agents

LLMs with Context and Control

LLM Orchestration: Software + LLM helps to route to software and LLMs.

**User Asks
Something**

**Agent
Responds**

Dialog/Retrieval Agents

LLMs with Context and Control

LLM Orchestration: Software + LLM helps to route to software and LLMs.

Retrieval: Tool runs algorithms (database, code execution, semantic search, return a constant value, etc) to provide context

**User Asks
Something**

**Agent
Responds**

Dialog/Retrieval Agents

LLMs with Context and Control

LLM Orchestration: Software + LLM helps to route to software and LLMs.

Retrieval: Tool runs algorithms (database, code execution, semantic search, return a constant value, etc) to provide context

Augmented: Based on tool responses, the software pipeline synthesizes some “context” to feed to LLM w/ question.

User Asks Something

Agent Responds

Dialog/Retrieval Agents

LLMs with Context and Control

LLM Orchestration: Software + LLM helps to route to software and LLMs.

User Asks Something

Agent Responds

Retrieval: Tool runs algorithms (database, code execution, semantic search, return a constant value, etc) to provide context

Augmented: Based on tool responses, the software pipeline synthesizes some “context” to feed to LLM w/ question.

Generation: Based on question, instructions, and enhanced context, the LLM returns a response.

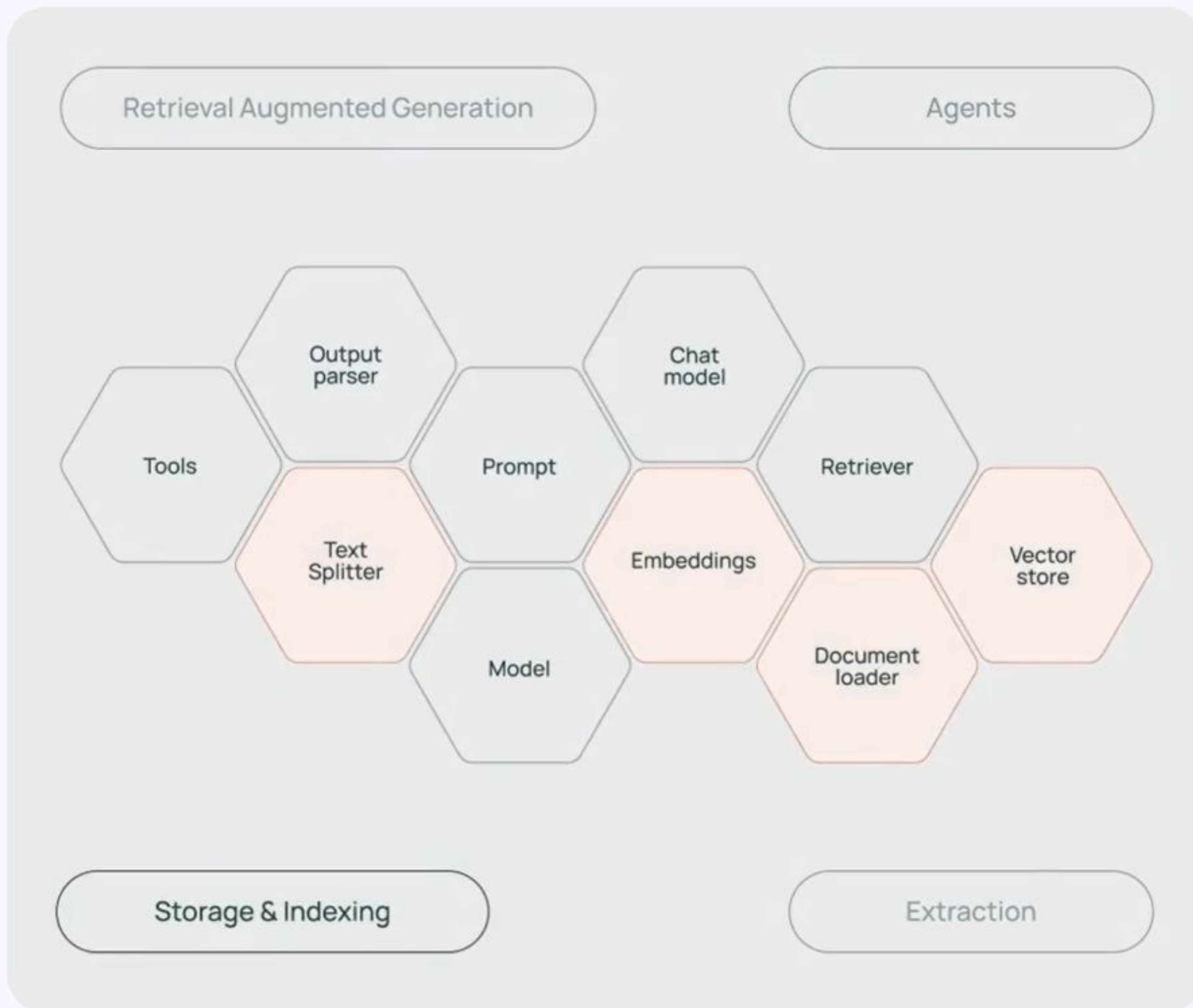
Chat Applications

Full Applications Build with LLMs



Chat Applications

Full Applications Build with LLMs



Build your app with LangChain

Build context-aware, reasoning applications with LangChain's flexible framework that leverages your company's data and APIs. Future-proof your application by making vendor optionality part of your LLM infrastructure design.

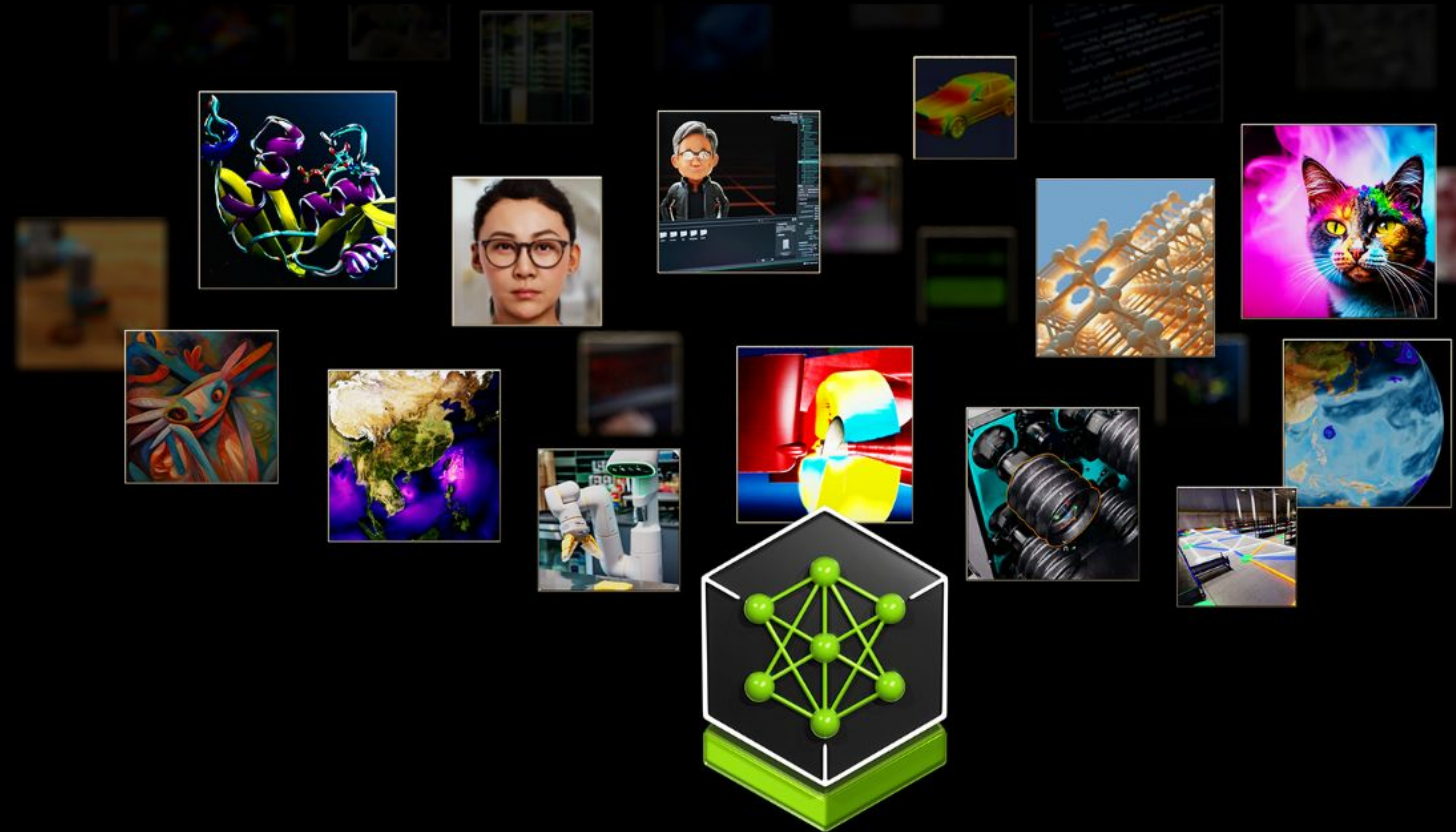
[Learn more about LangChain ↗](#)

Chat Applications

Full Applications Build with LLMs

Instantly Run and Deploy Generative AI

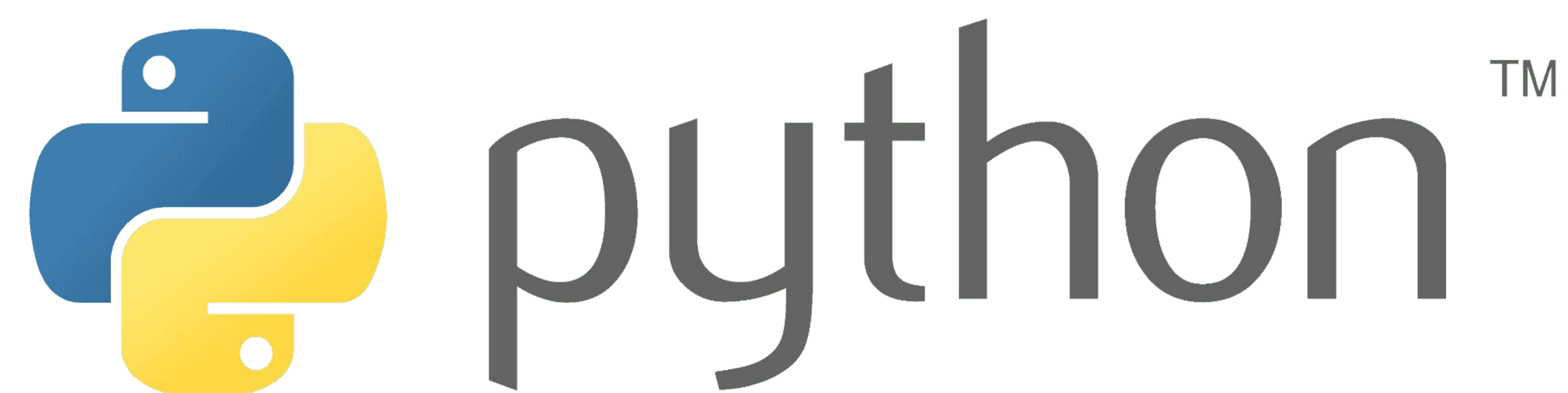
Explore the latest community-built AI models with APIs optimized and accelerated by NVIDIA, then deploy anywhere with NVIDIA NIM.



Prerequisites

RAG Agents in Production

- Prior LLM/LangChain Exposure
- Intermediate Python Experience
- Exposure to Web Engineering



Course Objectives

RAG Agents in Production

- Environment
- LLM Services
- Intro to LangChain (LCEL)
- Running State Chains
- Document Loading
- Embeddings
- Document Retrieval
- RAG Evaluation

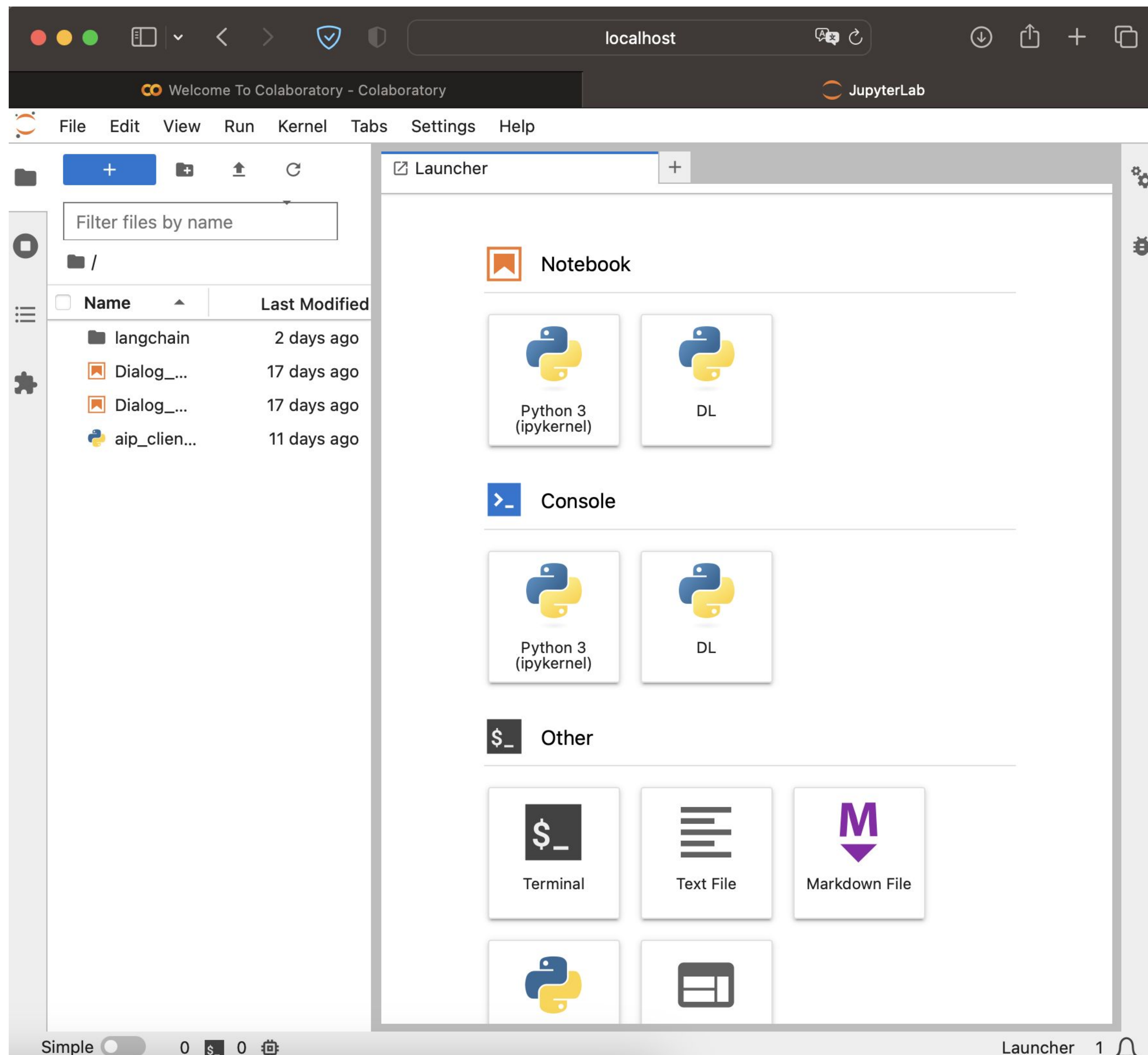




Building RAG Agents with LLMs

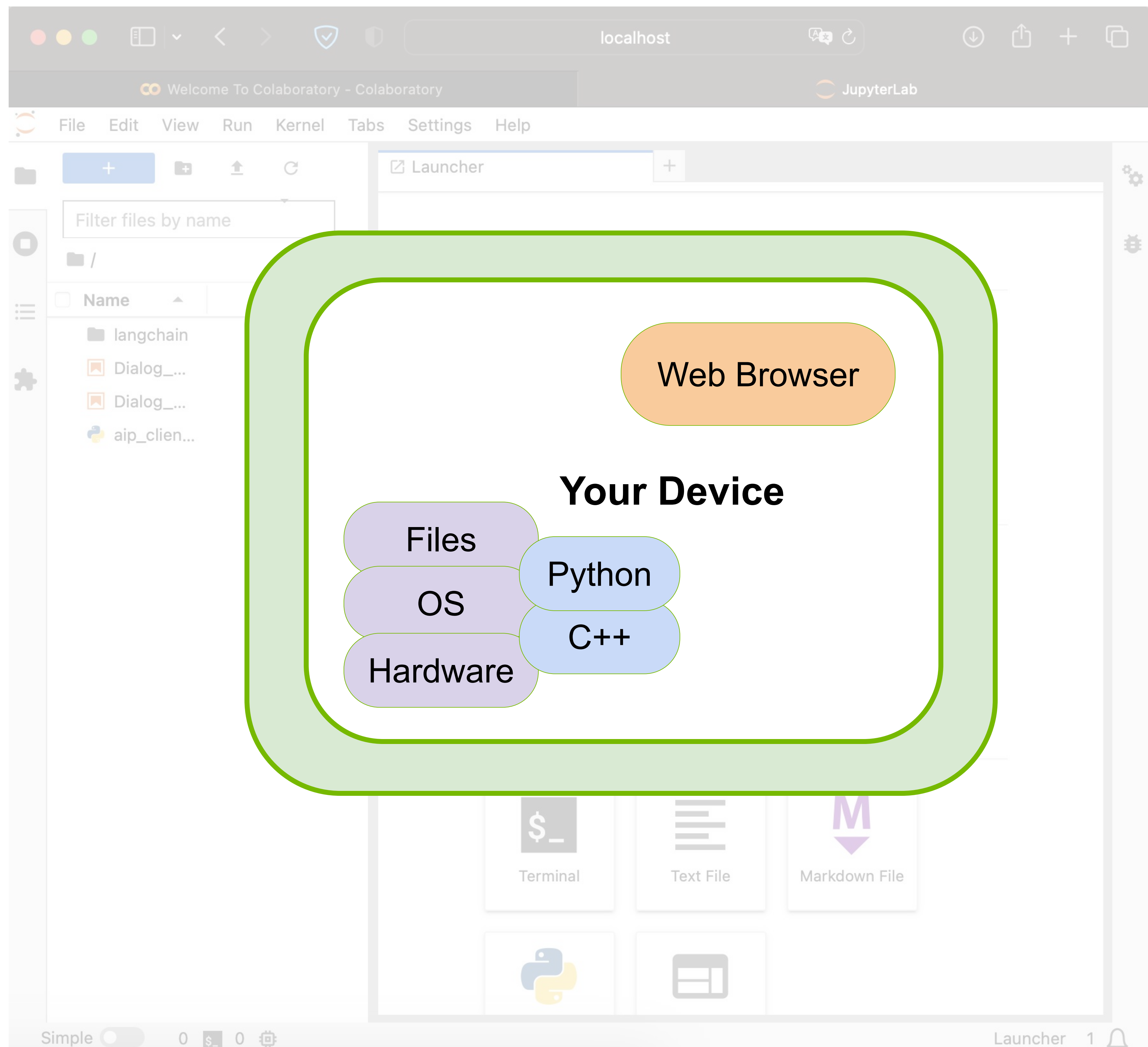
Part 1: Your Course Environment

Typical Jupyter Labs Interface



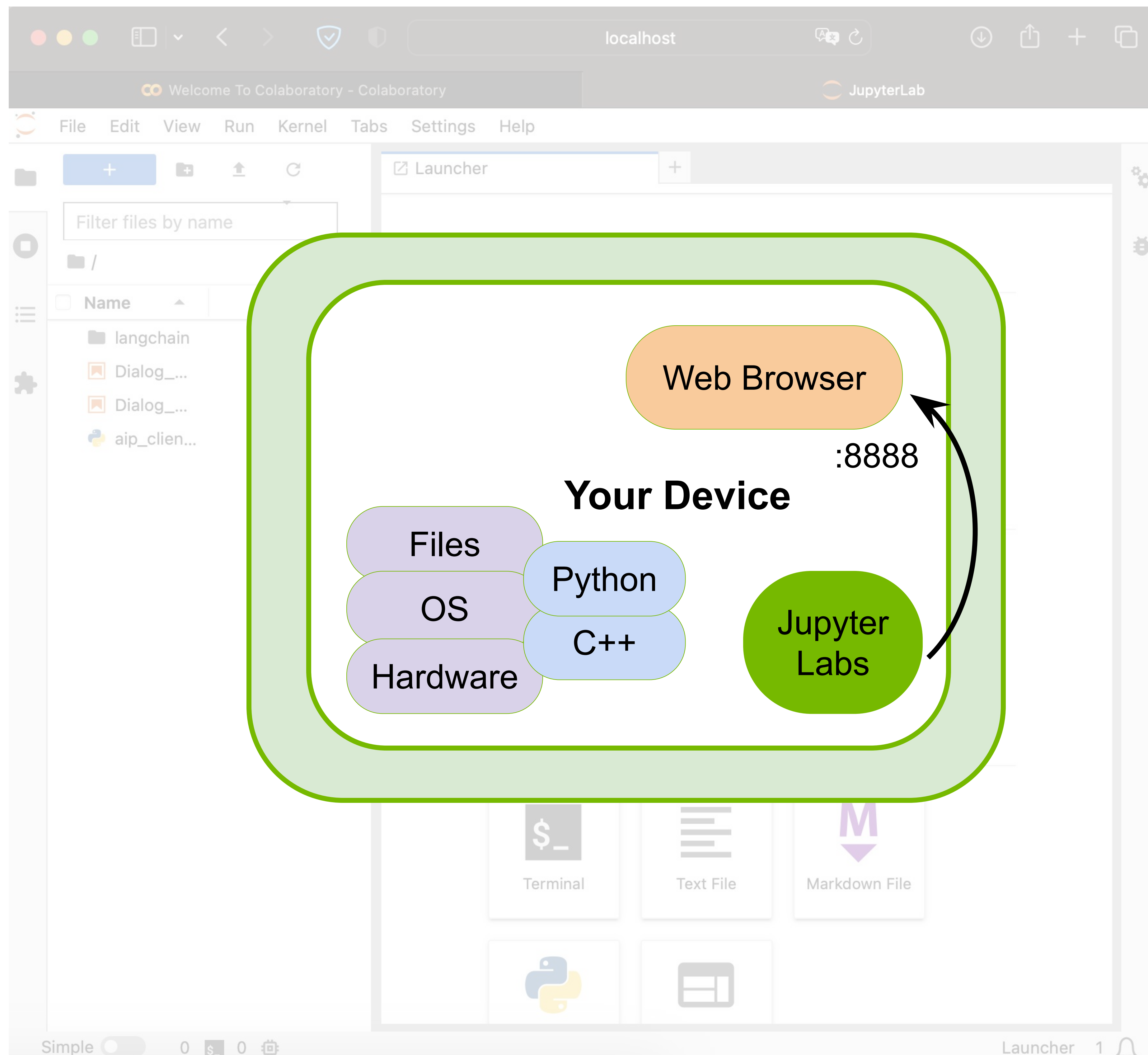
jupyter lab . &; open -a Safari <http://localhost:8888/lab>

Typical Jupyter Labs Interface



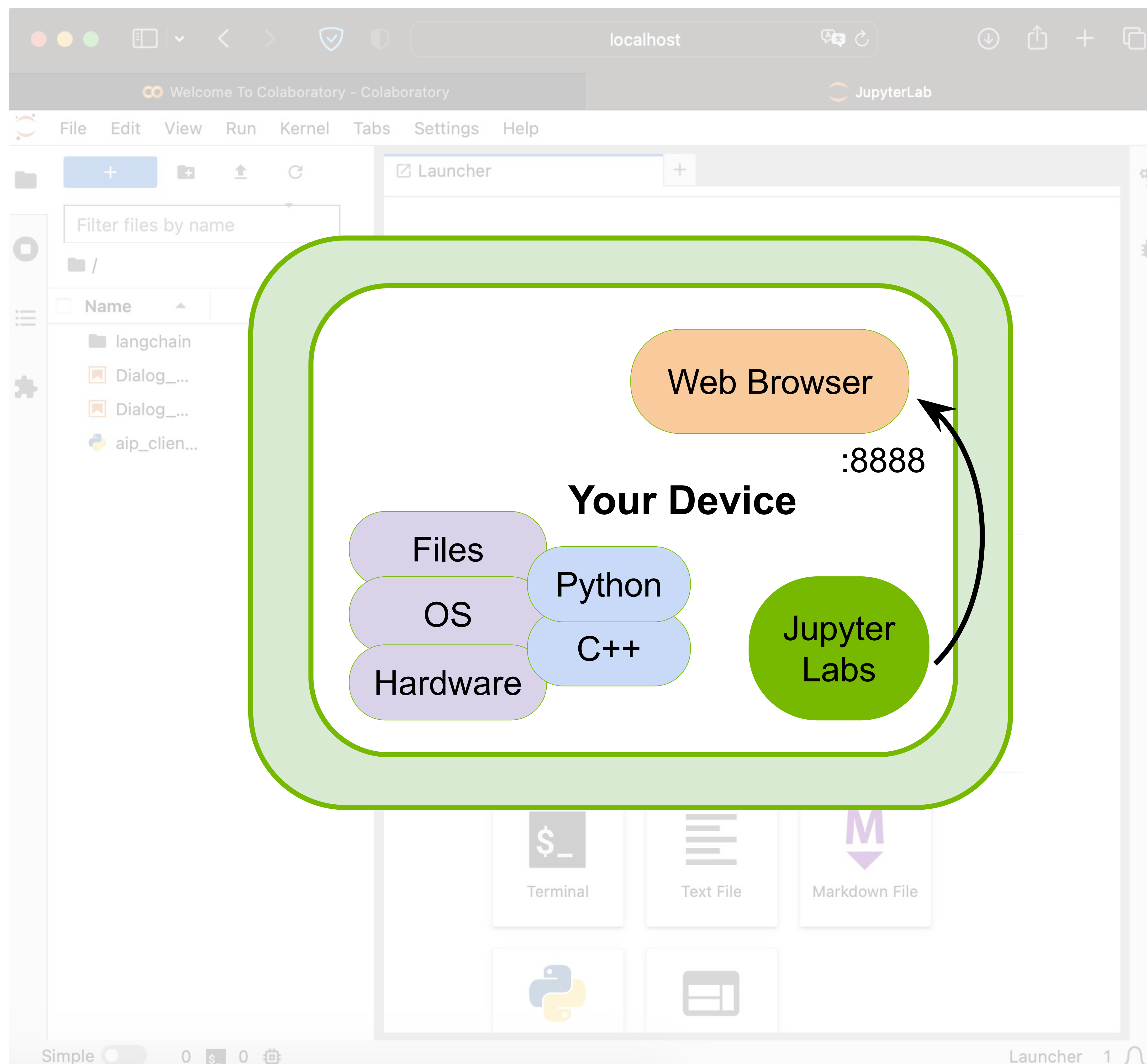
jupyter lab . &; open -a Safari <http://localhost:8888/lab>

Typical Jupyter Labs Interface

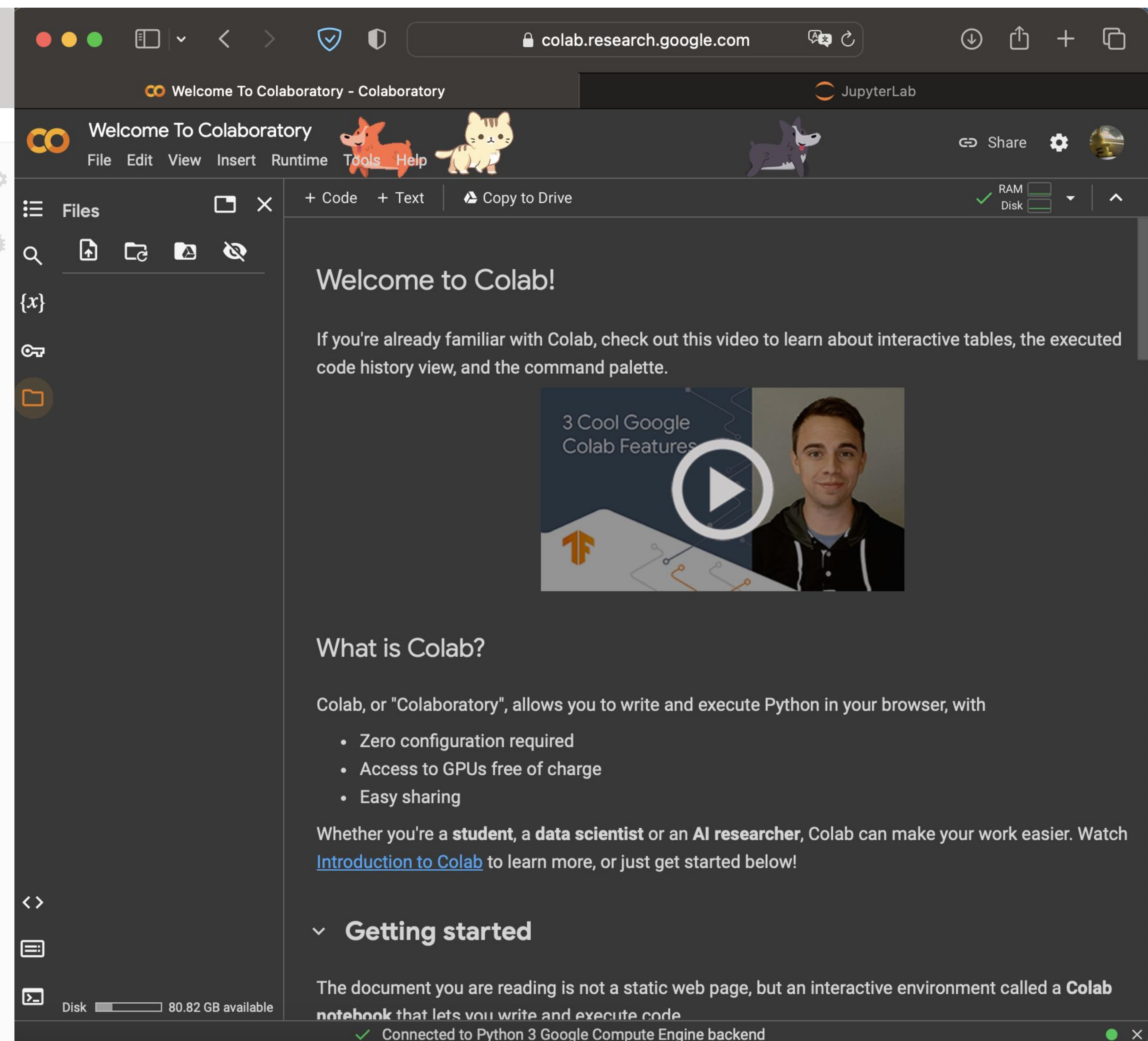


`jupyter lab . &; open -a Safari http://localhost:8888/lab`

Typical Jupyter Labs Interface

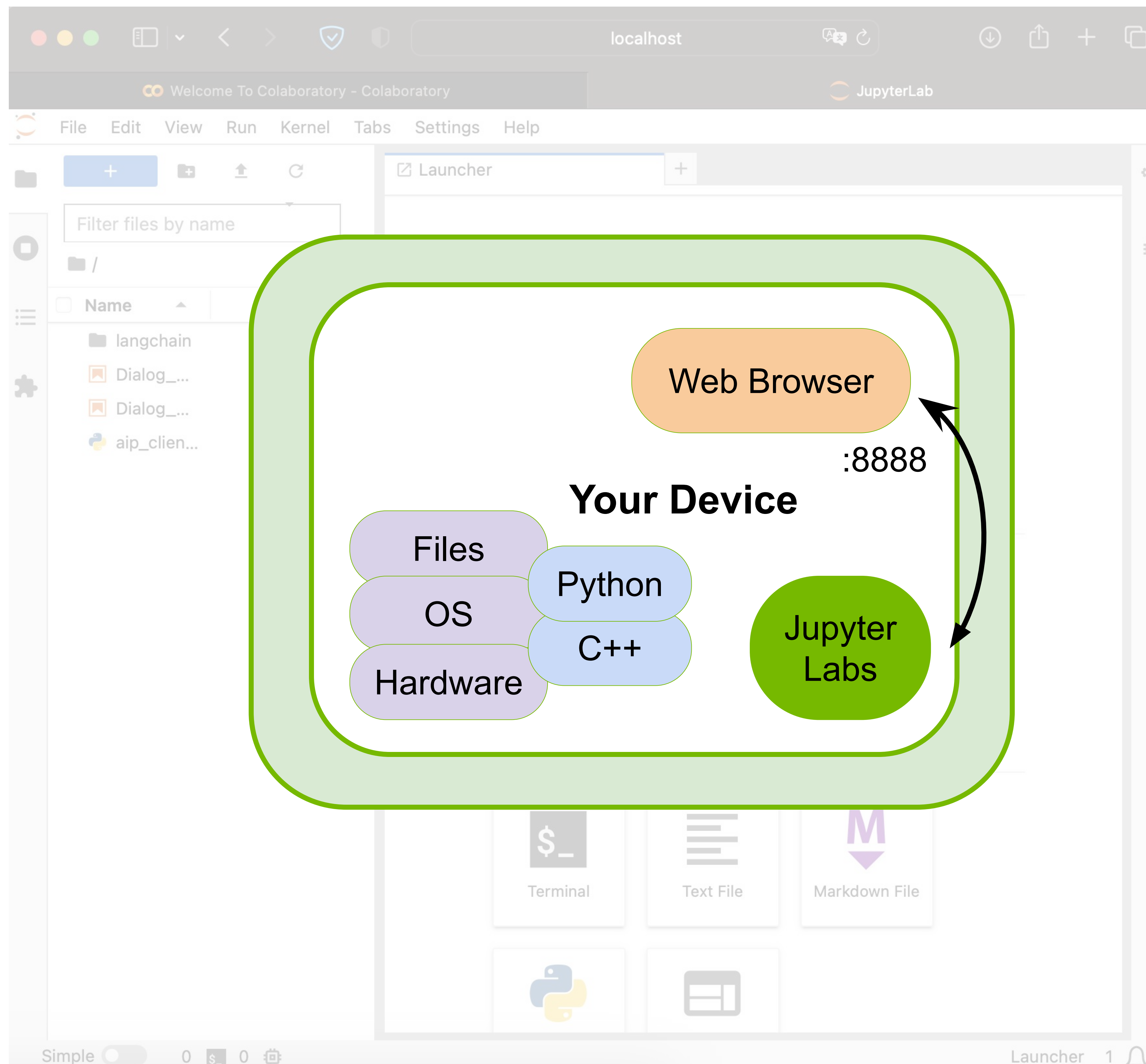


jupyter lab . &; open -a Safari <http://localhost:8888/lab>

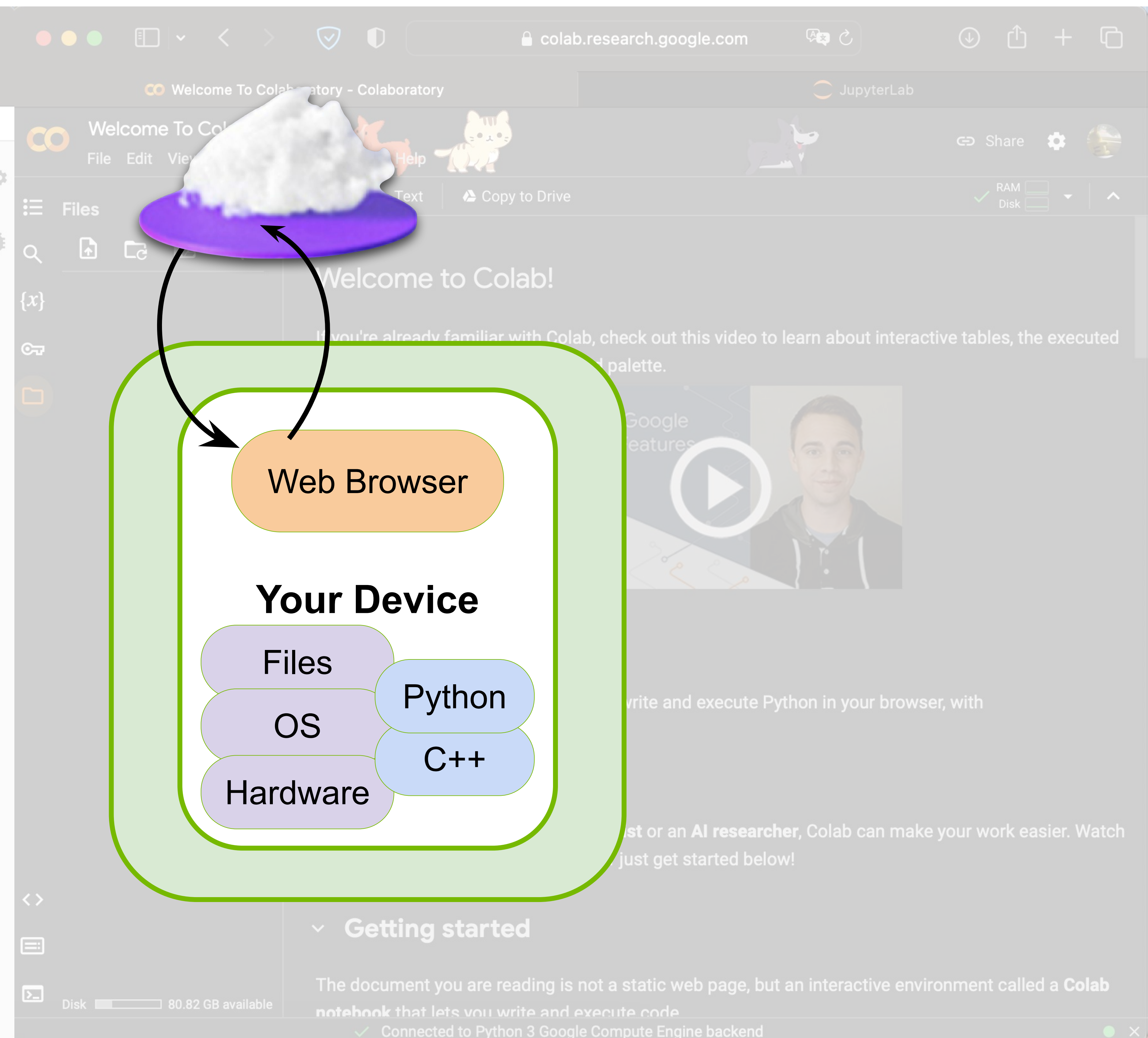


<https://colab.research.google.com>

Typical Jupyter Labs Interface

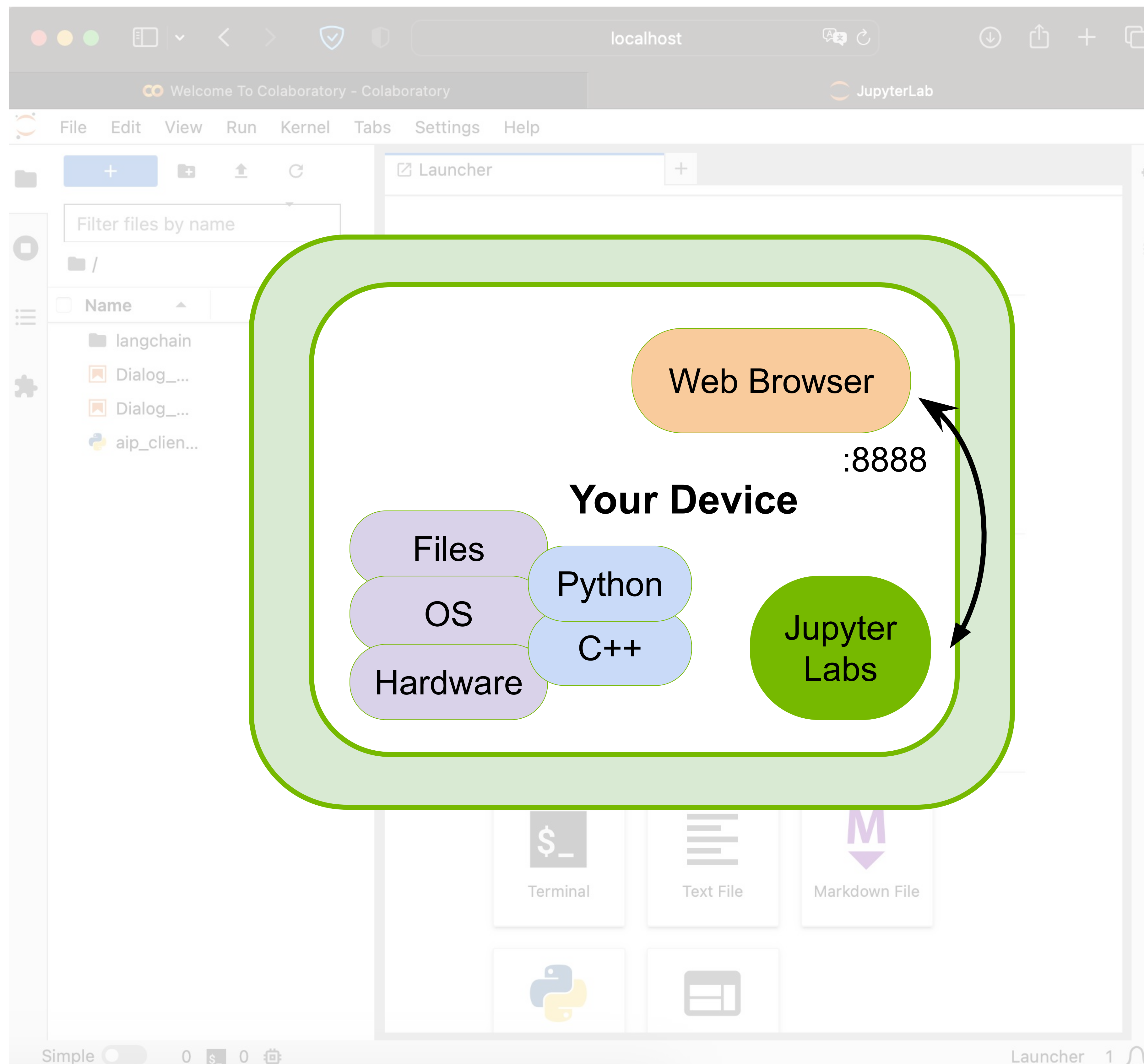


jupyter lab . &; open -a Safari <http://localhost:8888/lab>

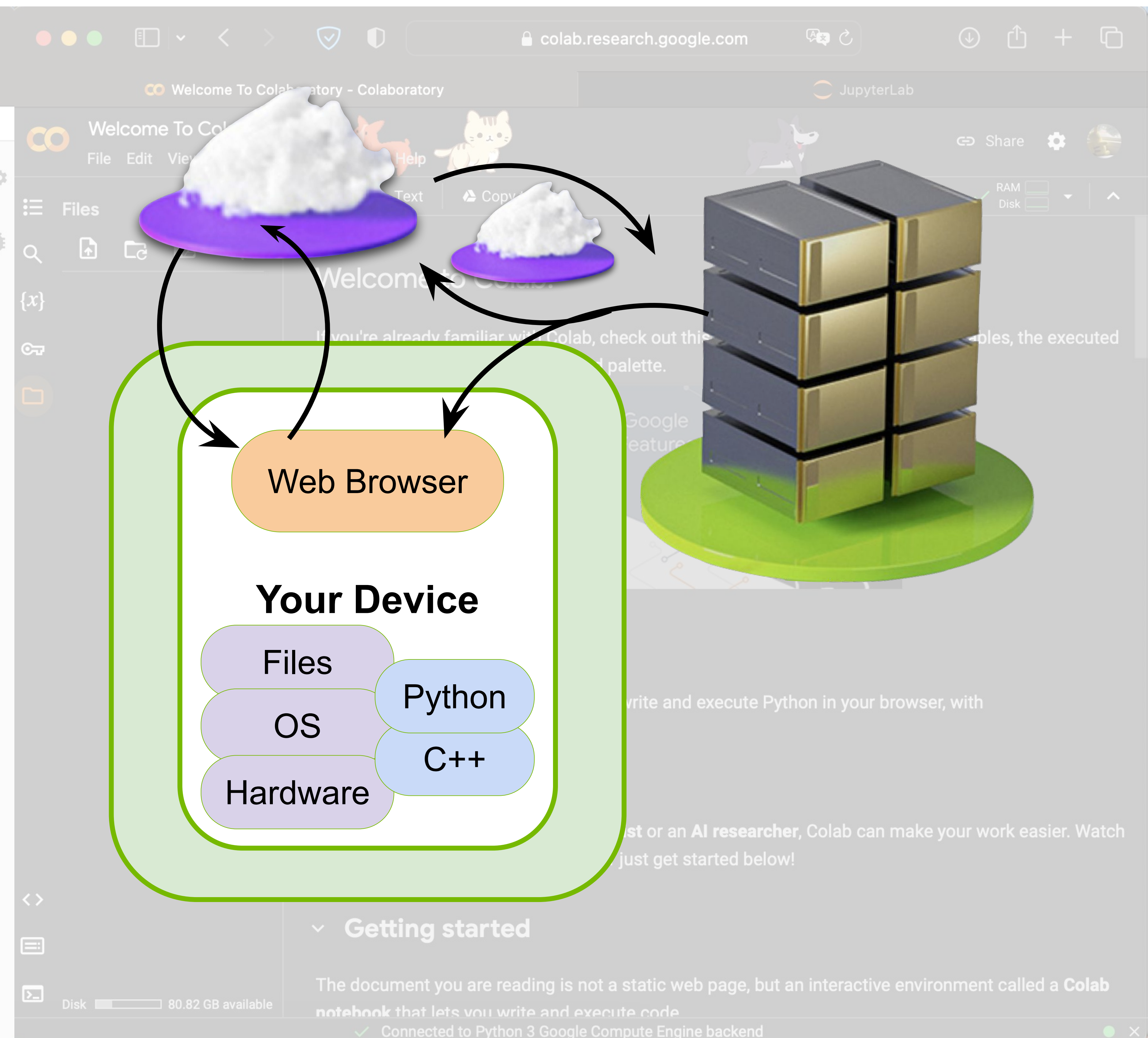


<https://colab.research.google.com>

Typical Jupyter Labs Interface

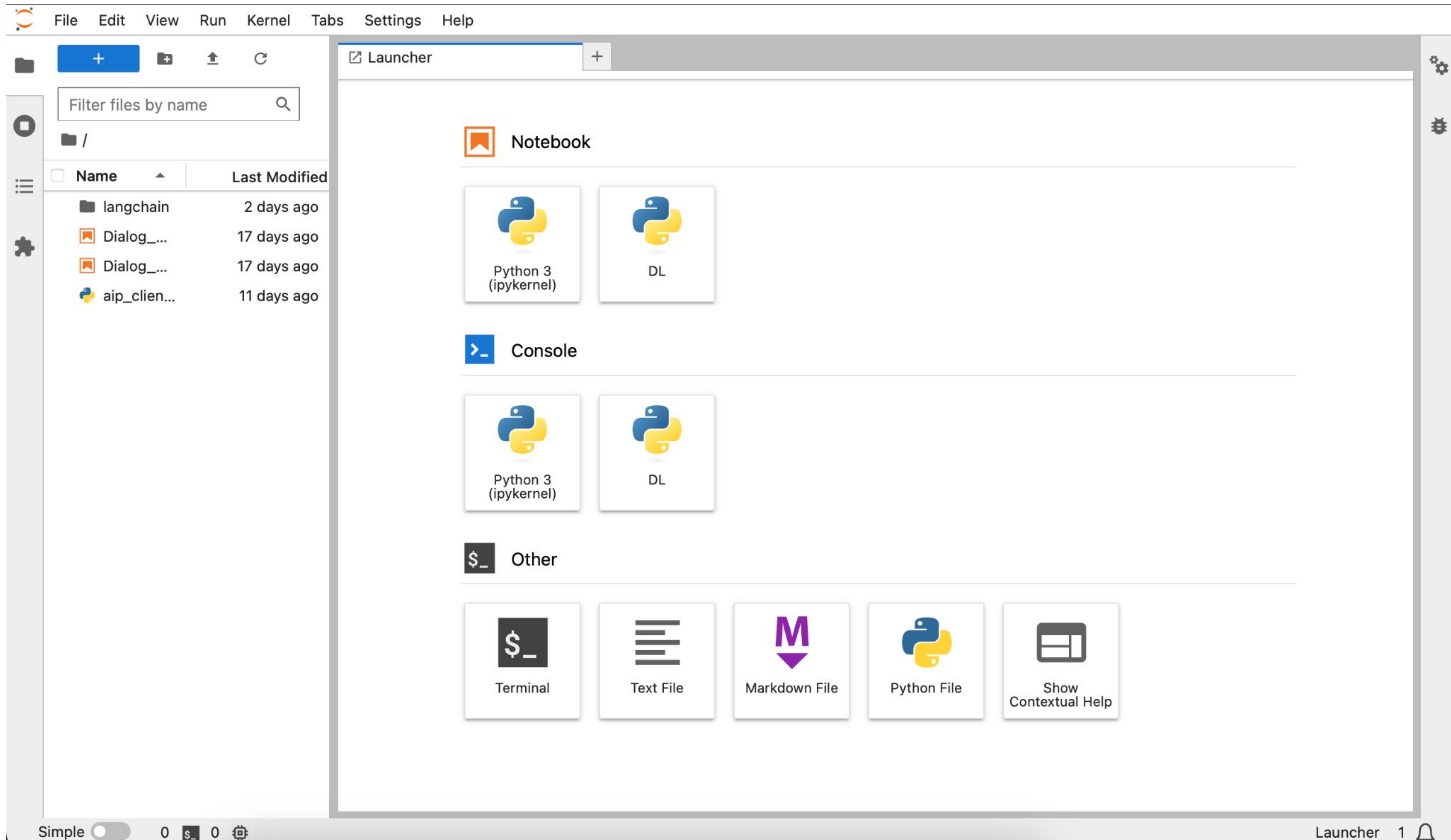


jupyter lab . &; open -a Safari <http://localhost:8888/lab>

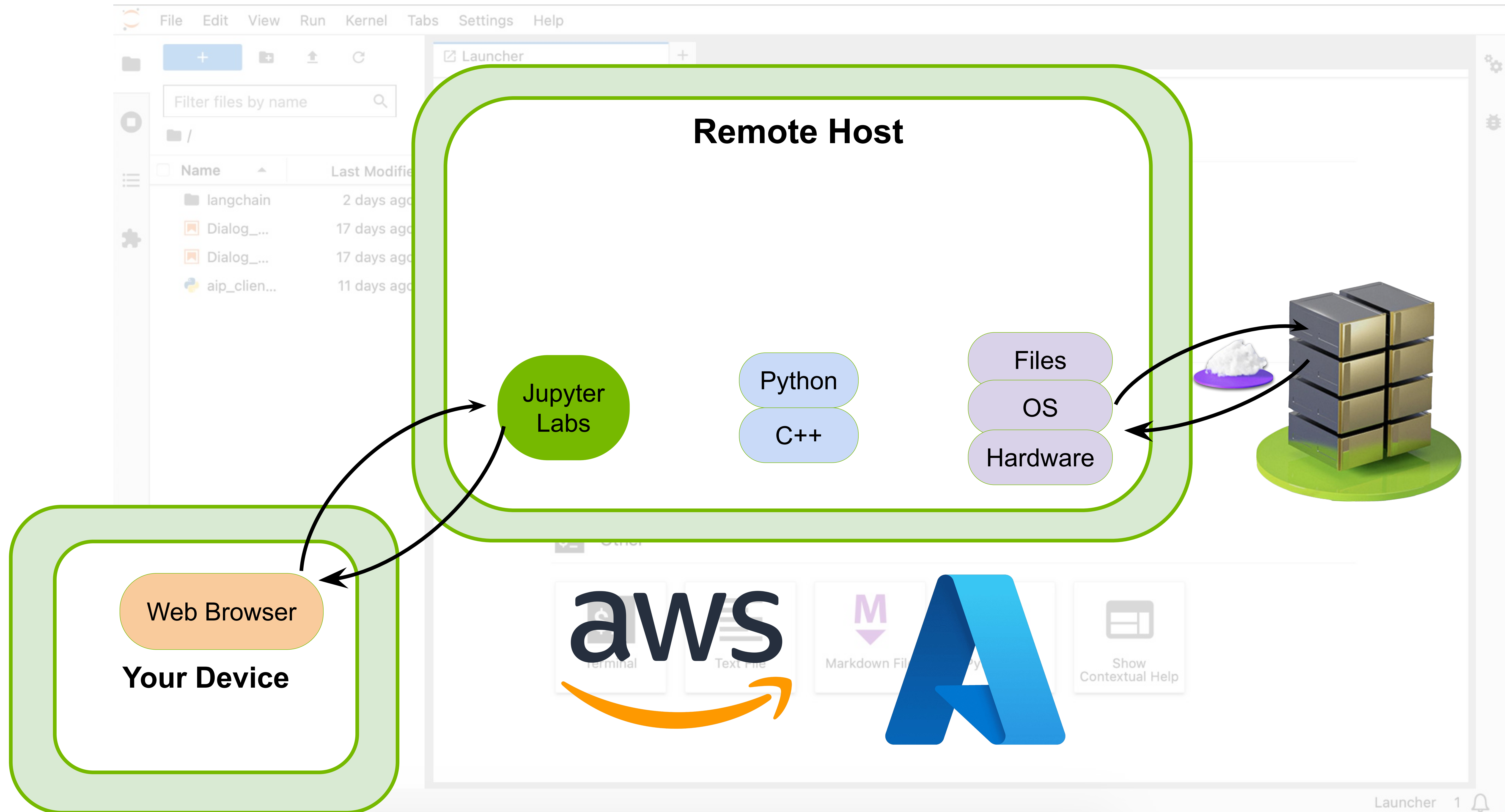


<https://colab.research.google.com>

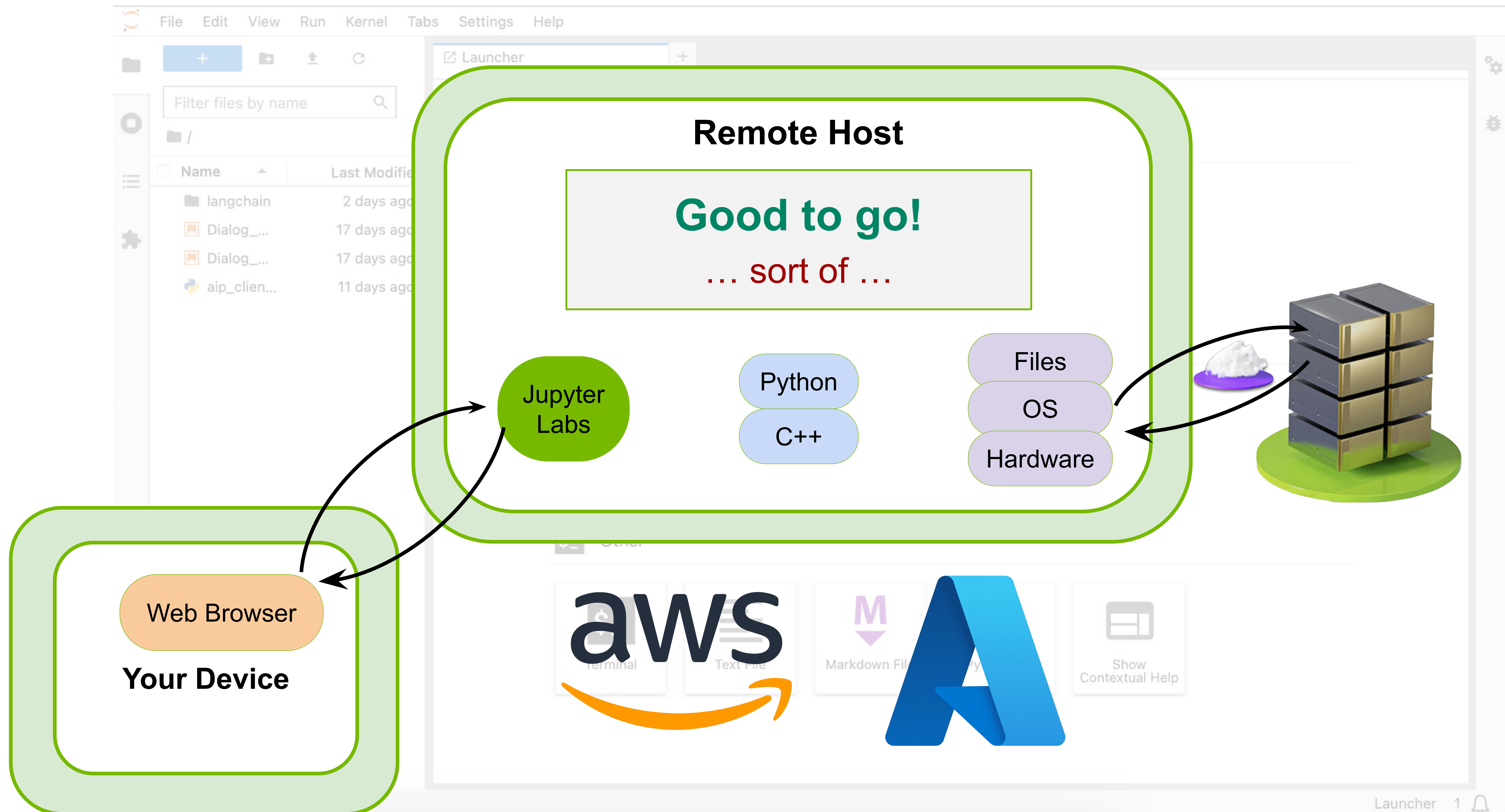
DLI Jupyter Labs Interface



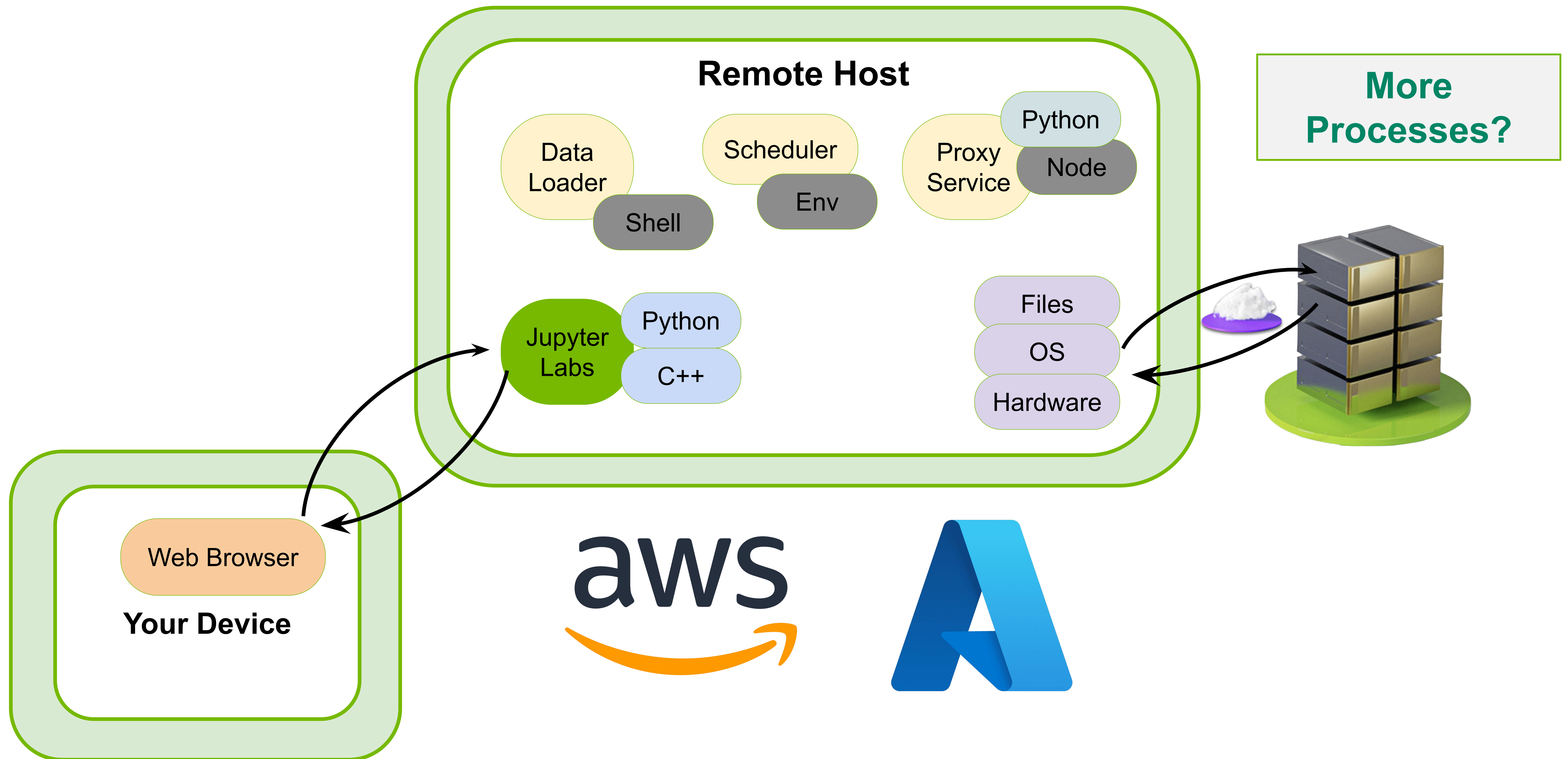
DLI Jupyter Labs Interface



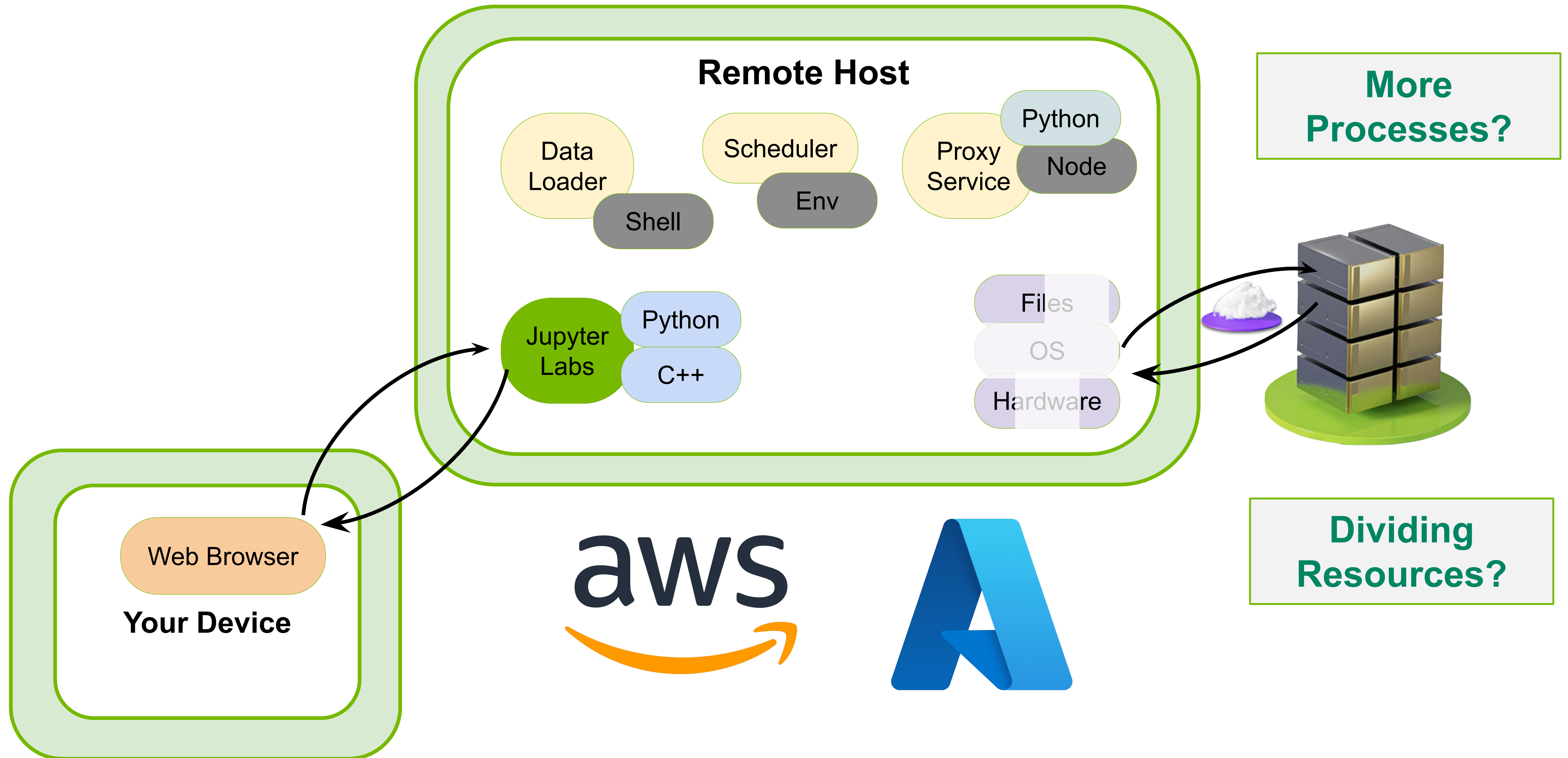
DLI Jupyter Labs Interface



DLI Jupyter Labs Interface

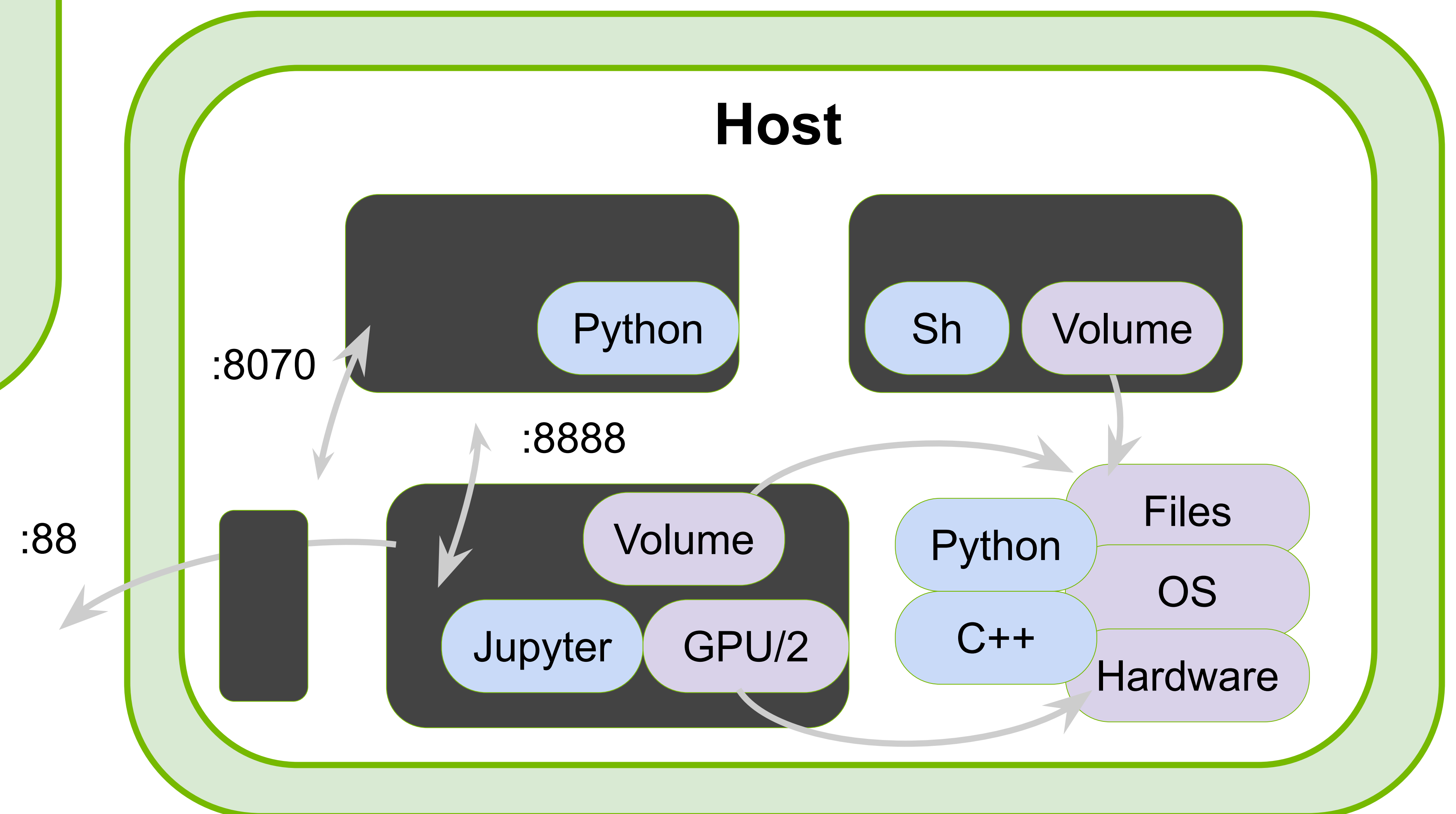
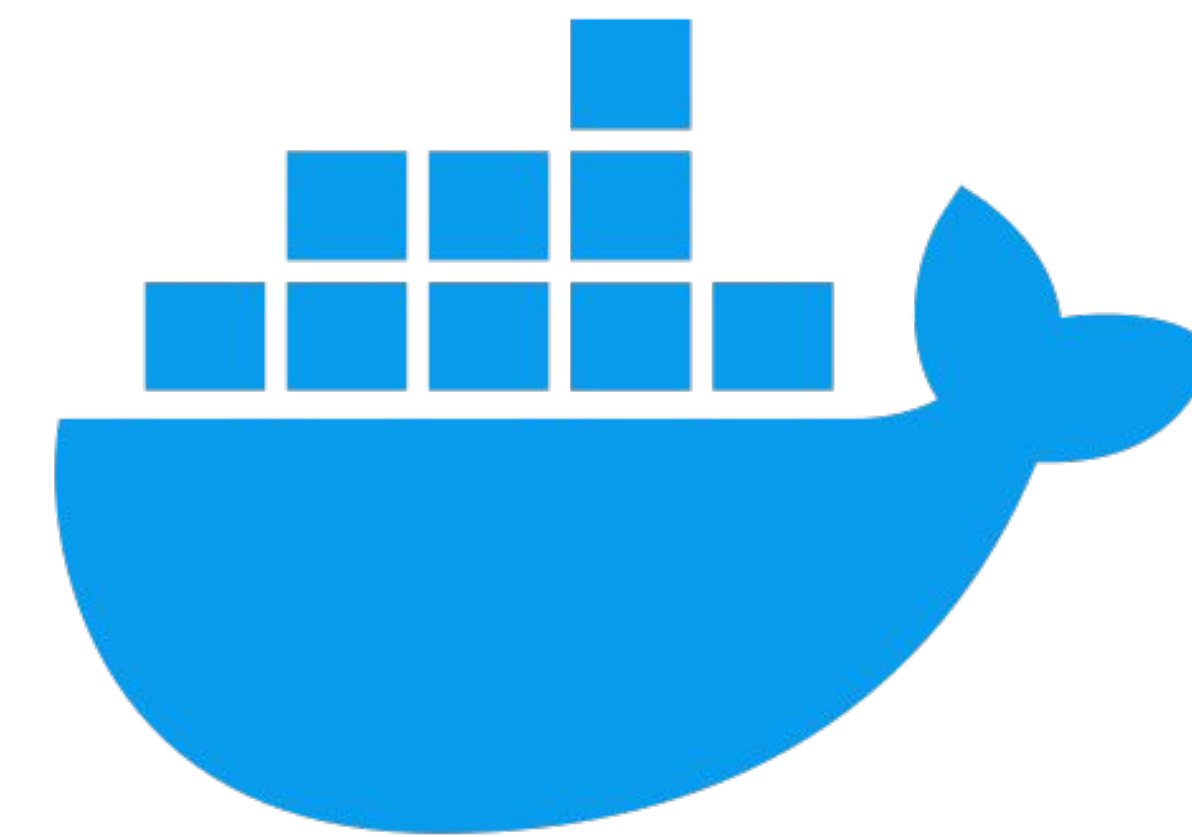
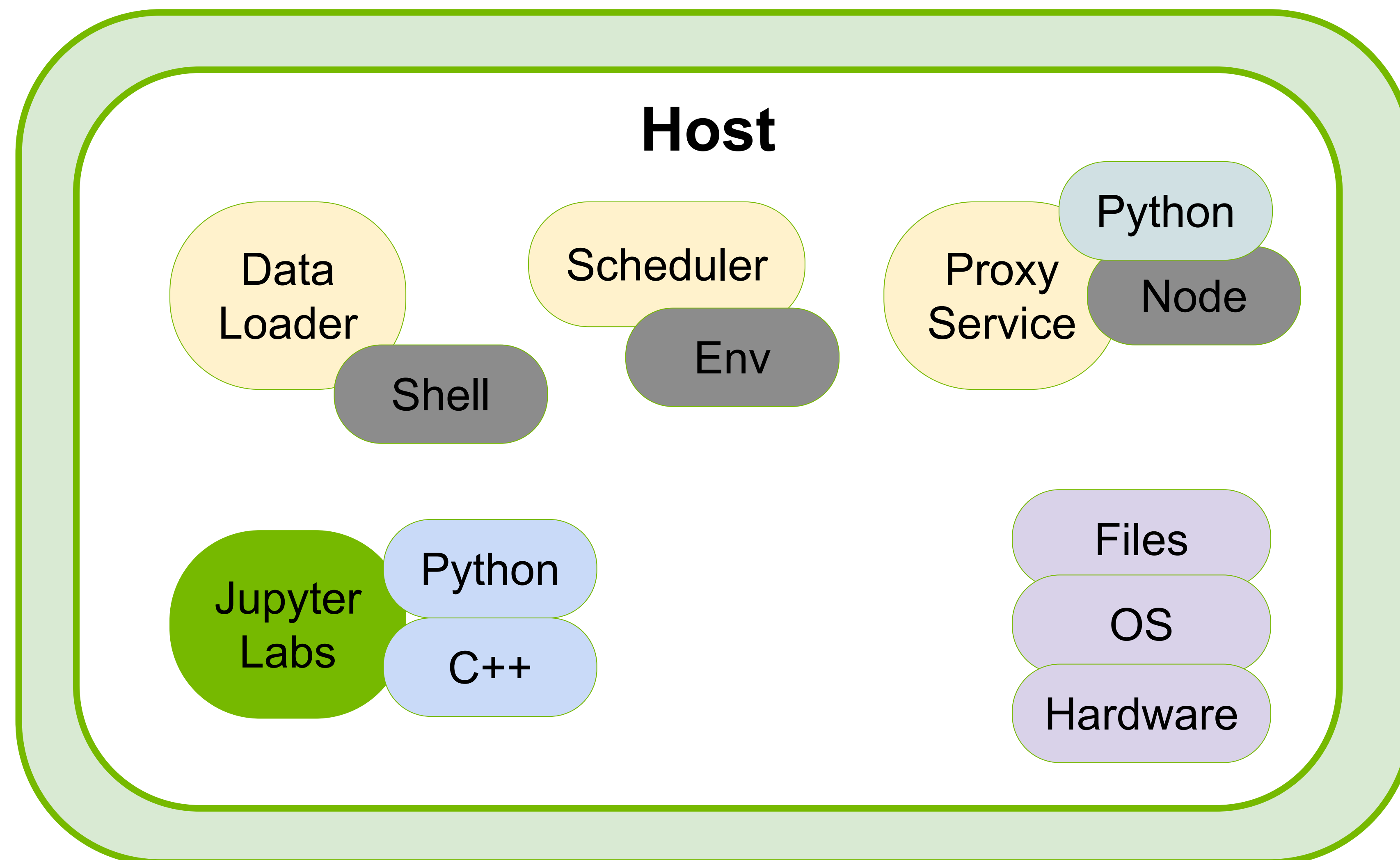


DLI Jupyter Labs Interface



Containerization with Docker

Compartmentalizing Functionality Into Microservices



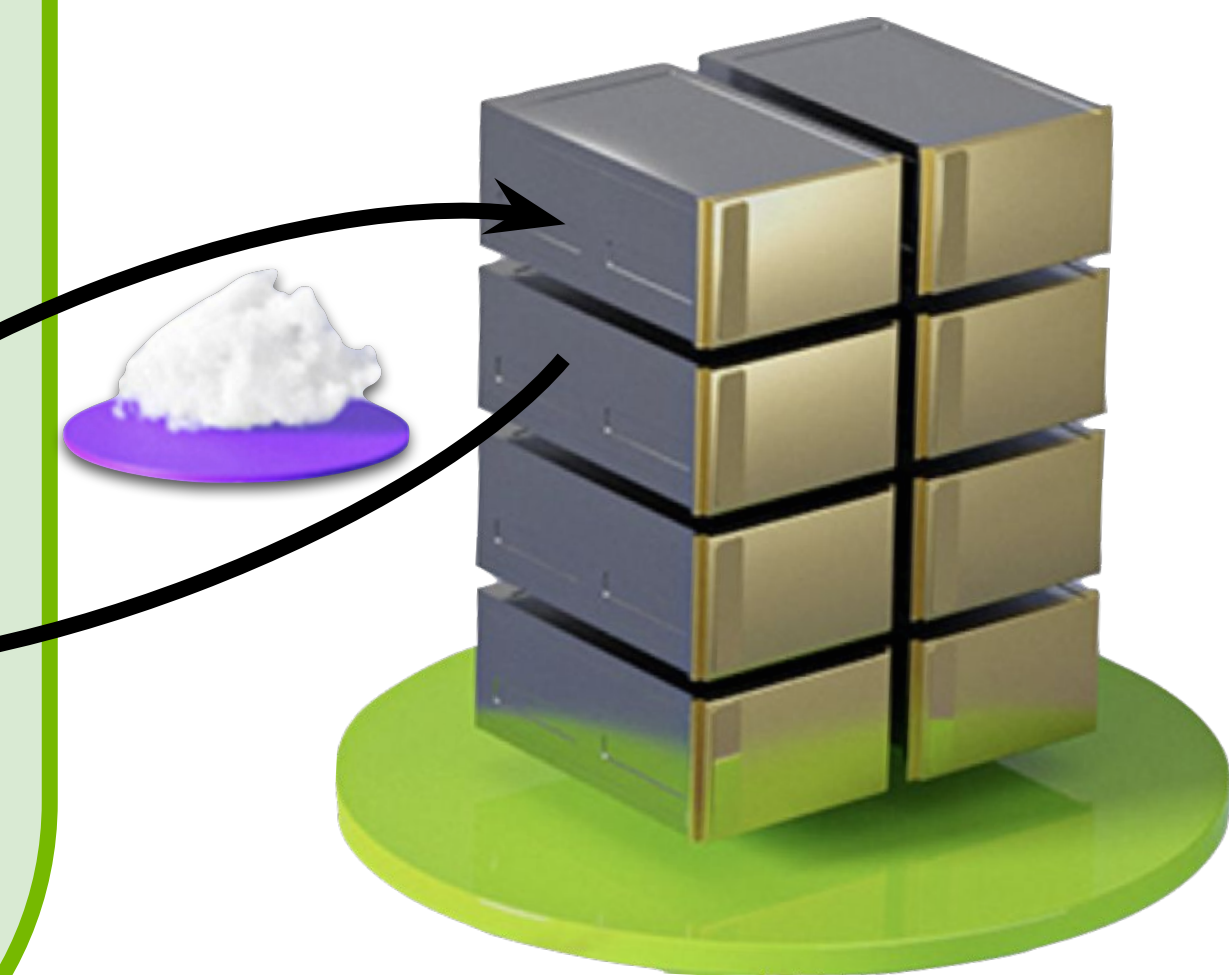
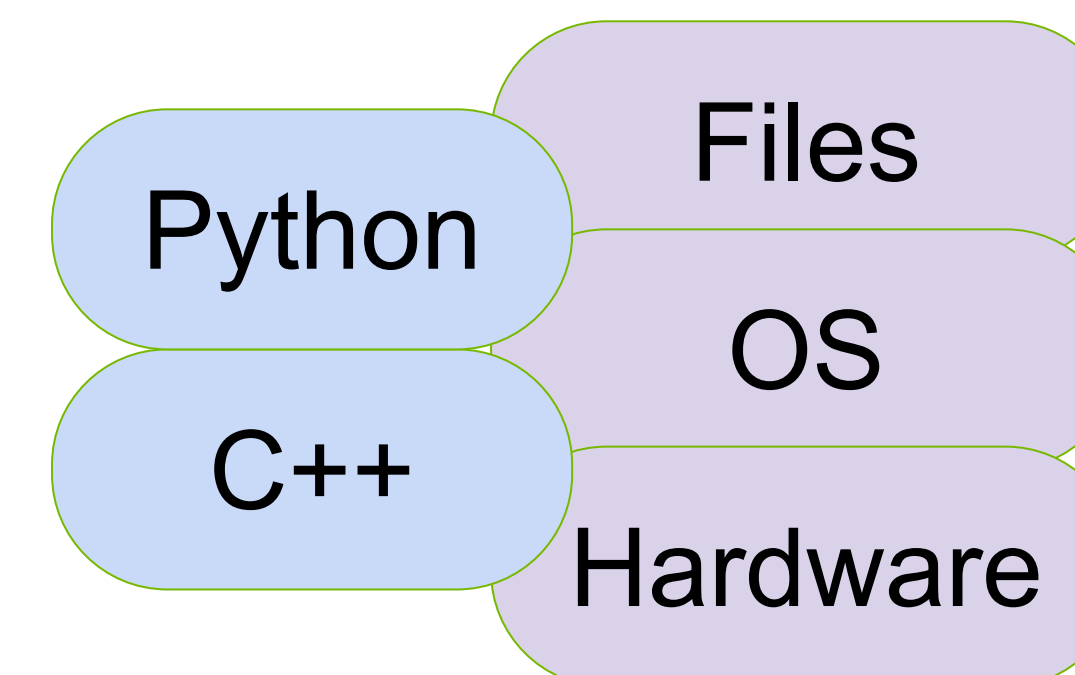
Microservices Workflow



Microservices Workflow

1. Allocate Resources
2. Define Services
3. Construct Containers
4. Start Processes

Remote Host

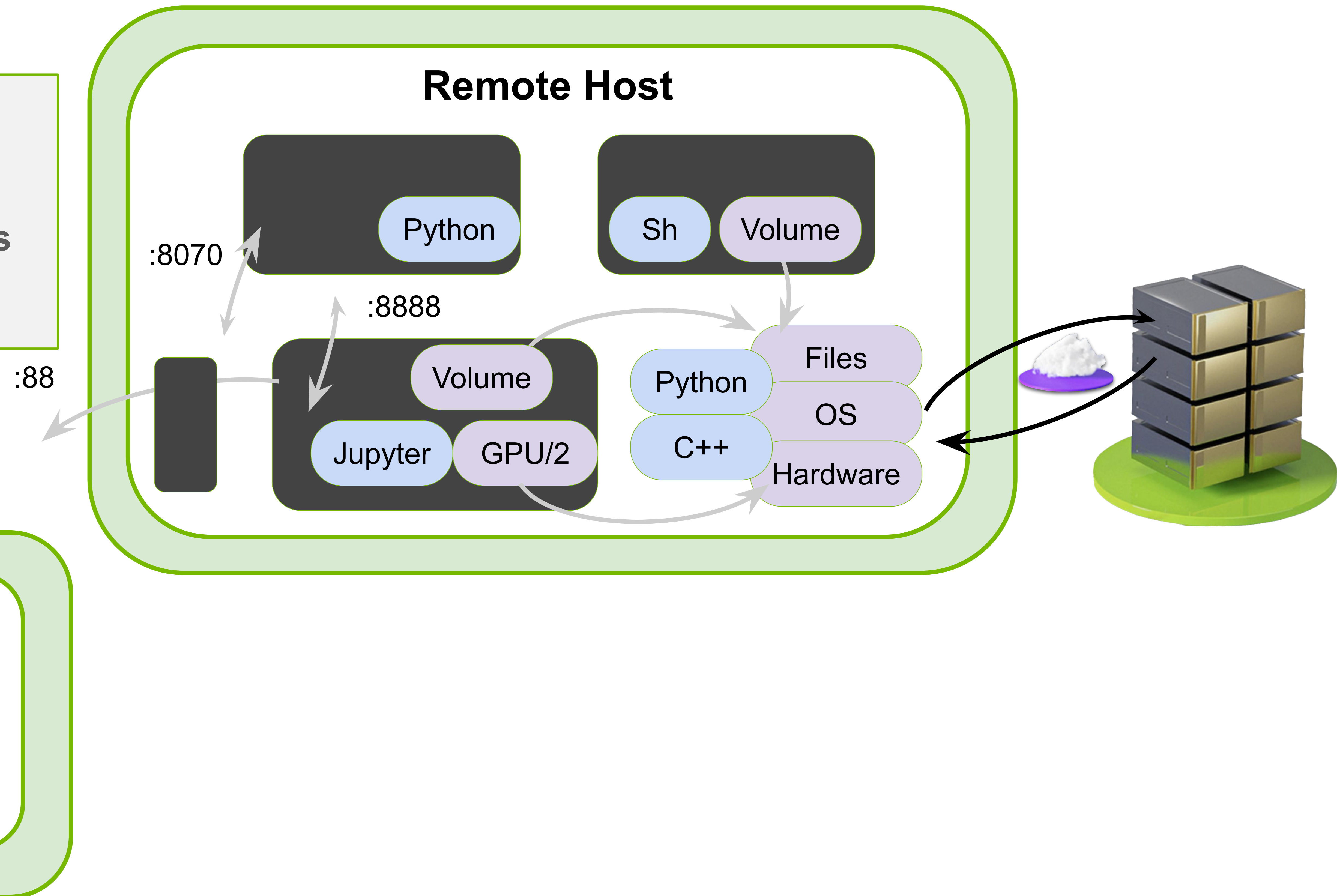


Web Browser

Your Device

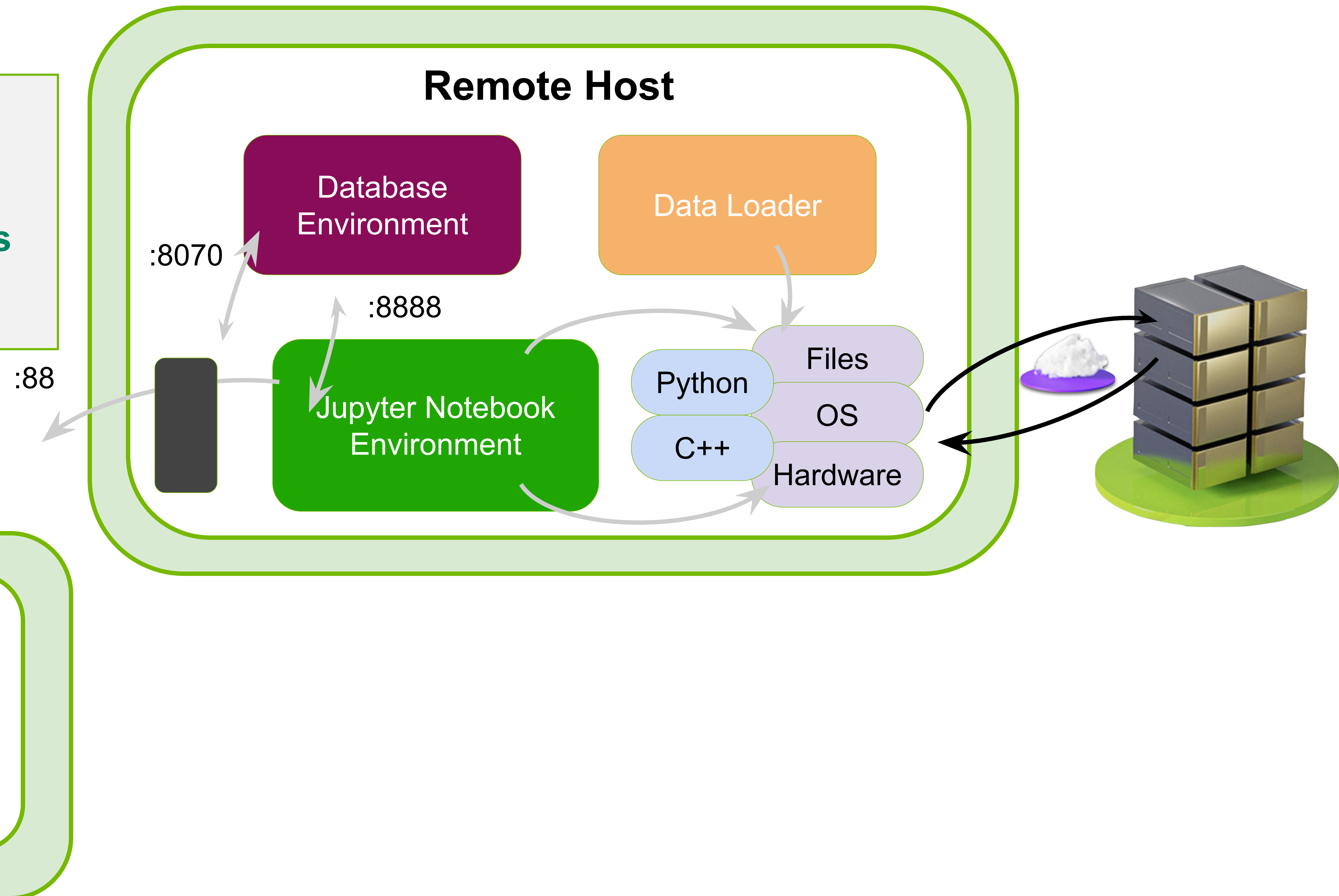
Microservices Workflow

1. Allocate Resources
2. Define Services
3. Construct Containers
4. Start Processes



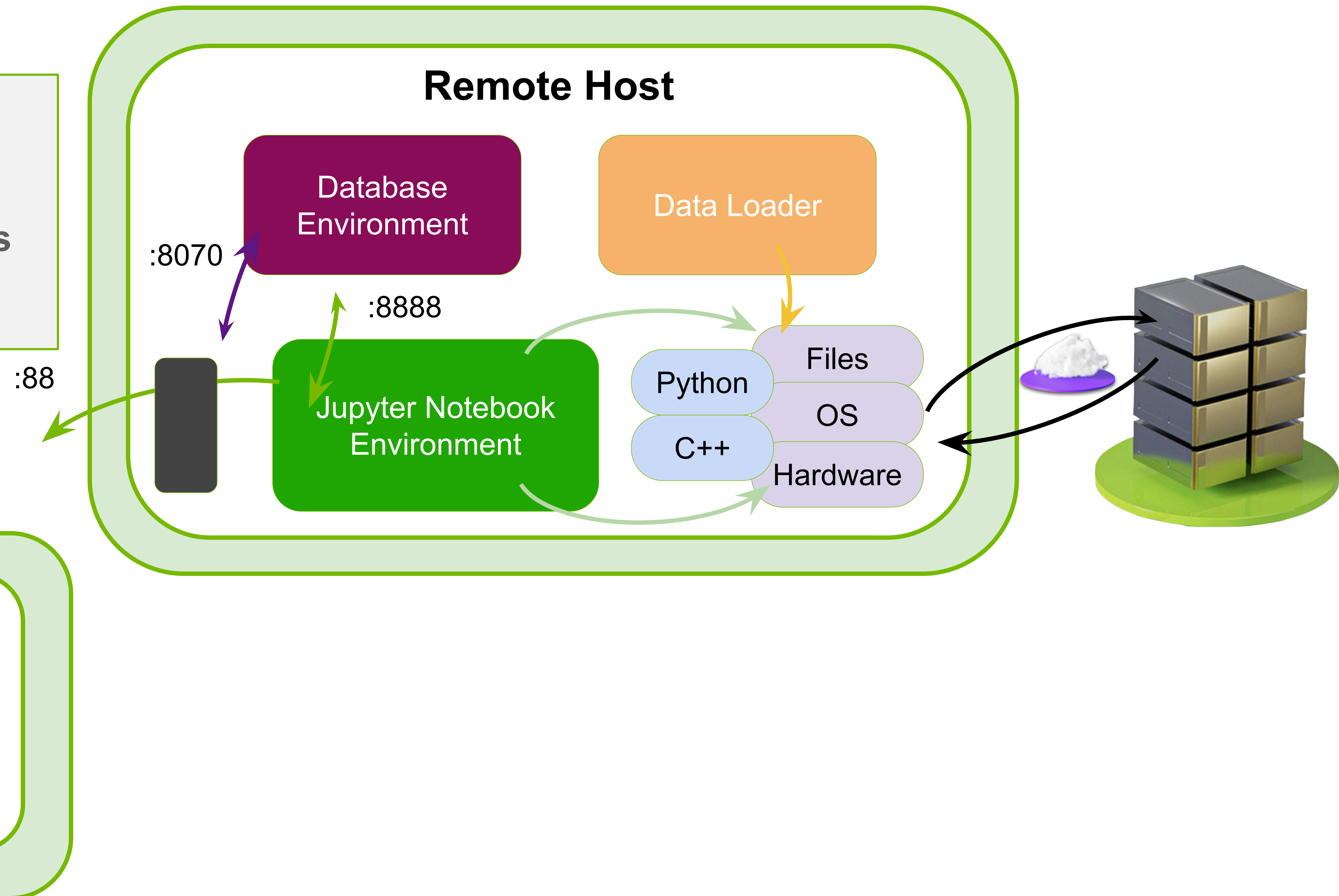
Microservices Workflow

1. Allocate Resources
2. Define Services
3. Construct Containers
4. Start Processes

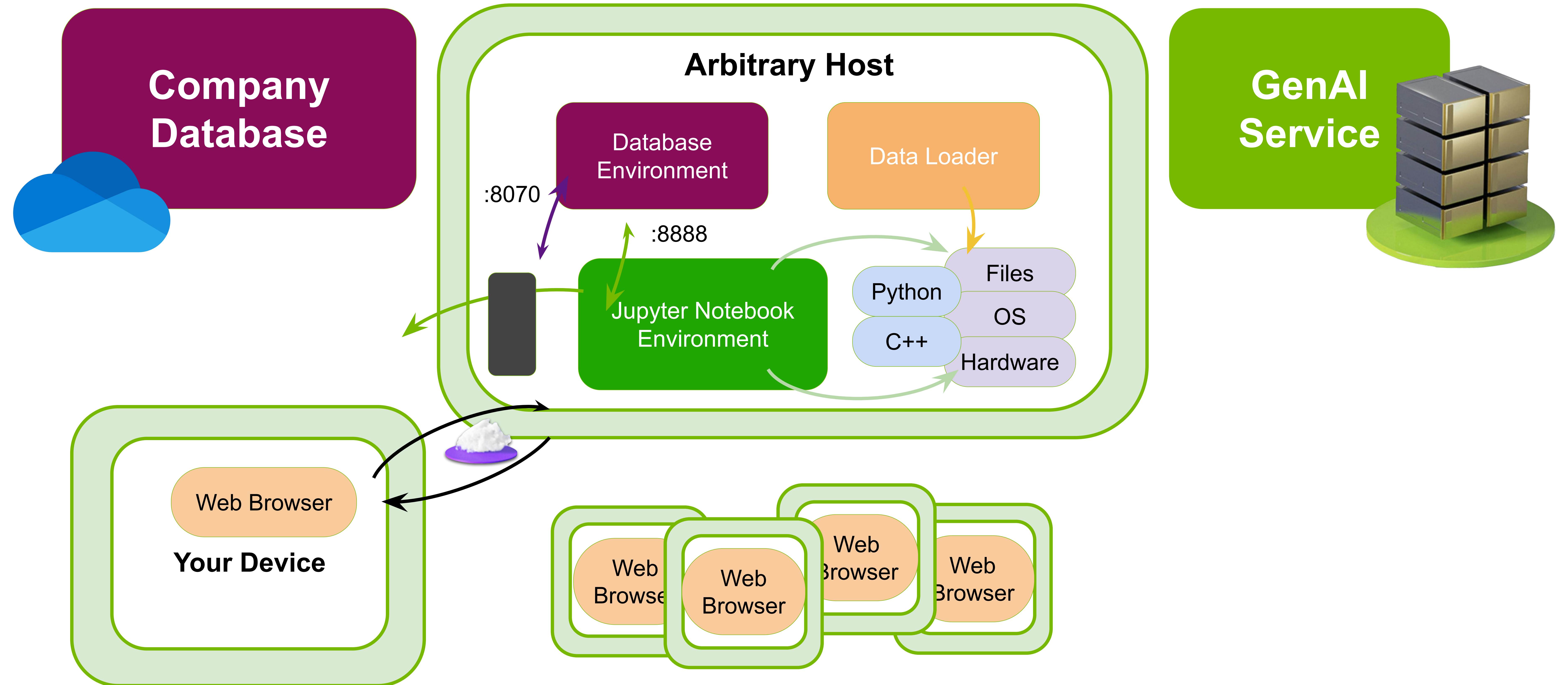


Microservices Workflow

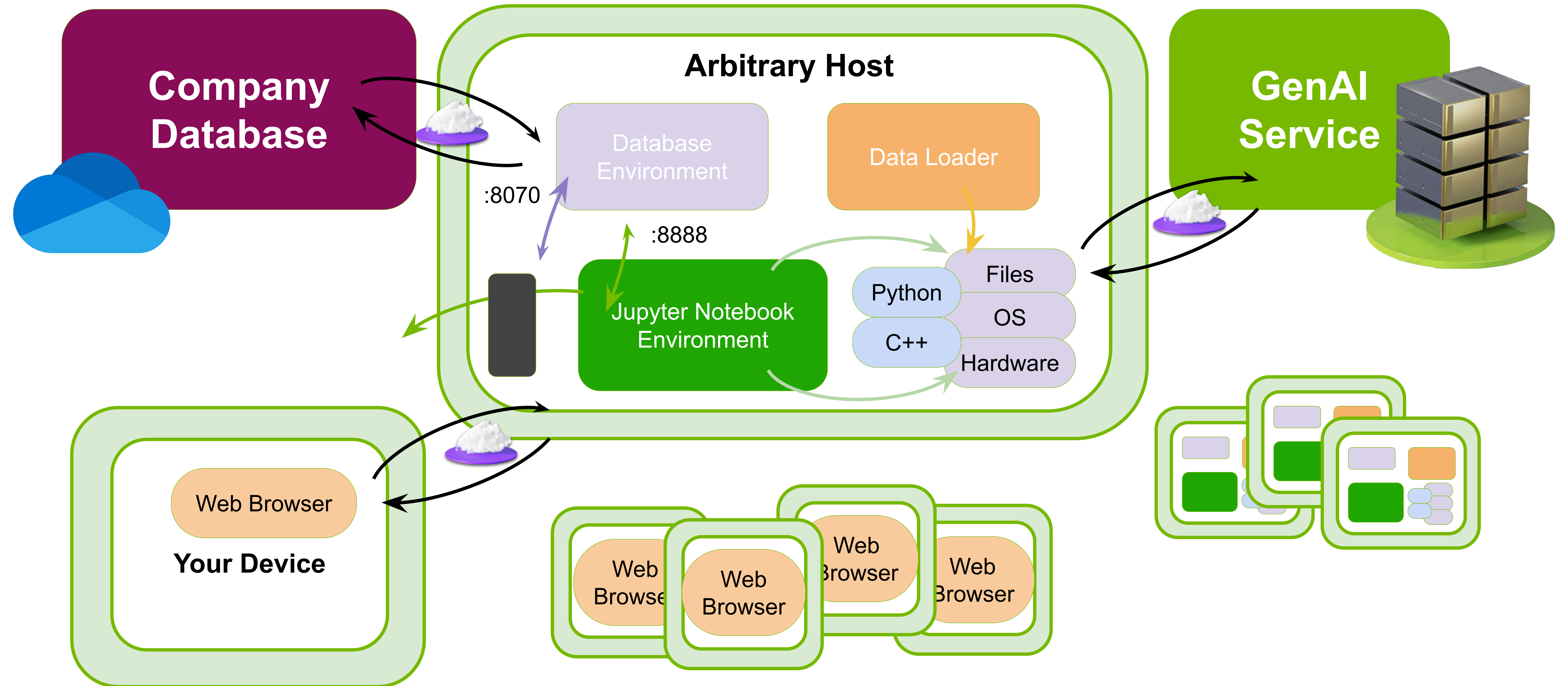
1. Allocate Resources
2. Define Services
3. Construct Containers
4. **Start Processes**



Scaling Containerized Applications

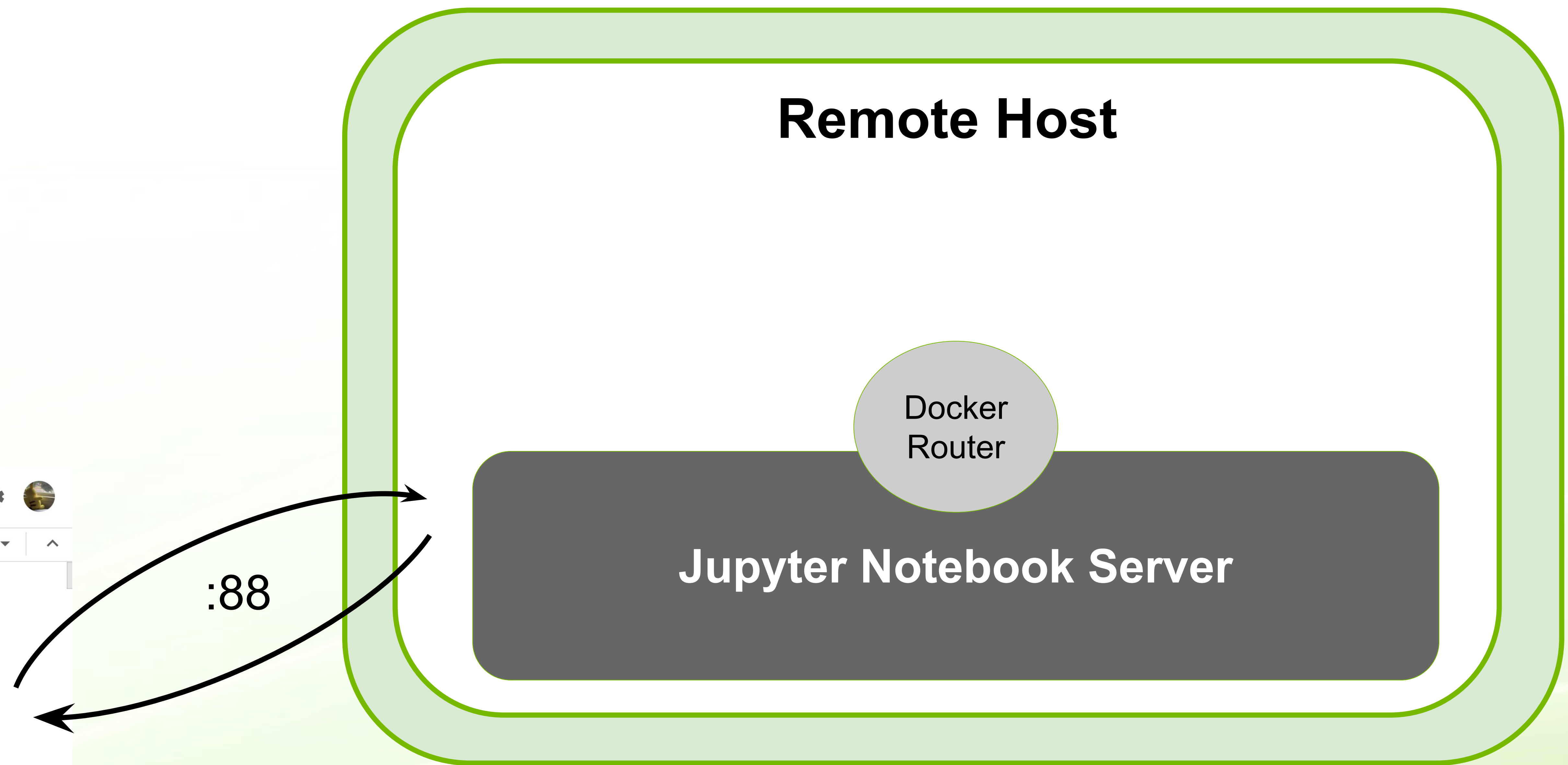
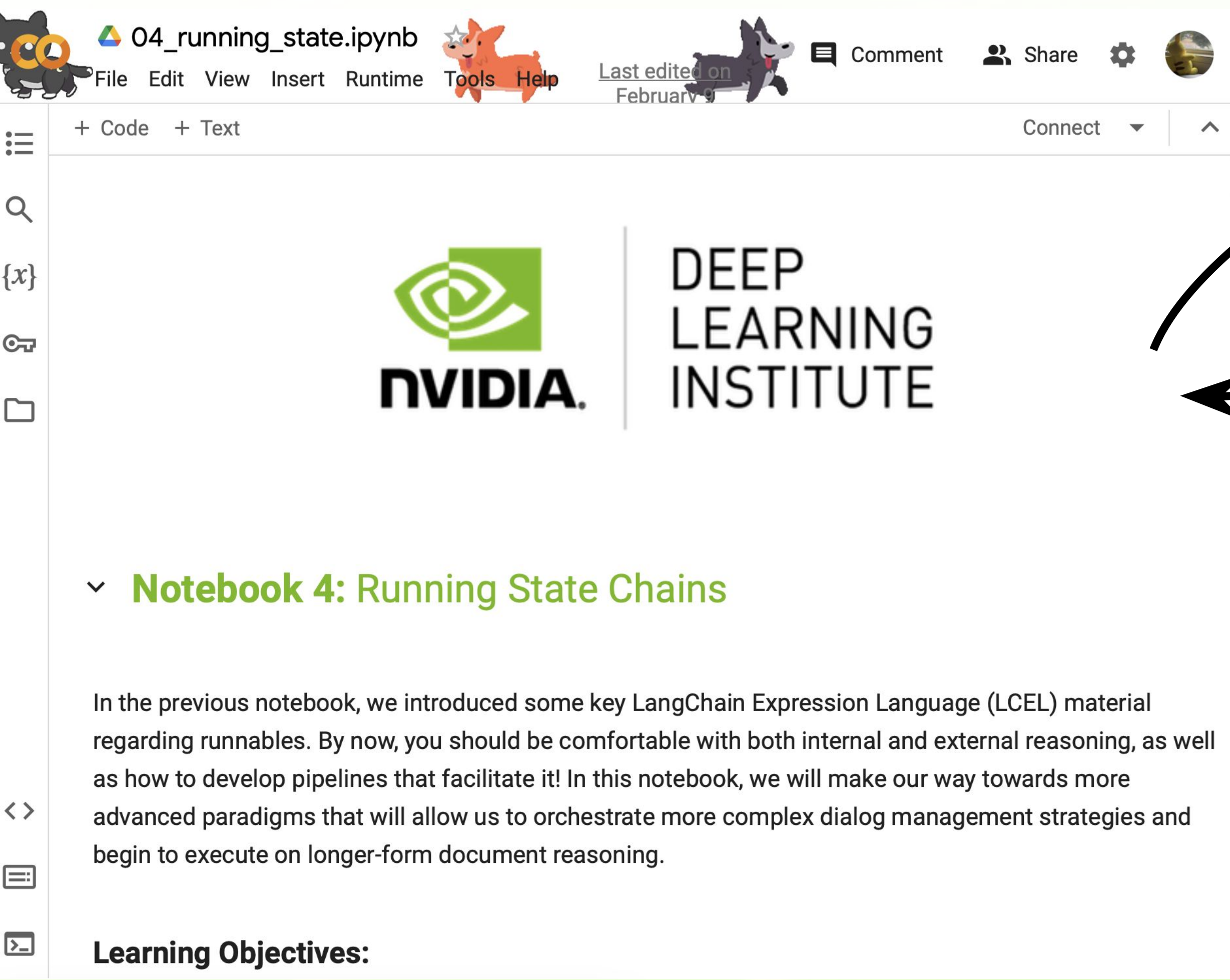


Scaling Containerized Applications



Our Environment

Jupyter Notebooks



Chatbot

Hello World!!

You typed: Hello World!!

Type a message...

Submit

Retry

Undo

Clear

04_running_state.ipynb

File Edit View Insert Runtime Tools Help

Last edited on February 9

Comment

Share

+ Code + Text

Connect

DEEP LEARNING INSTITUTE

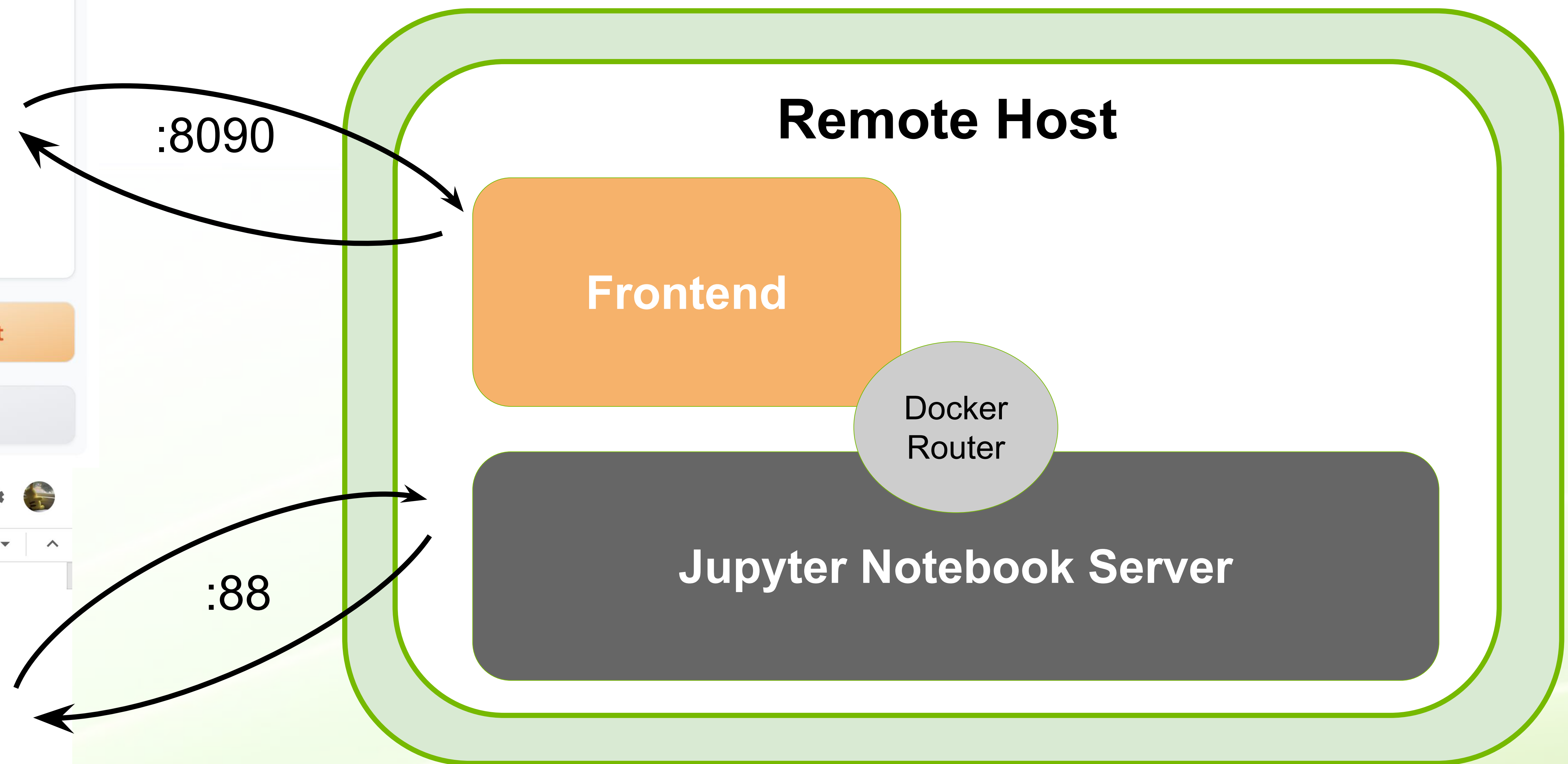
▼ Notebook 4: Running State Chains

In the previous notebook, we introduced some key LangChain Expression Language (LCEL) material regarding runnables. By now, you should be comfortable with both internal and external reasoning, as well as how to develop pipelines that facilitate it! In this notebook, we will make our way towards more advanced paradigms that will allow us to orchestrate more complex dialog management strategies and begin to execute on longer-form document reasoning.

Learning Objectives:

Our Environment

Jupyter Notebooks + Frontend



Gradio

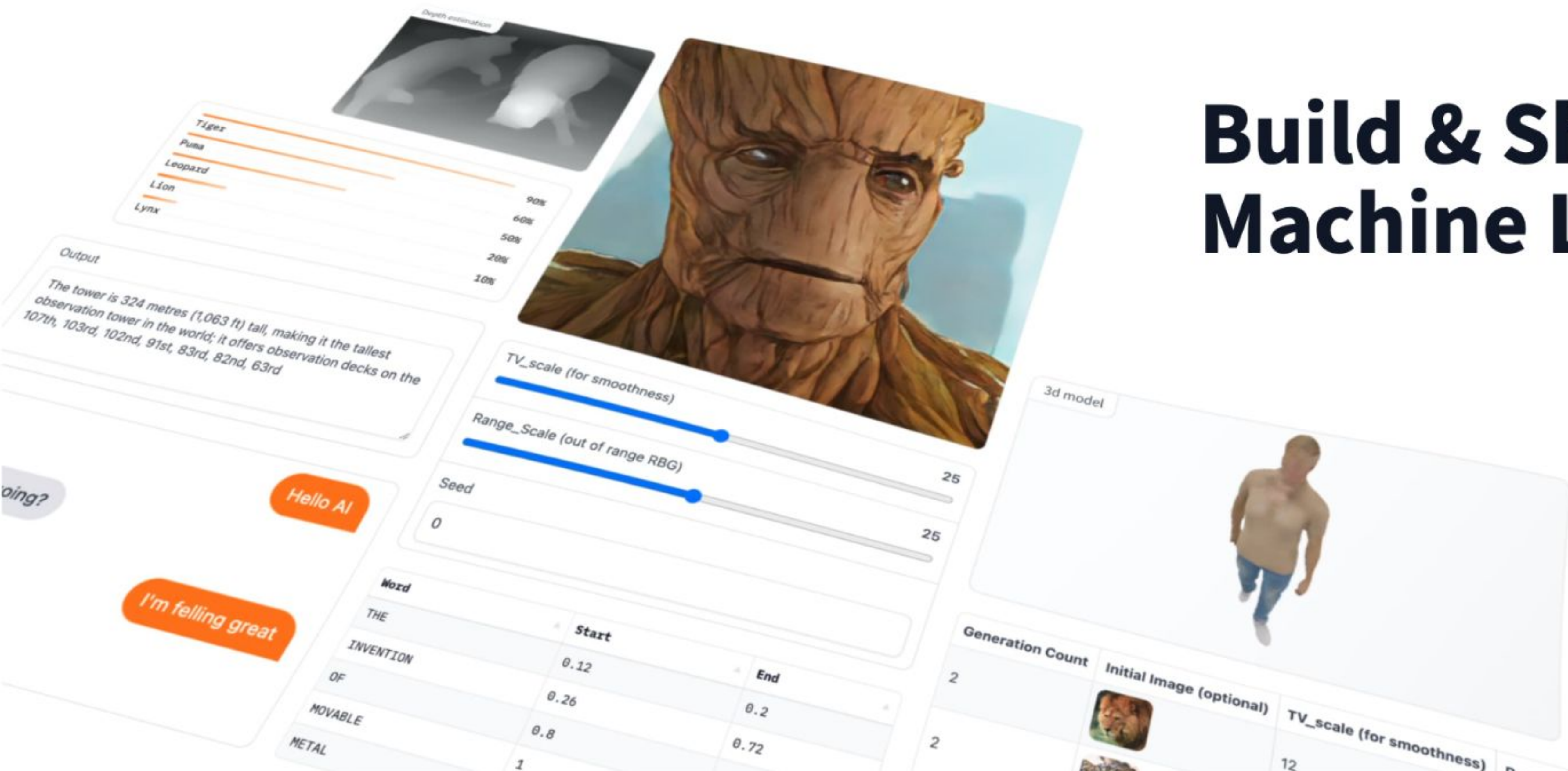
[Quickstart](#)[Docs](#)[Guides](#)[Playground^{NEW}](#)[Community](#)

Build & Share Delightful Machine Learning Apps

Gradio is the fastest way to demo your machine learning model with a friendly web interface so that anyone can use it, anywhere!

[Get Started](#)[Star](#)

23997



Simple Gradio ChatInterface

Chatbot

Hello World!!

You typed: Hello World!!

Type a message...

Submit

Retry

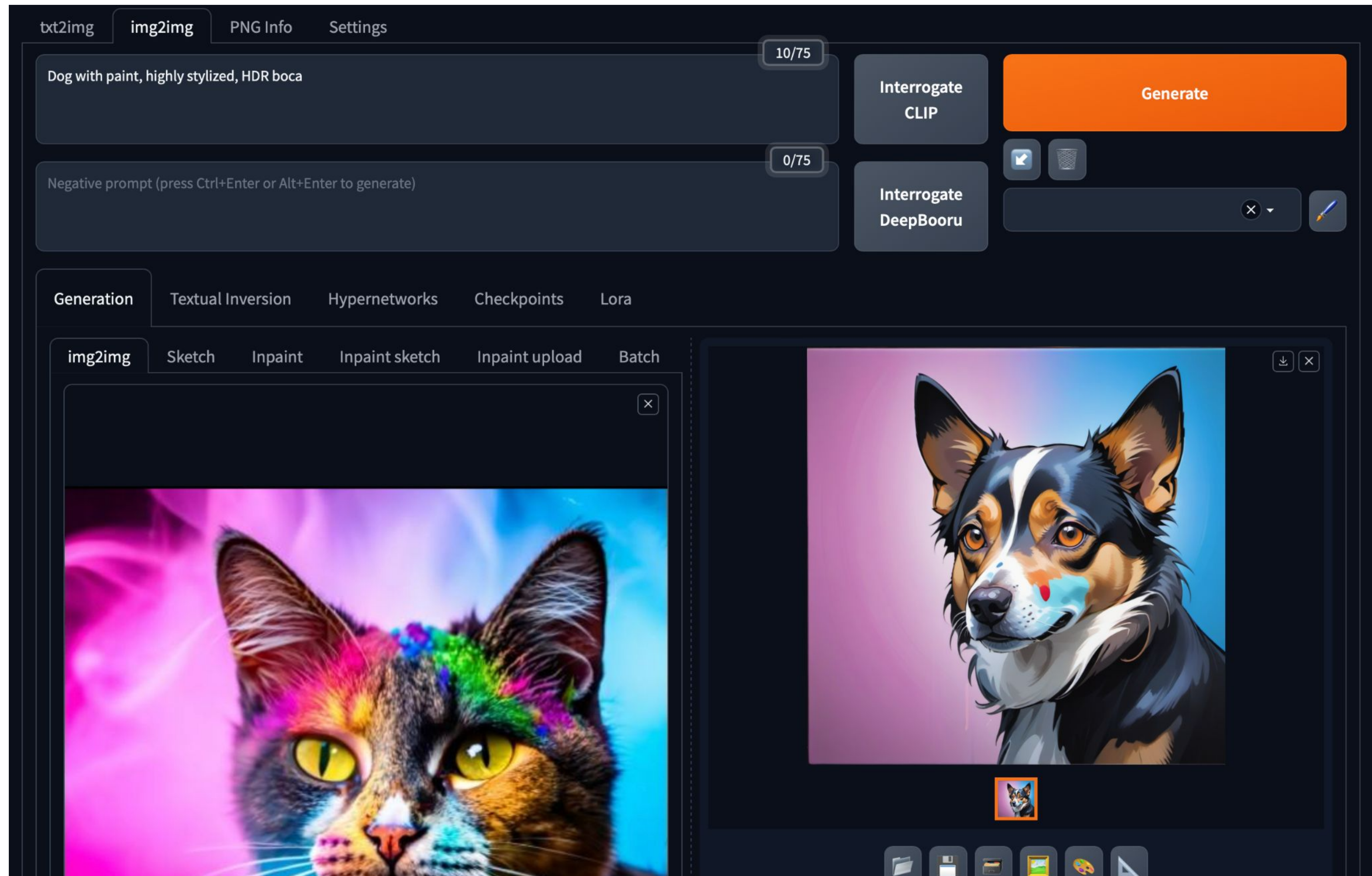
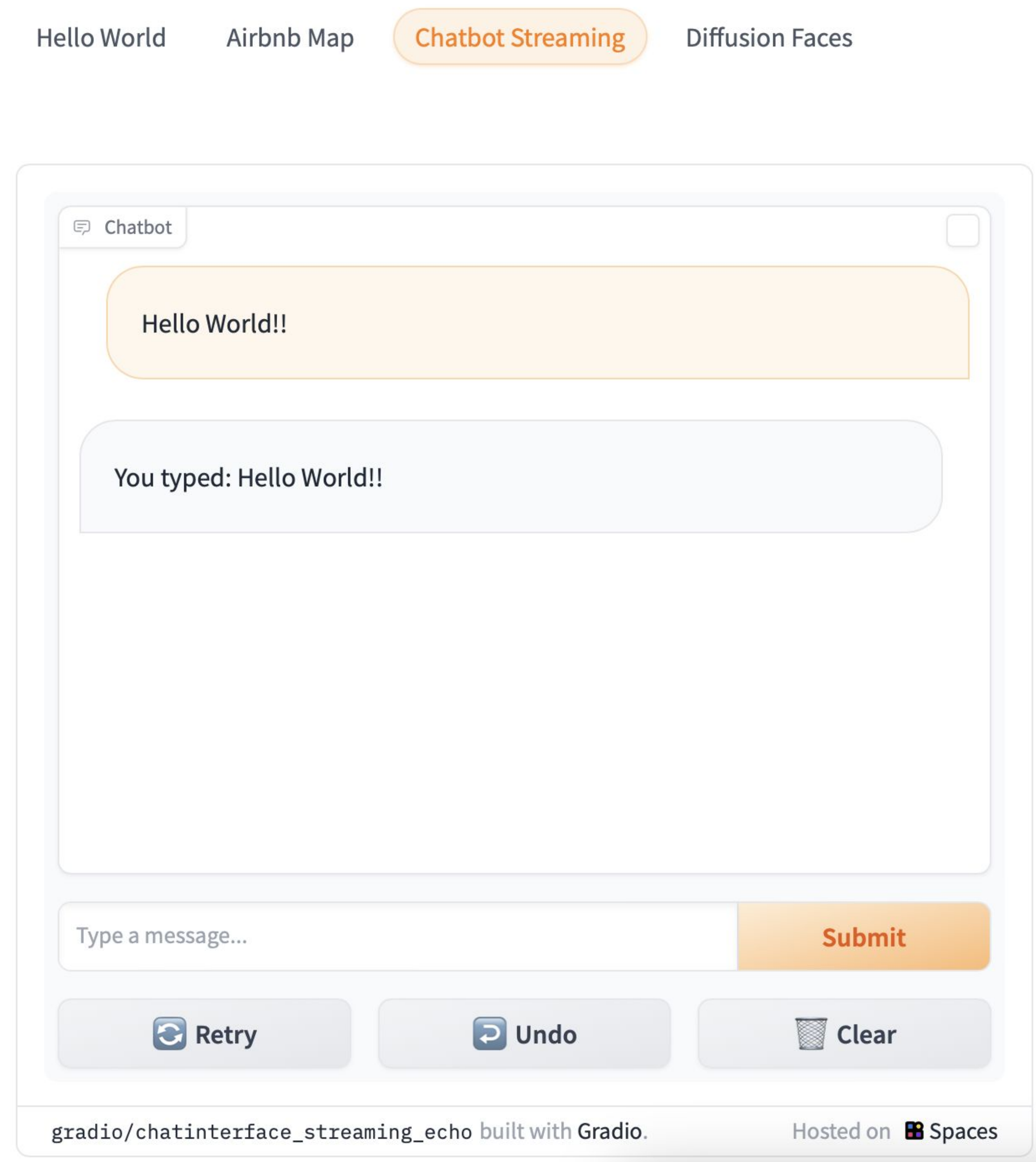
Undo

Clear

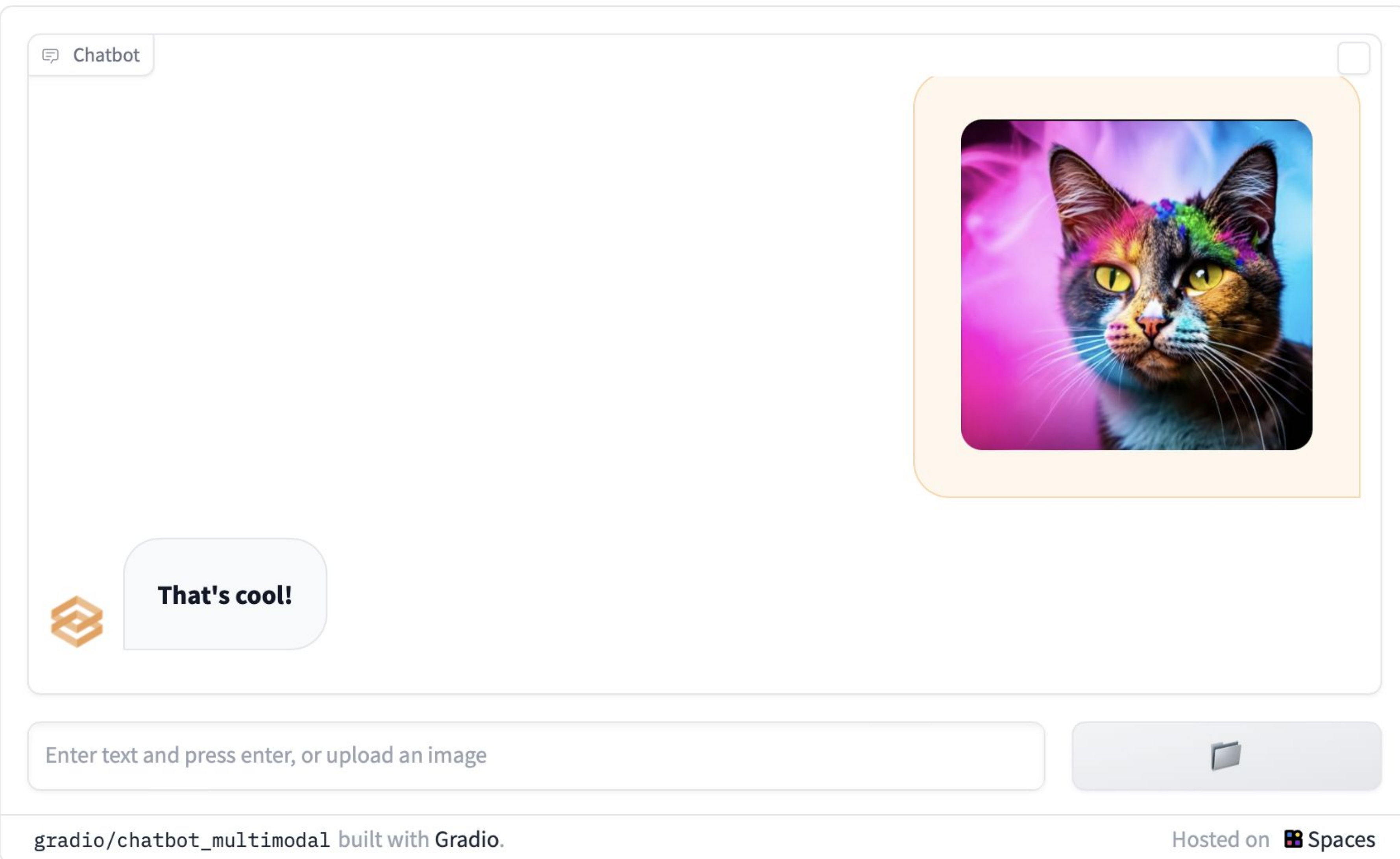
gradio/chatinterface_streaming_echo built with Gradio. Hosted on Spaces

```
1 import time
2 import gradio as gr
3
4 def slow_echo(message, history):
5     for i in range(len(message)):
6         time.sleep(0.05)
7         yield "You typed: " + message[: i+1]
8
9 demo = gr.ChatInterface(slow_echo).queue()
10
11 if __name__ == "__main__":
12     demo.launch()
13
```

Gradio in HuggingFace Spaces



Custom Gradio Block Interface



```
with gr.Blocks() as demo:
    chatbot = gr.Chatbot(
        [],
        elem_id="chatbot",
        bubble_full_width=False,
        avatar_images=(None, (os.path.join(os.path.dirname(__file__), "avatar.png"))),
    )

    with gr.Row():
        txt = gr.Textbox(
            scale=4,
            show_label=False,
            placeholder="Enter text and press enter, or upload an image",
            container=False,
        )
        btn = gr.UploadButton("📁", file_types=["image", "video", "audio"])

    txt_msg = txt.submit(add_text, [chatbot, txt], [chatbot, txt], queue=False).then(
        bot, chatbot, chatbot, api_name="bot_response"
    )
    txt_msg.then(lambda: gr.Textbox(interactive=True), None, [txt], queue=False)
    file_msg = btn.upload(add_file, [chatbot, btn], [chatbot], queue=False).then(
        bot, chatbot, chatbot
    )

demo.queue()
demo.launch()
```

Chatbot

Hello World!!

You typed: Hello World!!

Type a message...

Submit

Retry

Undo

Clear

04_running_state.ipynb

File Edit View Insert Runtime Tools Help

Last edited on February 9

Comment Share

+ Code + Text

Connect

DEEP LEARNING INSTITUTE

▼ Notebook 4: Running State Chains

In the previous notebook, we introduced some key LangChain Expression Language (LCEL) material regarding runnables. By now, you should be comfortable with both internal and external reasoning, as well as how to develop pipelines that facilitate it! In this notebook, we will make our way towards more advanced paradigms that will allow us to orchestrate more complex dialog management strategies and begin to execute on longer-form document reasoning.

Learning Objectives:

Our Environment

Jupyter Notebooks + Frontend

The diagram illustrates the environment architecture. A large light green rounded rectangle represents the **Remote Host**. Inside this host, there are two main components: an orange rounded rectangle labeled **Frontend** and a dark gray rounded rectangle labeled **Jupyter Notebook Server**. A gray oval labeled **Docker Router** is positioned between the Frontend and the Jupyter Notebook Server. Two curved arrows indicate connections from a local interface (shown on the left) to the Remote Host. The top arrow, labeled **:8090**, connects to the Frontend. The bottom arrow, labeled **:88**, connects to the Jupyter Notebook Server.

40



Building RAG Agents with LLMs

Part 2: LLM Services

Chatbot

Hello World!!

You typed: Hello World!!

Type a message...

Submit

Retry

Undo

Clear

04_running_state.ipynb

File Edit View Insert Runtime Tools Help

Last edited on February 9

Comment Share

+ Code + Text

Connect

 **DEEP LEARNING INSTITUTE**

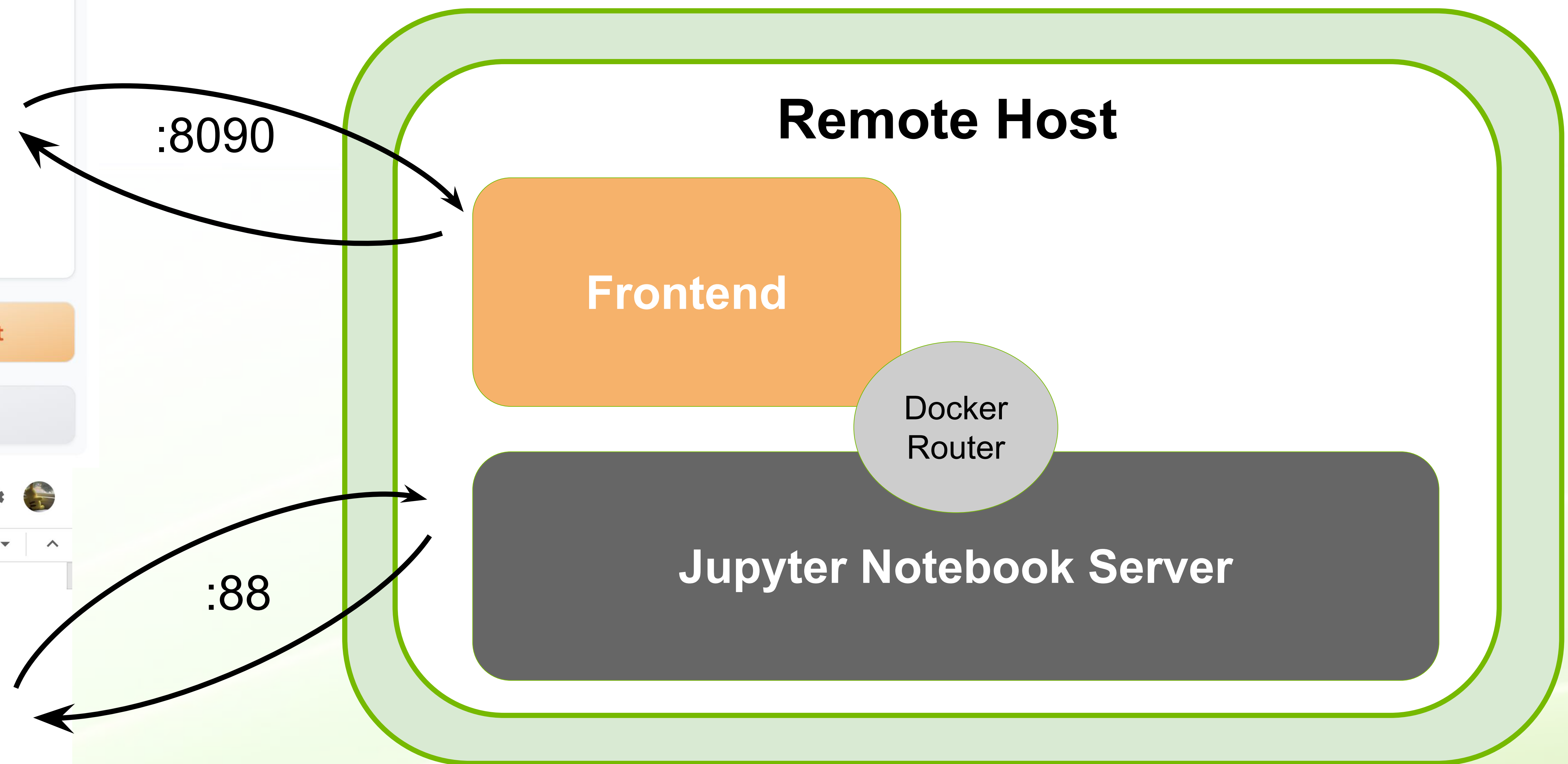
▼ **Notebook 4: Running State Chains**

In the previous notebook, we introduced some key LangChain Expression Language (LCEL) material regarding runnables. By now, you should be comfortable with both internal and external reasoning, as well as how to develop pipelines that facilitate it! In this notebook, we will make our way towards more advanced paradigms that will allow us to orchestrate more complex dialog management strategies and begin to execute on longer-form document reasoning.

Learning Objectives:

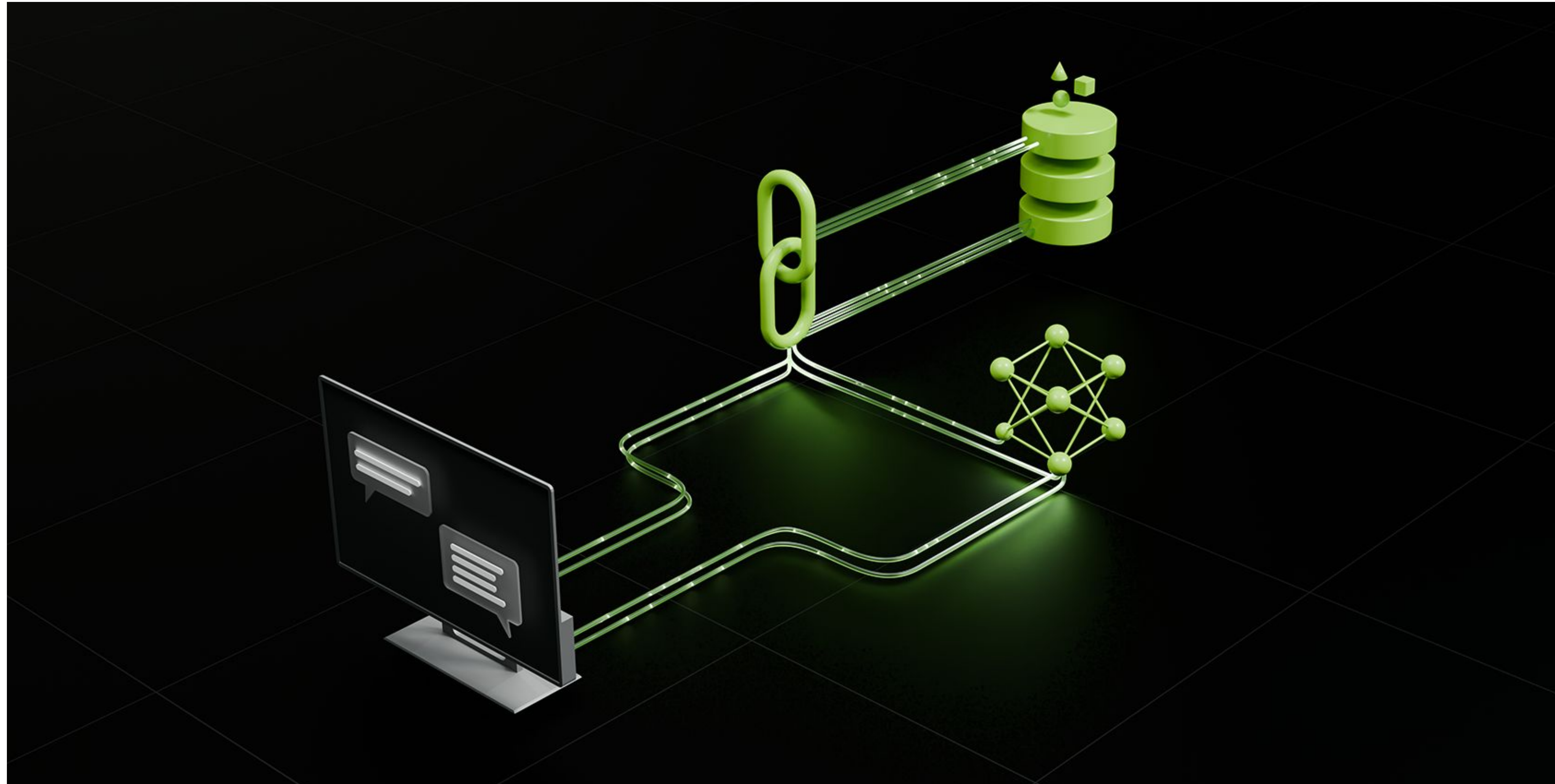
Our Environment

Jupyter Notebooks + Frontend



Dialog/Retrieval Agents

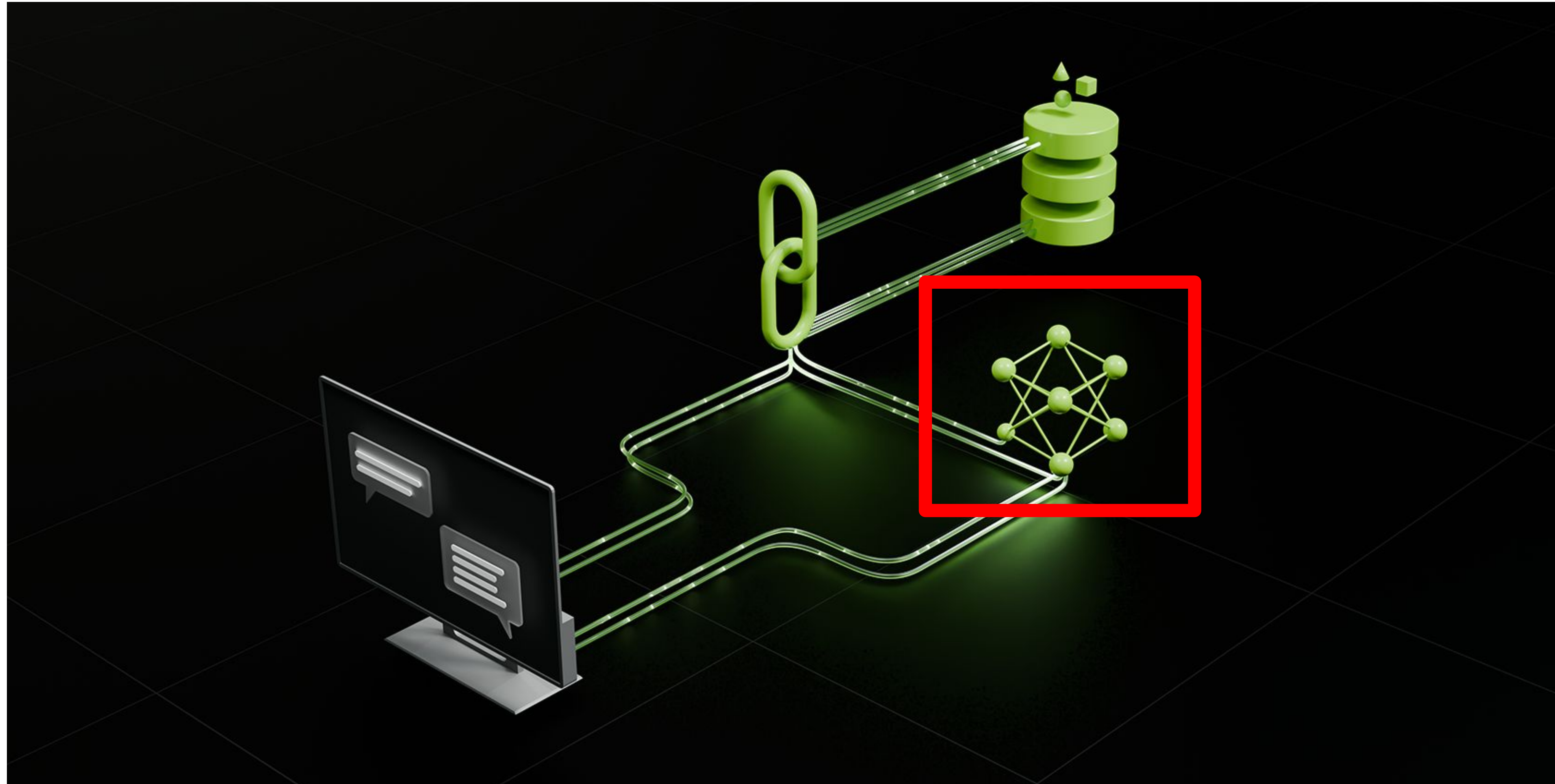
LLMs with Context and Control



Execute with dialog management and information retrieval in production

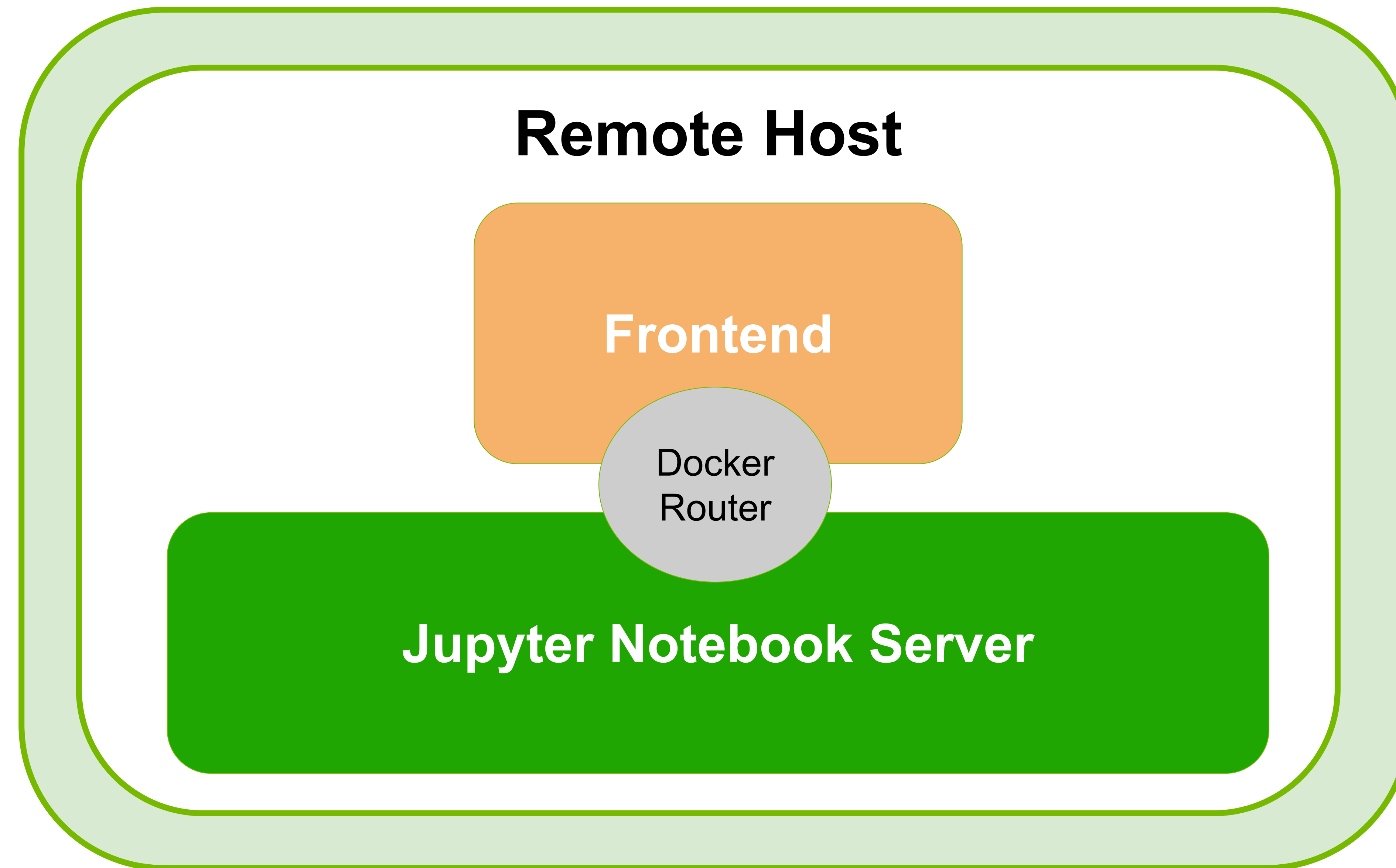
Dialog/Retrieval Agents

LLMs with Context and Control

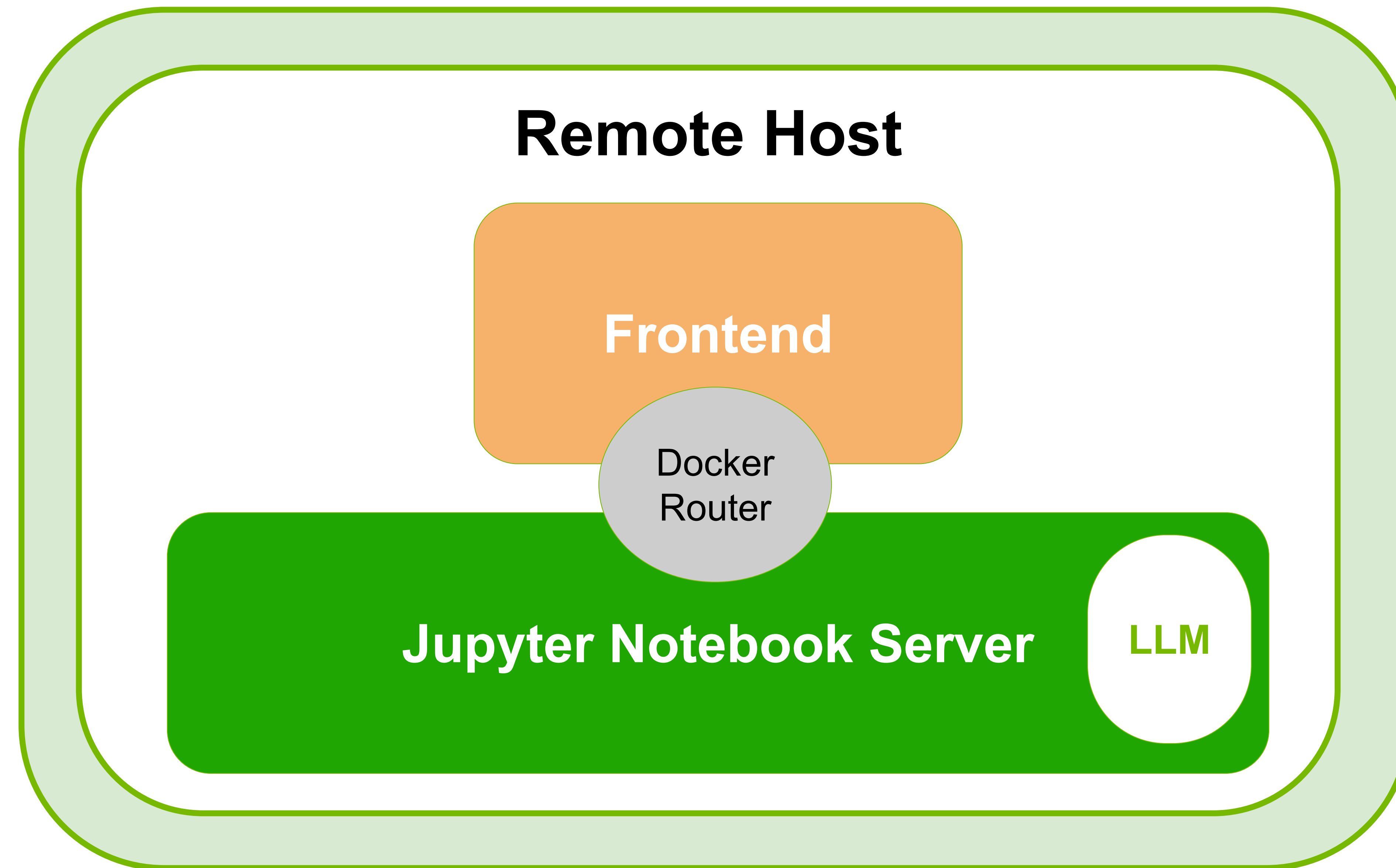


Execute with dialog management and information retrieval in production

Standalone Environment LLM



Standalone Environment LLM



Standalone Environment LLM

Jupyter Notebook

H200

A100

Jupyter Notebook

A10

4070

Jupyter Notebook

VRAM-bound

CPU-only



```
<s>[INST]<<SYS>>
{{system_message}}
<</SYS>>

{{instruction}} [/INST]
{{primer}}
```

```
<s>[INST]<<SYS>>
You are a code generator.
Please provide Python code
per the instruction.
<</SYS>>
```

```
Write a Fibonacci method [/INST]
```python
Implementation of Fibonacci w/
```

# Standalone Environment LLM

Jupyter Notebook

H200

A100

Jupyter Notebook

A10

4070

Jupyter Notebook

VRAM-bound

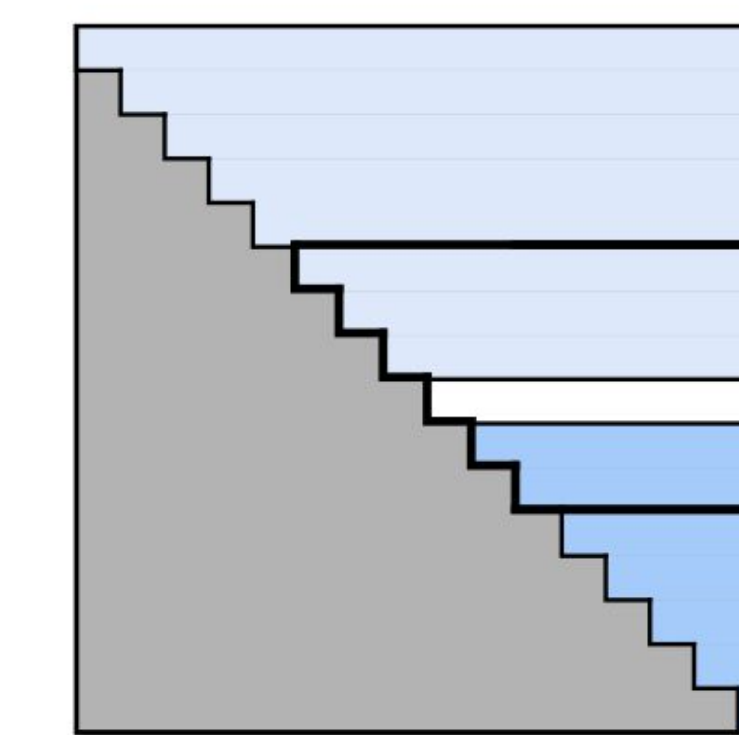
CPU-only



64	134	217
76	119	21
3	81	99

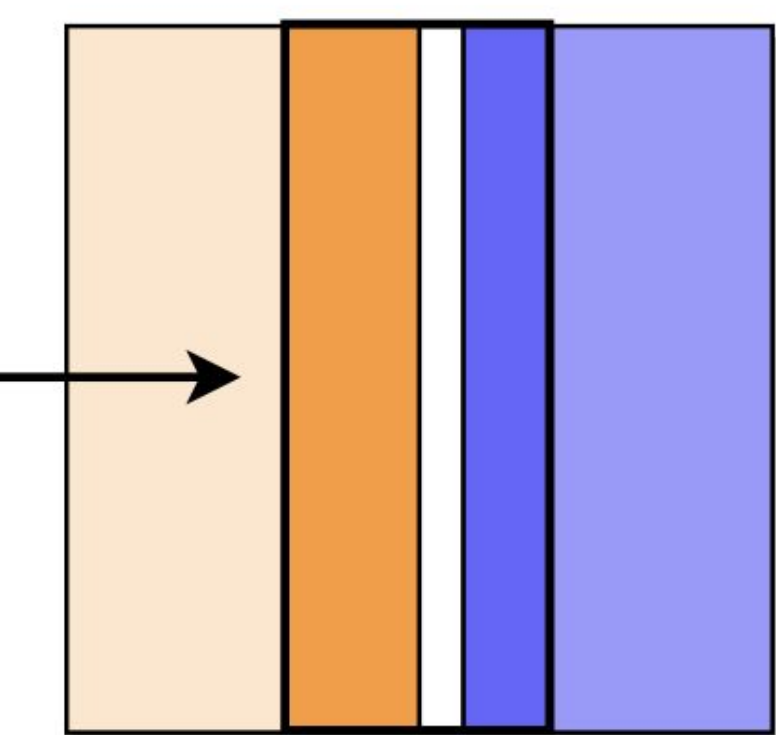
INT8

Inverse Layer Hessian  
(Cholesky Form)



computed initially

Weight Matrix / Block



block  $i$  quantized recursively  
column-by-column



# Standalone Environment LLM

Jupyter Notebook

H200

A100

Jupyter Notebook

A10

4070

Jupyter Notebook

VRAM-bound

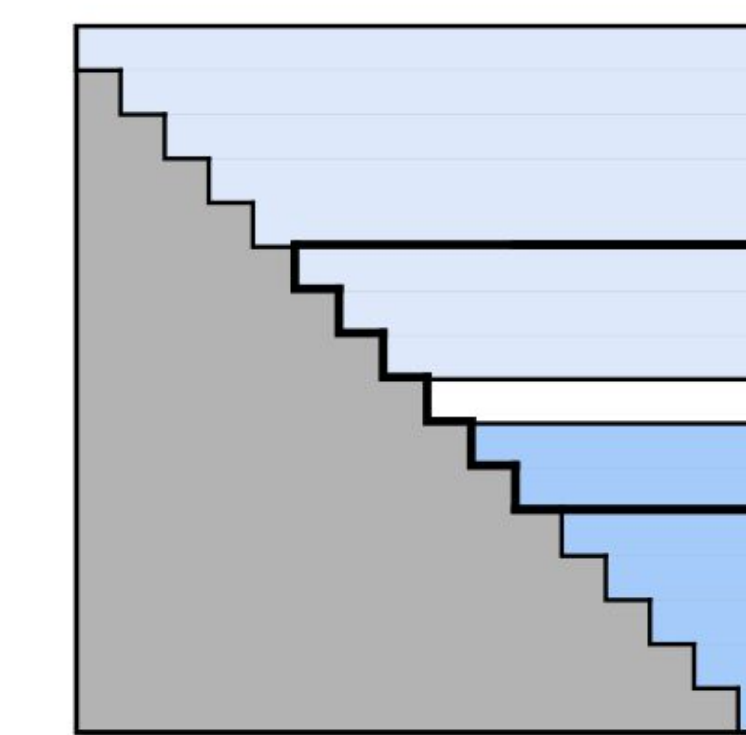
CPU-only



64	134	217
76	119	21
3	81	99

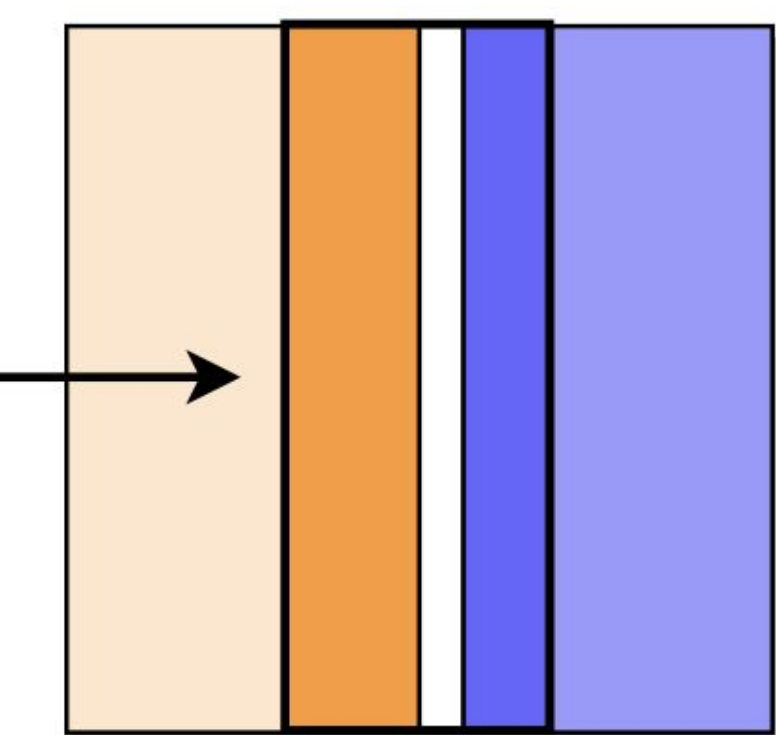
INT8

Inverse Layer Hessian  
(Cholesky Form)



computed initially

Weight Matrix / Block



block  $i$  quantized recursively  
column-by-column

quantized weights      unquantized weights  
that are updated

# Remote LLM Access

## Remote Host

Frontend

Jupyter Notebook

VRAM-bound

CPU-only

```
<s>[INST]<<SYS>>
{{system_message}}
<</SYS>>
```

```
{{instruction}} [/INST]
{{primer}}
```

```
<s>[INST]<<SYS>>
You are a code generator.
Please provide Python code
per the instruction.
<</SYS>>
```

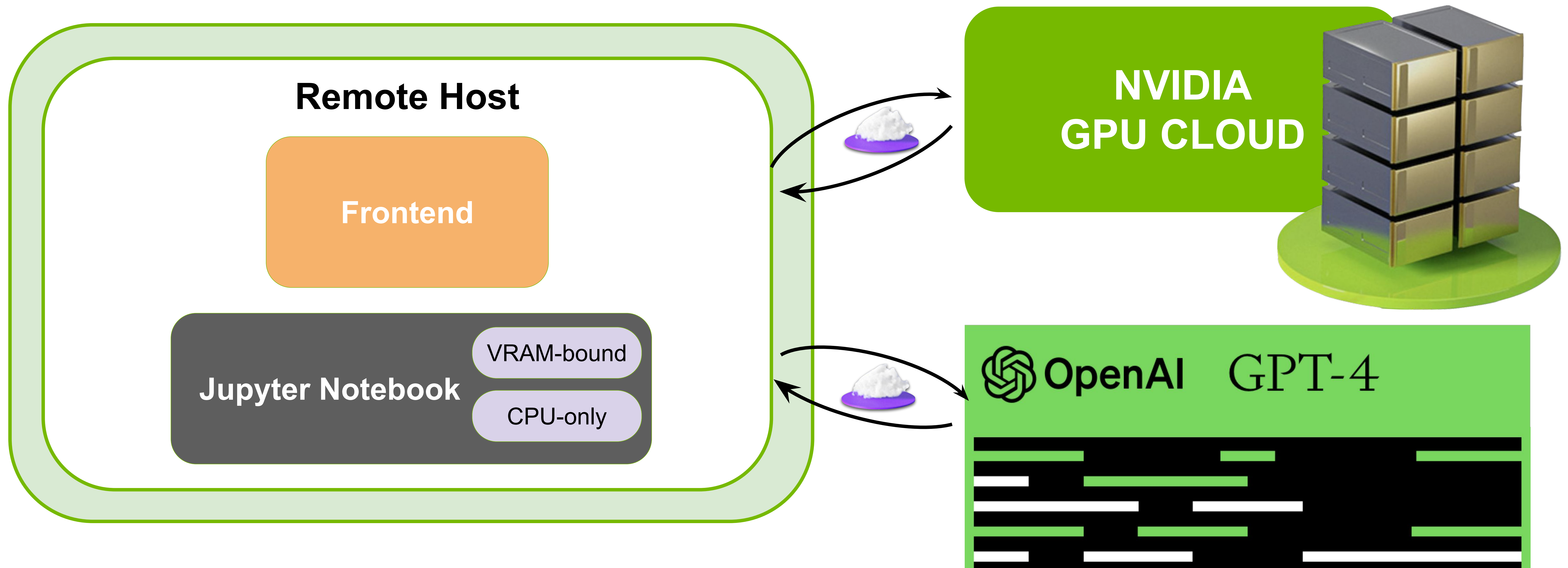
```
Write a Fibonacci method [/INST]
```python
## Implementation of Fibonacci w/
```

Llama

LLM



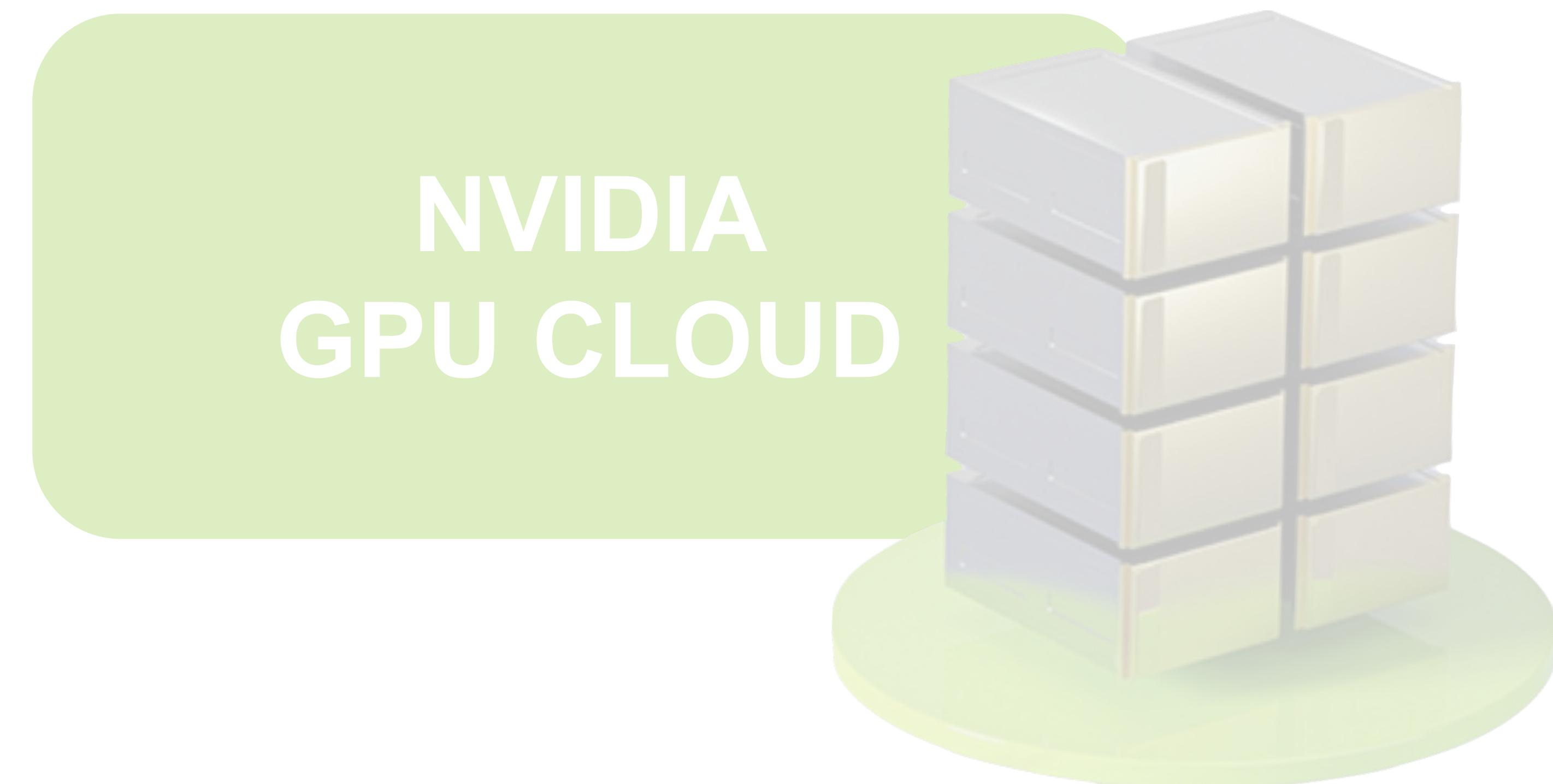
Large Model Hosting Platforms



Large Model Hosting Platforms

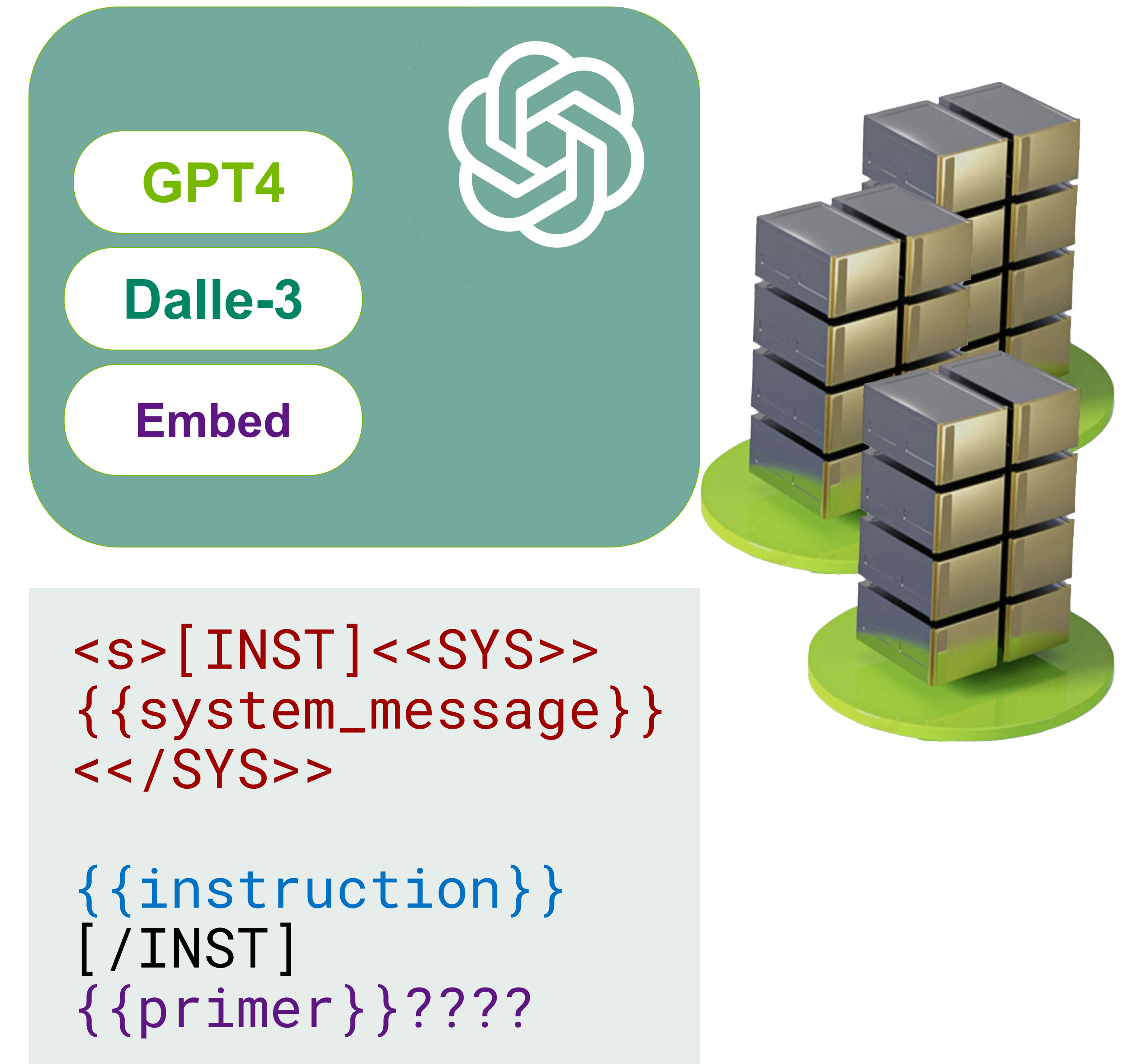


1. GPT4/3.5
2. Integrated Tools/Multimodal Support
3. Models/Query Router Internal
4. Path to Local Deployment Less Clear



1. Publicly Available Models
2. Models/Query Router Accessible
3. Path to Local Deployment/Self-Hosting
4. Pieces to Compose Complex Systems

Query Router Access



Query Router Access

```
{  
  "messages": [{  
    "content": "...",  
    "role": "system"  
  }, {  
    "content": "...",  
    "role": "user"  
  }],  
  "model": "gpt-4",  
  "temperature": 0.2,  
  "top_p": 0.7,  
  "max_tokens": 1024,  
  "stream": True  
}
```

/chat/completions

Facilitate
Monitor
Optimize
Load Balance

GPT4

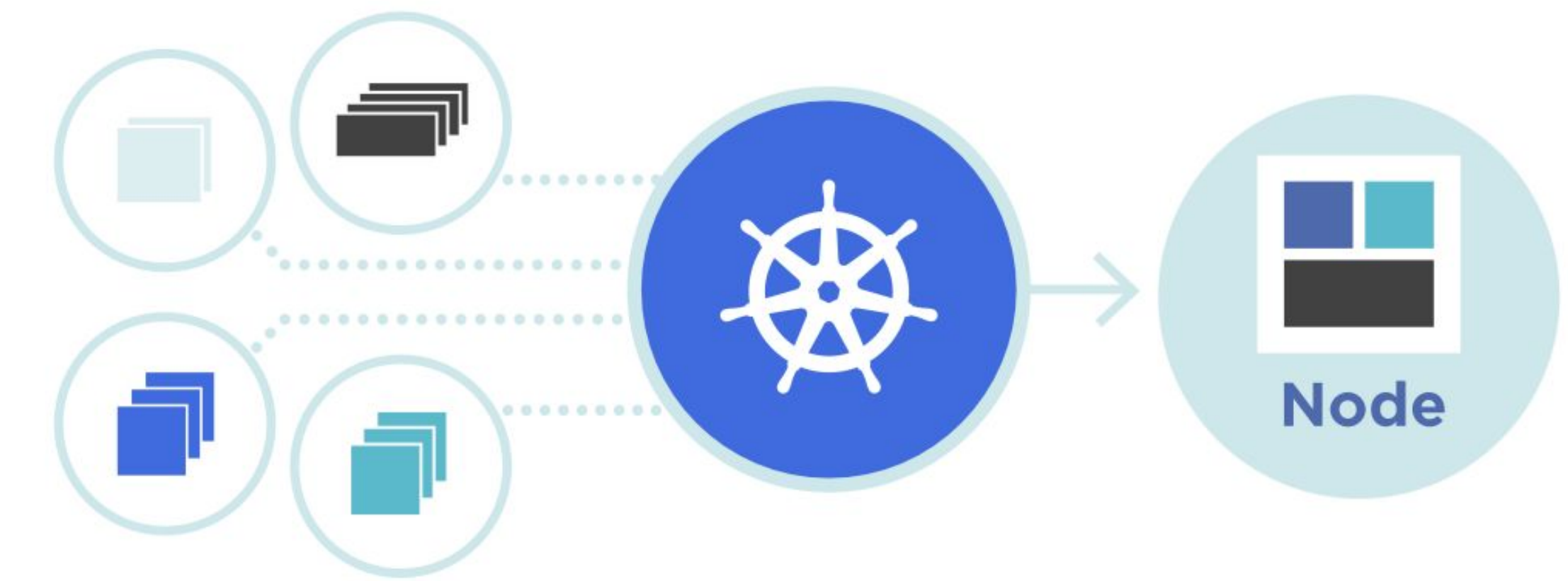
Dalle-3

Embed

```
<s>[INST]<<SYS>>  
{{system_message}}  
<</SYS>>
```

```
{{instruction}}  
[/INST]  
{{primer}}????
```

Query Router Access



```
{  
  "messages": [{  
    "content": "...",  
    "role": "system"  
  }, {  
    "content": "...",  
    "role": "user"  
  }],  
  "model": "gpt-4",  
  "temperature": 0.2,  
  "top_p": 0.7,  
  "max_tokens": 1024,  
  "stream": True  
}
```

/chat/completions

Facilitate
Monitor
Optimize
Load Balance

GPT4

Dalle-3

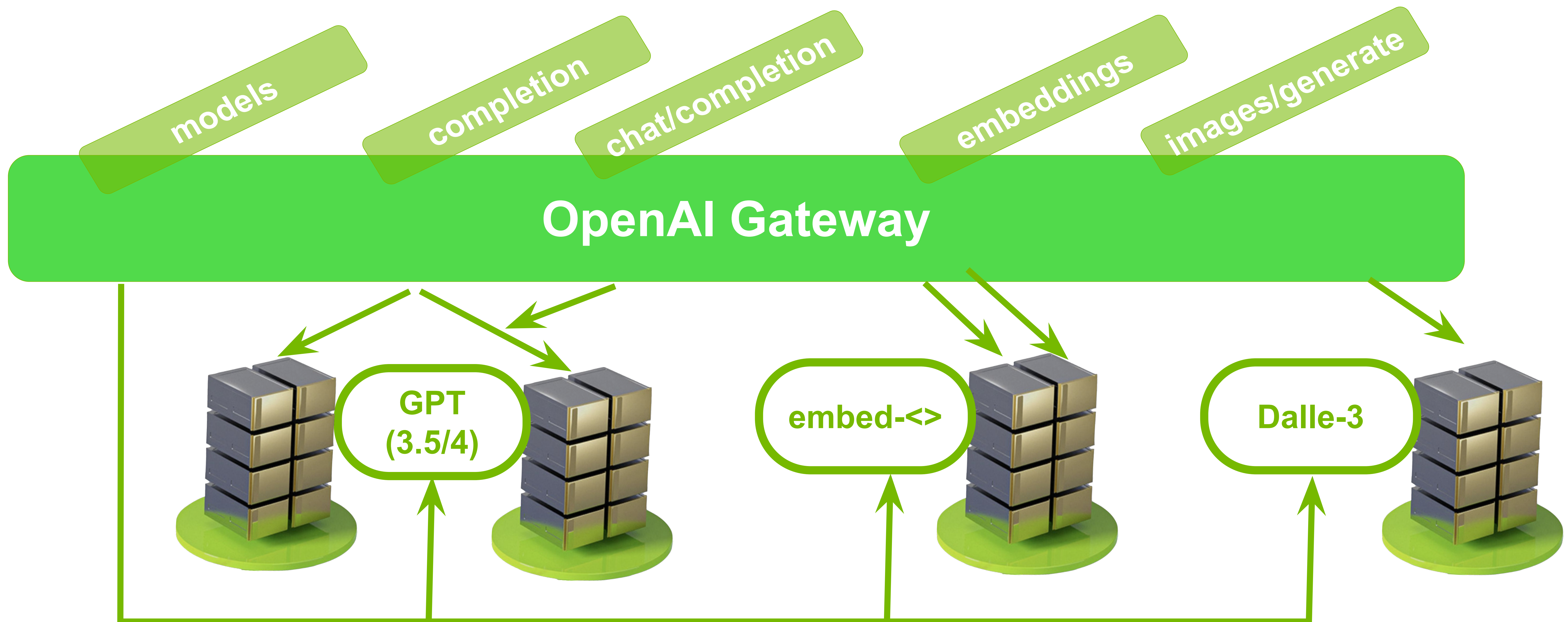
Embed

```
<s>[INST]<<SYS>>  
{{system_message}}  
<</SYS>>
```

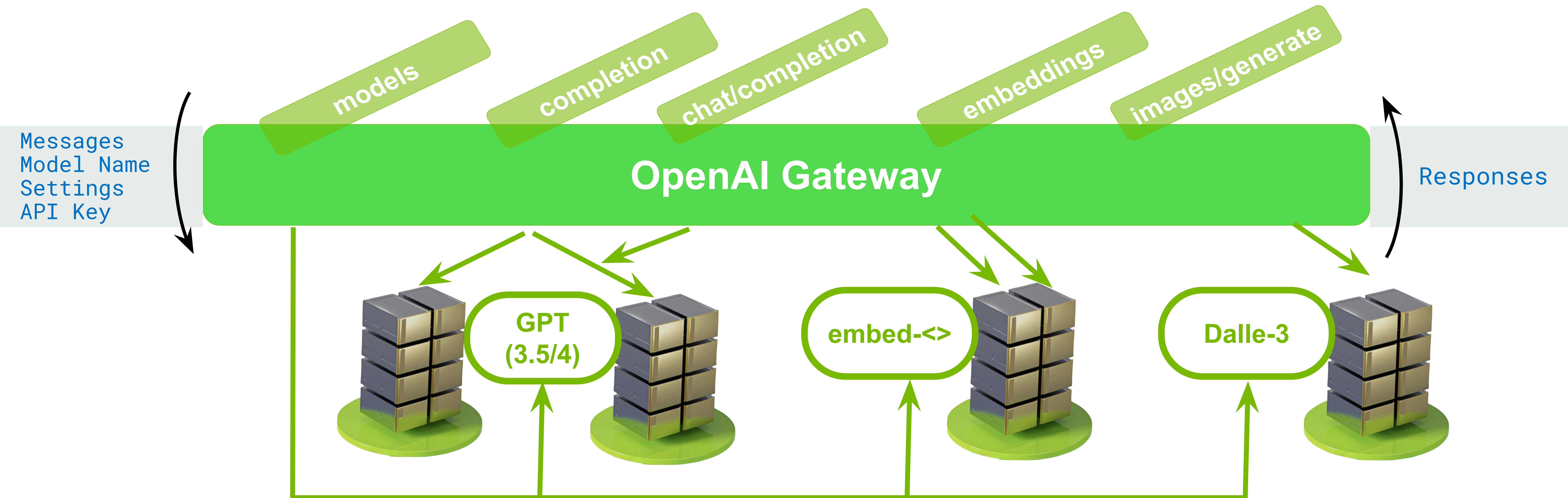
```
{{instruction}}  
[/INST]  
{{primer}}????
```



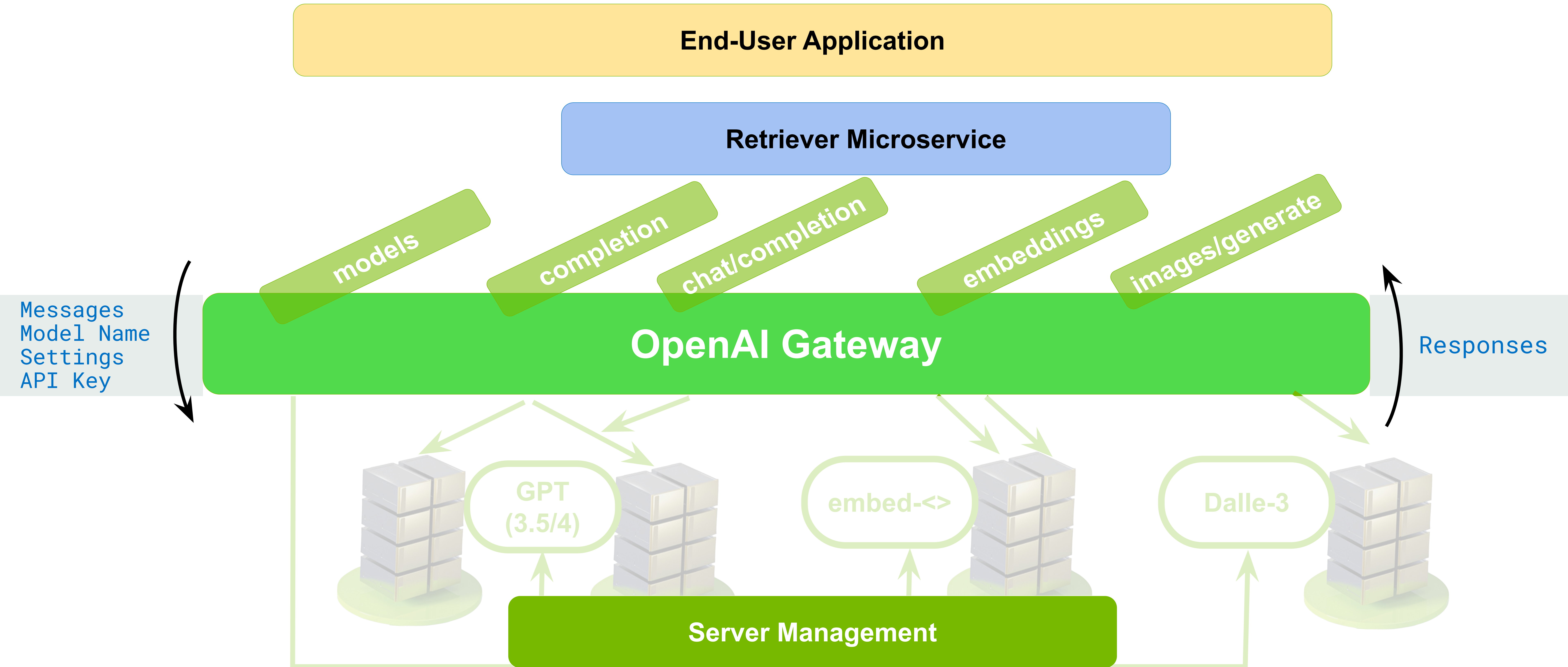
Large Model Hosting Platforms - OpenAI



Large Model Hosting Platforms - OpenAI



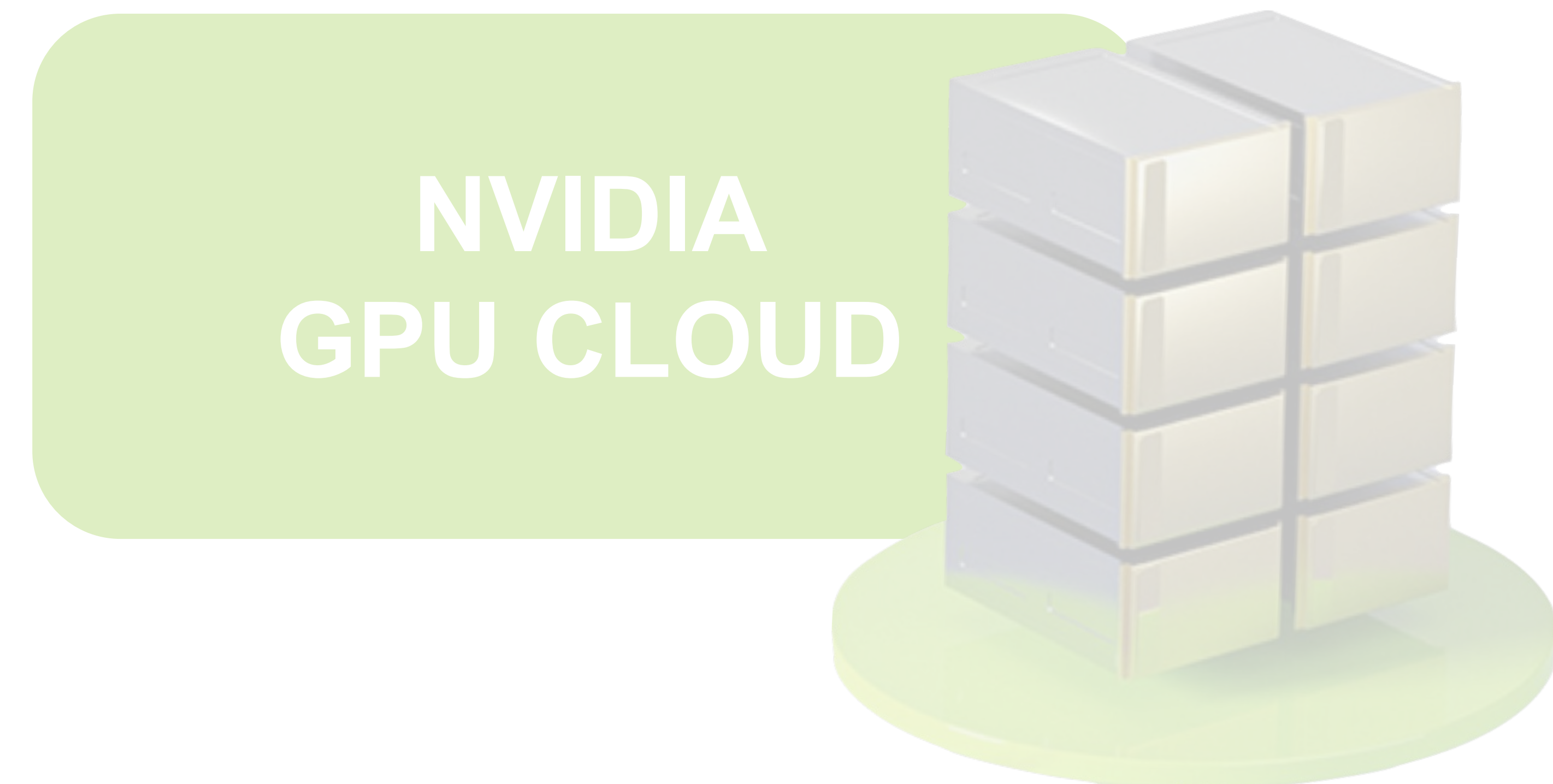
Large Model Hosting Platforms - OpenAI



Large Model Hosting Platforms



1. GPT4/3.5
2. Integrated Tools/Multimodal Support
3. Models/Query Router Internal
4. Path to Local Deployment Less Clear

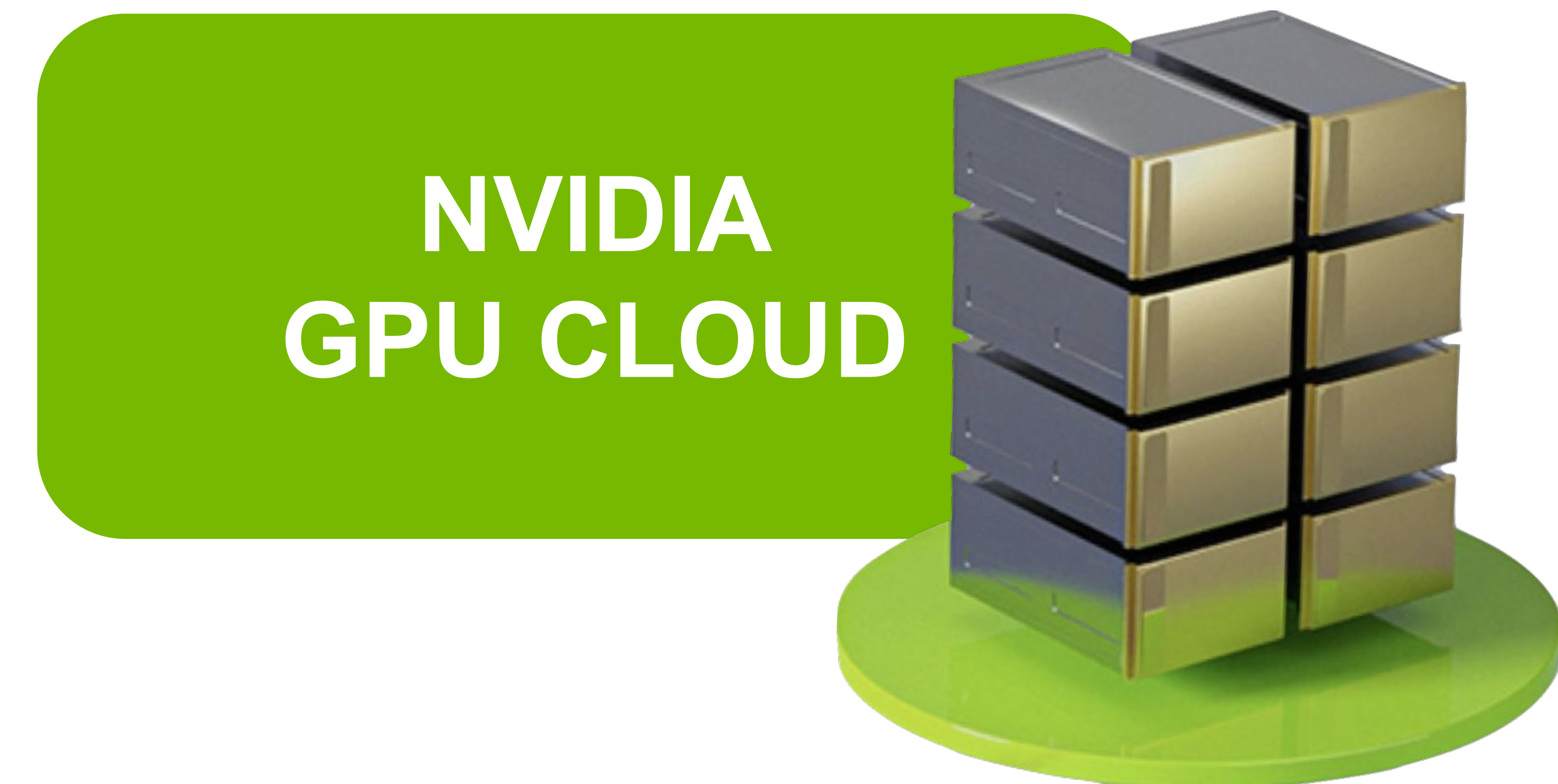


1. Publicly Available Models
2. Models/Query Router Accessible
3. Path to Local Deployment/Self-Hosting
4. Pieces to Compose Complex Systems

Large Model Hosting Platforms



1. GPT4/3.5
2. Integrated Tools/Multimodal Support
3. Models/Query Router Internal
4. Path to Local Deployment Less Clear



1. Publicly Available Models
2. Models/Query Router Accessible
3. Path to Local Deployment/Self-Hosting
4. Pieces to Compose Complex Systems

Query Router Access

```
{  
  "messages": [{  
    "content": "...",  
    "role": "system"  
  }, {  
    "content": "...",  
    "role": "user"  
  }],  
  "model": "mixtral",  
  "temperature": 0.2,  
  "top_p": 0.7,  
  "max_tokens": 1024,  
  "stream": True  
}
```

/chat/completions

**Facilitate
Monitor
Optimize
Load Balance**

```
<s>[INST]<<SYS>>  
{{system_message}}  
<</SYS>>
```

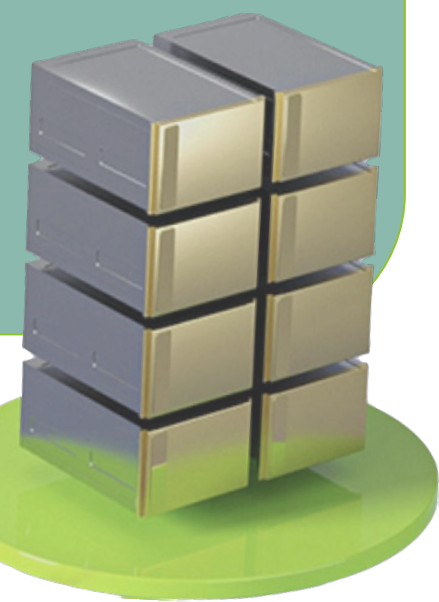
```
{{instruction}} [/INST]  
{{primer}}
```

```
{  
  "context": {{context}}  
  "model": "query"/"doc"  
}
```

GPT4

Dalle-3

Embed



Mixtral

SDXL

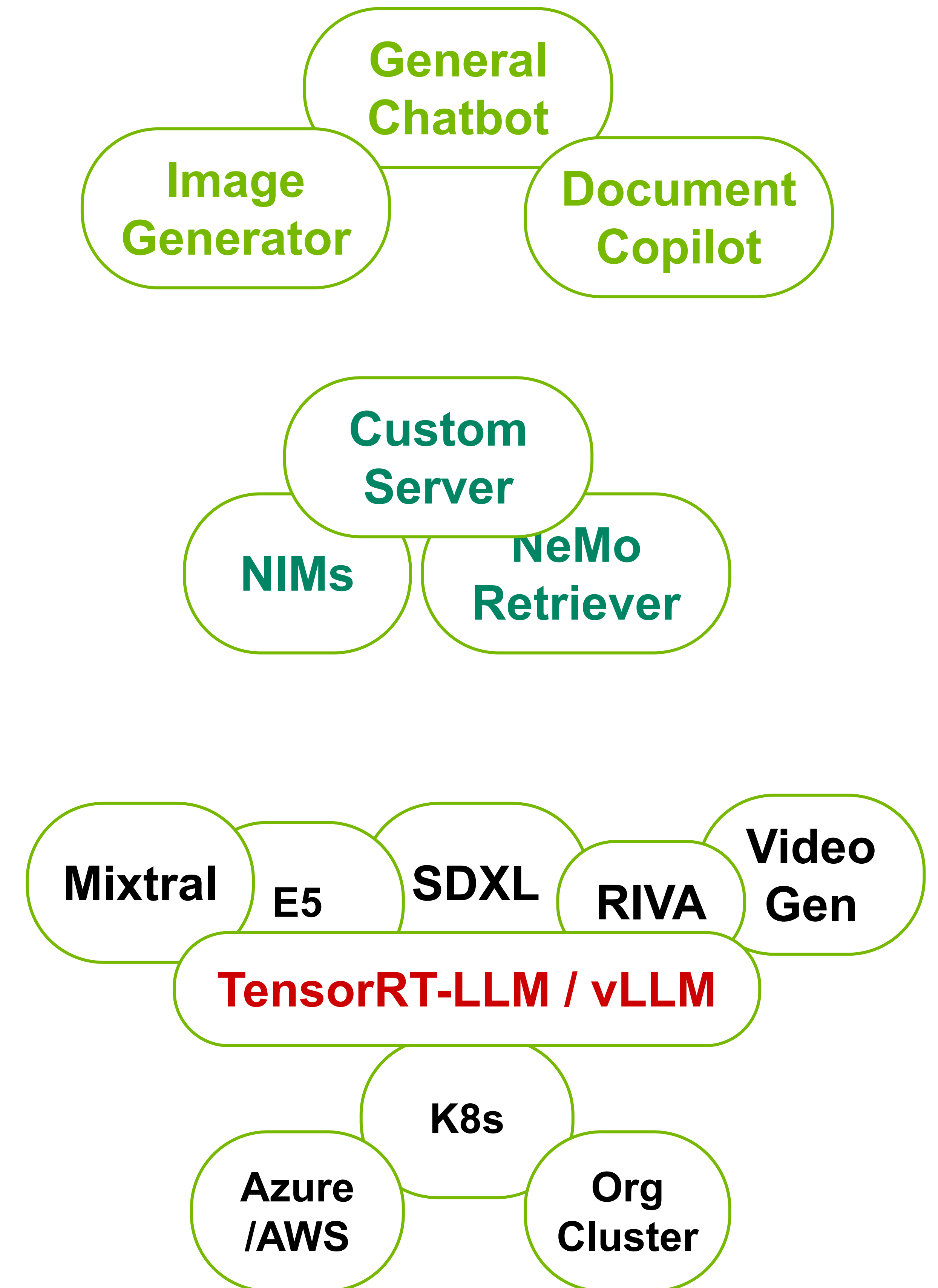
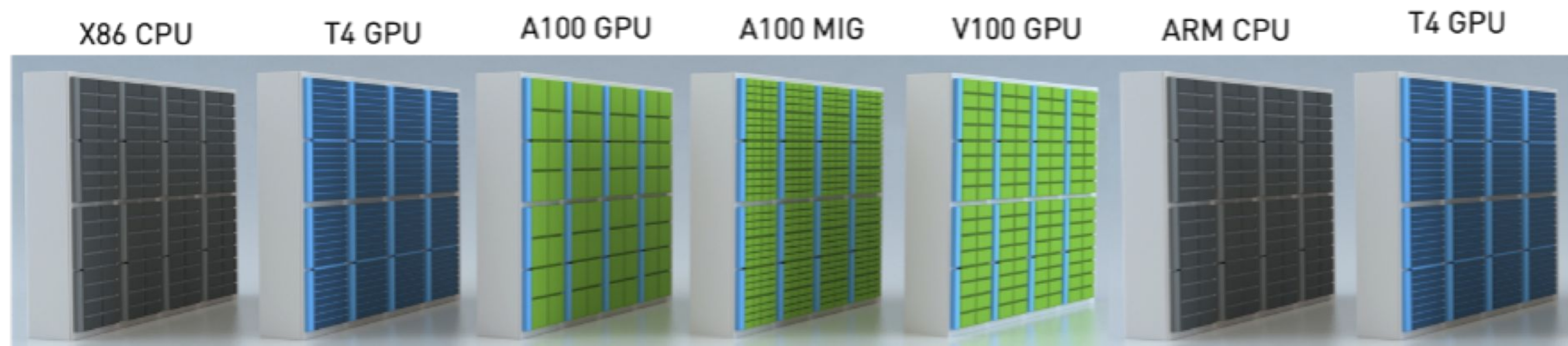
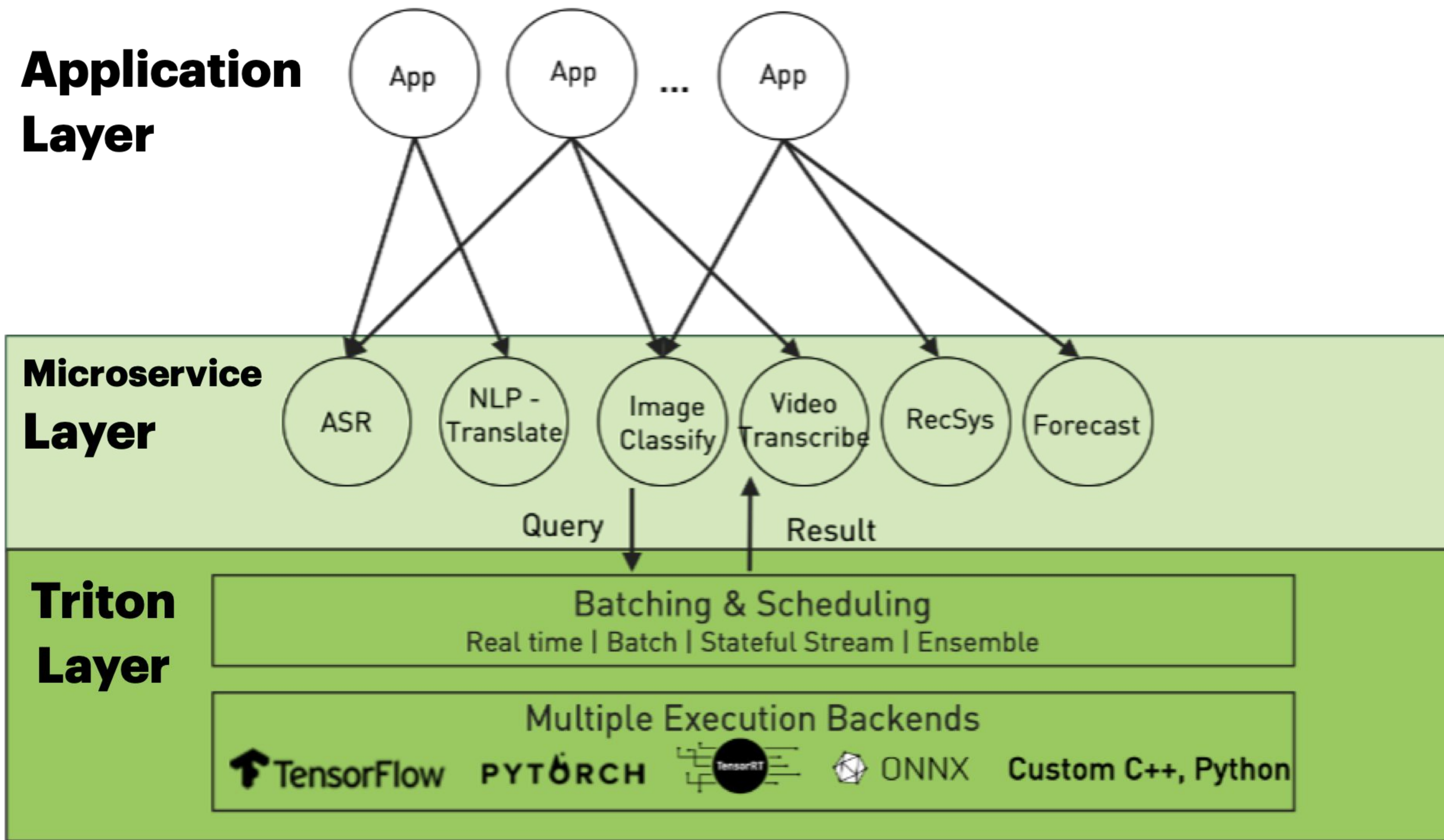
LLM

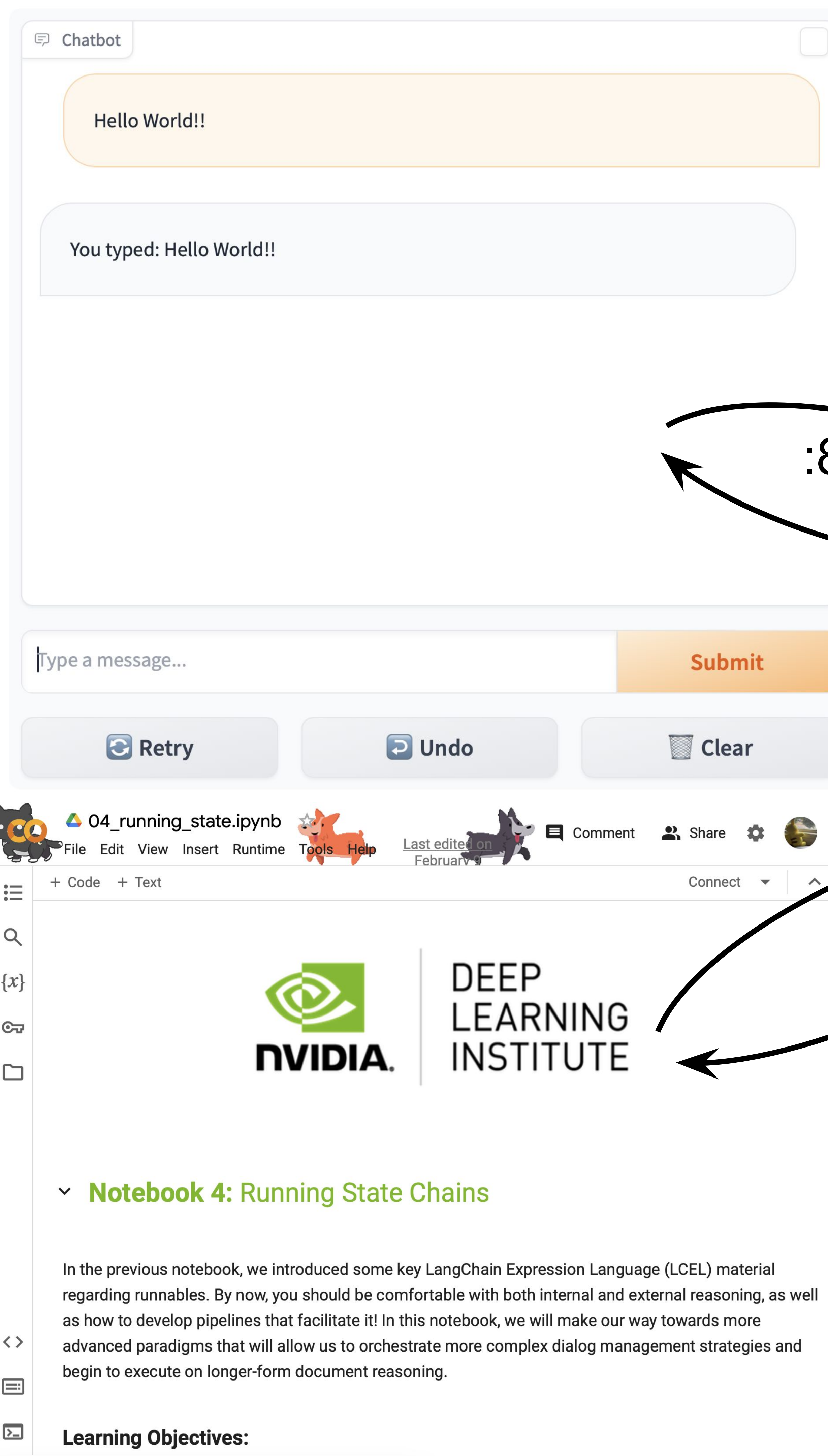
E5



Full Deployment Stack

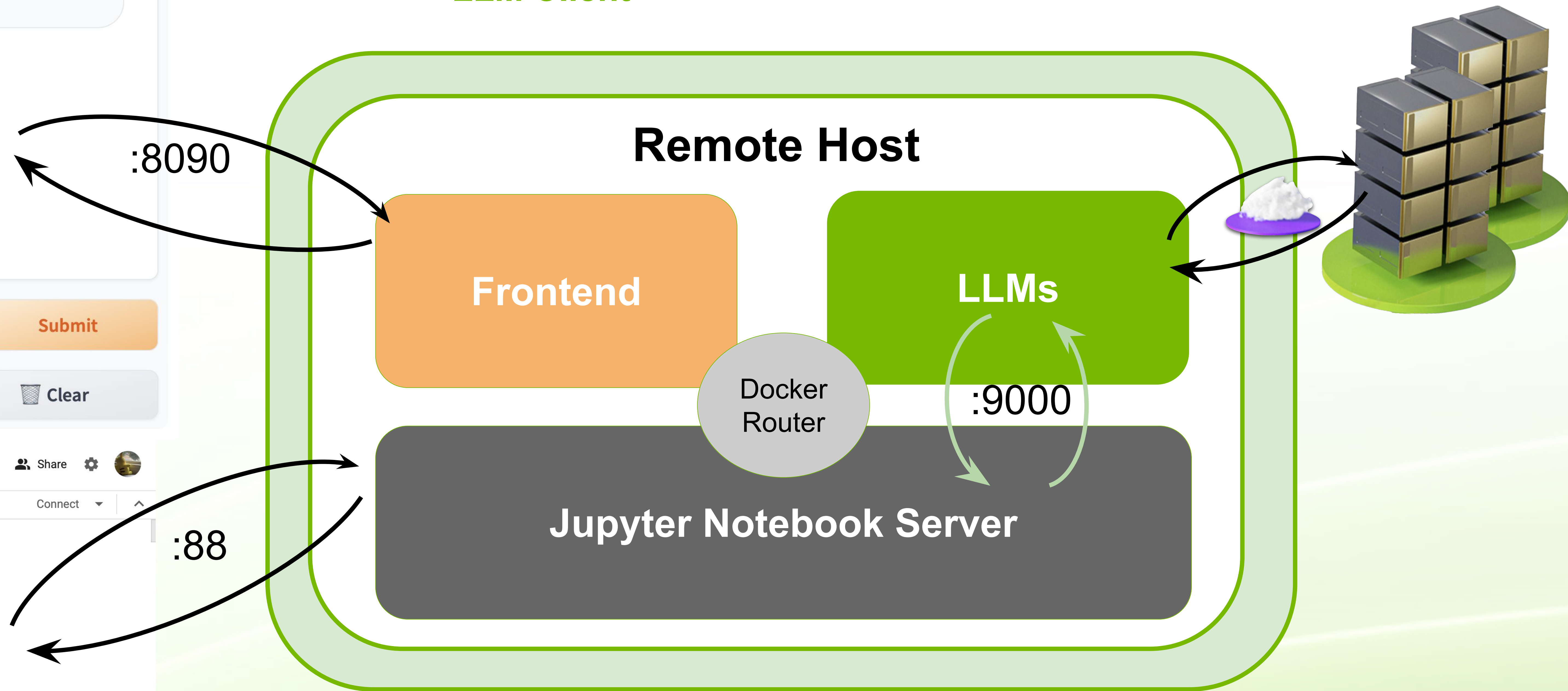
Millions of Apps | Billions of Users | Trillions of Queries





Our Environment

Jupyter Notebooks + Frontend + LLM Client



NVIDIA Foundation Model Endpoints



[Explore](#)

[Docs](#)

[Login](#)

Discover

MODELS

Reasoning

Visual Design

Retrieval

Speech

Biology

INDUSTRIES

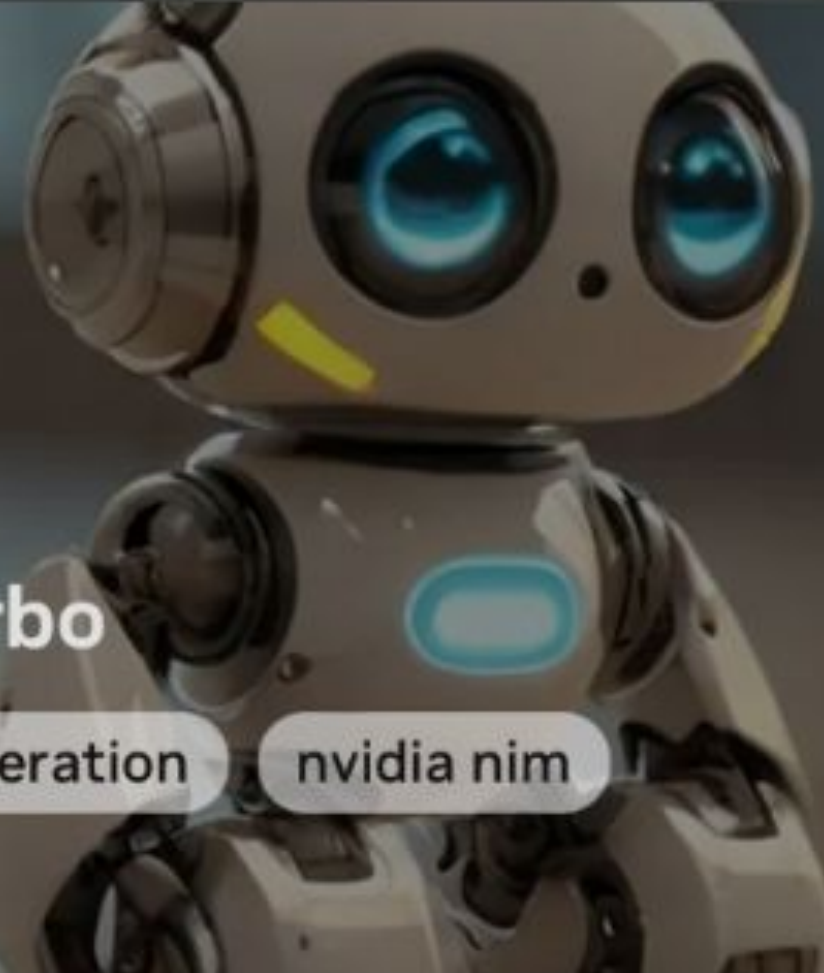
Gaming

Healthcare

Industrial

Top Open Foundation Models

The leading open models built by the community, optimized and accelerated by NVIDIA's enterprise-ready inference runtime



stabilityai

sdxl-turbo

image generation

nvidia nim



google

gemma-7b

chat

language generation



meta

llama2-70b

chat

language generation

Trending Now

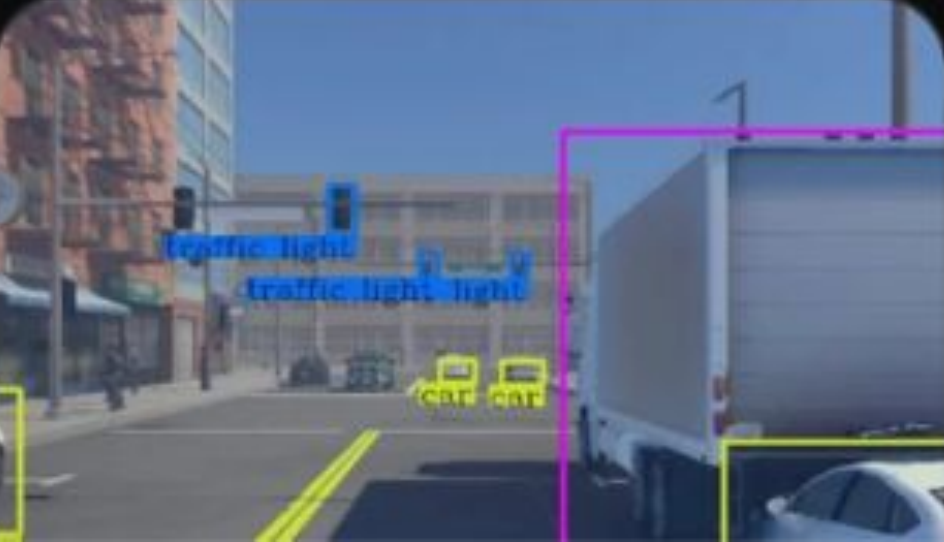
The latest and most popular additions to the list



mistralai /

mixtral-8x7b-instruct-v0.1

An MOE LLM that follows instructions, completes requests, and generates creative text.



microsoft /

kosmos-2

Groundbreaking multimodal model designed to understand and reason about visual elements in images.



NVIDIA

GPU CLOUD

64 

NVIDIA Foundation Model Endpoints



Search NVIDIA AI

< Back

mistralai / **mixtral-8x7b-instruct-v0.1**

An MOE LLM that follows instructions, completes requests, and generates creative text.

advanced reasoning

chat

large language models

text-to-text



**NVIDIA
GPU CLOUD**

NVIDIA Foundation Model Endpoints

mistralai / **mixtral-8x7b-instruct-v0.1**

An MOE LLM that follows instructions, completes requests, and generates creative text.

advanced reasoning

chat

large language models

text-to-text

Preview

JSON

Reset Chat



NVIDIA's GPU Technology Conference (GTC) is an annual event that showcases the latest developments in graphics processing units (GPUs) and artificial intelligence (AI). Here are some of the things you can expect to see at GTC:

1. Keynote speeches: GTC typically features keynote speeches from NVIDIA's CEO, Jensen Huang, as well as other industry leaders and experts. These speeches cover the latest trends and breakthroughs in AI, data science, high-performance computing, and graphics.
2. Technical sessions: GTC offers a wide range of technical sessions, including

Type text here...

Python

Node

Shell

Get API Key Copy Code

```
from openai import OpenAI

client = OpenAI(
    base_url = "https://integrate.api.nvidia.com/v1",
    api_key = "$API_KEY_REQUIRED_IF_EXECUTING_OUTSIDE_NGC"
)

completion = client.chat.completions.create(
    model="mistralai/mixtral-8x7b-instruct-v0.1",
    messages=[{"role": "user", "content": ""}],
    temperature=0.5,
    top_p=1,
    max_tokens=1024,
    stream=True
)

for chunk in completion:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="")
```



NVIDIA
GPU CLOUD

NVIDIA Foundation Model Endpoints

mistralai / **mixtral-8x7b-instruct-v0.1**

An MOE LLM that follows instructions, completes requests, and generates creative text.

advanced reasoning

chat

large language models

text-to-text

Python

Node

Shell

[Get API Key](#) [Copy Code](#)

```
curl https://integrate.api.nvidia.com/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $API_KEY_REQUIRED_IF_EXECUTING_OUTSIDE_NGC" \
-d '{
  "model": "mistralai/mixtral-8x7b-instruct-v0.1",
  "messages": [{"role": "user", "content": ""}],
  "temperature": 0.5,
  "top_p": 1,
  "max_tokens": 1024,
  "stream": true
}'
```

```
from openai import OpenAI

client = OpenAI(
    base_url = "https://integrate.api.nvidia.com/v1",
    api_key = "$API_KEY_REQUIRED_IF_EXECUTING_OUTSIDE_NGC"
)

completion = client.chat.completions.create(
    model="mistralai/mixtral-8x7b-instruct-v0.1",
    messages=[{"role": "user", "content": ""}],
    temperature=0.5,
    top_p=1,
    max_tokens=1024,
    stream=True
)

for chunk in completion:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="")
```



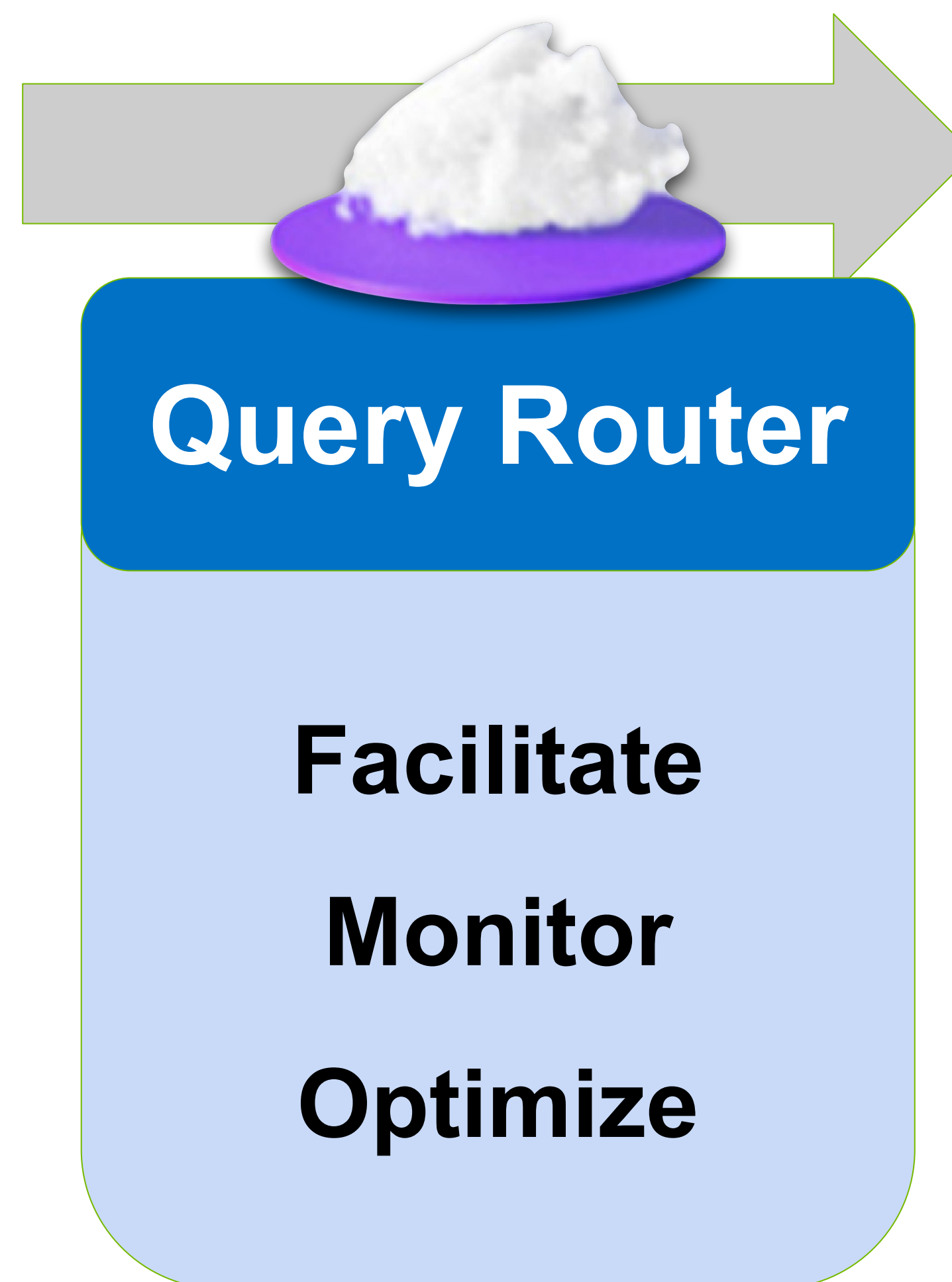
**NVIDIA
GPU CLOUD**

From Raw Requests to LangChain Model



From Raw Requests to LangChain Model

```
> llm("Hello world")  
  ChatMessage(content="hello")  
  
> embedder("Hello")  
  [0.4535437, 0.0800435, ...]
```



```
<s>[INST]<<SYS>>  
{{system_message}}  
<</SYS>>
```

```
{{instruction}} [/INST]  
{{primer}}
```

```
{  
  "context": {{context}}  
  "model": "query"/"doc"  
}
```



LLM Interfaces

The Whole Stack

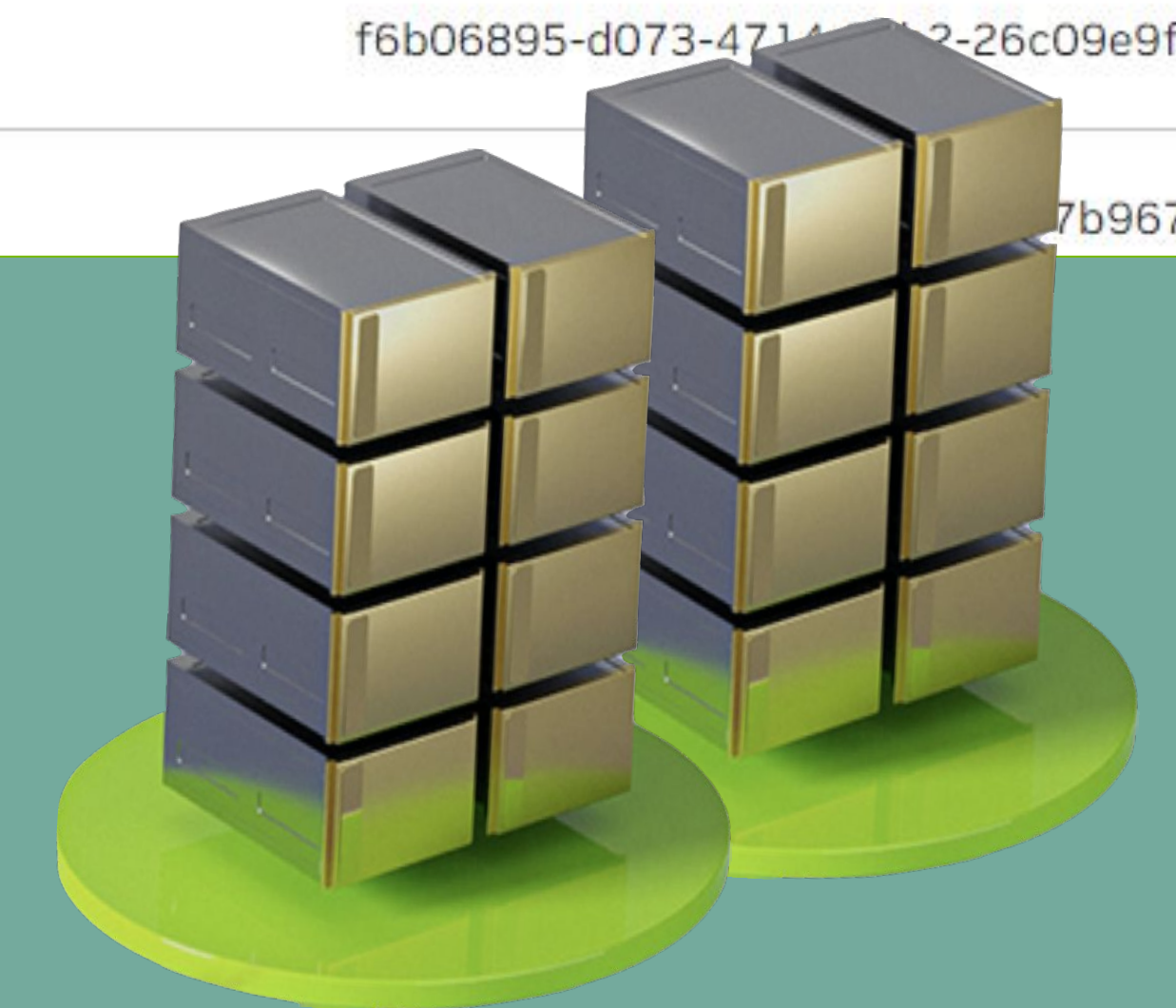
api.nvcf.
nvidia.com
/v2/nvcf

NVIDIA. NGC CLOUD FUNCTIONS	
CATALOG	
CONSOLE	
PRIVATE REGISTRY	
ORGANIZATION	
CLOUD FUNCTIONS	
Functions	
Create Function	

Cloud Functions > Functions	
Functions	
My Functions	Shared Functions
Created by NVIDIA	
Function Name	Function ID
ai-llama2-70b	2fddadfb-7e76-4c8a-9b82-f7d3fab94471
ai-mixtral-8x7b-instruct	a1e53ece-bff4-44d1-8b13-c009e5bf47f6
ai-mistral-7b-instruct-v2	d7618e99-db93-4465-af4d-330213a7f51f
ai-codellama-70b	f6b06895-d073-4714-8b13-26c09e9f6597
ai-gemma-7b	7b9675f9



Scaled
Function
Deployment
Solution



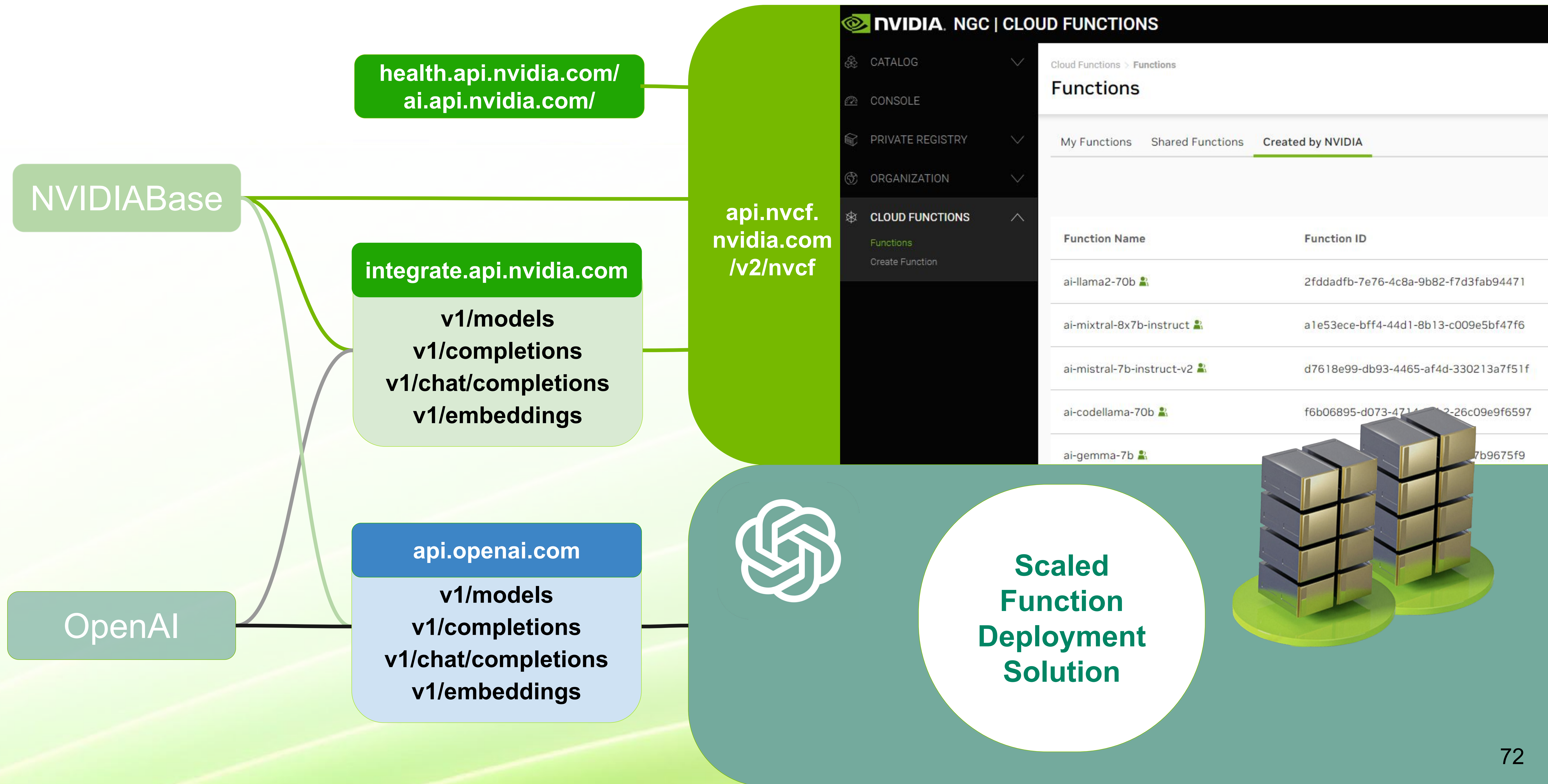
LLM Interfaces

The Whole Stack



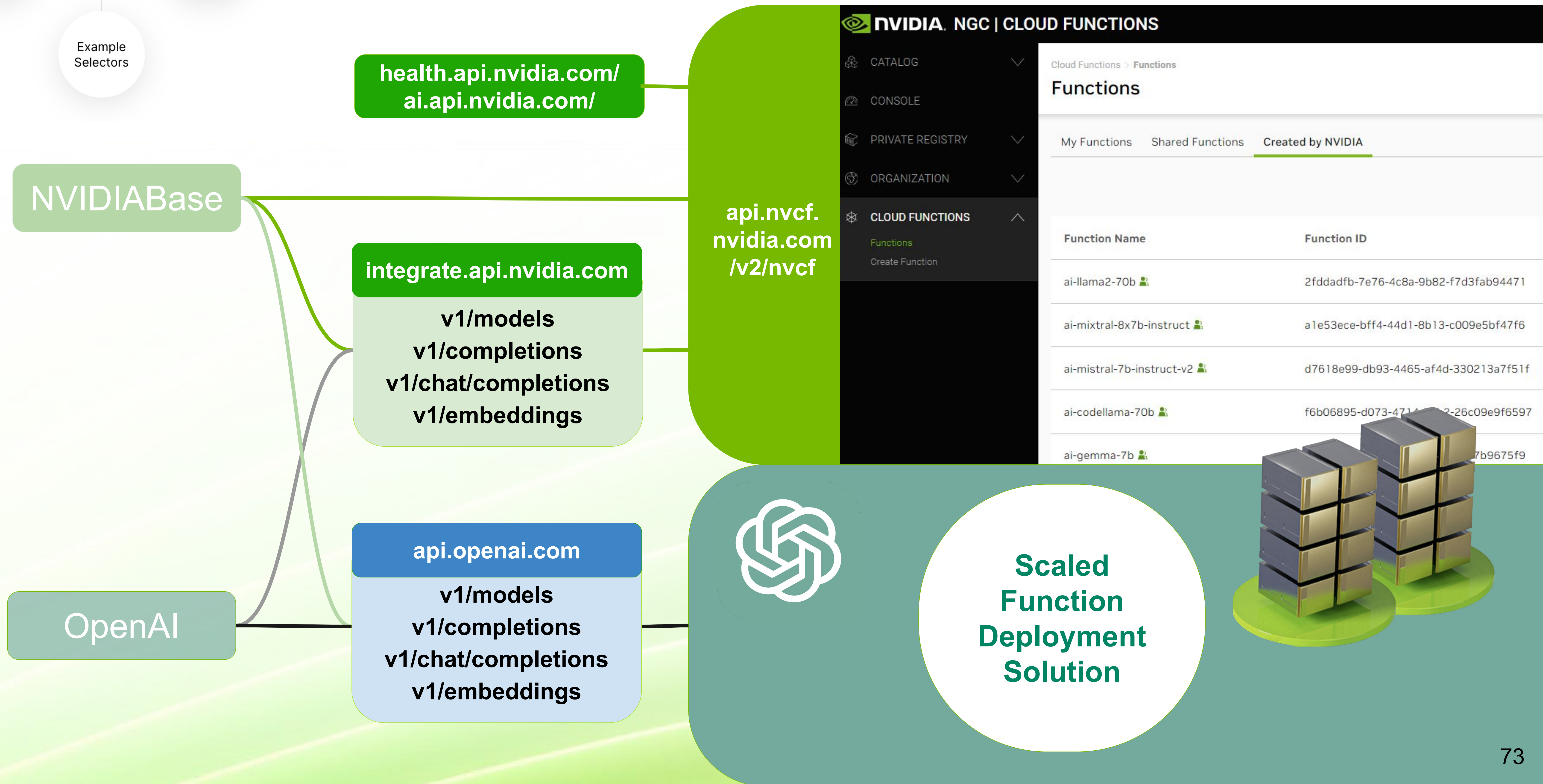
LLM Interfaces

The Whole Stack



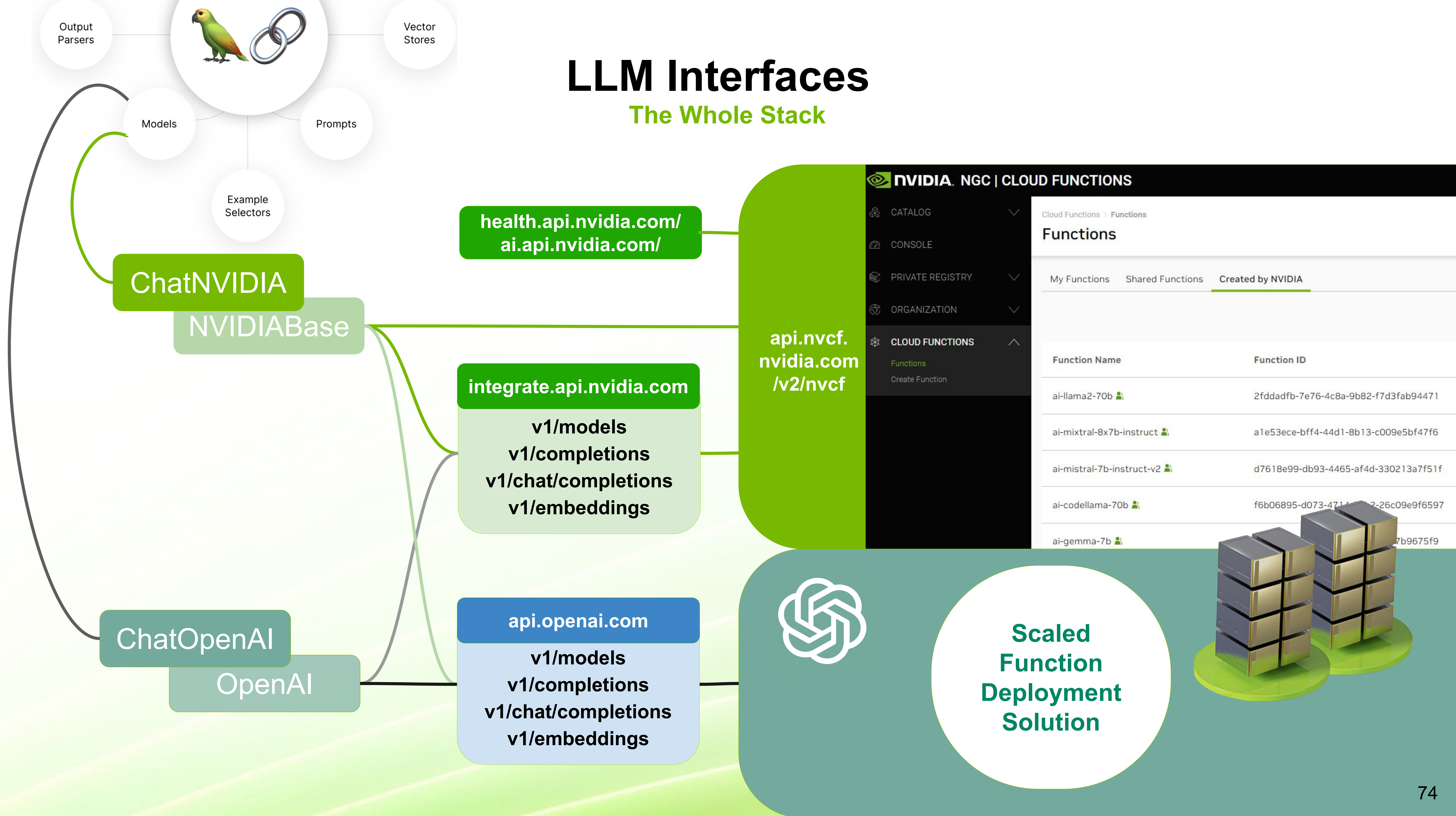
LLM Interfaces

The Whole Stack



LLM Interfaces

The Whole Stack





Building RAG Agents with LLMs

Part 3: Intro to LangChain

LangChain Structure



Chain Building

Just The LLM



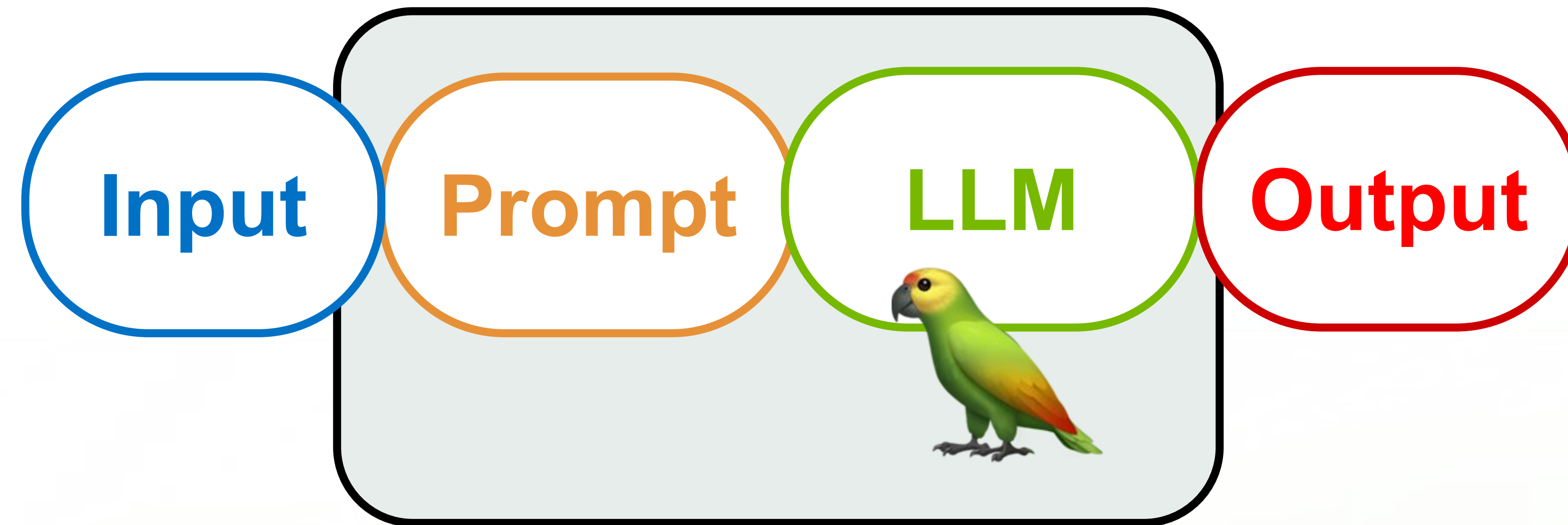
Chain Building

Just The LLM



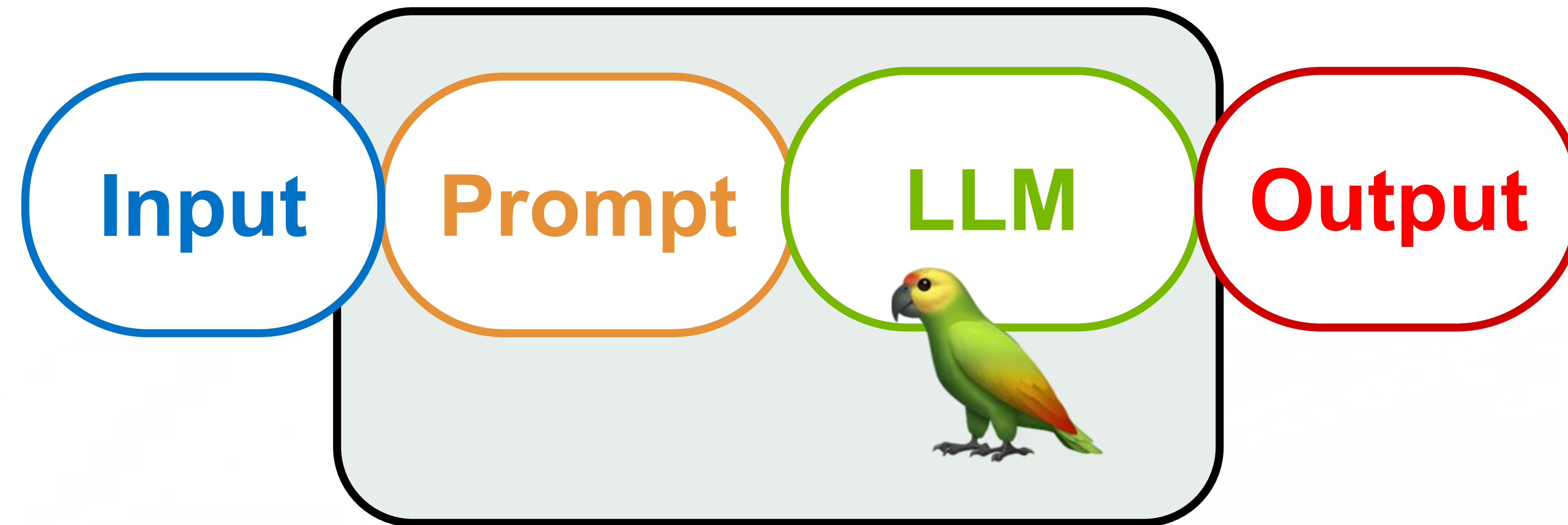
Chain Building

Simple Prompt+LLM Chain



Chain Building

Simple Prompt+LLM Chain

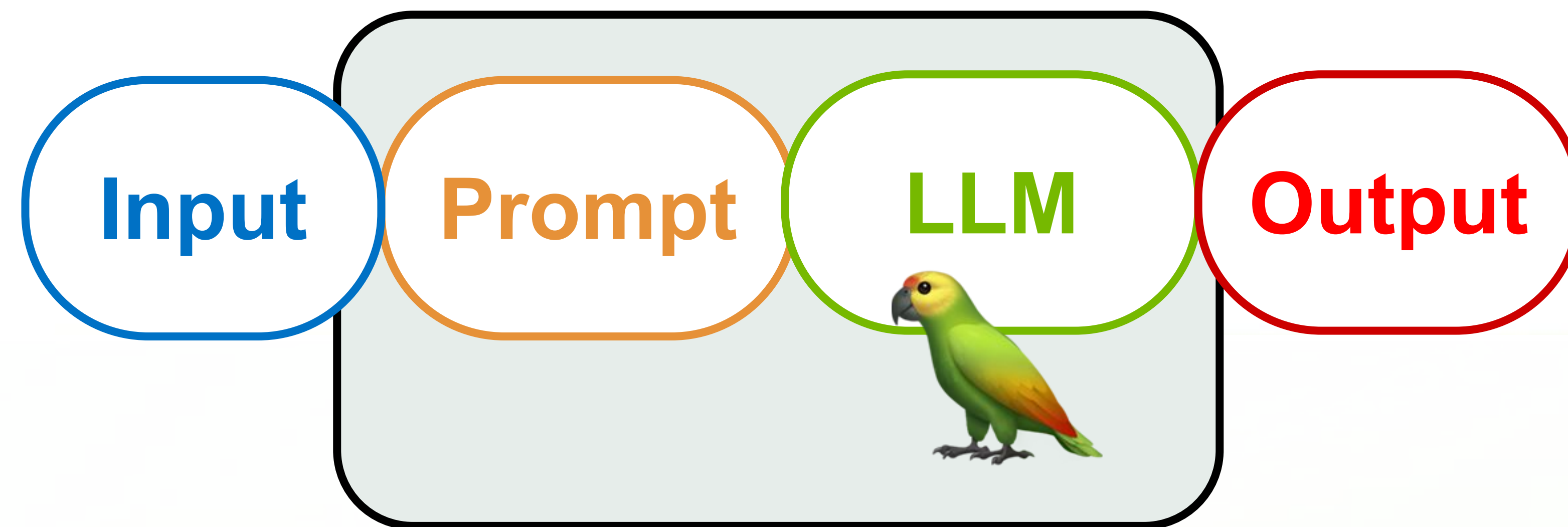


```
chain = prompt | chat  
chain.invoke(inputs)
```



Chain Building

Simple Prompt+LLM Chain



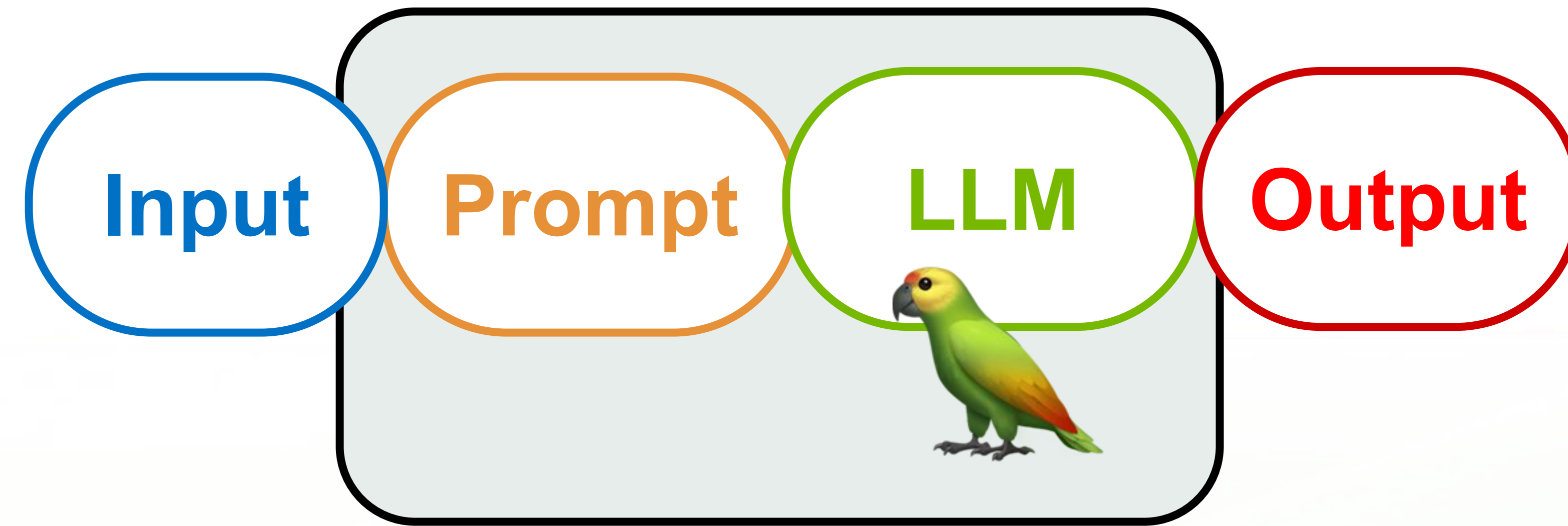
```
## StrOutputParser()  
def get_content(value):  
    return getattr(value, "content", value)
```

```
chain = prompt | chat | get_content  
chain.invoke(inputs)
```



Chain Building

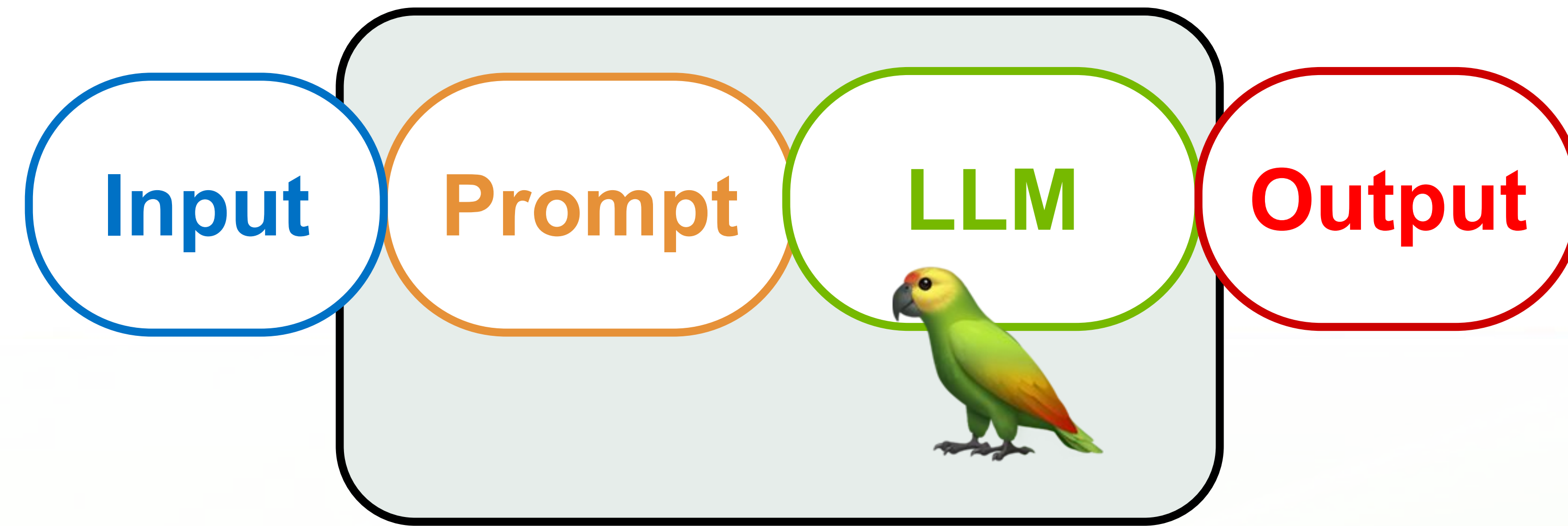
Invoking Runnables



```
chain = prompt | chat | StrOutputParser()
```

Chain Building

Invoking Runnables



```
chain = prompt | chat | StrOutputParser()
```

```
msg = chain.invoke(...)
```

```
for token in chain.stream(...)
```

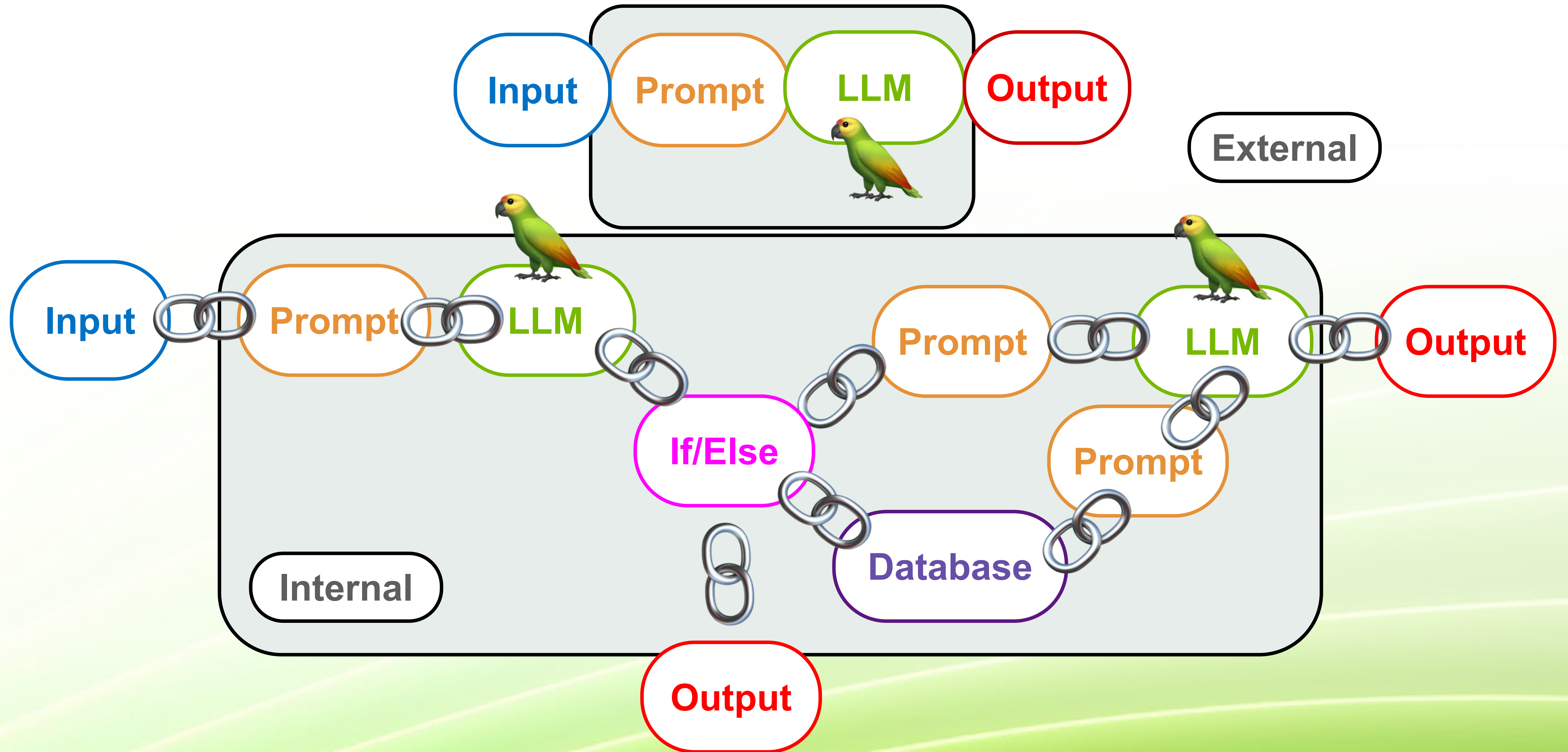
Chain Building

Building Information Pipelines

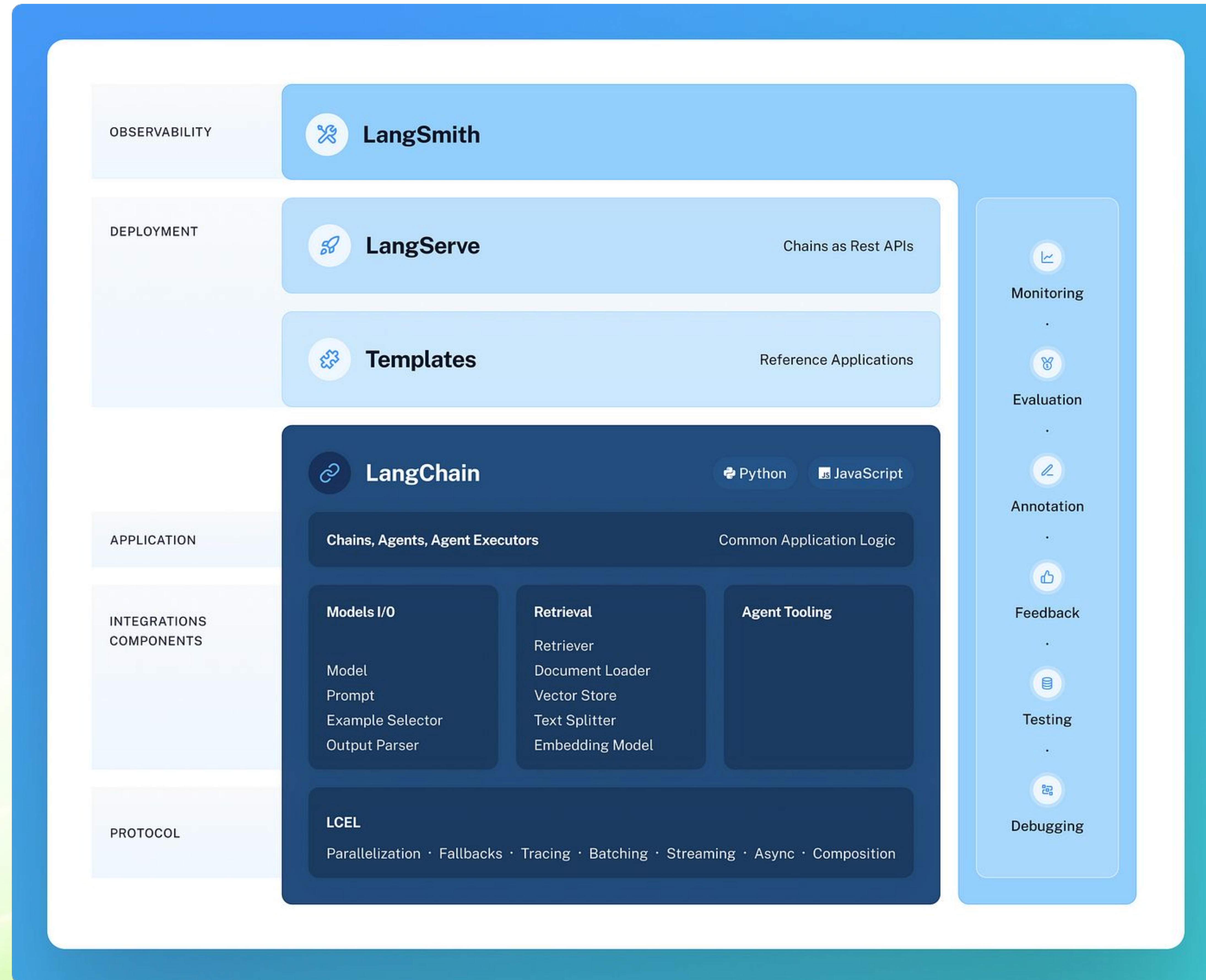


Chain Building

Building Information Pipelines



LangChain Extended Ecosystem



Chatbot

Hello World!!

You typed: Hello World!!

Type a message...

Submit

Retry

Undo

Clear

04_running_state.ipynb

File Edit View Insert Runtime Tools Help

Last edited on February 9

Connect

DEEP LEARNING INSTITUTE

▼ Notebook 4: Running State Chains

In the previous notebook, we introduced some key LangChain Expression Language (LCEL) material regarding runnables. By now, you should be comfortable with both internal and external reasoning, as well as how to develop pipelines that facilitate it! In this notebook, we will make our way towards more advanced paradigms that will allow us to orchestrate more complex dialog management strategies and begin to execute on longer-form document reasoning.

Learning Objectives:

Our Environment

Jupyter Notebooks + Frontend + LLM Client

The diagram illustrates the environment architecture, showing the interaction between the Frontend, LLMs, Jupyter Notebook Server, and Docker Router, all running on a Remote Host.

- Remote Host**: The container for all components.
- Frontend**: An orange box representing the user interface.
- LLMs**: A green box representing the Large Language Models.
- Jupyter Notebook Server**: A dark gray box representing the server for Jupyter notebooks.
- Docker Router**: A gray circle representing the router for Docker containers.

Connections and Ports:

- The **Frontend** connects to the **Jupyter Notebook Server** via port **:8090**.
- The **Frontend** connects to the **Jupyter Notebook Server** via port **:88**.
- The **Frontend** connects to the **Jupyter Notebook Server** via port **:9012**.
- The **LLMs** connect to the **Jupyter Notebook Server** via port **:9000**.
- The **LLMs** connect to the **Jupyter Notebook Server** via port **:8090**.

External connections are shown with a cloud icon and server racks, indicating network connectivity.

87

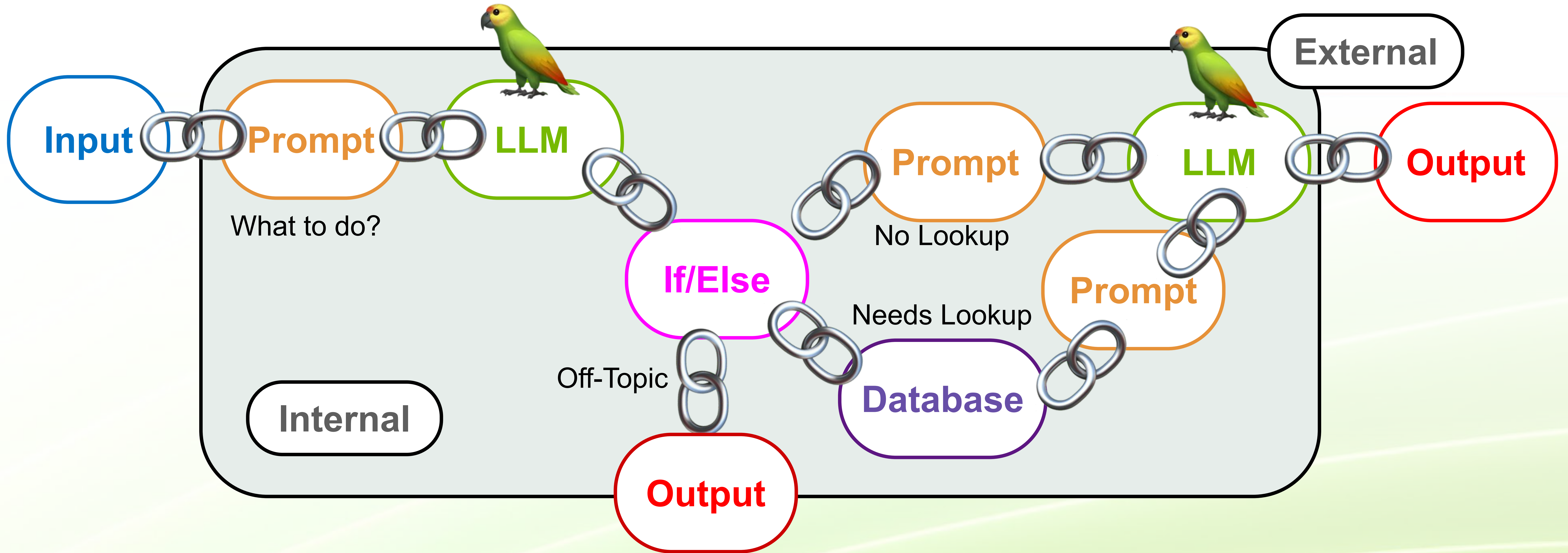


Building RAG Agents with LLMs

Part 4: Running State Chain

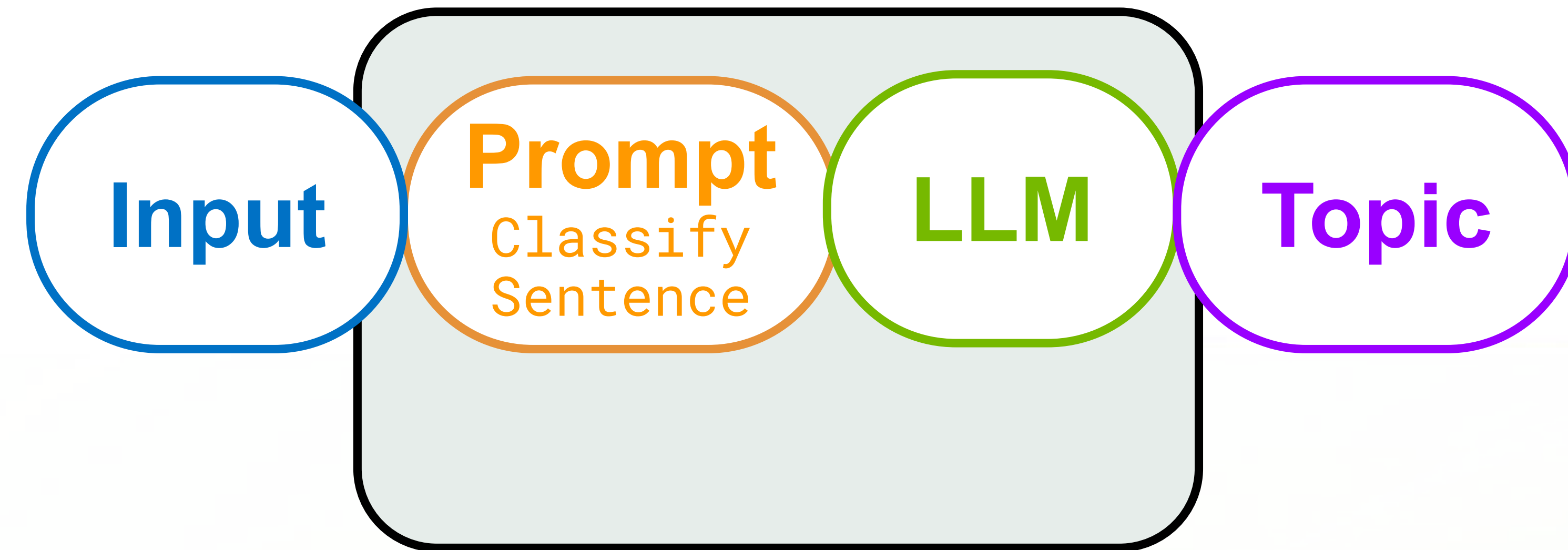
Chain Building

Recall Our Assumptions



Chain Building

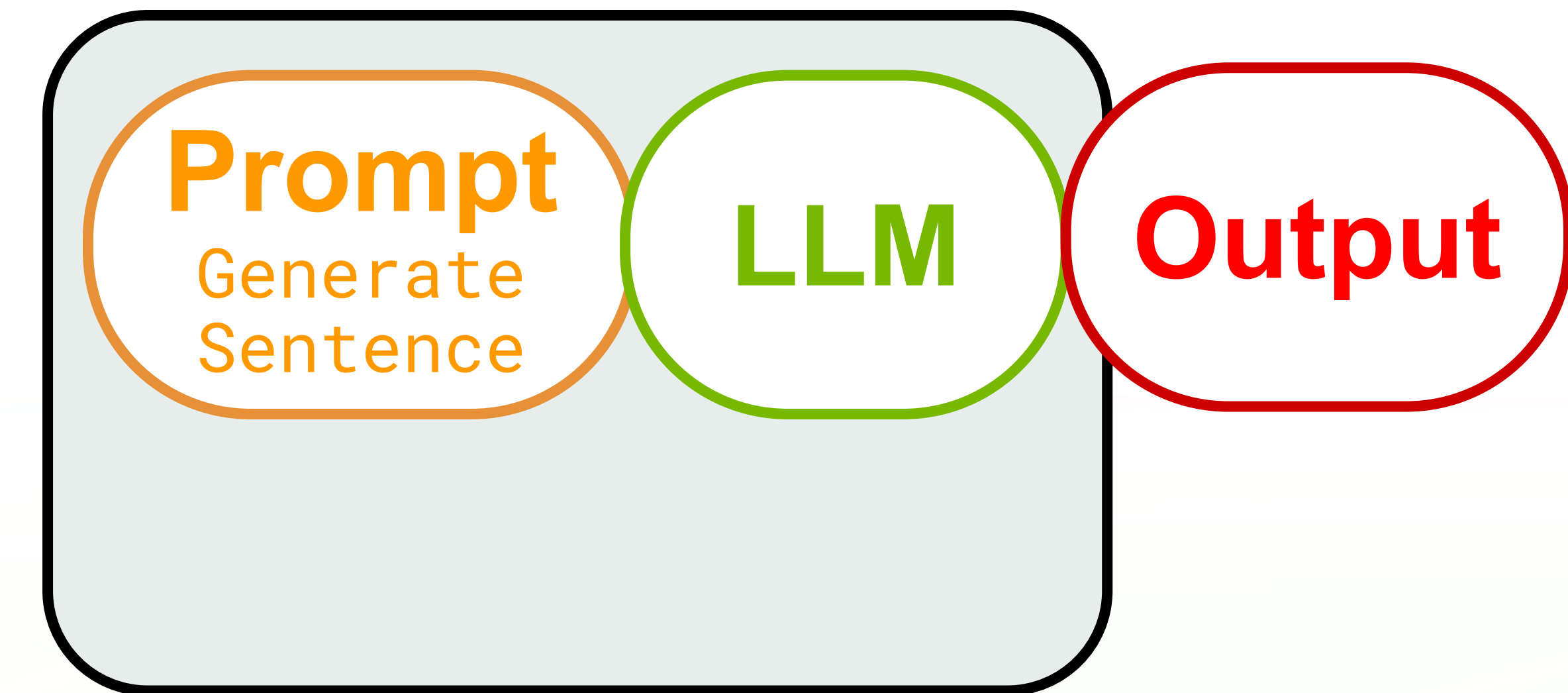
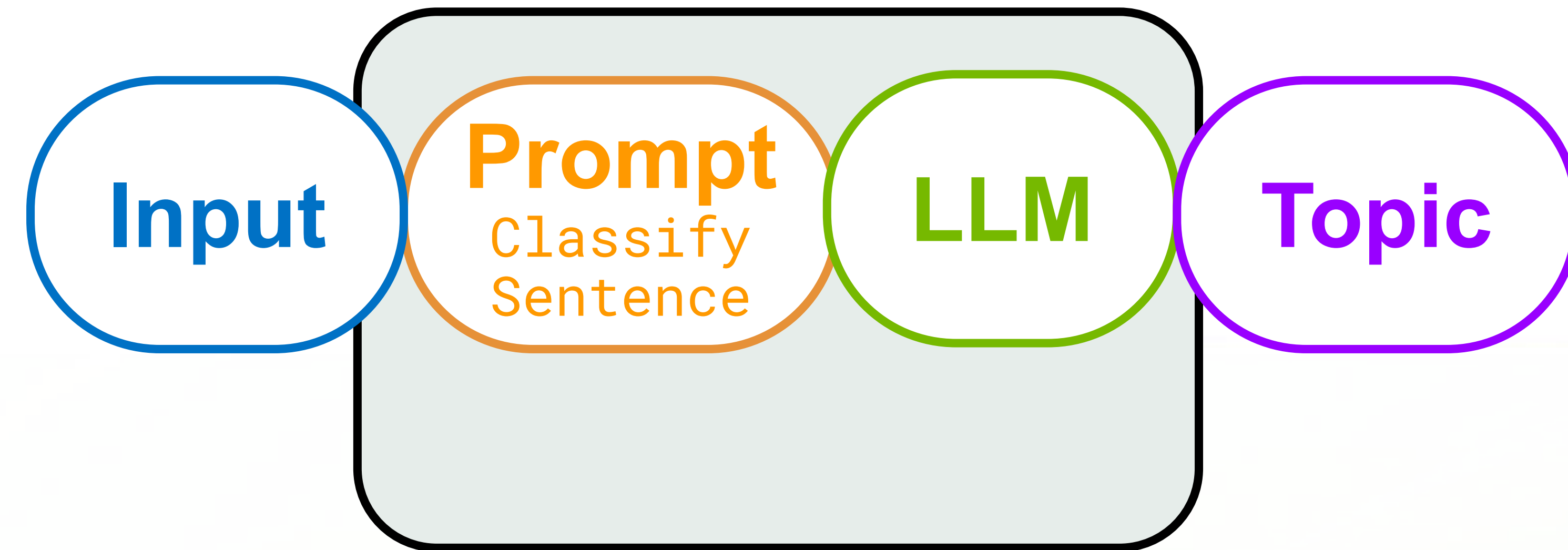
Towards Running State



```
inputs = {'input' : 'sentence'}  
cls_chain = cls_prompt | llm  
topic = cls_chain.invoke(inputs)
```

Chain Building

Towards Running State

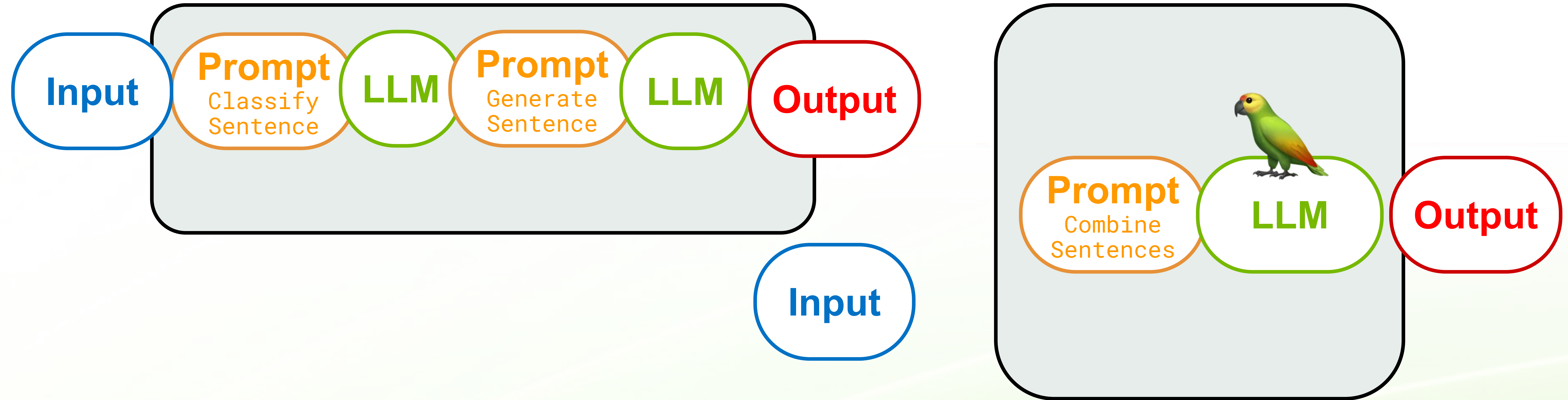


```
inputs = {'input' : 'sentence'}  
cls_chain = cls_prompt | llm  
topic = cls_chain.invoke(inputs)
```

```
gen_chain = out_prompt | llm  
out_chain = cls_chain | gen_chain  
for token in out_chain.stream(inputs):  
    print(token, end="")
```

Chain Building

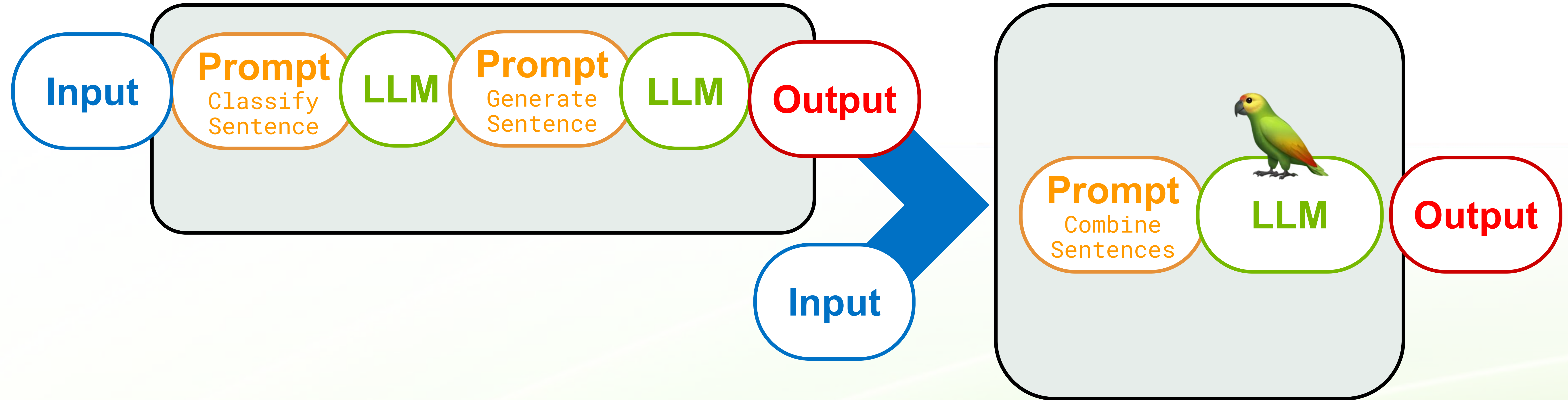
Towards Running State



```
inputs = {'input' : 'sentence'}  
new_sentence = out_chain.invoke(inputs)
```

Chain Building

Towards Running State

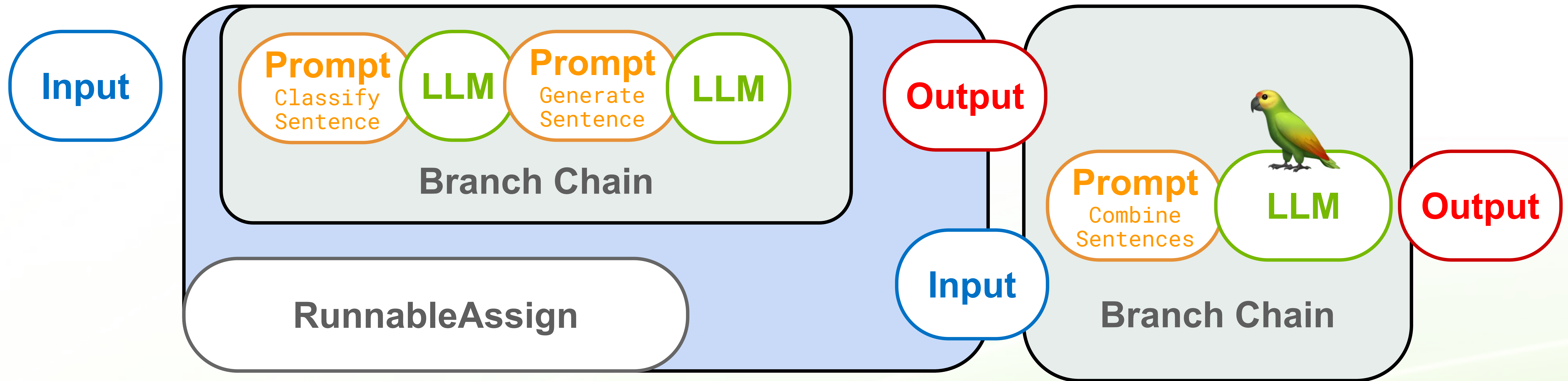


```
inputs = {'input' : 'sentence'}  
new_sentence = out_chain.invoke(inputs)
```

```
inputs.update({'new' : 'new_sentence'})  
for token in merge_chain.stream(inputs):  
    print(token, end="")
```

Chain Building

Towards Running State



```
out_chain = RunnableAssign({  
    new_sentence : cls_chain | gen_chain  
}) | merge_chain
```

Typical Running State

Regular Fibonacci w/ While Loop

```
def fn():  
    ## Initial State  
    n = 8  
    fib = [0,1]  
  
    ## Running State  
    while len(fib) < n:  
        fib += [fib[-1] + fib[-2]]  
  
    ## Final State  
    print("\nFinal State:", locals())  
  
    ## Variable Pulled From State  
    print("\nFinal Output Variable:", fib)  
    return fib  
  
fn();
```

Initial State: {'n': 8, 'fib': [0, 1]}

Running State: {'n': 8, 'fib': [0, 1, 1]}

Running State: {'n': 8, 'fib': [0, 1, 1, 2]}

Running State: {'n': 8, 'fib': [0, 1, 1, 2, 3]}

Running State: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5]}

Running State: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5, 8]}

Running State: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5, 8, 13]}

Final State: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5, 8, 13]}

Final Output Variable: [0, 1, 1, 2, 3, 5, 8, 13]

Running State Chain Components

Towards LCEL While Loop

```
def get_initial_state():  
    n = 8  
    fib = [0,1]  
    return locals()  
  
initial_state = get_initial_state()  
initial_state  
  
{'n': 8, 'fib': [0, 1]}
```

State

n = 8
fib = [0,1]

Running State Chain Components

Towards LCEL While Loop

```
running_state = {'n': 8, 'fib': [0, 1]}
```

```
running_state['n'] = 7
```

```
running_state['fib'] += [1]
```

```
running_state['msg'] = "Hello"
```

```
{'n': [7], 'fib': [0, 1]}
```

```
{'n': [7], 'fib': [0, 1, 1]}
```

```
{'n': [7], 'fib': [0, 1, 1], 'msg': 'Hello'}
```

State

n = 8

fib = [0,1]

State

n = 7

fib = [0,1,1]

msg="Hello"

Running State Chain Components

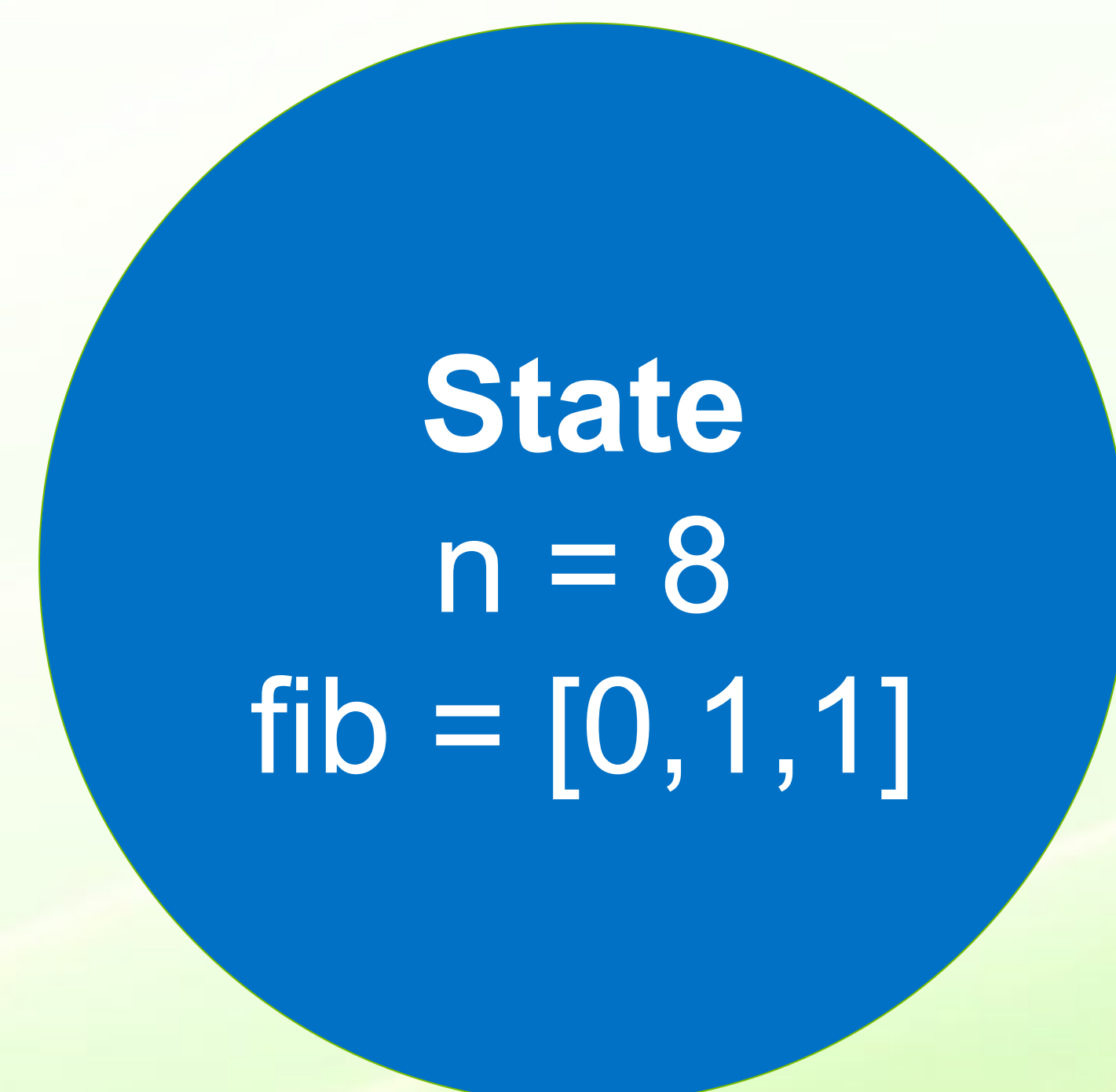
Towards LCEL While Loop

```
from langchain_core.runnables import RunnableLambda

def next_fib_fn(d):
    return d['fib'] + [d['fib'][-1] + d['fib'][-2]]

fib_branch = RunnableLambda(next_fib_fn)
fib_branch.invoke(running_state)

[0, 1, 1, 2]
```



Running State Chain Components

Towards LCEL While Loop

```
from langchain_core.runnables import RunnableLambda

def next_fib_fn(d):
    return d['fib'] + [d['fib'][-1] + d['fib'][-2]]

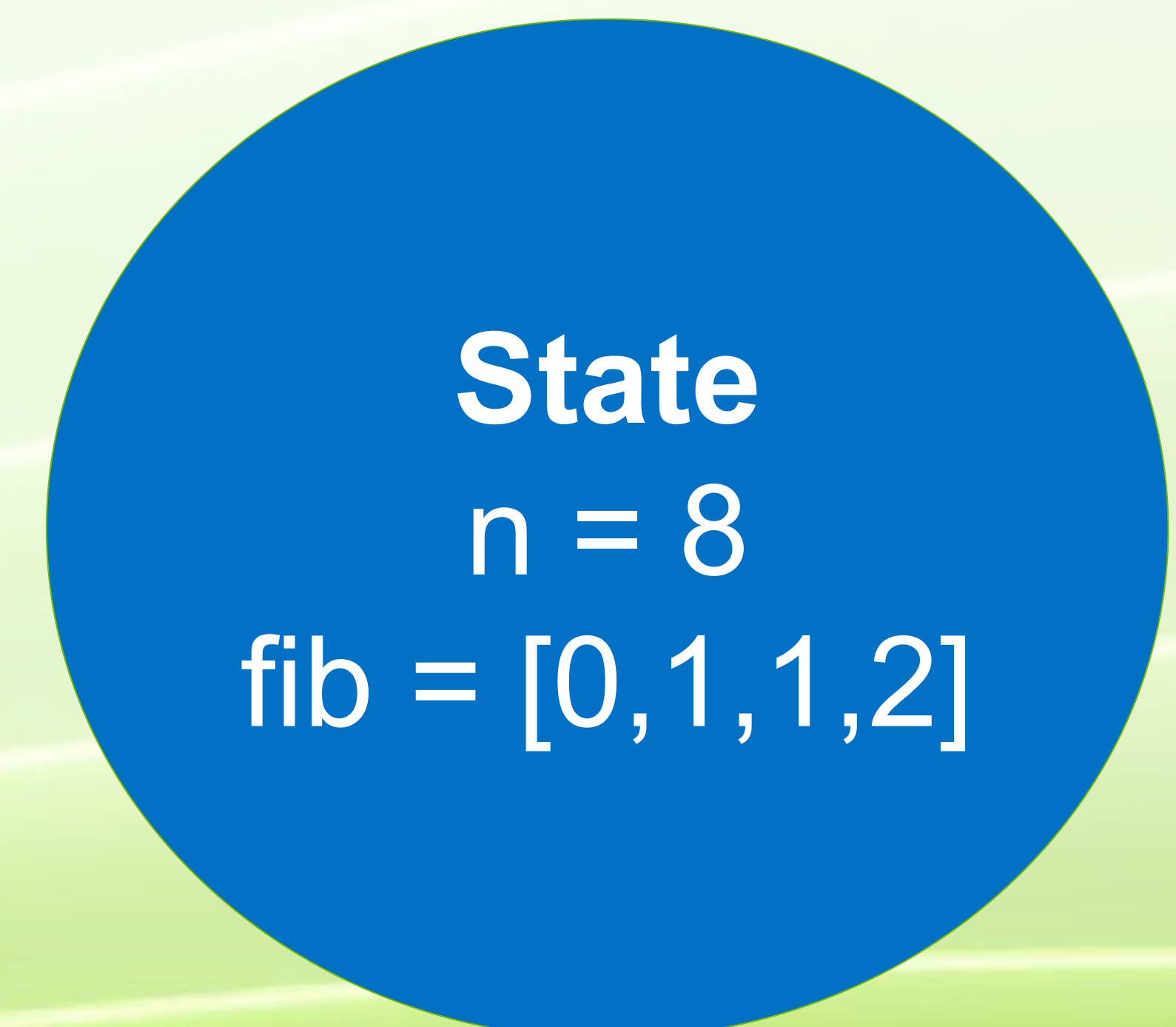
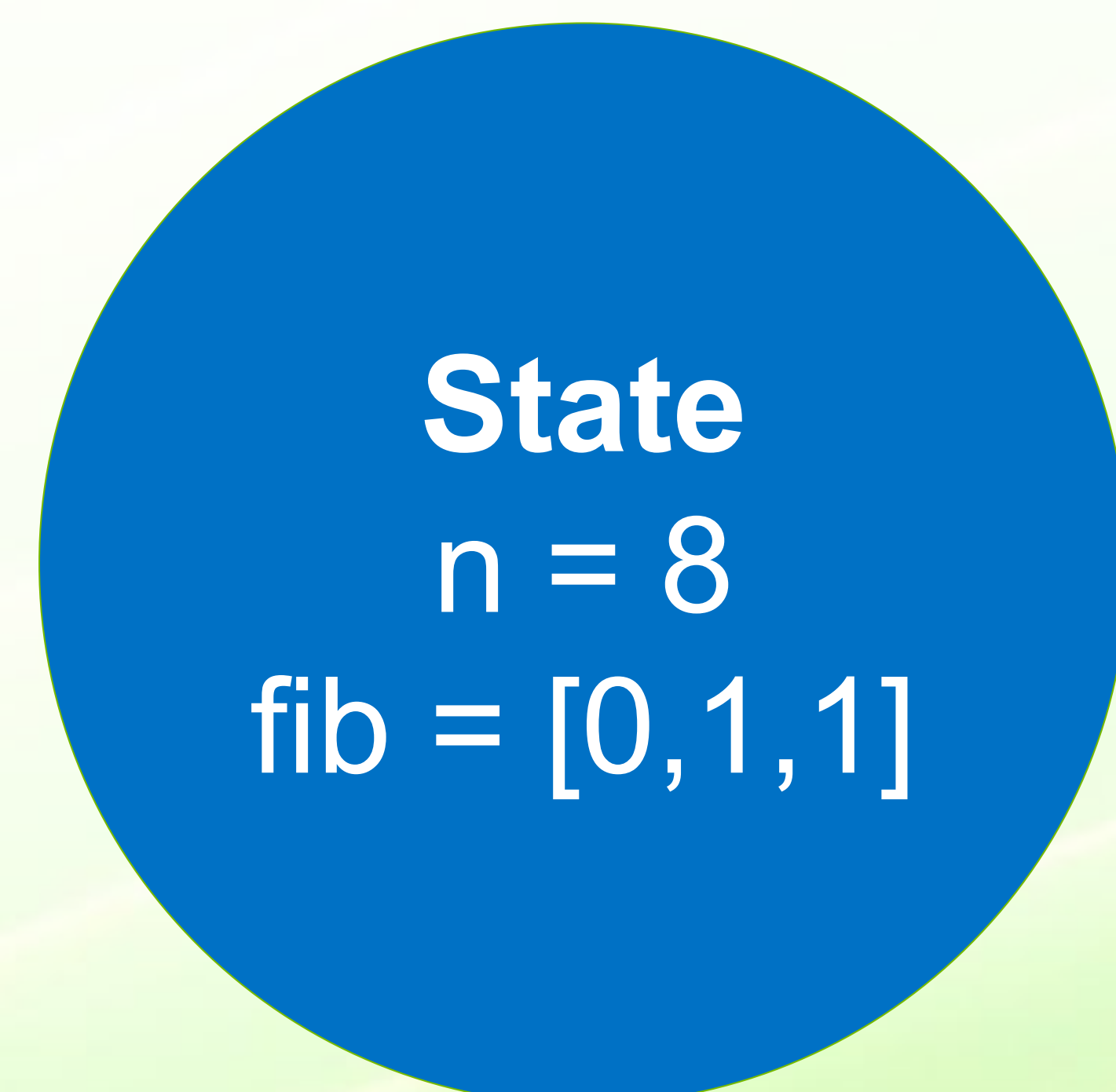
fib_branch = RunnableLambda(next_fib_fn)
fib_branch.invoke(running_state)

[0, 1, 1, 2]
```

```
from langchain_core.runnables import RunnableAssign

rstate_chain = RunnableAssign({'fib' : fib_branch})
running_state = initial_state.copy()
running_state = rstate_chain.invoke(running_state)

running_state: {'n': 8, 'fib': [0, 1, 1, 2]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2, 3]}
```



Running State Chain Components

Towards LCEL While Loop

```
from langchain_core.runnables import RunnableLambda

def next_fib_fn(d):
    return d['fib'] + [d['fib'][-1] + d['fib'][-2]]

fib_branch = RunnableLambda(next_fib_fn)
fib_branch.invoke(running_state)

[0, 1, 1, 2]
```

```
from langchain_core.runnables import RunnableAssign

rstate_chain = RunnableAssign({'fib' : fib_branch})
running_state = initial_state.copy()
running_state = rstate_chain.invoke(running_state)
running_state = rstate_chain.invoke(running_state)

running_state: {'n': 8, 'fib': [0, 1, 1, 2]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2, 3]}
```

State
n = 8
fib = [0,1,1]

State
n = 8
fib = [0,1,1,2]

State
n = 8
fib =
[0,1,1,2, 3]

Final Running State Loop

```
from langchain_core.runnables import RunnableAssign

def next_fib_fn(d):
    fib = d.get('fib')
    return fib + [fib[-1] + fib[-2]]

rstate_chain = RunnableAssign({'fib' : next_fib_fn})

running_state = {
    'n': 8,
    'fib': [0, 1]
}
while len(running_state['fib']) < running_state['n']:
    running_state = rstate_chain.invoke(running_state)

running_state: {'n': 8, 'fib': [0, 1, 1]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2, 3]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5, 8]}
running_state: {'n': 8, 'fib': [0, 1, 1, 2, 3, 5, 8, 13]}
```

Typical Running State Loop

Comparing Typical with Running State

```
def fn():
    ## Initial State
    n = 8
    fib = [0,1]

    ## Running State
    while len(fib) < n:
        fib += [fib[-1] + fib[-2]]

    ## Final State
    print("\nFinal State:", locals())

    ## Variable Pulled From State
    print("\nFinal Output Variable:", fib)
    return fib
```

```
from langchain_core.runnables import RunnableAssign

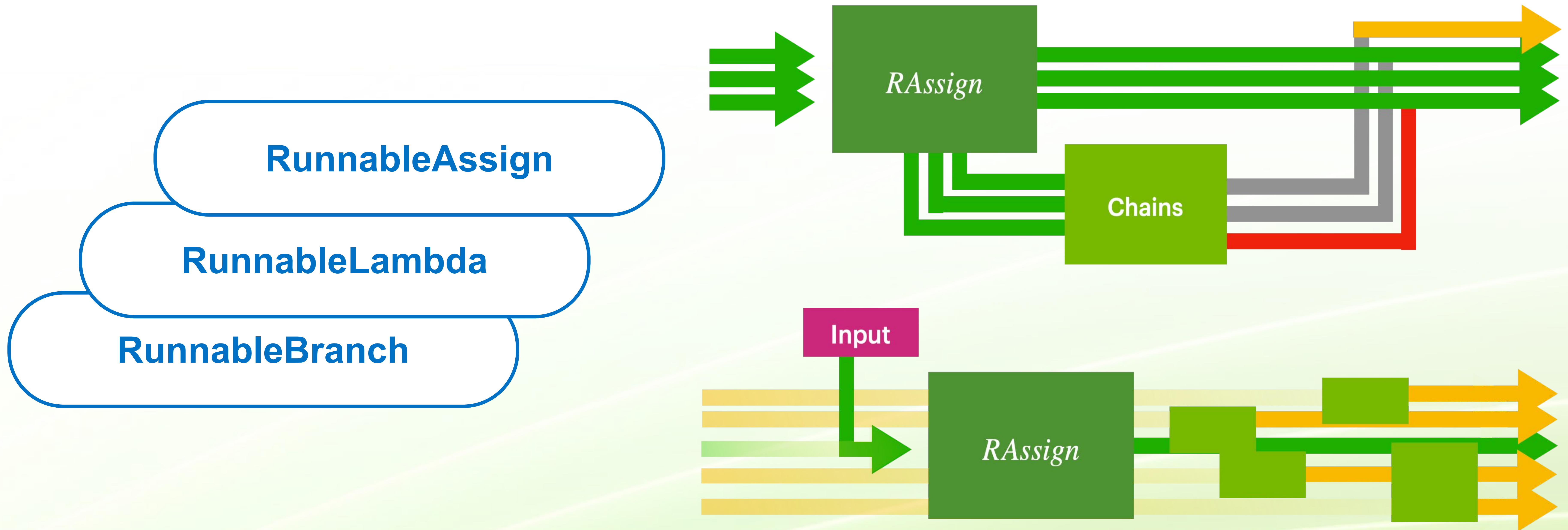
def next_fib_fn(d):
    fib = d.get('fib')
    return fib + [fib[-1] + fib[-2]]

rstate_chain = RunnableAssign({'fib' : next_fib_fn})

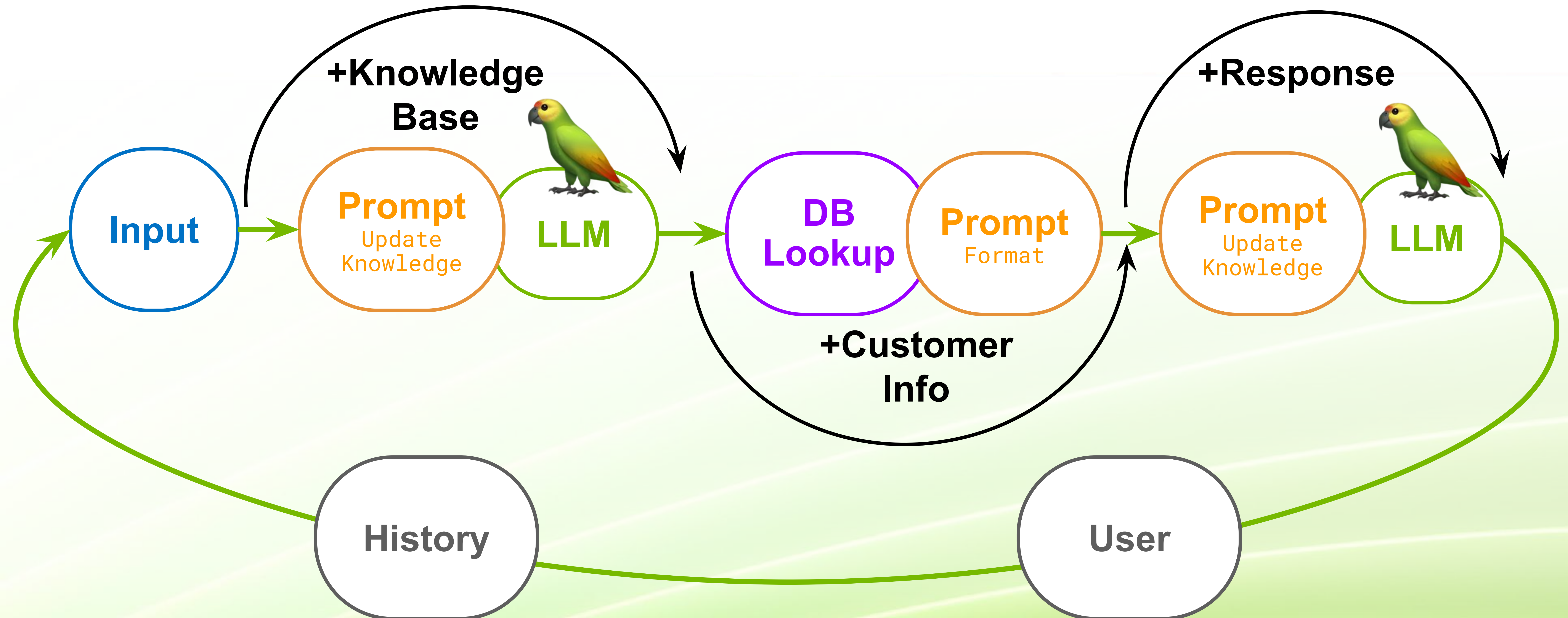
running_state = {
    'n': 8,
    'fib': [0, 1]
}
while len(running_state['fib']) < running_state['n']:
    running_state = rstate_chain.invoke(running_state)
```

Final Running State Loop

Big Picture

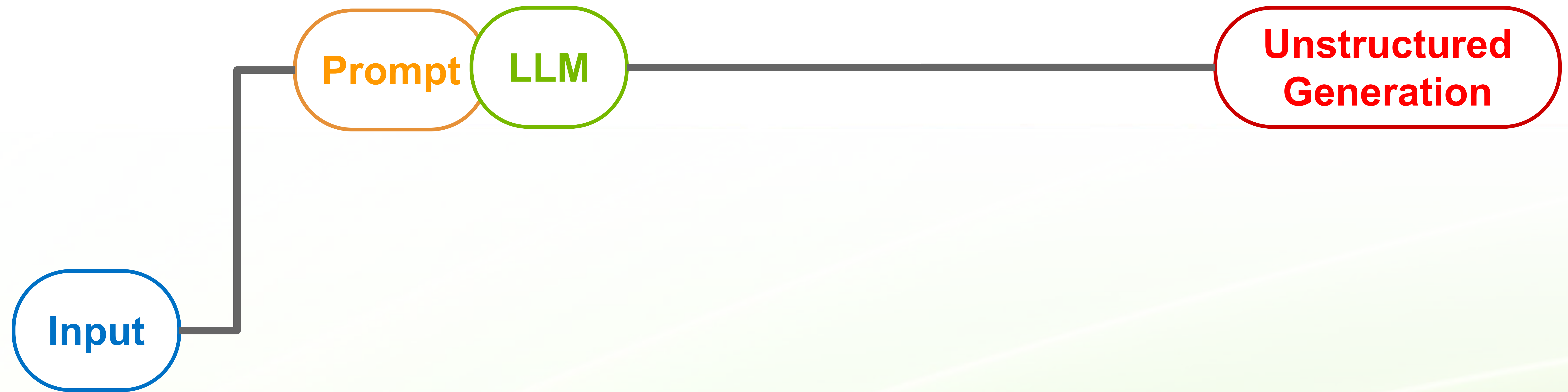


Airline Chatbot



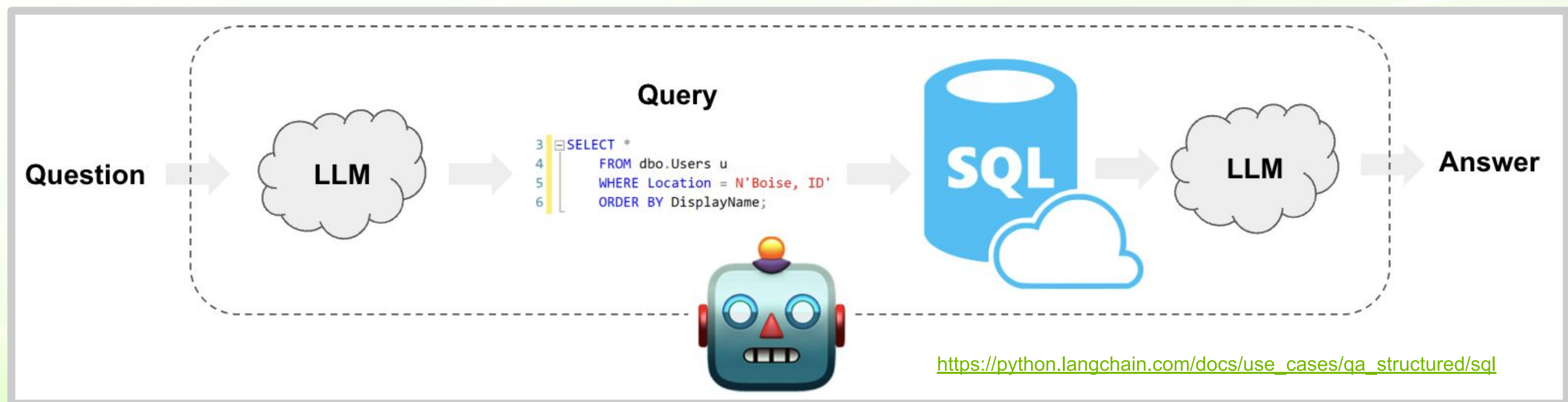
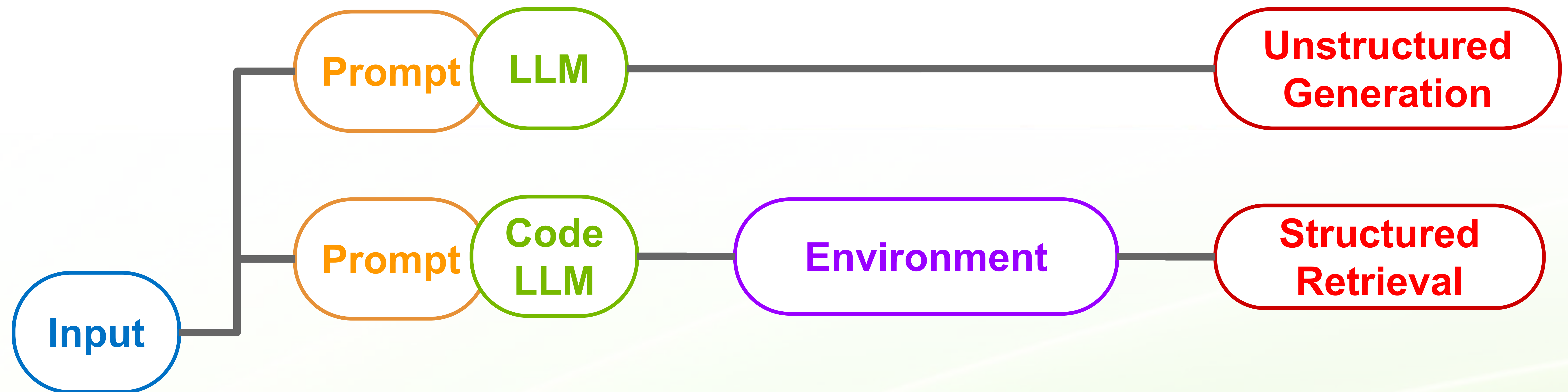
Modern Chain Paradigms

Towards Powerful Running State



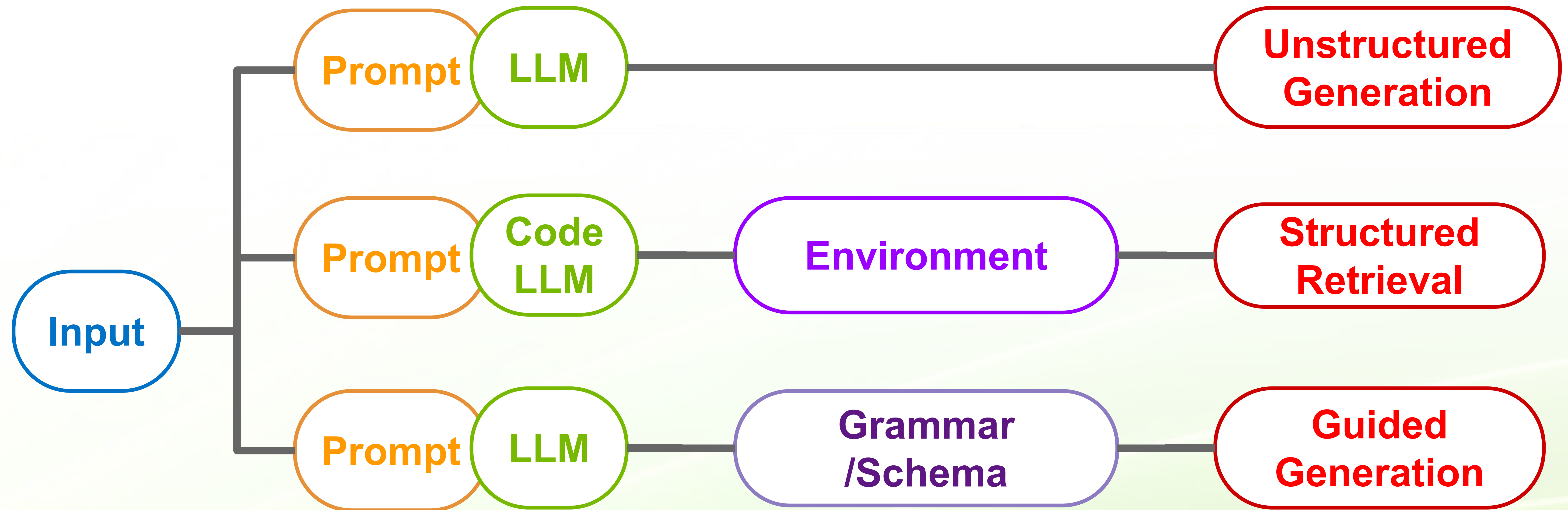
Modern Chain Paradigms

Towards Powerful Running State

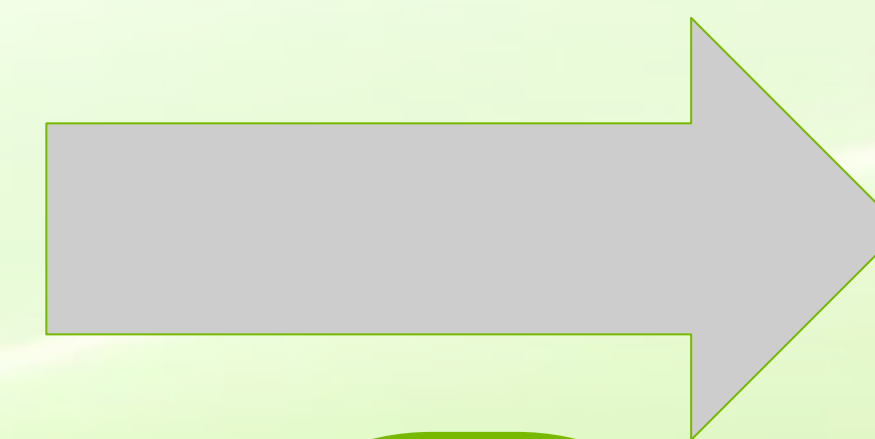


Modern Chain Paradigms

Towards Powerful Running State



```
{ "first_name" : "unknown",  
  "last_name" : "unknown",  
  "confirmation" : -1 }  
Update given new info:  
"Sure, my name is Jane Doe!"
```

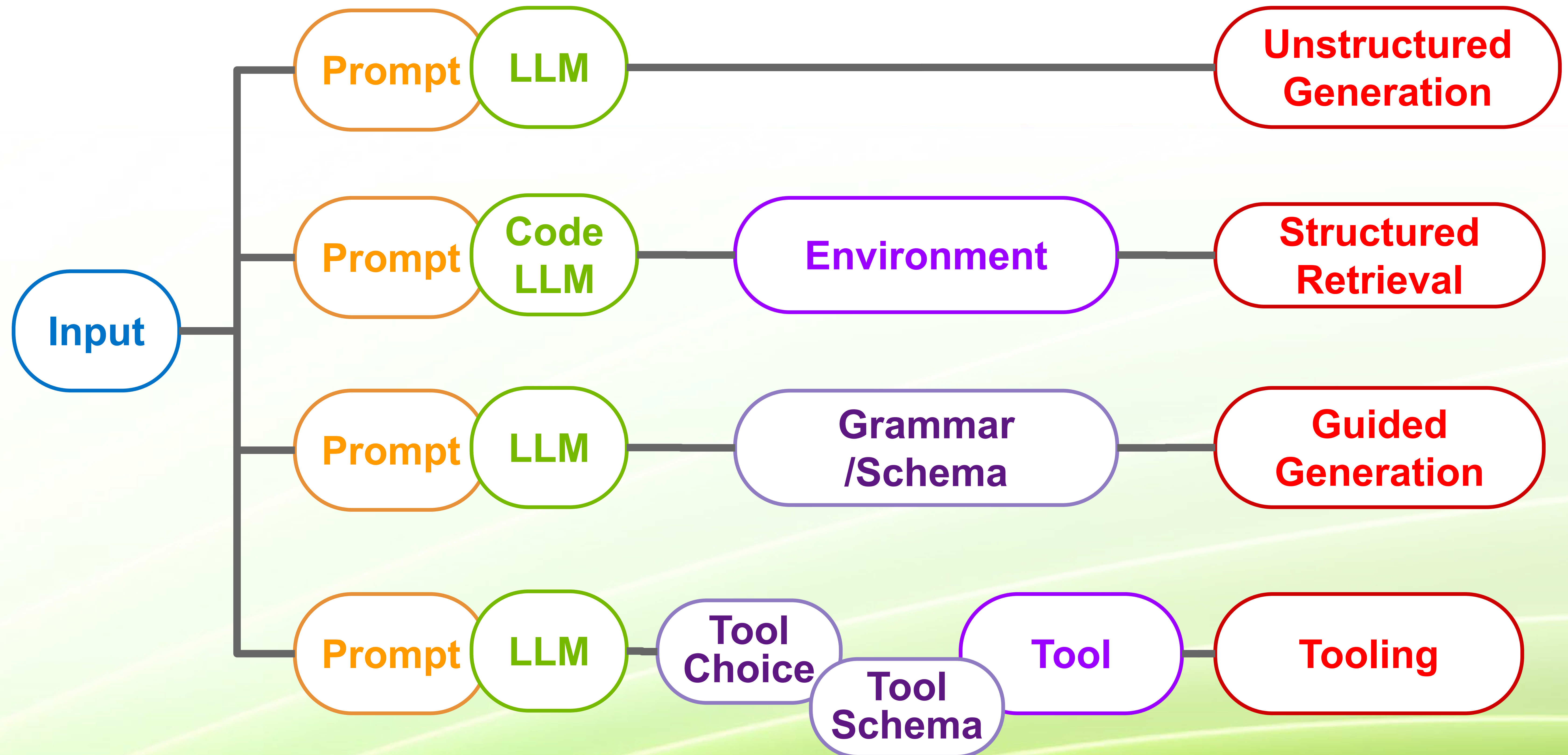


LLM

```
{ "first_name" : "Jane",  
  "last_name" : "Doe",  
  "confirmation" : -1 }
```

Modern Chain Paradigms

Towards Powerful Running State



Final Objective

Knowledge Base + Running State Chain

```
{  
  "first_name" : "unknown",  
  "last_name" : "unknown",  
  "confirmation" : -1,  
  ...  
}
```

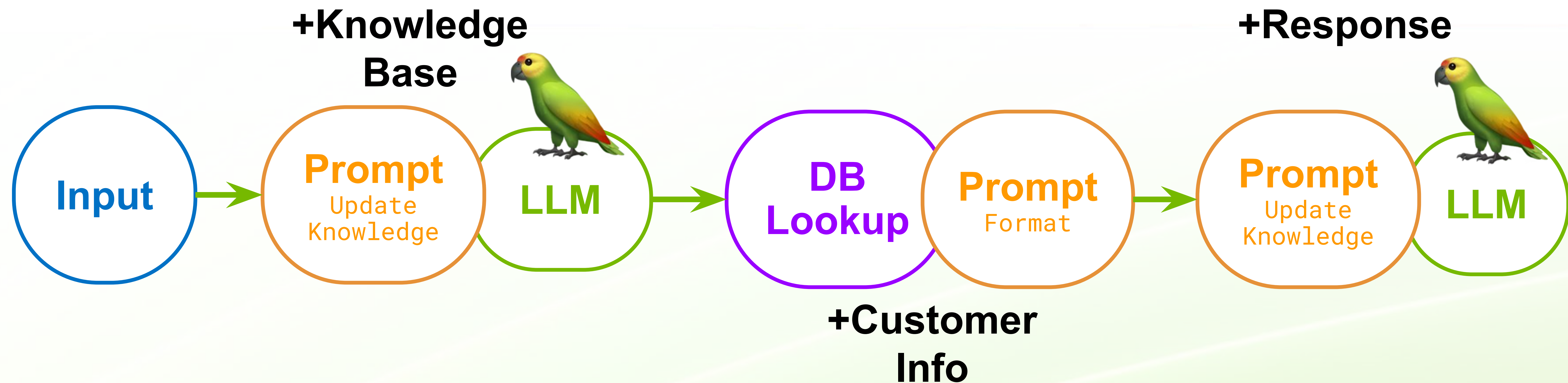
Update given new info:
"Sure, my name is Jane Doe!"



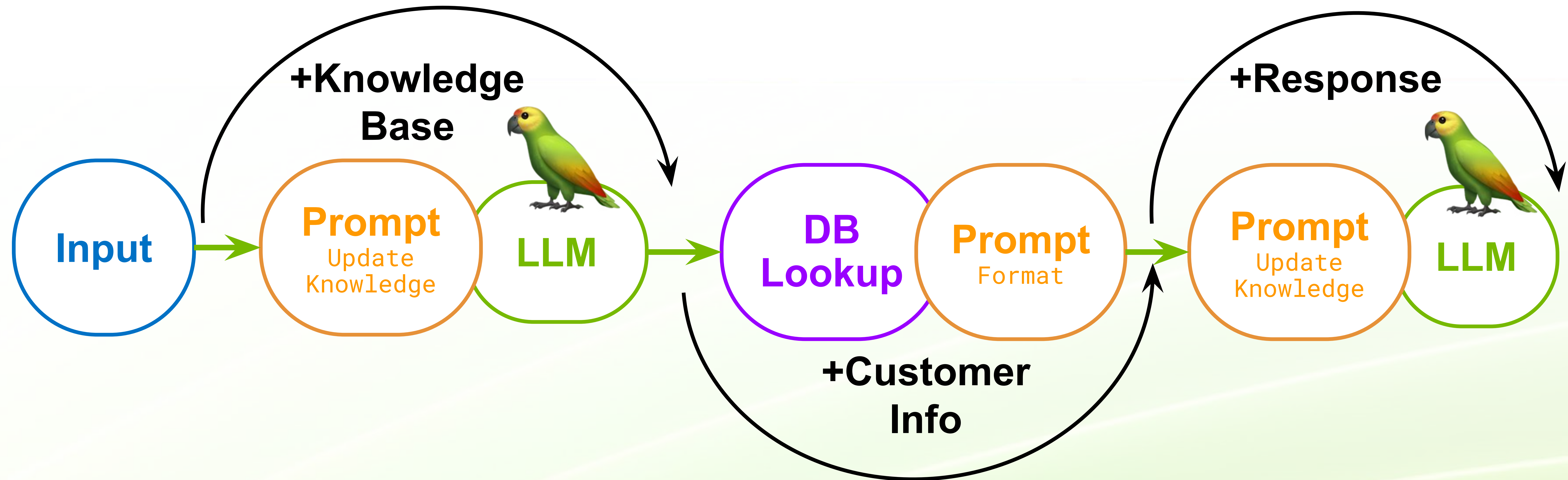
```
{  
  "first_name" : "Jane",  
  "last_name" : "Doe",  
  "confirmation" : -1,  
  ...  
}
```

```
## Your goal is to invoke the following through natural conversation  
get_flight_info({  
  "first_name" : "Jane",  
  "last_name" : "Doe",  
  "confirmation" : 12345,  
}) -> "Jane Doe's flight from San Jose to New Orleans departs at 12:30 PM tomorrow and lands at 9:30 PM."
```

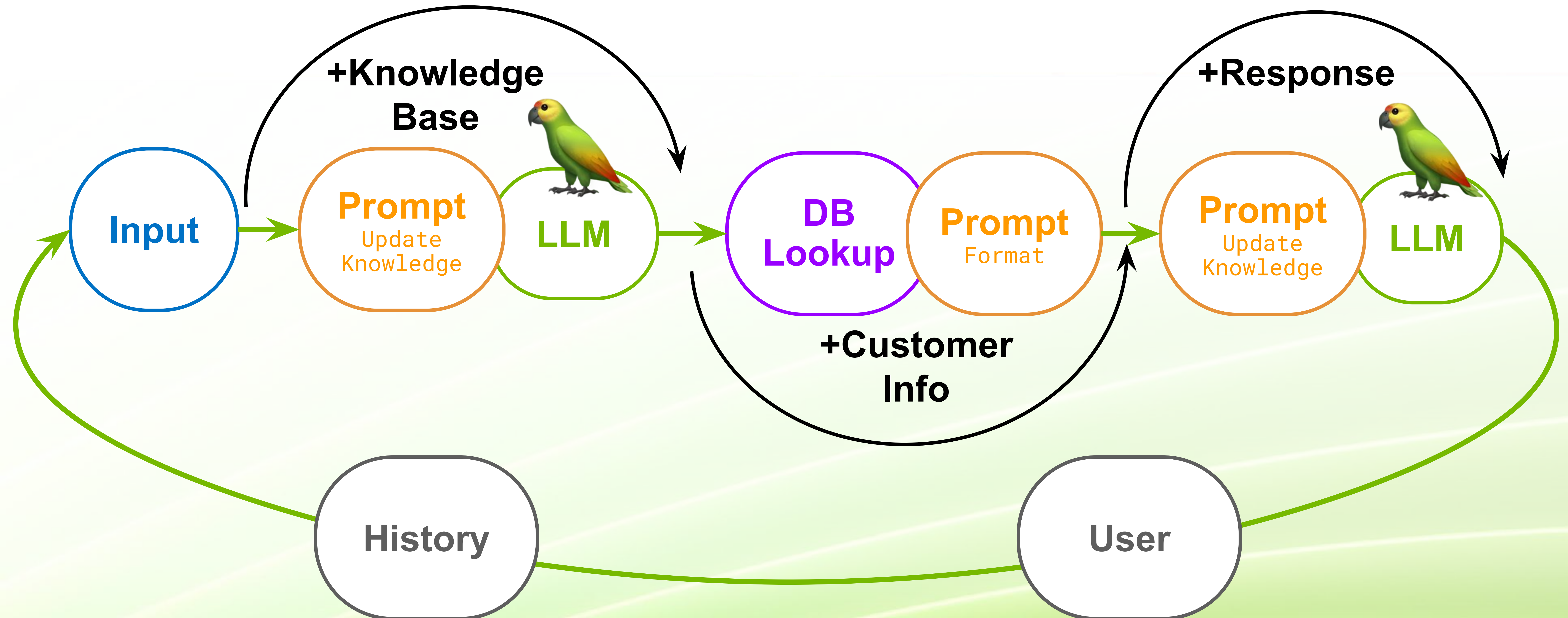
Airline Chatbot



Airline Chatbot



Airline Chatbot



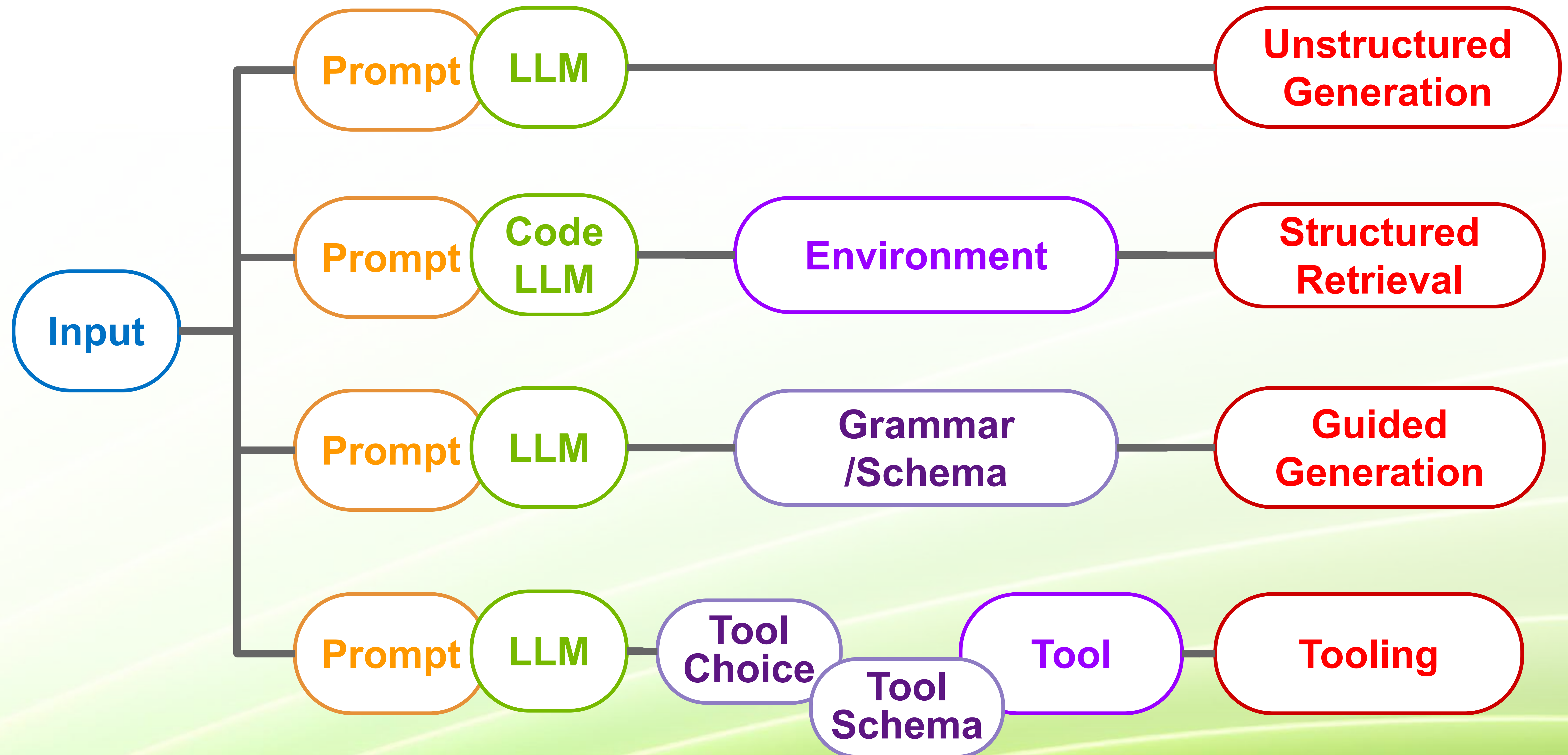


Building RAG Agents with LLMs

Part 5: Working with Documents

Modern Chain Paradigms

Towards Powerful Running State



Document Reasoning

NVIDIA DEVELOPER

Home

Blog

Forums

Docs

Downloads

Training

Technical Blog

Search blog

Filter

Learn the latest advancements in technology and get hands-on training at GTC 2024. March 18-21.

Generative AI / LLMs

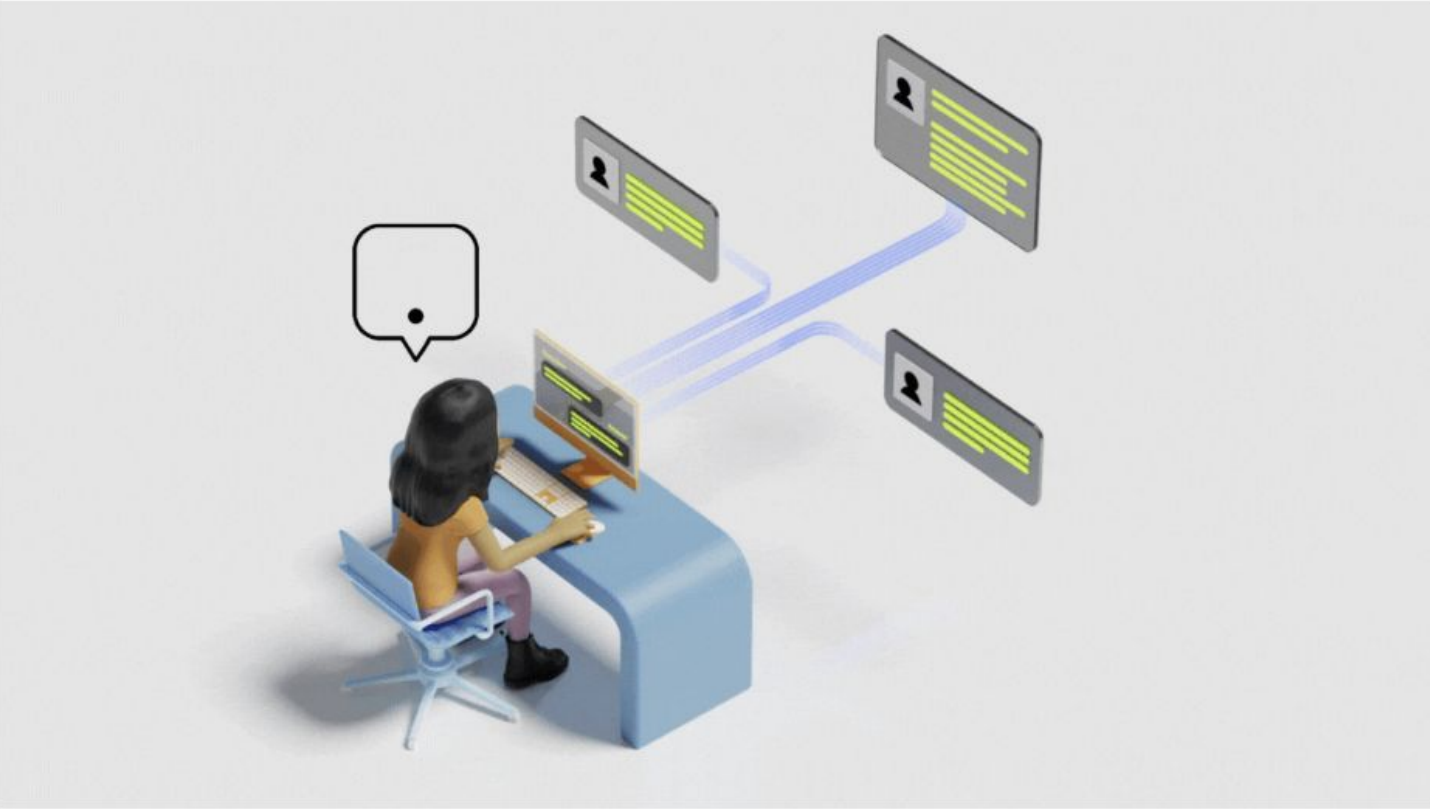
RAG 101: Retrieval-Augmented Generation Questions Answered

Dec 18, 2023

+5 Like

Discuss (1)

By [Hayden Wolff](#), [Daniel Rohrer](#) and [Jiwei Liu](#)



Data scientists, AI engineers, MLOps engineers, and IT infrastructure professionals must consider a variety of factors when designing and deploying a RAG pipeline: from core components like LLM to evaluation approaches.

The key point is that RAG is a system, not just a model or set of models. This system consists of several stages, which were discussed at a high level in [RAG 101: Demystifying Retrieval-Augmented Generation Pipelines](#). All these stages provide opportunities to make design decisions according to your needs.

Here's a list of top questions and answers.

When should you fine-tune the LLM vs. using RAG?

In the world of LLMs, choosing between fine-tuning, Parameter-Efficient Fine-Tuning (PEFT), prompt engineering, and retrieval-augmented generation (RAG) depends on the specific needs and constraints of your application.

- *Fine-tuning* customizes a pretrained LLM for a specific domain by updating most or all of its parameters with a domain-specific dataset. This approach is resource-intensive but yields high accuracy for specialized use cases.
- *PEFT* modifies a pretrained LLM with fewer parameter updates, focusing on a subset of the model. It strikes a balance between accuracy and resource usage, offering improvements over prompt engineering with manageable data and computational demands.
- *Prompt engineering* manipulates the input to an LLM to steer its output, without altering the model's

Generative AI / LLMs

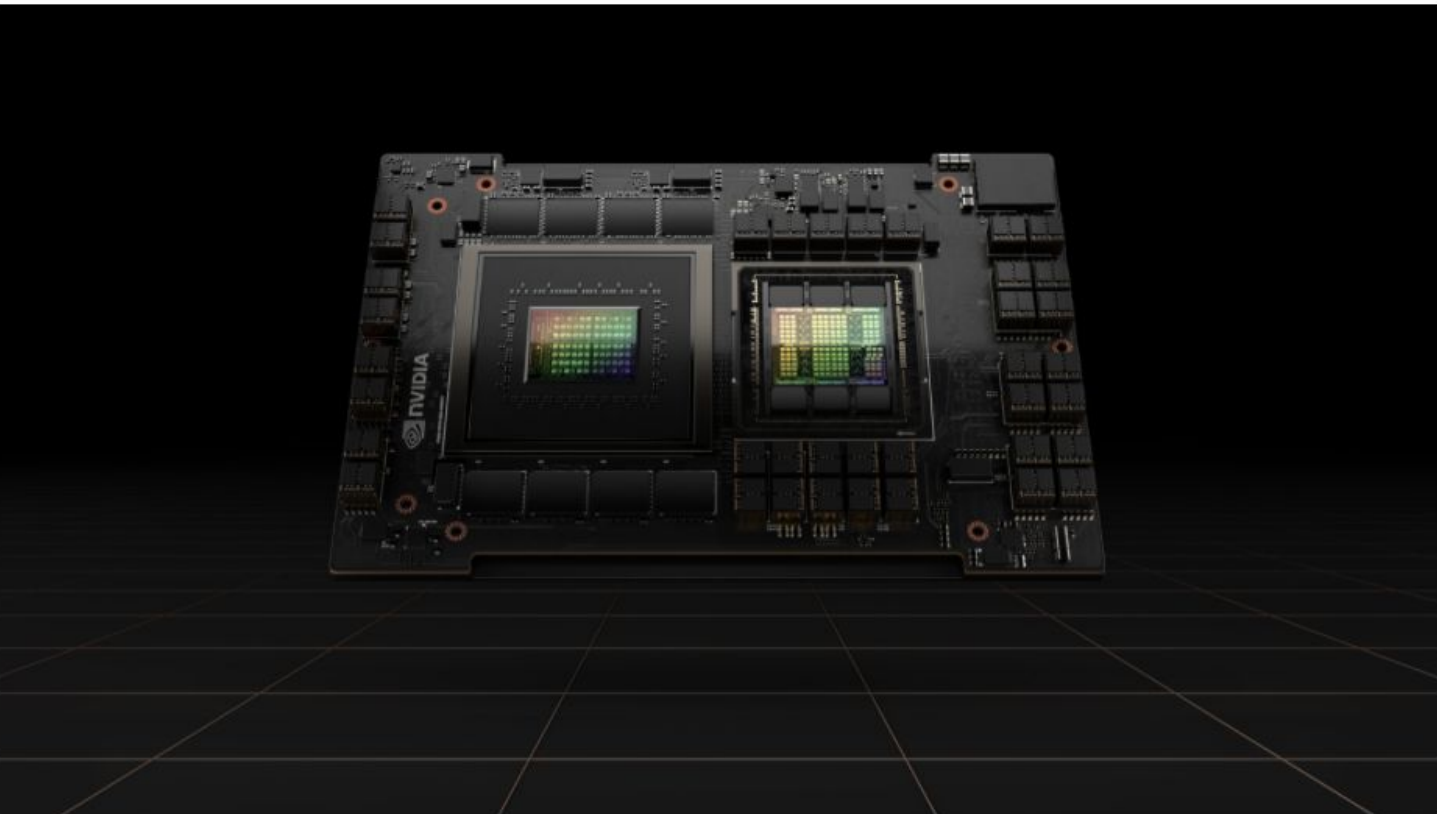
Deploying Retrieval-Augmented Generation Applications on NVIDIA GH200 Delivers Accelerated Performance

Dec 18, 2023

+14 Like

Discuss (2)

By [Rohil Bhargava](#), [Jiwei Liu](#) and [Ivan Goldwasser](#)



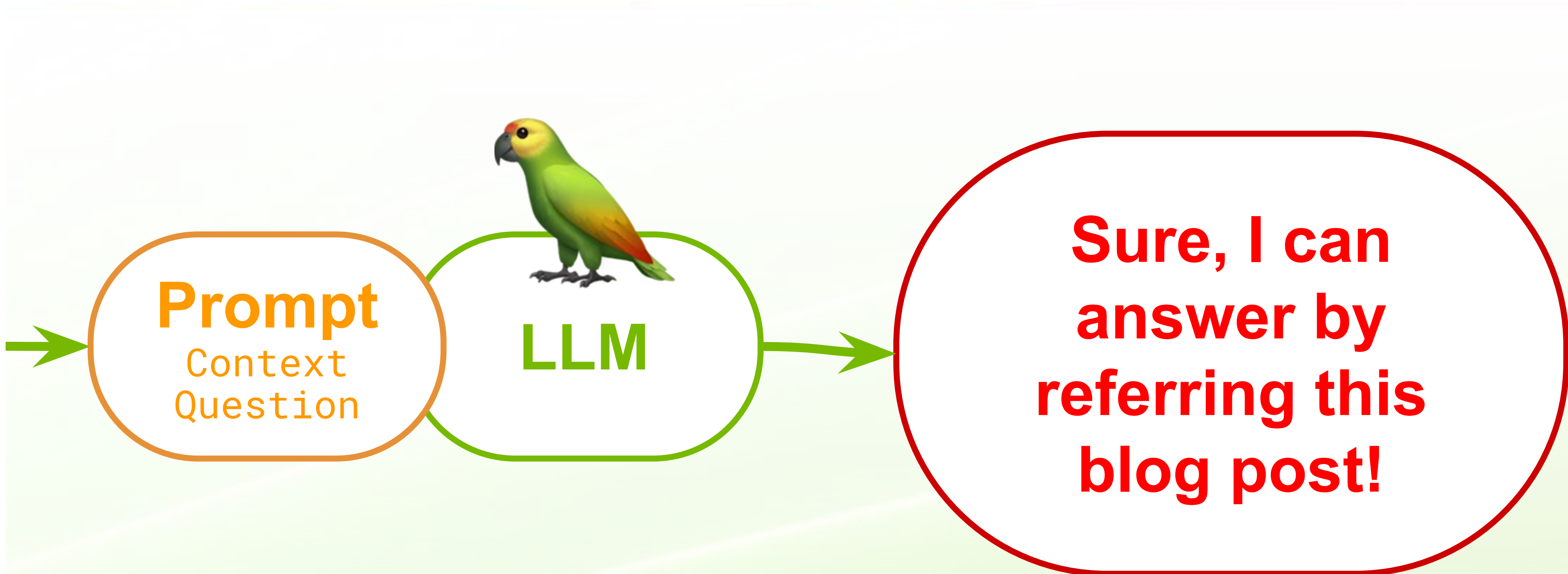
Large language model (LLM) applications are essential in enhancing productivity across industries through natural language. However, their effectiveness is often limited by the extent of their training data, resulting in poor performance when dealing with real-time events and new knowledge the LLM isn't trained on.

Retrieval-augmented generation (RAG) solves these problems. By augmenting models with an external knowledge base, RAG can ground LLMs with relevant data. This enhances the quality of their response, improving accuracy, and reducing hallucinations. RAG not only extends the utility of LLMs but also provides a cheaper alternative to time-consuming re-training runs.

Deploying and scaling RAG applications for tens of thousands to millions of users comes with its own set of challenges, specifically around GPU memory management. Developers need access to state-of-the-art infrastructure with robust memory capabilities that runs real-time RAG applications performantly and within stringent service level agreements (SLAs).

This is where the NVIDIA end-to-end full-stack accelerated computing solution shines. In this post, we discuss how the [NVIDIA GH200 Grace Hopper Superchip](#) can help solve these issues.

Comparing the GH200 to NVIDIA A100 Tensor Core GPUs, we observed up to a 2.7x increase in speed for embedding generation, 2.9x for index build, 3.3x for vector search time, and 5.7x for Llama-2-70B (FP8).

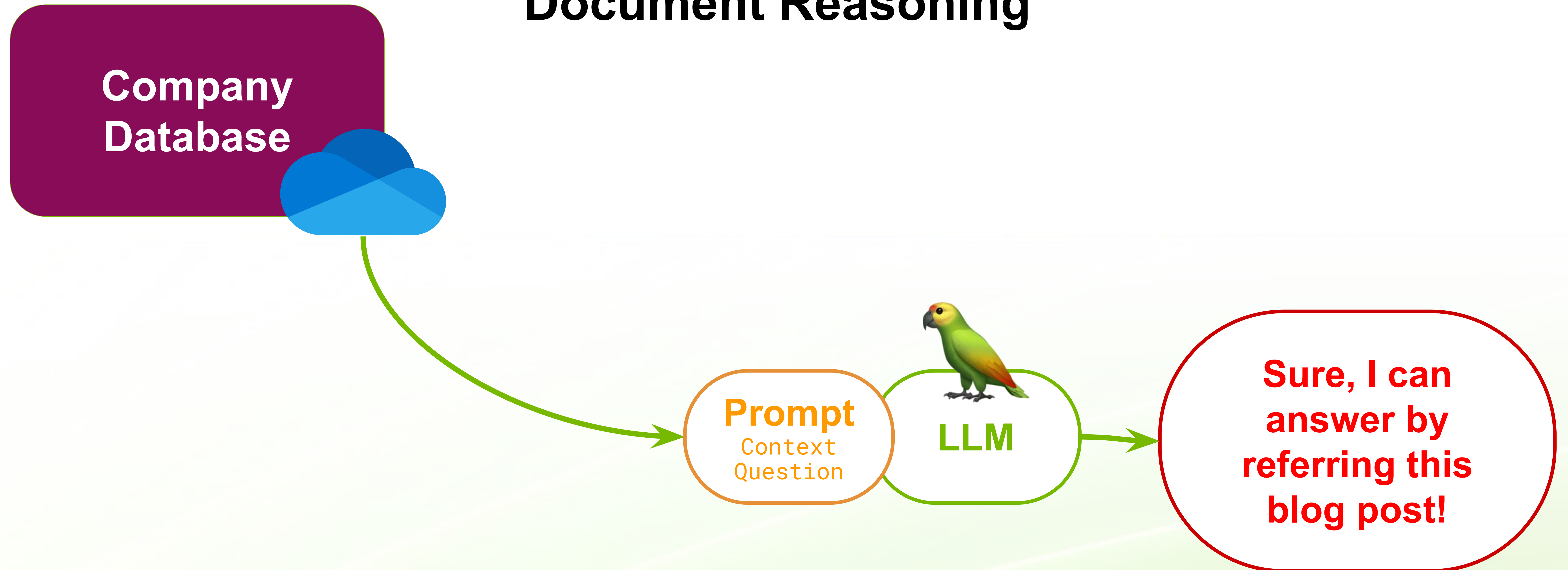


```
graph LR; A[Prompt<br/>Context<br/>Question] --> B((LLM)); B --> C([Sure, I can answer by referring this blog post!])
```

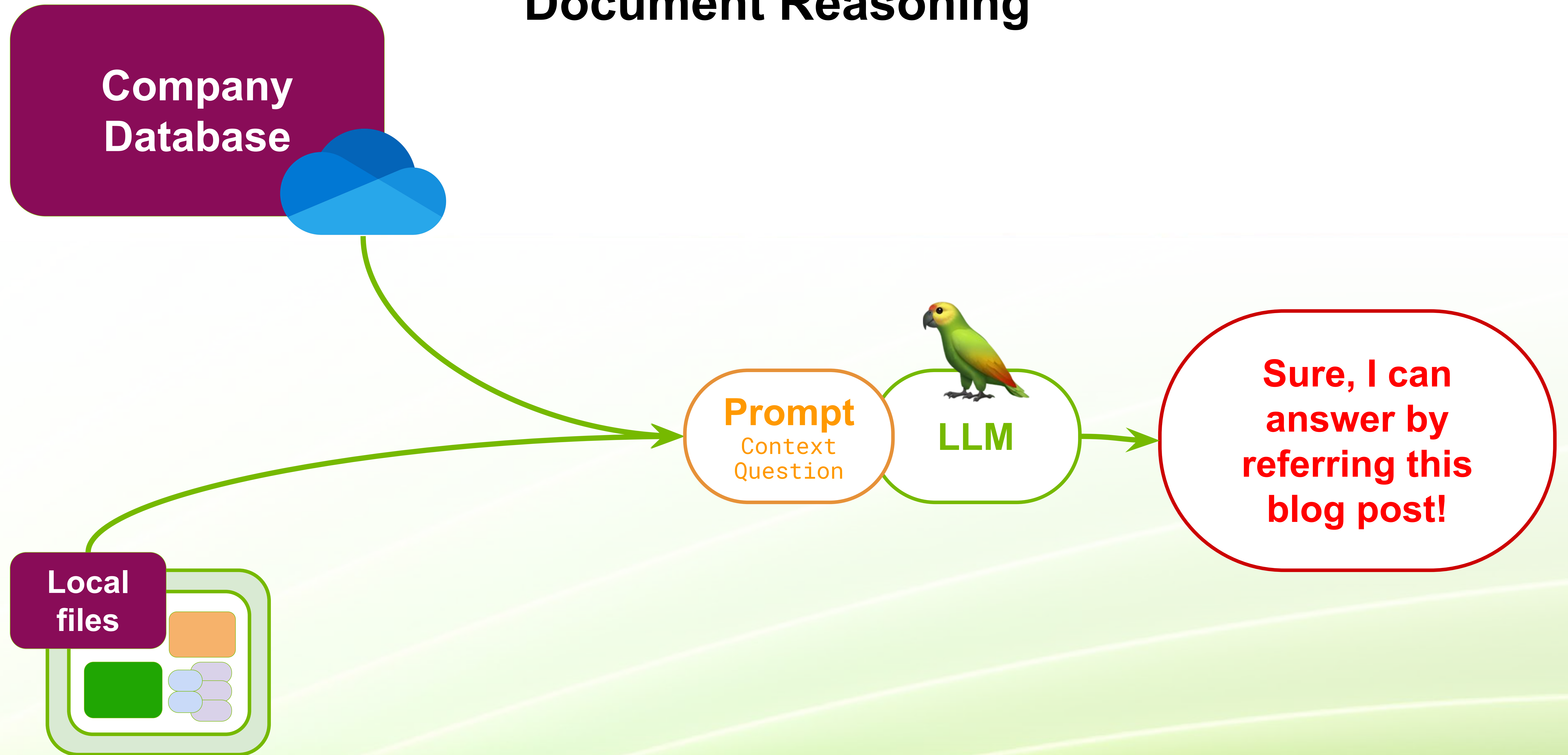
<https://developer.nvidia.com/blog/>

115

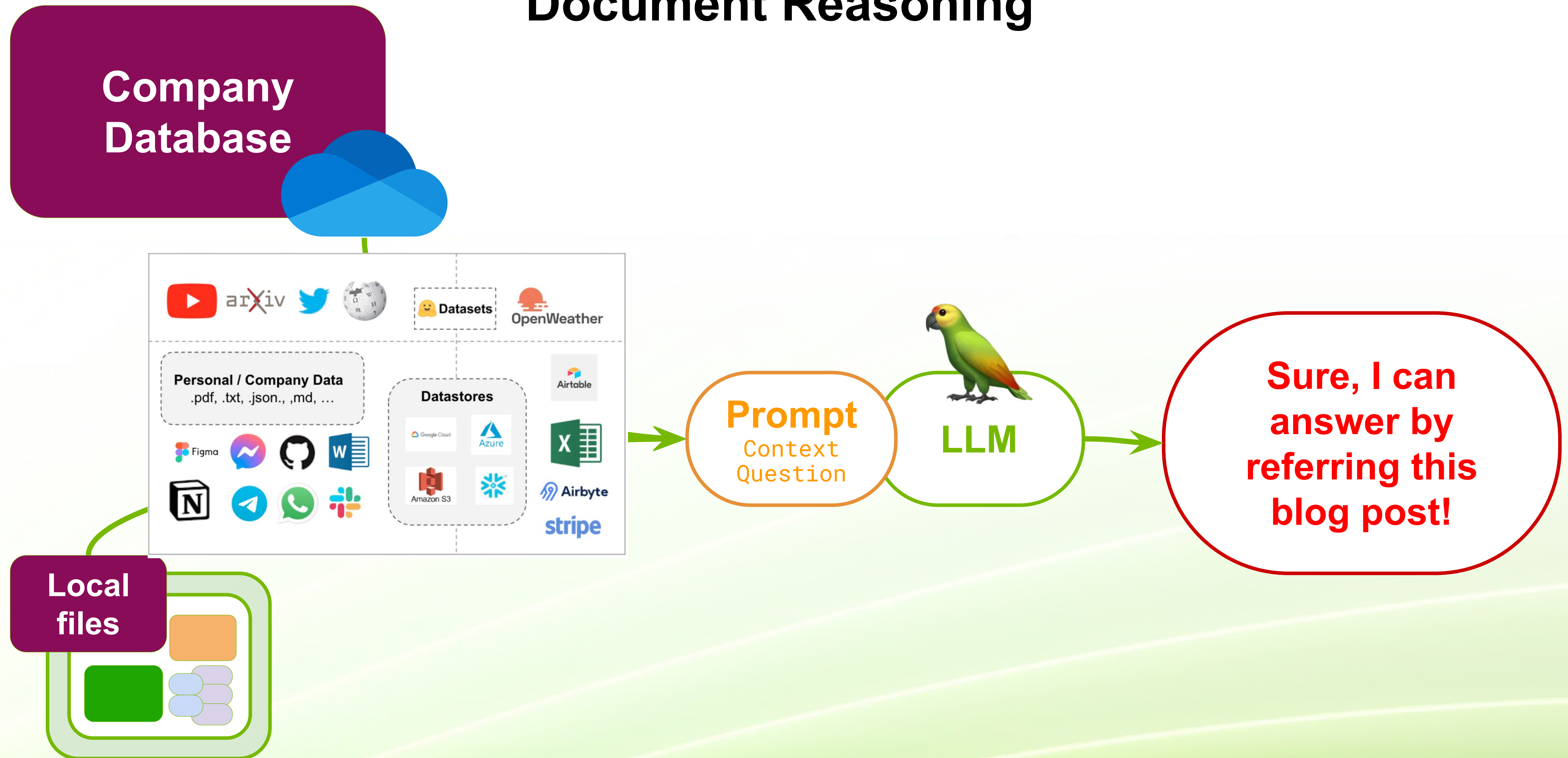
Document Reasoning



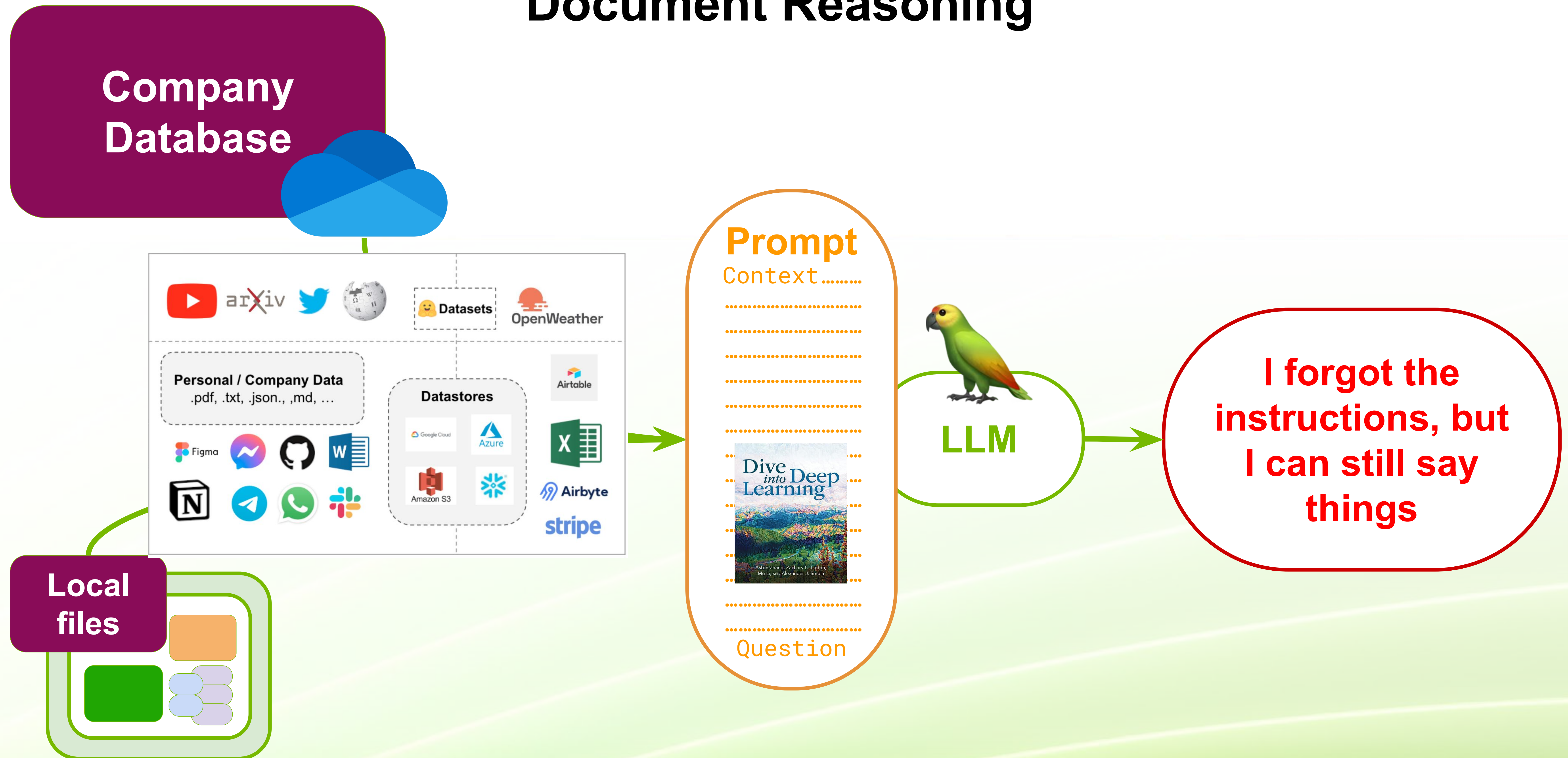
Document Reasoning



Document Reasoning



Document Reasoning



Chunking

arXiv:2307.09288v2 [cs.CL] 19 Jul 2023

LLAMA 2: Open Foundation and Fine-Tuned Chat Models

Hugo Touvron* Louis Martin† Kevin Stone†
Peter Albert Amjad Almahairi Yasmine Babaei Nikolay Bashlykov Soumya Batra
Prajwal Bhargava Shruti Bhosale Dan Bikel Lukas Blecher Cristian Canton Ferrer Moya Chen
Guillem Cucurull David Esiobu Jude Fernandes Jeremy Fu Wenyin Fu Brian Fuller
Cynthia Gao Vedanuj Goswami Naman Goyal Anthony Hartshorn Saghar Hosseini Rui Hou
Hakan Inan Marcin Kardas Viktor Kerkez Madian Khabsa Isabel Kloumann Artem Korenev
Punit Singh Koura Marie-Anne Lachaux Thibaut Lavril Jenya Lee Diana Liskovich
Yinghai Lu Yuning Mao Xavier Martinet Todor Mihaylov Pushkar Mishra
Igor Molybog Yixin Nie Andrew Poulton Jeremy Reizenstein Rashi Rungta Kalyan Saladi
Alan Schelten Ruan Silva Eric Michael Smith Ranjan Subramanian Xiaoqing Ellen Tan Binh Tang
Ross Taylor Adina Williams Jian Xiang Kuan Puxin Xu Zheng Yan Iliyan Zarov Yuchen Zhang
Angela Fan Melanie Kambadur Sharan Narang Aurelien Rodriguez Robert Stojnic
Sergey Edunov Thomas Scialom*

GenAI, Meta

Abstract

In this work, we develop and release Llama 2, a collection of pretrained and fine-tuned large language models (LLMs) ranging in scale from 7 billion to 70 billion parameters. Our fine-tuned LLMs, called LLAMA 2-CHAT, are optimized for dialogue use cases. Our models outperform open-source chat models on most benchmarks we tested, and based on our human evaluations for helpfulness and safety, may be a suitable substitute for closed-source models. We provide a detailed description of our approach to fine-tuning and safety improvements of LLAMA 2-CHAT in order to enable the community to build on our work and contribute to the responsible development of LLMs.

*Equal contribution, corresponding authors: {tscialom, htouvron}@meta.com
†Second author

Contributions for all the authors can be found in Section A.1.

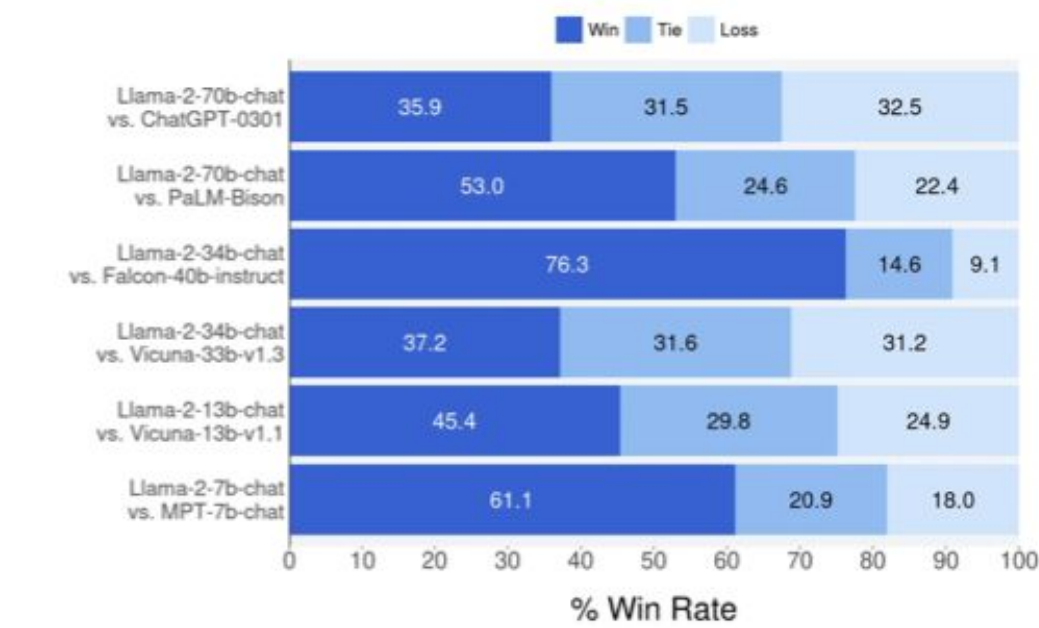


Figure 1: Helpfulness human evaluation results for LLAMA 2-CHAT compared to other open-source and closed-source models. Human raters compared model generations on ~4k prompts consisting of both single and multi-turn prompts. The 95% confidence intervals for this evaluation are between 1% and 2%. More details in Section 3.4.2. While reviewing these results, it is important to note that human evaluations can be noisy due to limitations of the prompt set, subjectivity of the review guidelines, subjectivity of individual raters, and the inherent difficulty of comparing generations.

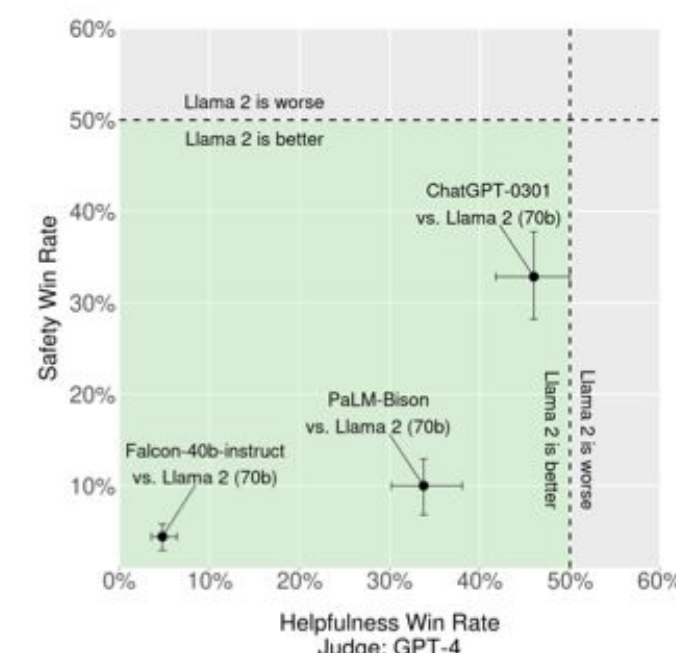


Figure 2: Win-rate % for helpfulness and safety between commercial-licensed base-lines and LLAMA 2-CHAT, according to GPT-4. To complement the human evaluation, we used a more capable model, not subject to our own guidance. Green area indicates our model is better according to GPT-4. To remove ties, we used $win/(win + loss)$. The orders in which the model responses are presented to GPT-4 are randomly swapped to alleviate bias.

1 Introduction

Large Language Models (LLMs) have shown great promise as highly capable AI assistants that excel in complex reasoning tasks requiring expert knowledge across a wide range of fields, including in specialized domains such as programming and creative writing. They enable interaction with humans through intuitive chat interfaces, which has led to rapid and widespread adoption among the general public.

The capabilities of LLMs are remarkable considering the seemingly straightforward nature of the training methodology. Auto-regressive transformers are pretrained on an extensive corpus of self-supervised data, followed by alignment with human preferences via techniques such as Reinforcement Learning with Human Feedback (RLHF). Although the training methodology is simple, high computational requirements have limited the development of LLMs to a few players. There have been public releases of pretrained LLMs (such as BLOOM (Scao et al., 2022), LLaMA-1 (Touvron et al., 2023), and Falcon (Penedo et al., 2023)) that match the performance of closed pretrained competitors like GPT-3 (Brown et al., 2020) and GPT-4 (Hoffmann et al., 2022), but none of these models are suitable substitutes for closed “product” LLMs, such as ChatGPT, BARD, and Claude. These closed product LLMs are heavily fine-tuned to align with human preferences, which greatly enhances their usability and safety. This step can require significant costs in compute and human annotation, and is often not transparent or easily reproducible, limiting progress within the community to advance AI alignment research.

In this work, we develop and release Llama 2, a family of pretrained and fine-tuned LLMs, LLAMA 2 and LLAMA 2-CHAT, at scales up to 70B parameters. On the series of helpfulness and safety benchmarks we tested, LLAMA 2-CHAT models generally perform better than existing open-source models. They also appear to be on par with some of the closed-source models, at least on the human evaluations we performed (see Figures 1 and 3). We have taken measures to increase the safety of these models, using safety-specific data annotation and tuning, as well as conducting red-teaming and employing iterative evaluations. Additionally, this paper contributes a thorough description of our fine-tuning methodology and approach to improving LLM safety. We hope that this openness will enable the community to reproduce fine-tuned LLMs and continue to improve the safety of those models, paving the way for more responsible development of LLMs. We also share novel observations we made during the development of LLAMA 2 and LLAMA 2-CHAT, such as the emergence of tool usage and temporal organization of knowledge.

Document Stuffing

LLAMA 2: Open Foundation and Fine-Tuned Chat Models

Hugo Touvron* Louis Martin† Kevin Stone†
Peter Albert Amjad Almahairi Yasmine Babaei Nikolay Bashlykov Soumya Batra
Prajwal Bhargava Shruti Bhosale Dan Bikel Lukas Blecher Cristian Canton Ferrer Moya Chen
Guillem Cucurull David Esiobu Jude Fernandes Jeremy Fu Wenyin Fu Brian Fuller
Cynthia Gao Vedanuj Goswami Naman Goyal Anthony Hartshorn Saghar Hosseini Rui Hou
Hakan Inan Marcin Kardas Viktor Kerkez Madian Khabsa Isabel Kloumann Artem Korenev
Punit Singh Koura Marie-Anne Lachaux Thibaut Lavril Jenya Lee Diana Liskovich
Yinghai Lu Yuning Mao Xavier Martinet Todor Mihaylov Pushkar Mishra
Igor Molybog Yixin Nie Andrew Poulton Jeremy Reizenstein Rashi Rungta Kalyan Saladi
Alan Schelten Ruan Silva Eric Michael Smith Ranjan Subramanian Xiaoqing Ellen Tan Binh Tang
Ross Taylor Adina Williams Jian Xiang Kuan Puxin Xu Zheng Yan Iliyan Zarov Yuchen Zhang
Angela Fan Melanie Kambadur Sharan Narang Aurore Rodriguez Robert Stojanovic

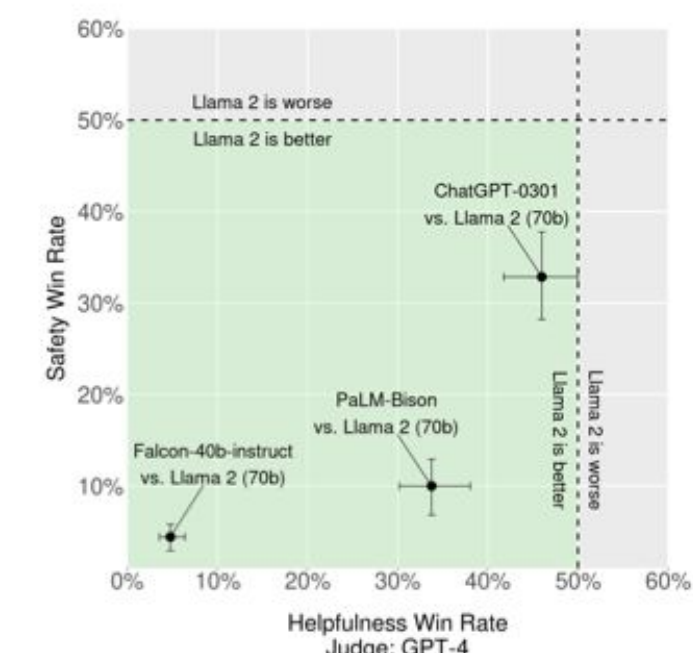


Figure 2: Win-rate % for helpfulness and safety between commercial-licensed base models and Llama 2. According to GPT-4 evaluation, we are not subject to indicate our model is better. To remove the orders in the chart presented to alleviate bias.

1 Introduction

Large Language Models (LLMs) have shown great promise as complex reasoning tasks requiring expert knowledge across a wide range of domains such as programming and creative writing. They enable chat interfaces, which has led to rapid and widespread adoption.

The capabilities of LLMs are remarkable considering the training methodology. Auto-regressive transformers are pretrained on a large corpus of text, followed by alignment with human preferences via techniques such as Reinforcement Learning with Human Feedback (RLHF). Although the training methodology is simple, high computational requirements have limited the development of LLMs to a few players. There have been public releases of pretrained LLMs (such as BLOOM (Scao et al., 2022), LLaMa-1 (Touvron et al., 2023)) that match the performance of closed pretrained competitors like

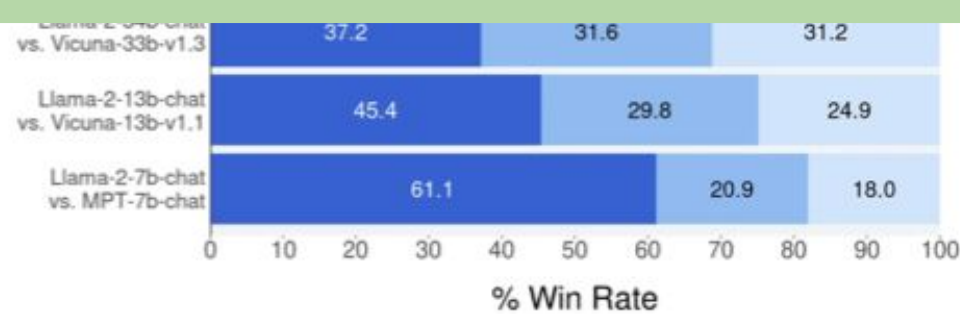


Figure 1: Helpfulness human evaluation results for LLaMA 2-CHAT compared to other open-source and closed-source models. Human raters compared model generations on ~4k prompts consisting of both single and multi-turn prompts. The 95% confidence intervals for this evaluation are between 1% and 2%. More details in Section 3.4.2. While reviewing these results, it is important to note that human evaluations can be noisy due to limitations of the prompt set, subjectivity of the review guidelines, subjectivity of individual raters, and the inherent difficulty of comparing generations.

Abstract

In this work, we develop and release Llama 2, a collection of pretrained and fine-tuned large language models (LLMs) ranging in scale from 7 billion to 70 billion parameters. Our fine-tuned LLMs, called LLaMA 2-CHAT, are optimized for dialogue use cases. Our models outperform open-source chat models on most benchmarks we tested, and based on our human evaluations for helpfulness and safety, may be a suitable substitute for closed-source models. We provide a detailed description of our approach to fine-tuning and safety improvements of LLaMA 2-CHAT in order to enable the community to build on our work and contribute to the responsible development of LLMs.

The capabilities of LLMs are remarkable considering the seemingly straightforward nature of the training methodology. Auto-regressive transformers are pretrained on an extensive corpus of self-supervised data, followed by alignment with human preferences via techniques such as Reinforcement Learning with Human Feedback (RLHF). Although the training methodology is simple, high computational requirements have limited the development of LLMs to a few players. There have been public releases of pretrained LLMs (such as BLOOM (Scao et al., 2022), LLaMa-1 (Touvron et al., 2023), and Falcon (Penedo et al., 2023)) that match the performance of closed pretrained competitors like GPT-3 (Brown et al., 2020) and Chinchilla (Hoffmann et al., 2022), but none of these models are suitable substitutes for closed “product” LLMs, such as ChatGPT, BARD, and Claude. These closed product LLMs are heavily fine-tuned to align with human preferences, which greatly enhances their usability and safety. This step can require significant costs to compute and human annotation, and is often not transparent or easily reproducible, limiting progress within the community to advance AI alignment research.

In this work, we develop and release Llama 2, a family of pretrained and fine-tuned LLMs, LLaMA 2 and LLaMA 2-CHAT, at scales up to 70B parameters. On the series of helpfulness and safety benchmarks we tested, LLaMA 2-CHAT models generally perform better than existing open-source models. They also appear to be on par with some of the closed-source models, at least on the human evaluations we performed (see Figures 1 and 3). We have taken measures to increase the safety of these models, using safety-specific data annotation and tuning, as well as conducting red-teaming and employing iterative evaluations. Additionally, this paper contributes a thorough description of our fine-tuning methodology and approach to improving

Prompt

(DOCS 1-4)

Use Them
Please

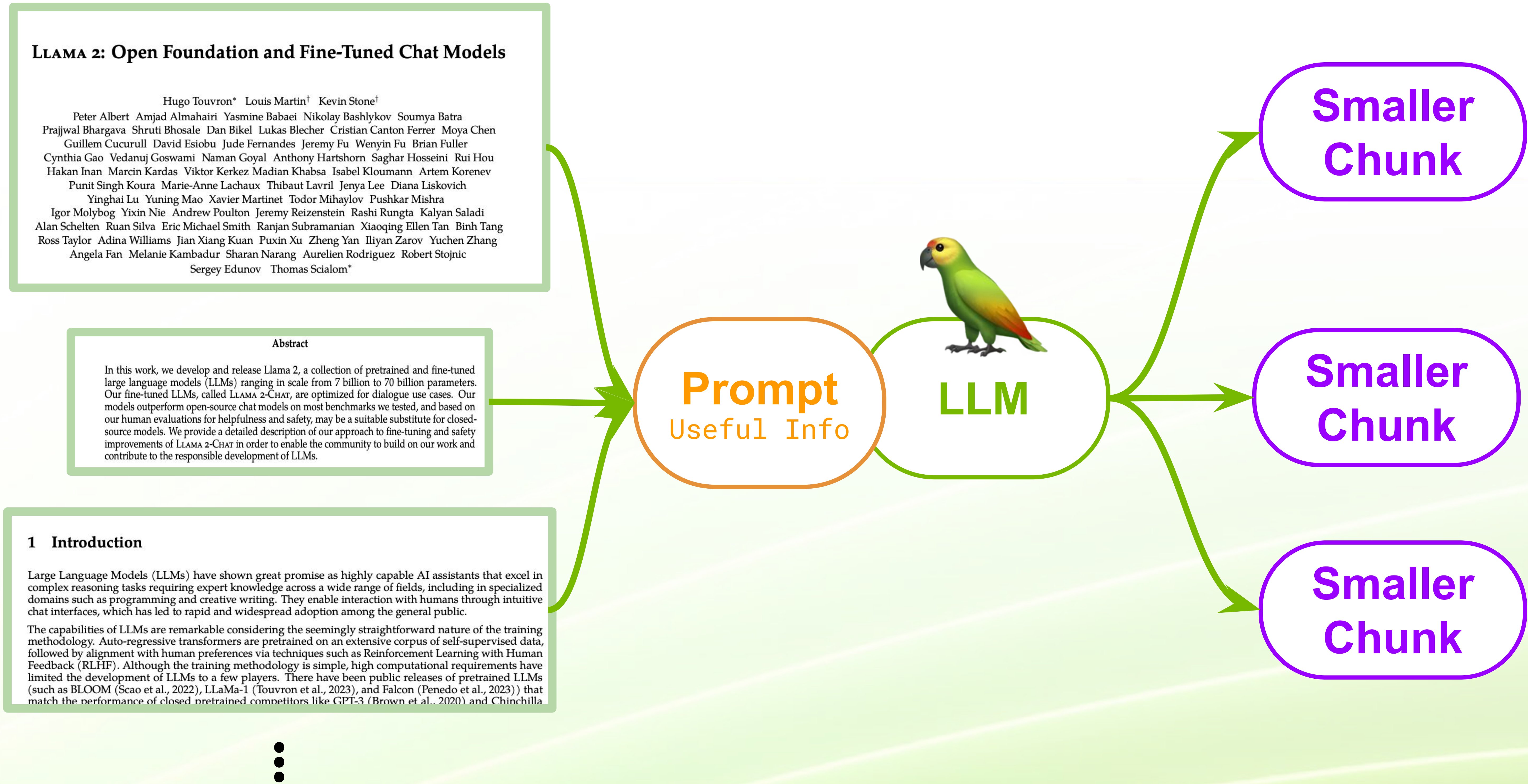
Question



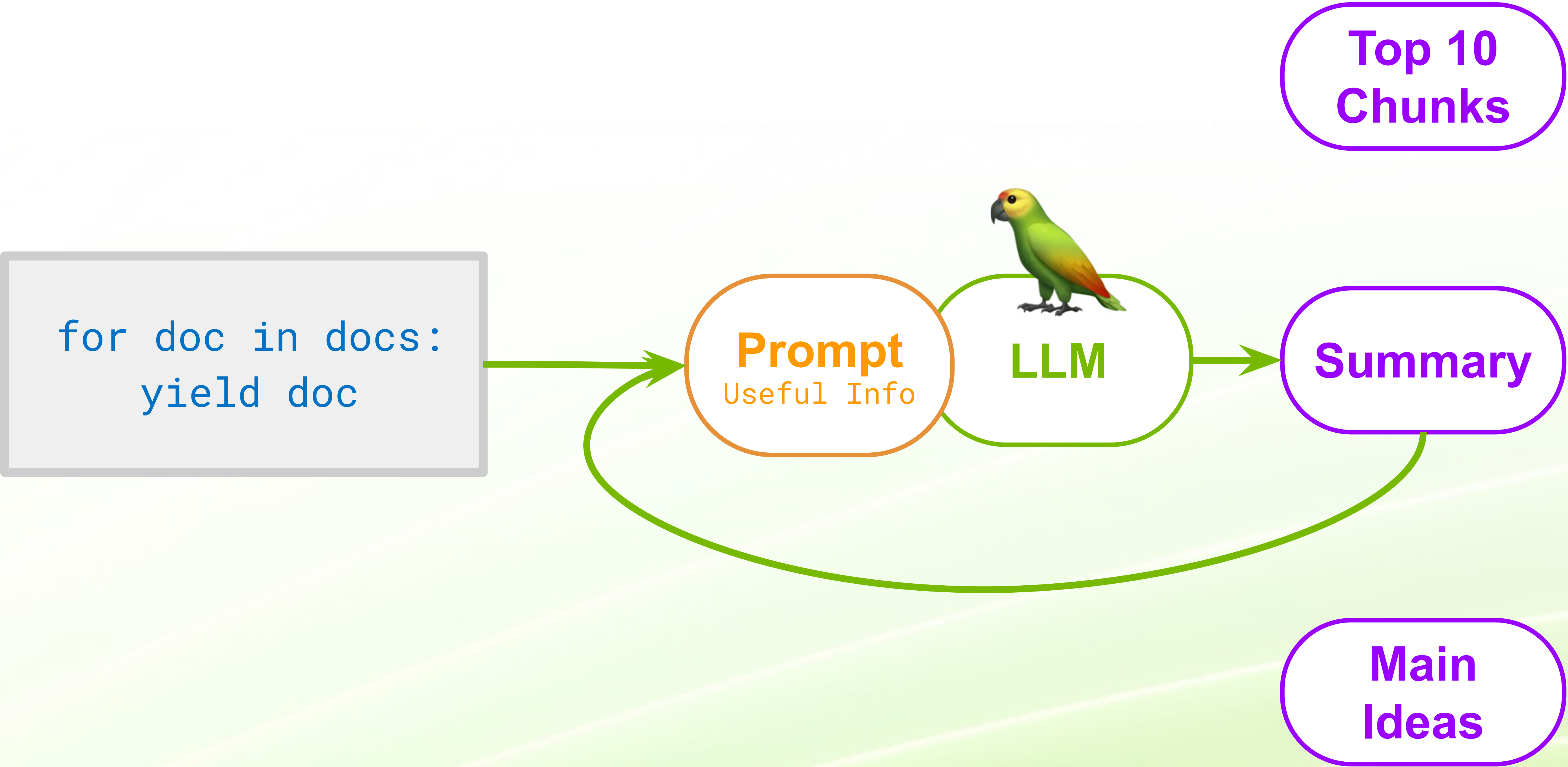
LLM

Oh yes, the intro
tells me
everything! You
see...

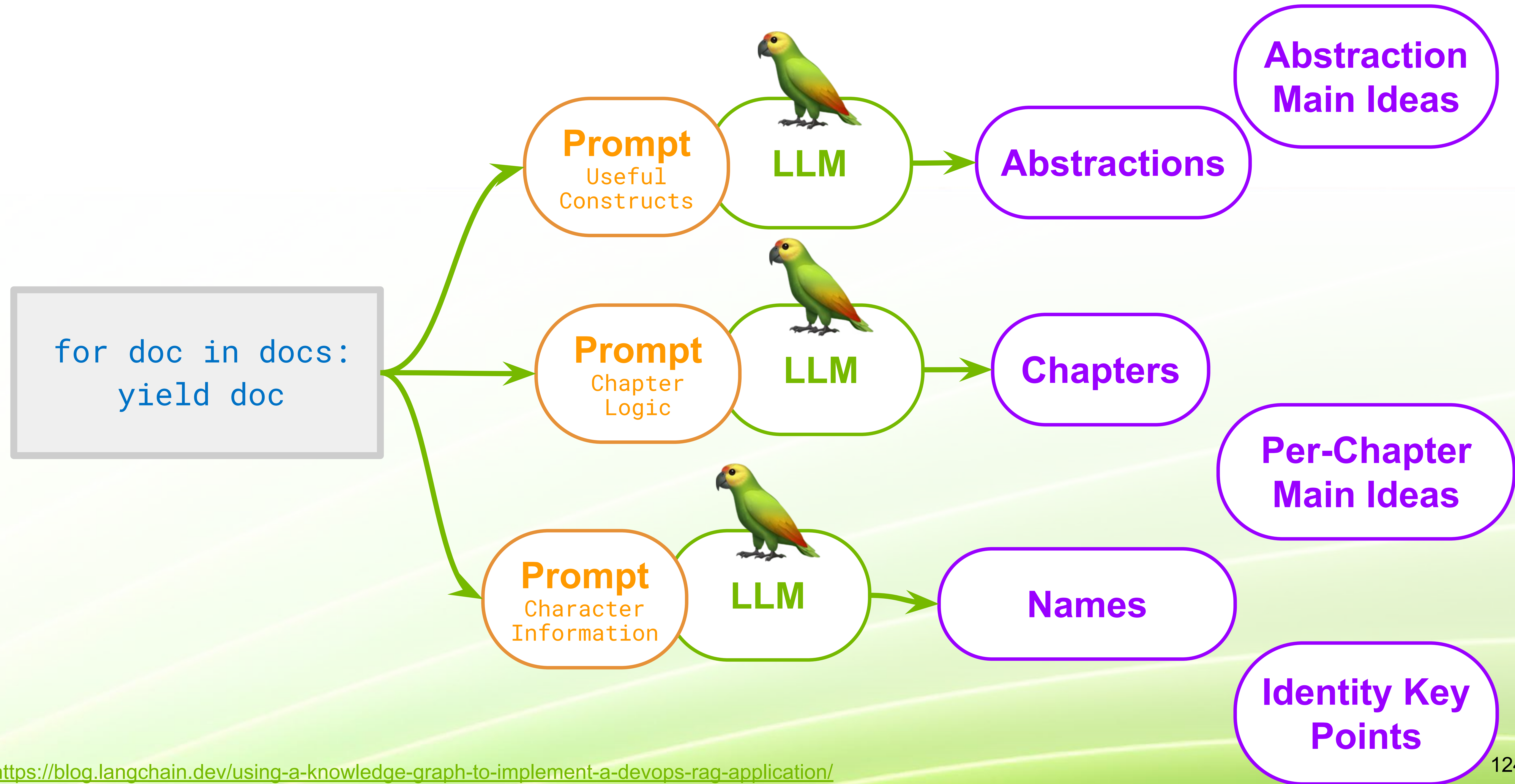
Map Reduce Chain



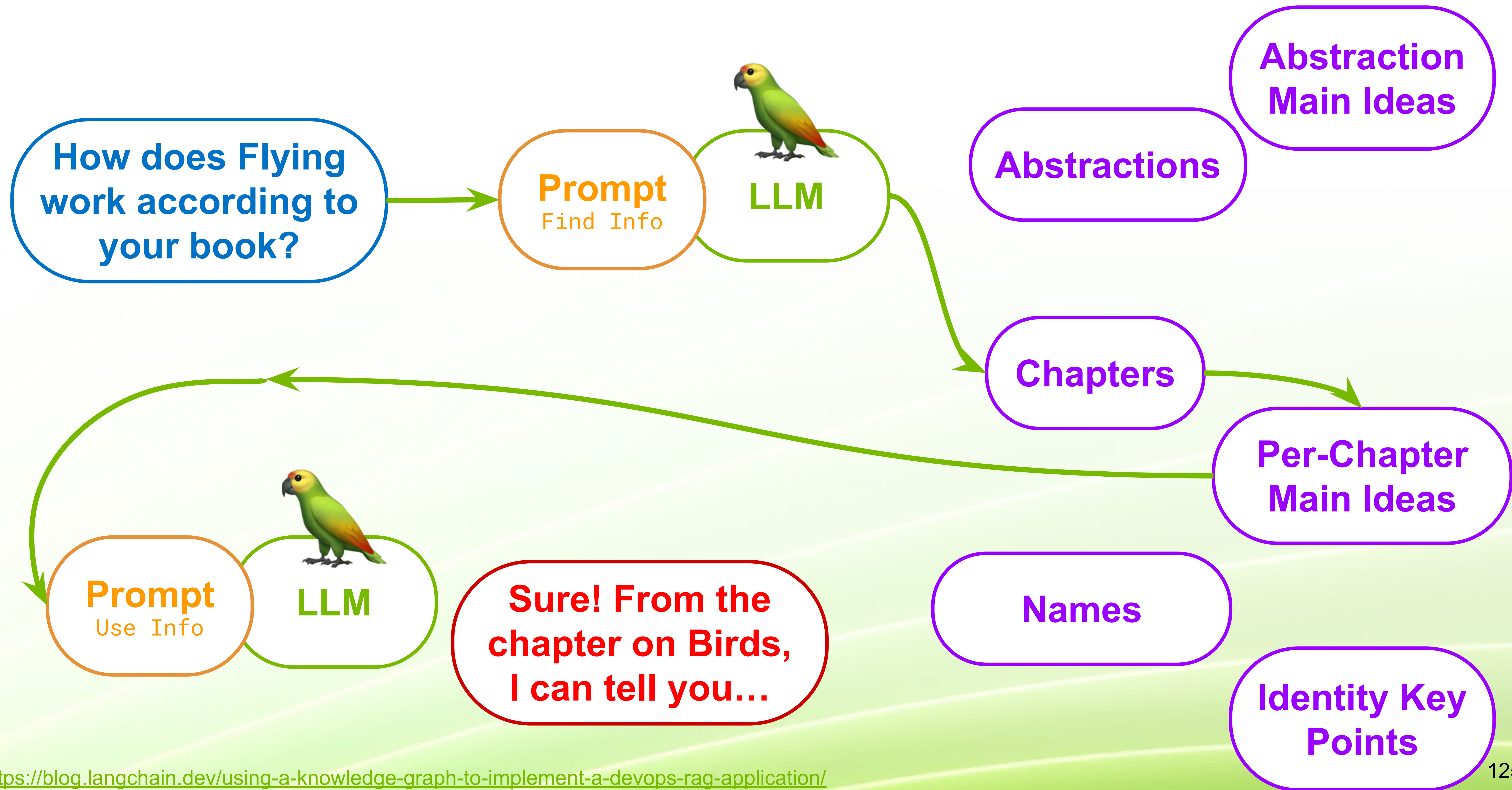
Refinement Chain



Knowledge Graph Construction

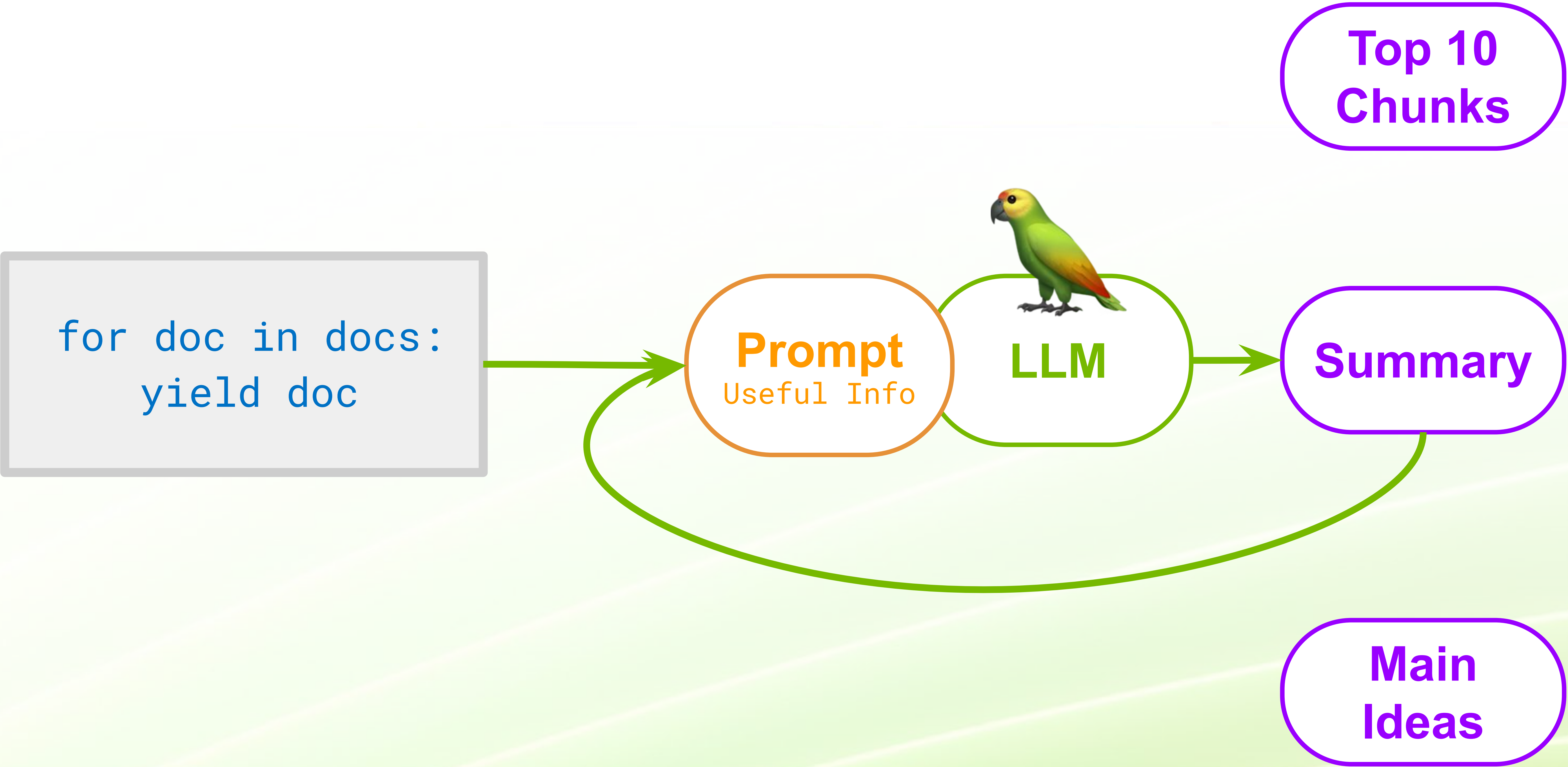


Knowledge Graph Traversal



Refinement Chain

Your Assignment



Refinement Chain

Your Assignment

RunnableLambda

```
for doc in docs:  
    yield doc
```

Prompt
Useful Info



LLM

Summary

Optional Tangent: LangGraph

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langgraph.graph import END, MessageGraph
```

```
model = ChatOpenAI(temperature=0)
```

```
graph = MessageGraph()
```

```
graph.add_node("oracle", model)
graph.add_edge("oracle", END)
```

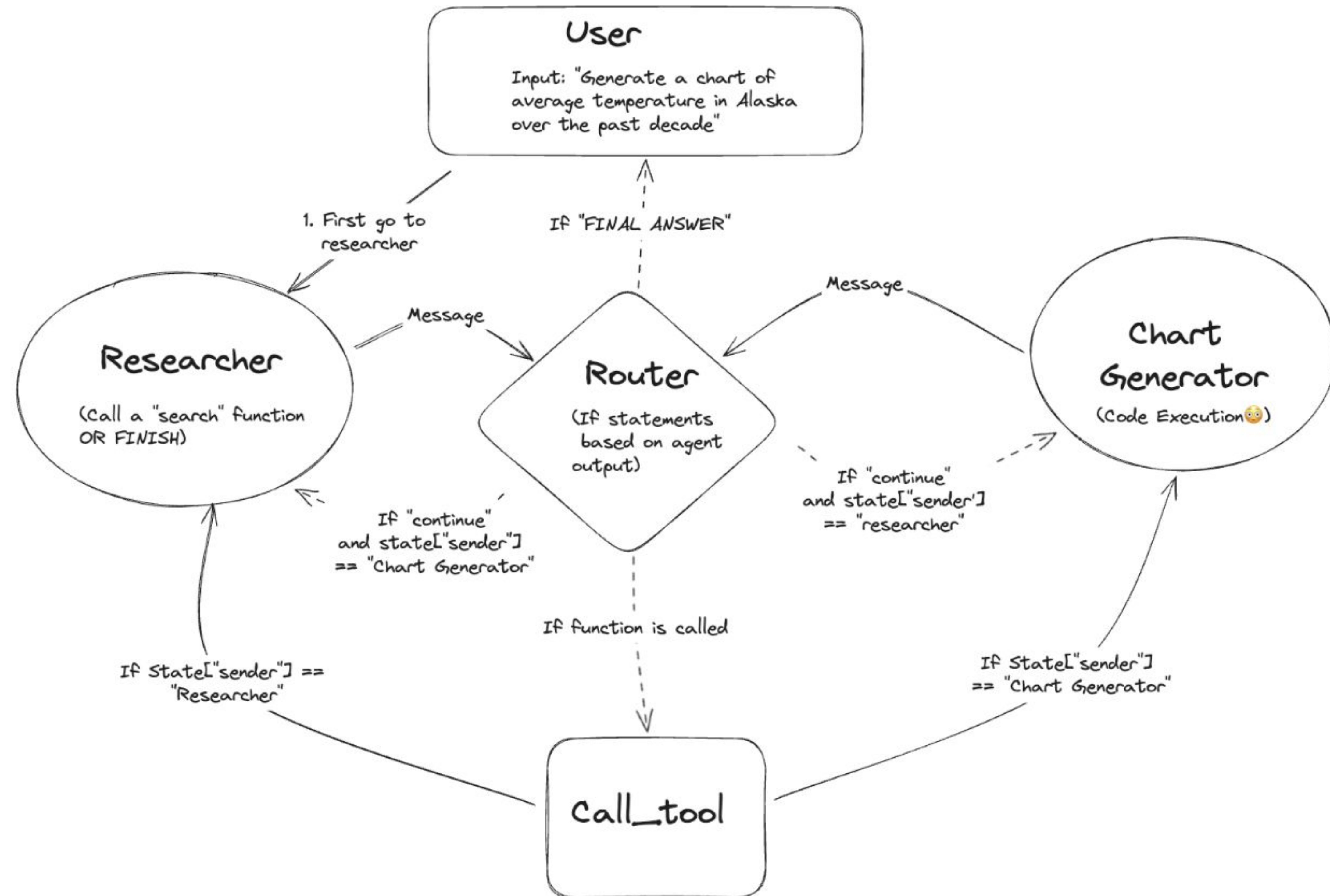
```
graph.set_entry_point("oracle")
```

```
runnable = graph.compile()
```

Let's run it!

```
runnable.invoke(HumanMessage("What is 1 + 1?"))
```

```
[HumanMessage(content='What is 1 + 1?'), AIMessage(content='1 + 1 equals 2.')]
```



Refinement Chain

Your Assignment

RunnableLambda

```
for doc in docs:  
    yield doc
```

Prompt
Useful Info



LLM

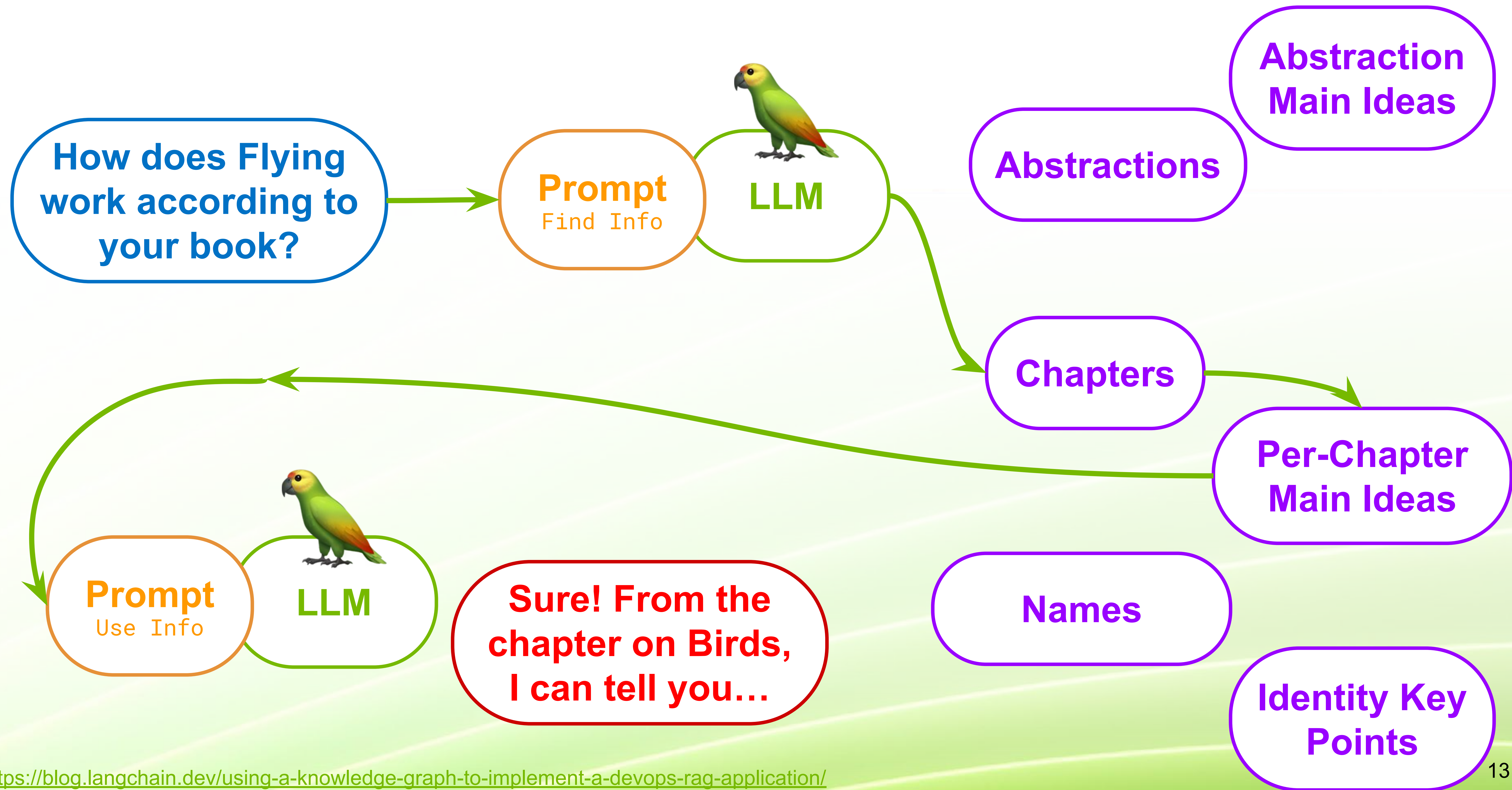
Summary



Building RAG Agents with LLMs

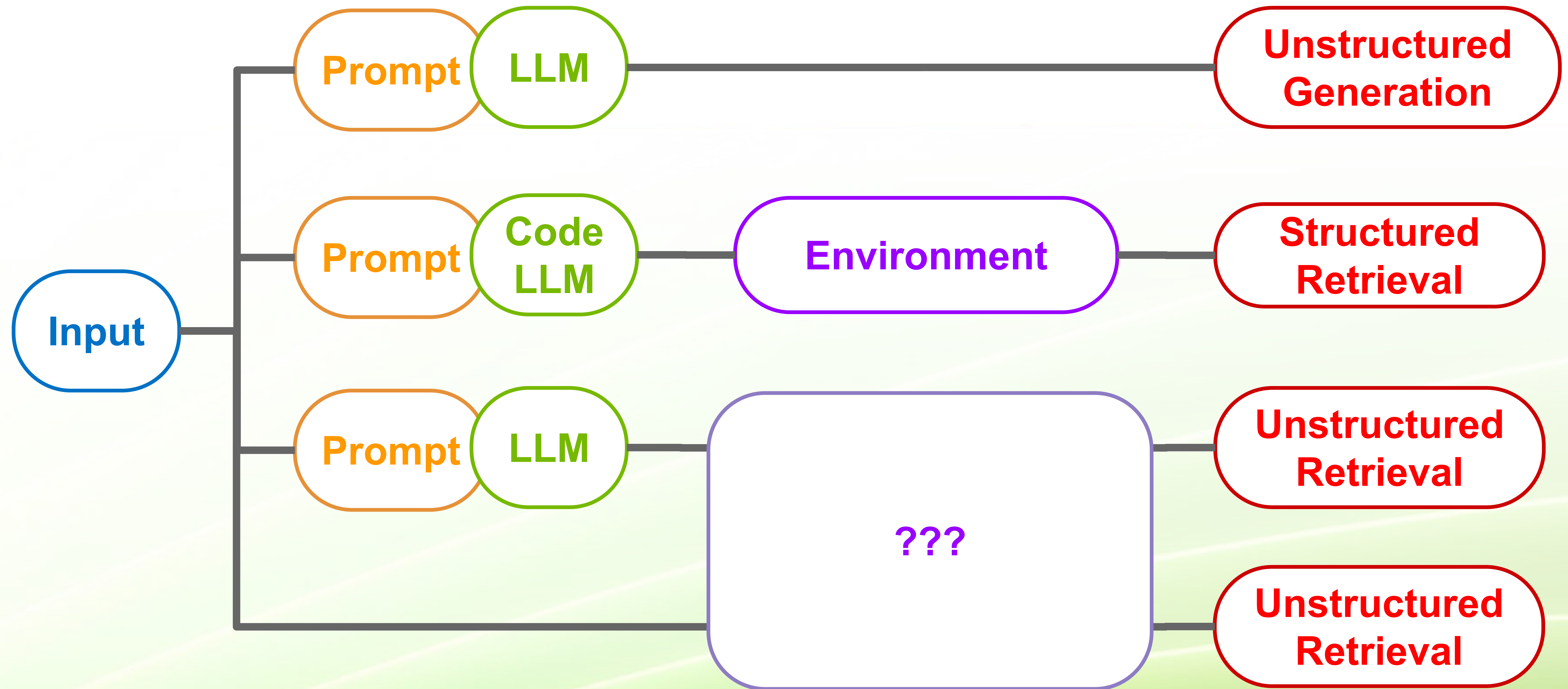
Part 6: Embedding Model for Retrieval

Knowledge Graph Traversal



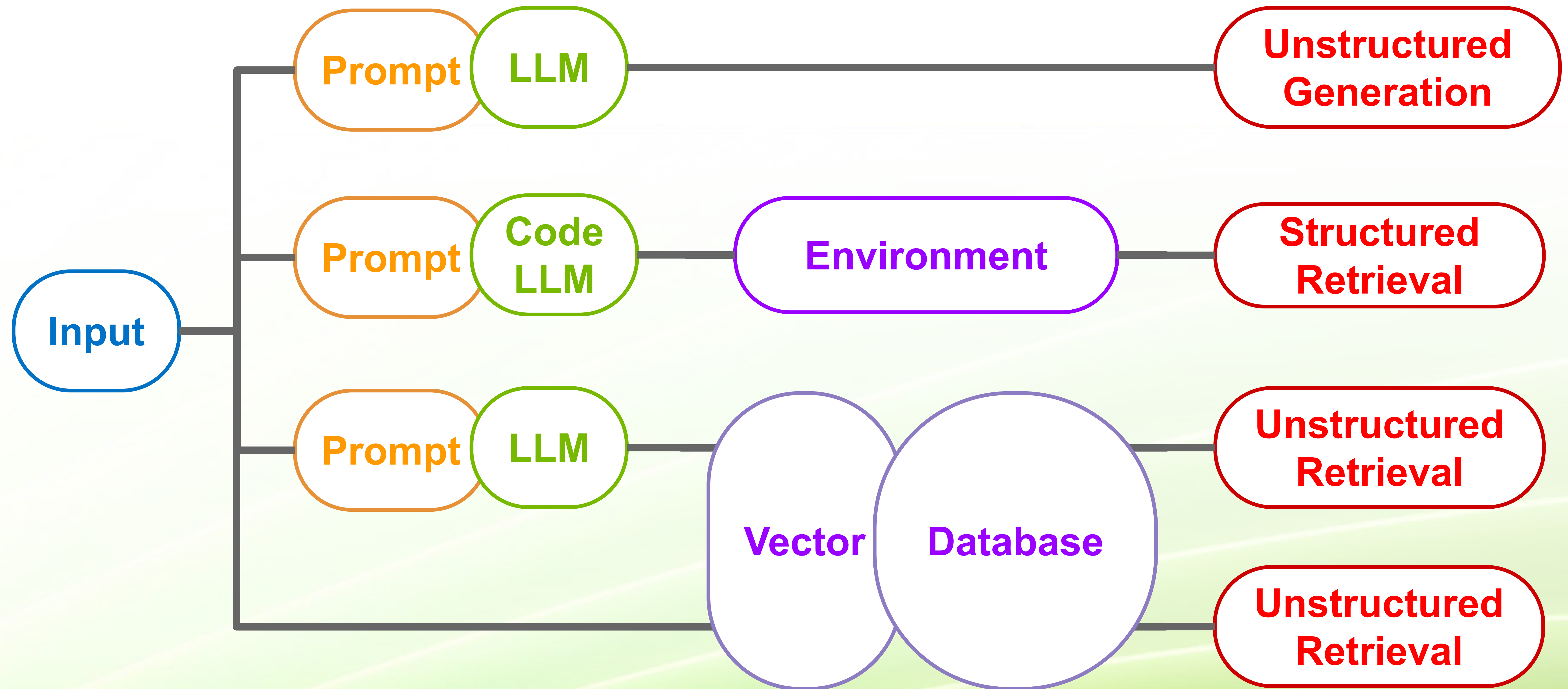
Modern Chain Paradigms

Towards Powerful Running State



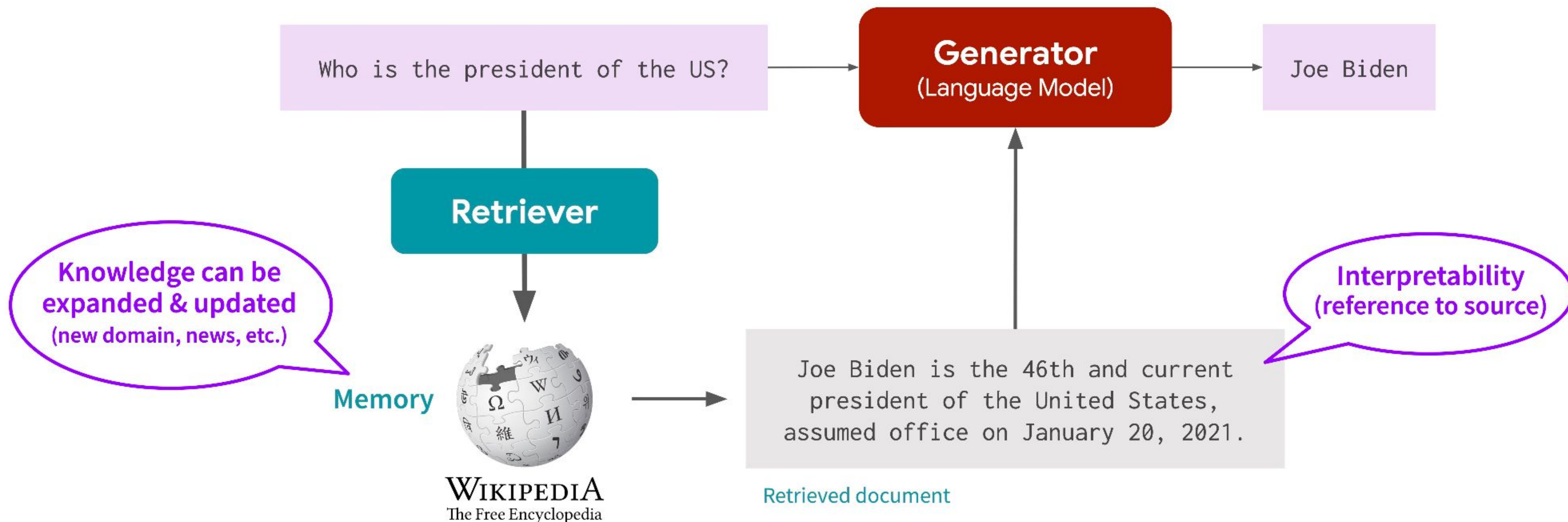
Modern Chain Paradigms

Towards Powerful Running State



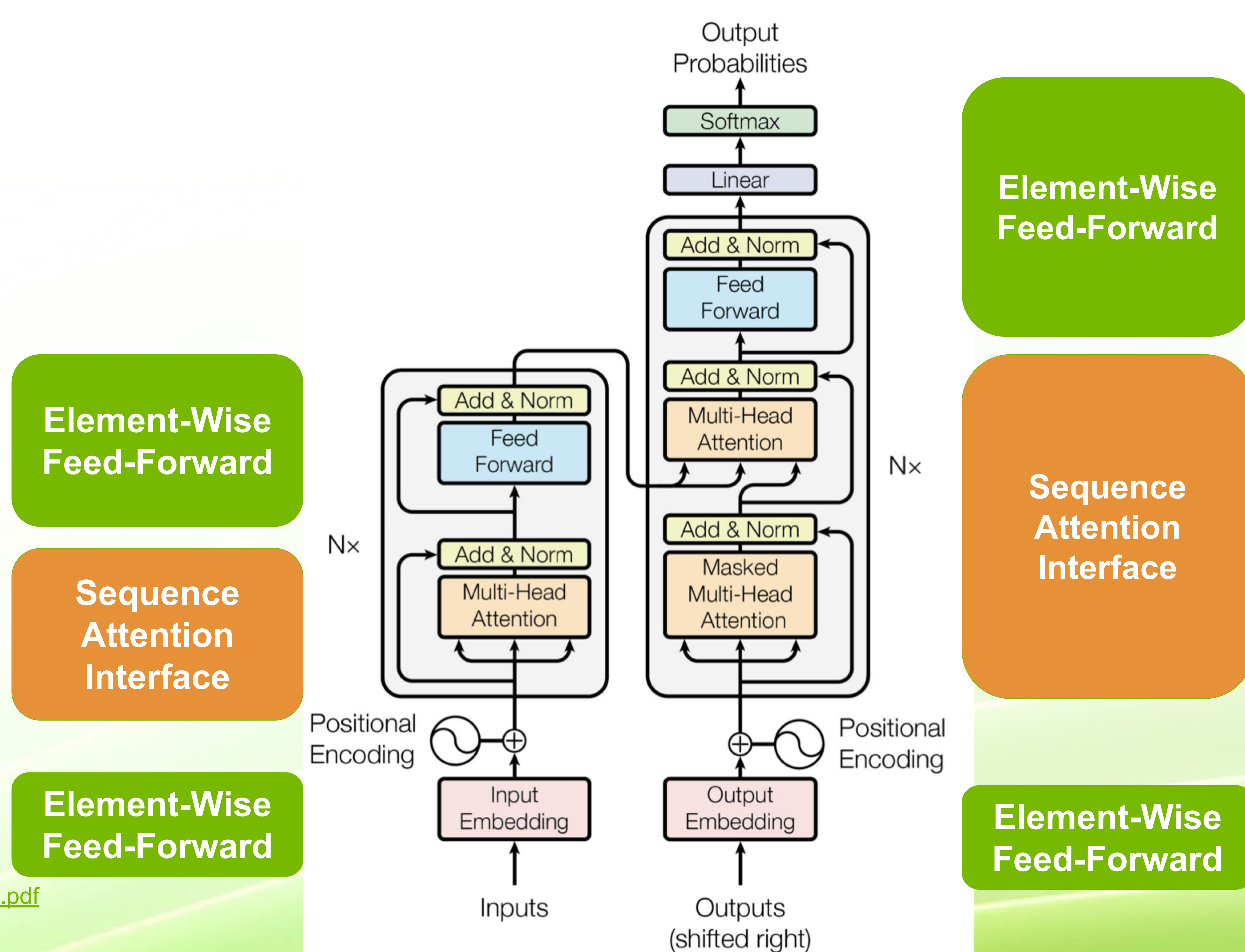
Retrieval-Augmented Generation

Pulling in Information from a Database



Transformer Architecture

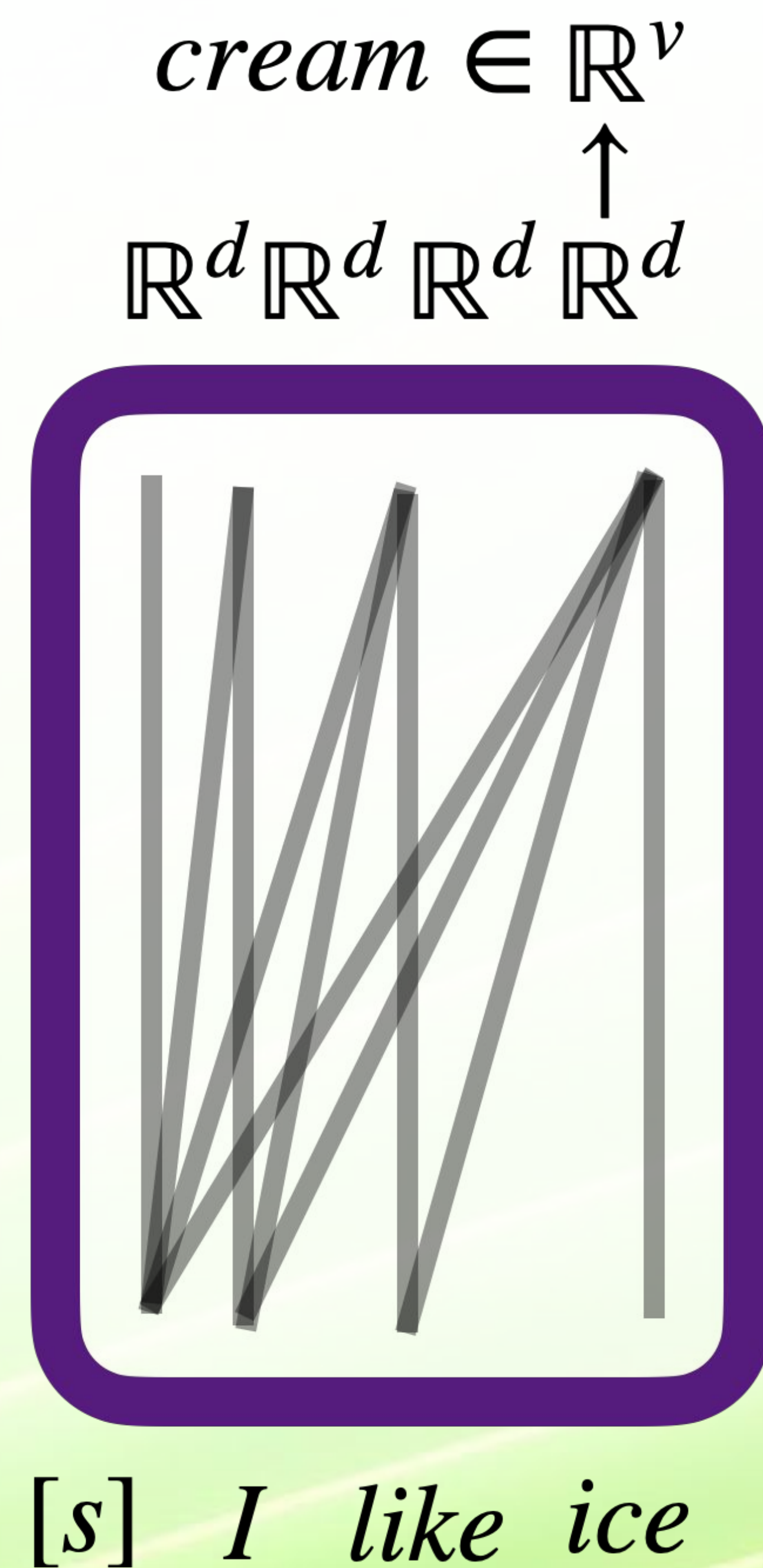
Primary Backbone of LLMs



Transformer Architecture

Autoregressing vs Embedding Flavors

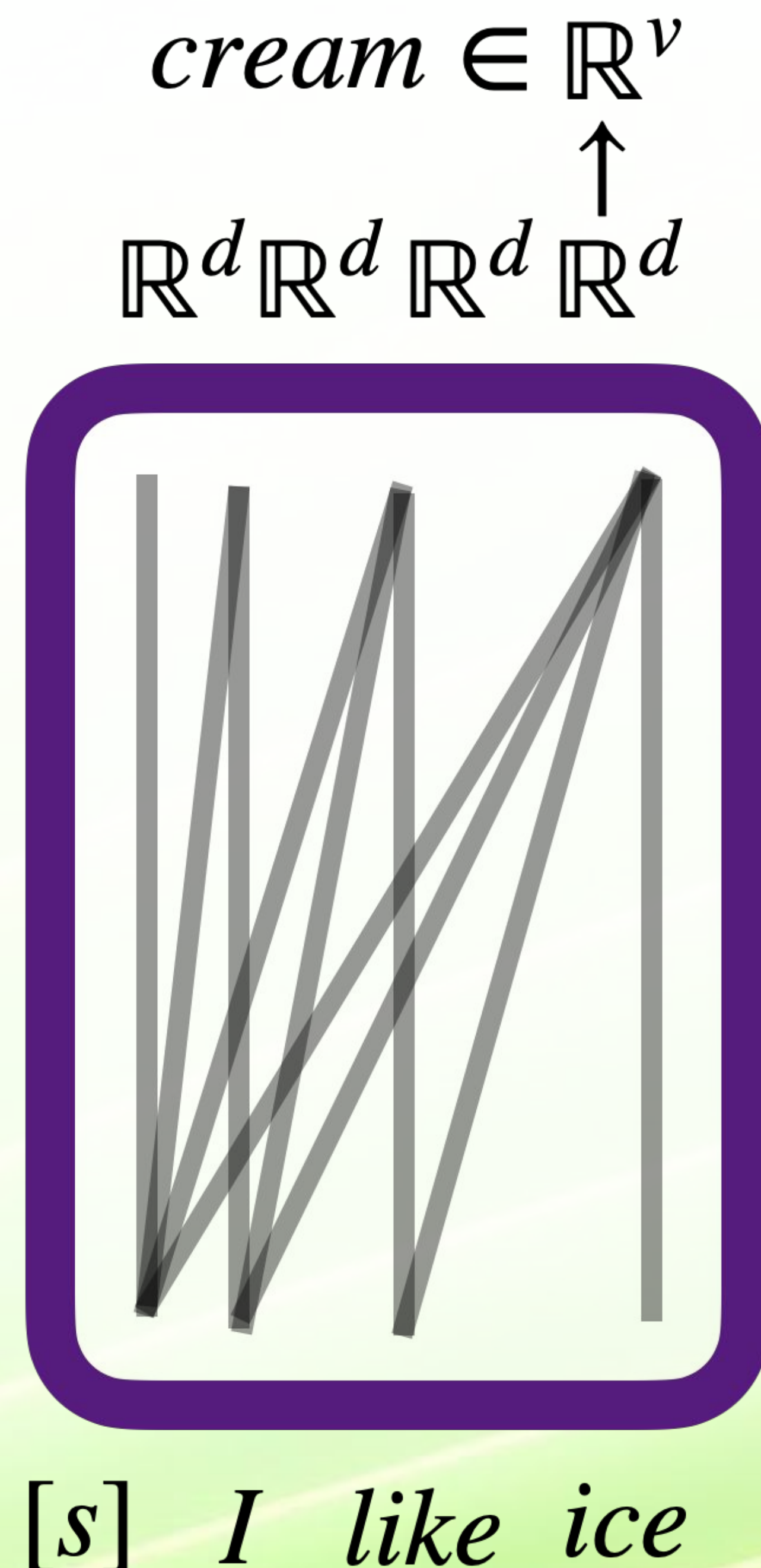
Decoder For
Autoregressive Forecasting
(Common, Parallelizable)



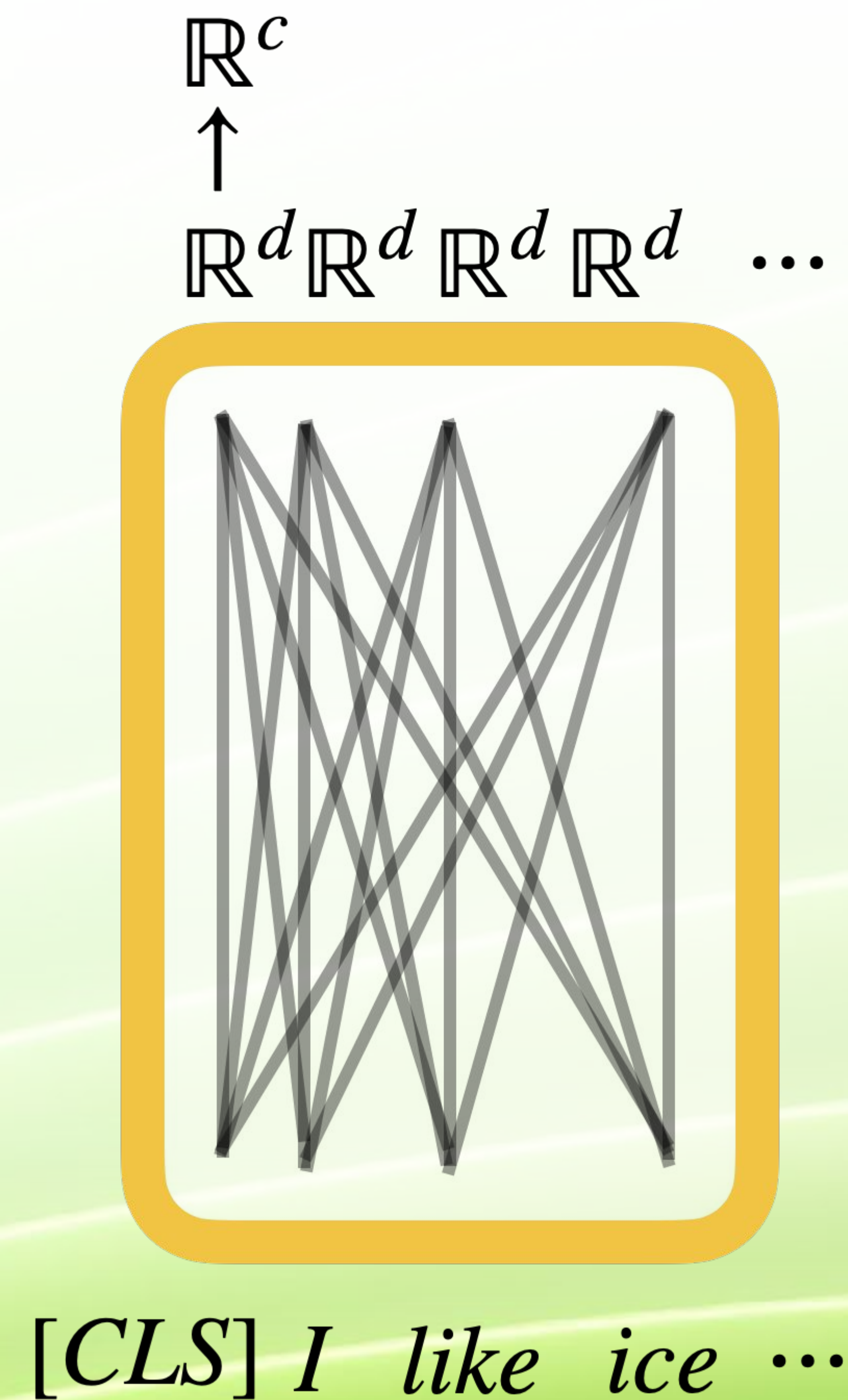
Transformer Architecture

Autoregressing vs Embedding Flavors

Decoder For
Autoregressive Forecasting
(Common, Parallelizable)

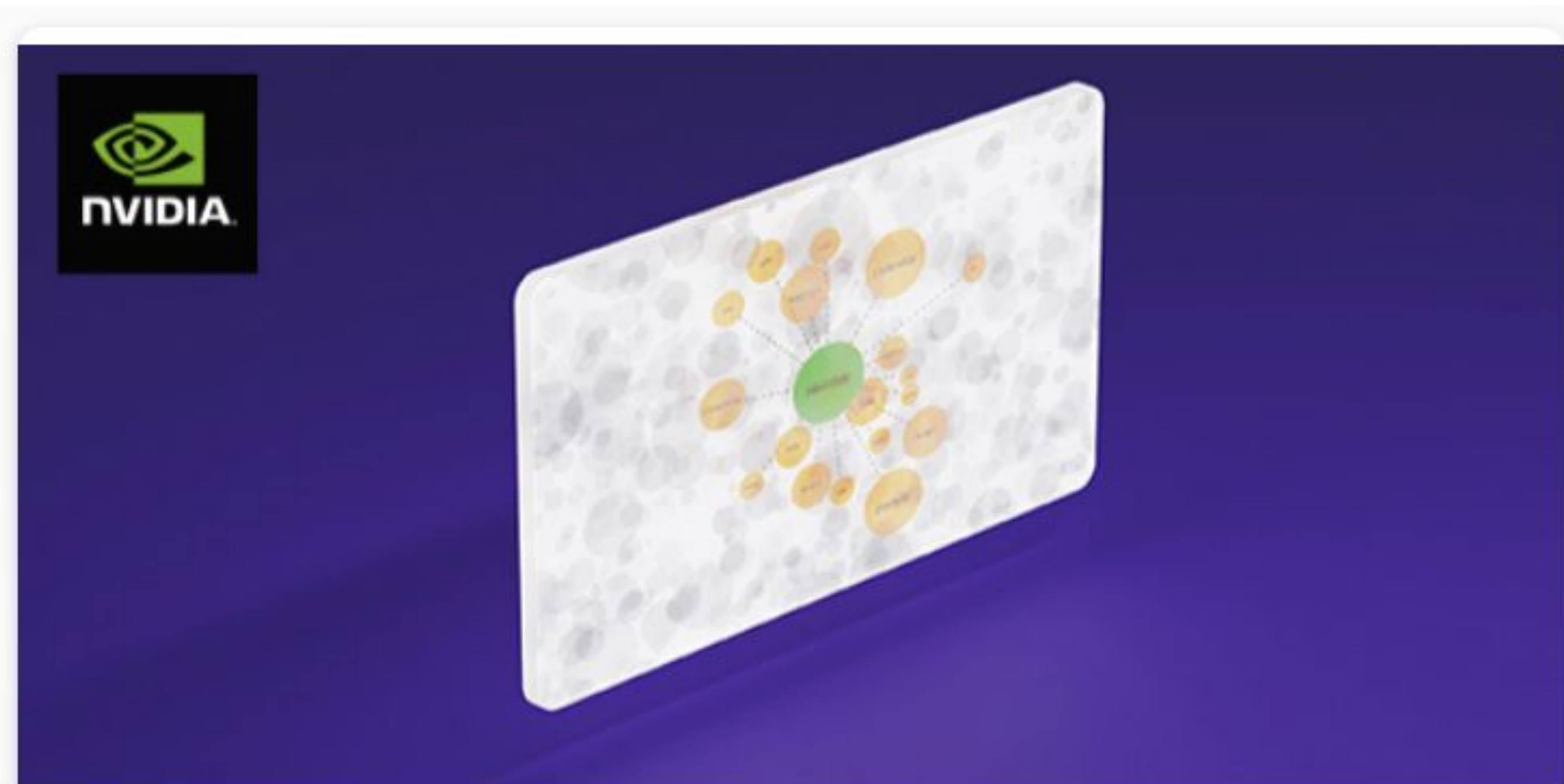


Encoder For
Token/Sequence Prediction
(Common, Fast, Lightweight)



Retrieval QA Embedding

Asymmetric Query/Document Model



Description

NVIDIA Retrieval QA Embedding is an embedding model that represents words, phrases, or other entities as vectors of numbers and understands the relation between words and phrases.

Publisher

NVIDIA

```
"data": [  
  {  
    "index": 0,  
    "embedding": [  
      -0.02204068750143051,  
      -0.05545914173126221,  
      0.00420820526778698,  
      0.07957681268453598,  
      0.0136950071901083,  
      -0.005691236350685358,  
      -0.030427008867263794,  
      0.009445813484489918,  
      -0.02140694297850132,  
      -0.010374147444963455,  
      0.0348891019821167,  
      0.01636182703077793,  
      0.04929877817630768,  
      0.04269032925367355,
```

Embedding and Comparing

Querying for Semantically Similar Entries

High-performance computing.

Happy Holidays!

DLSS Gaming Statistics

Any cool video games lately?

Biological vision structure

What's with GPUs these days?

Mitochondria, powerhouse

Embedding and Comparing

Querying for Semantically Similar Entries

High-performance computing.

Happy Holidays!

DLSS Gaming Statistics

Any cool video games lately?

Biological vision structure

What's with GPUs these days?

Mitochondria, powerhouse

Database

Text Image Video	▸ [1, 0, 3, 5]
Text Image Video	▸ [9, 4, 6, 9]
Text Image Video	▸ [4, 6, 2, 5]
Text Image Video	▸ [6, 8, 6, 1]
Text Image Video	▸ [7, 0, 3, 5]
Text Image Video	▸ [1, 4, 6, 9]
Text Image Video	▸ [4, 6, 2, 4]

Embedding and Comparing

Querying for Semantically Similar Entries

High-performance computing.

Happy Holidays!

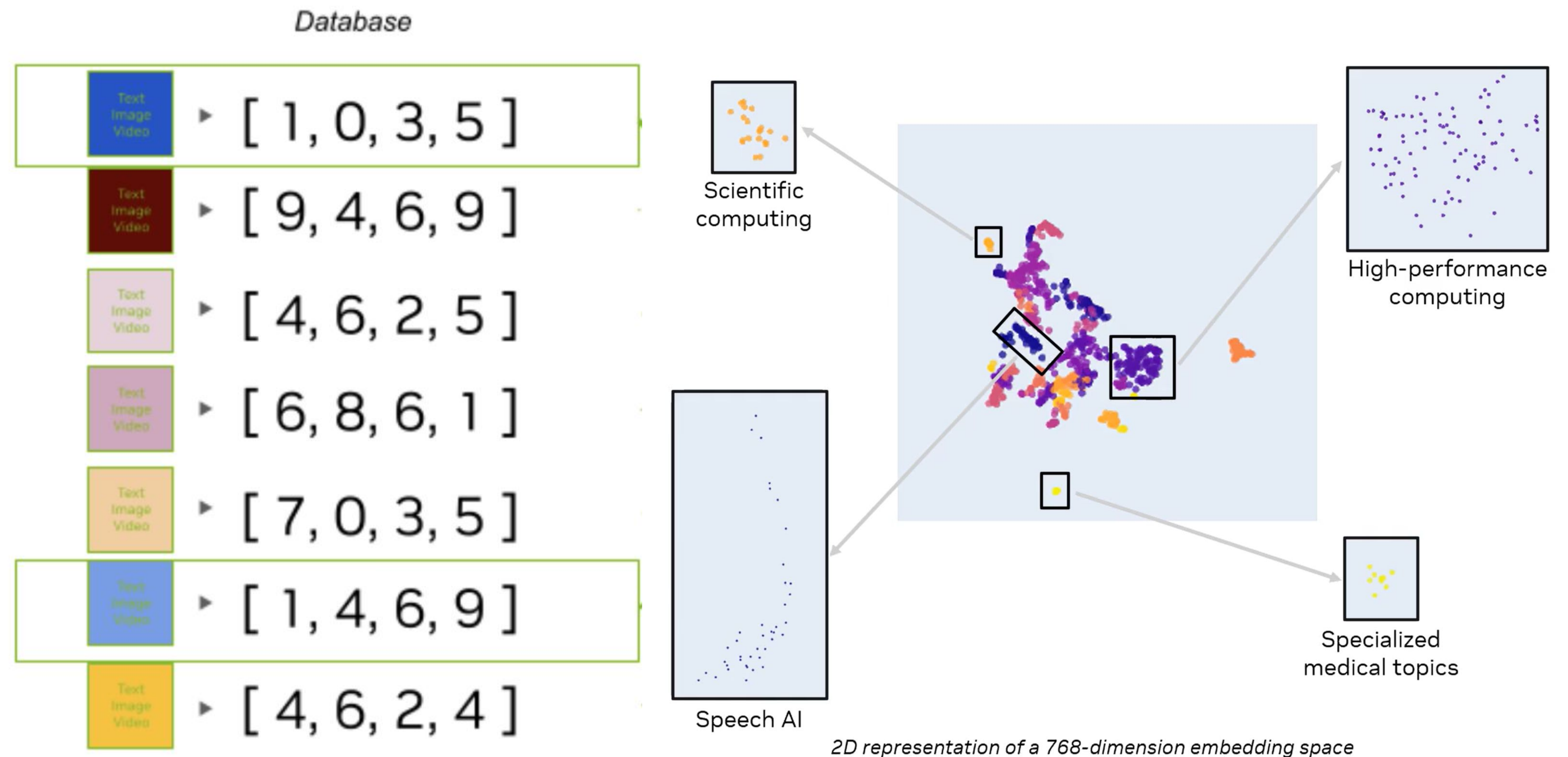
DLSS Gaming Statistics

Any cool video games lately?

Biological vision structure

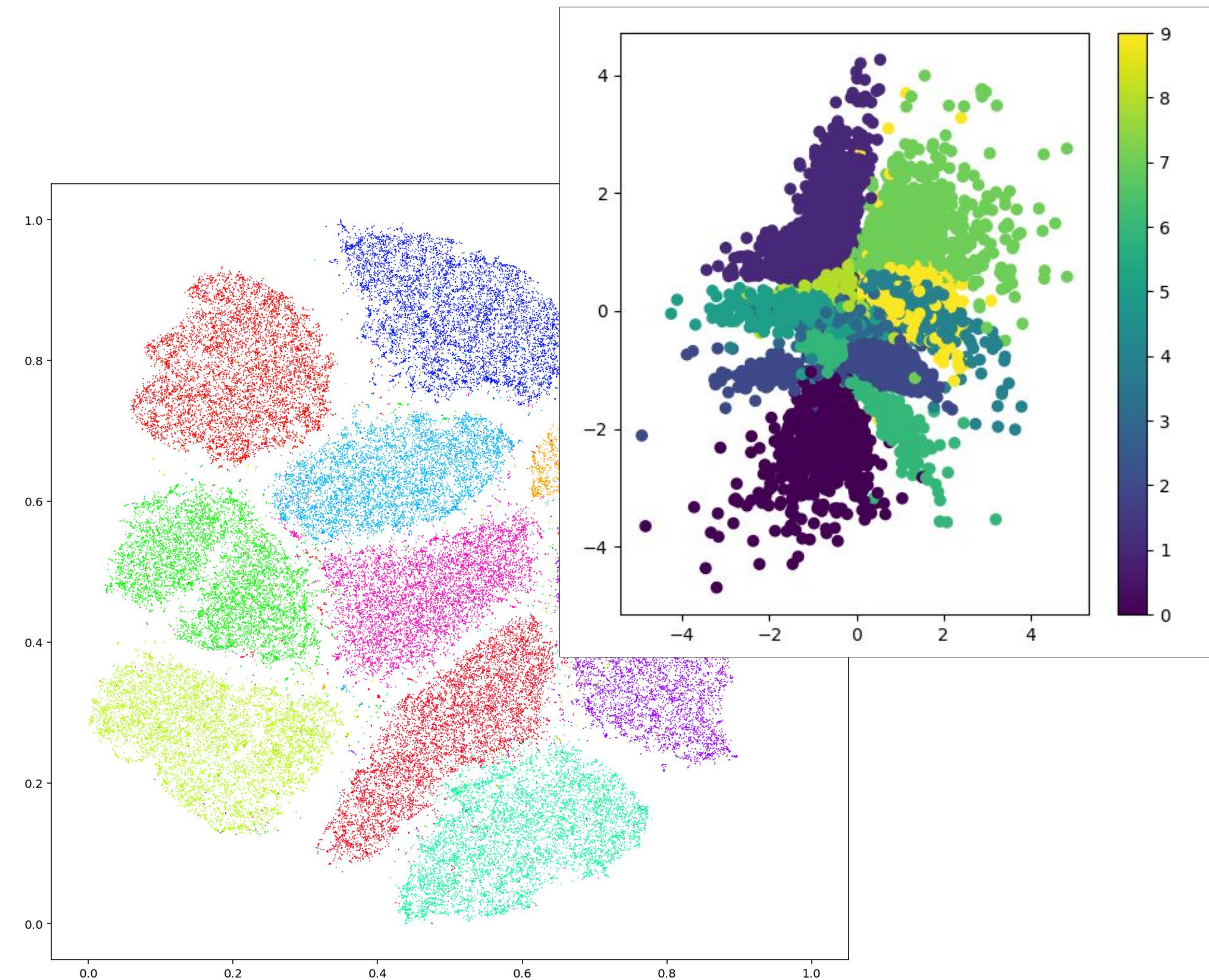
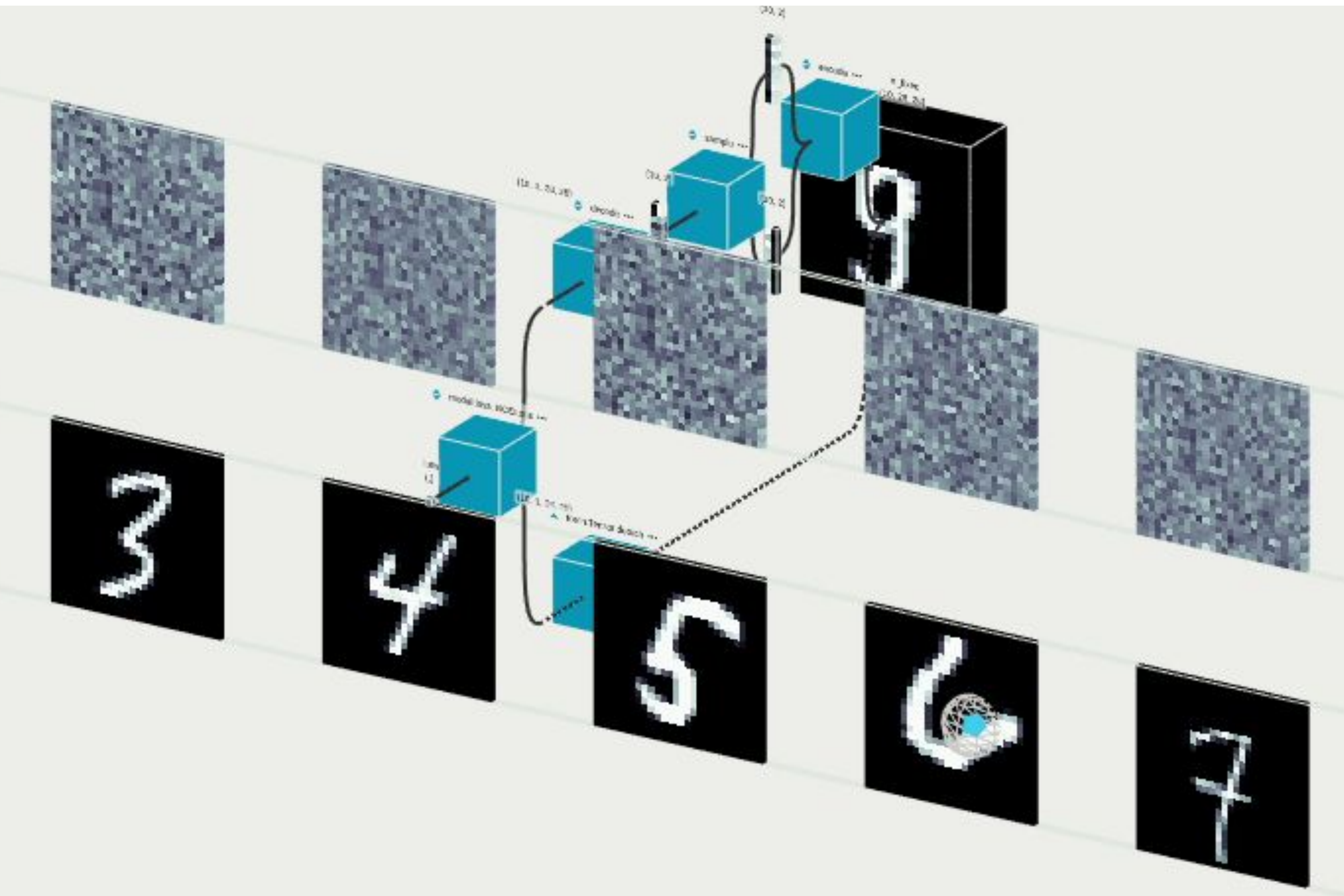
What's with GPUs these days?

Mitochondria, powerhouse



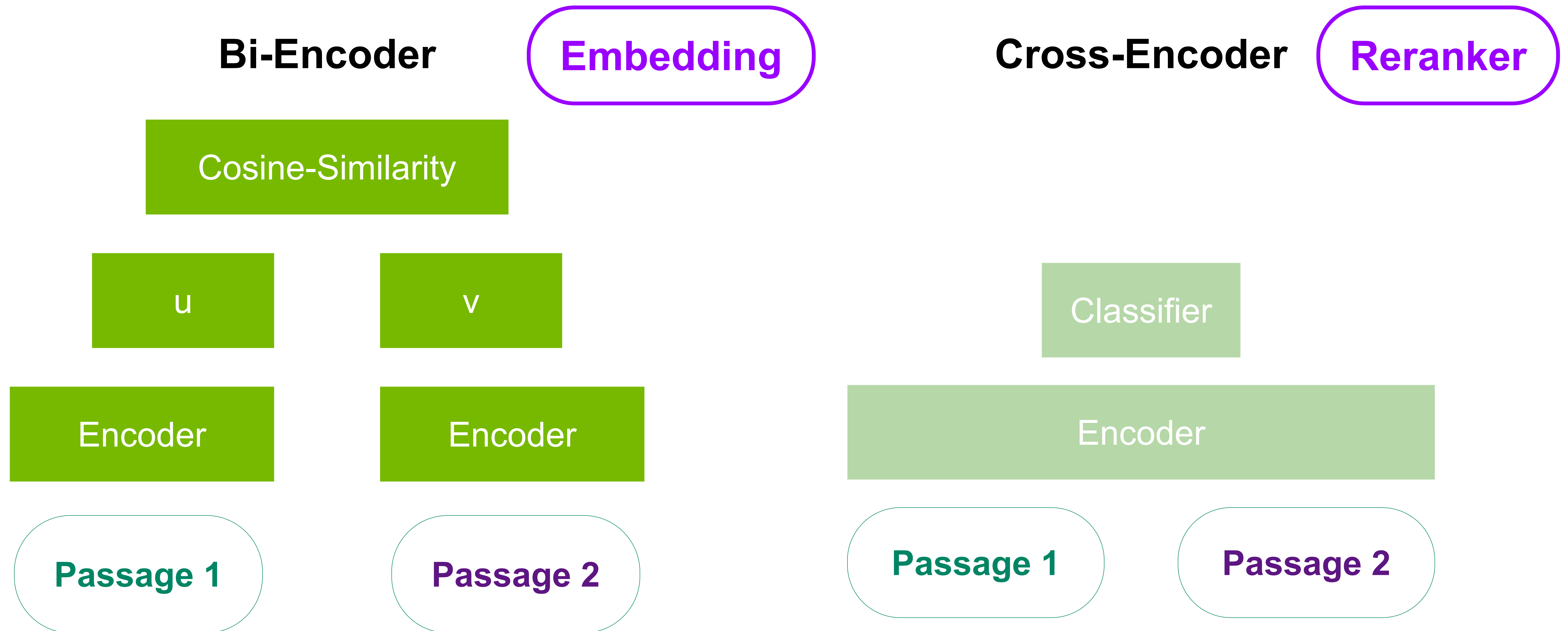
Embedding Spaces

Learned Semantically Dense Values



Language Embedding Schemes

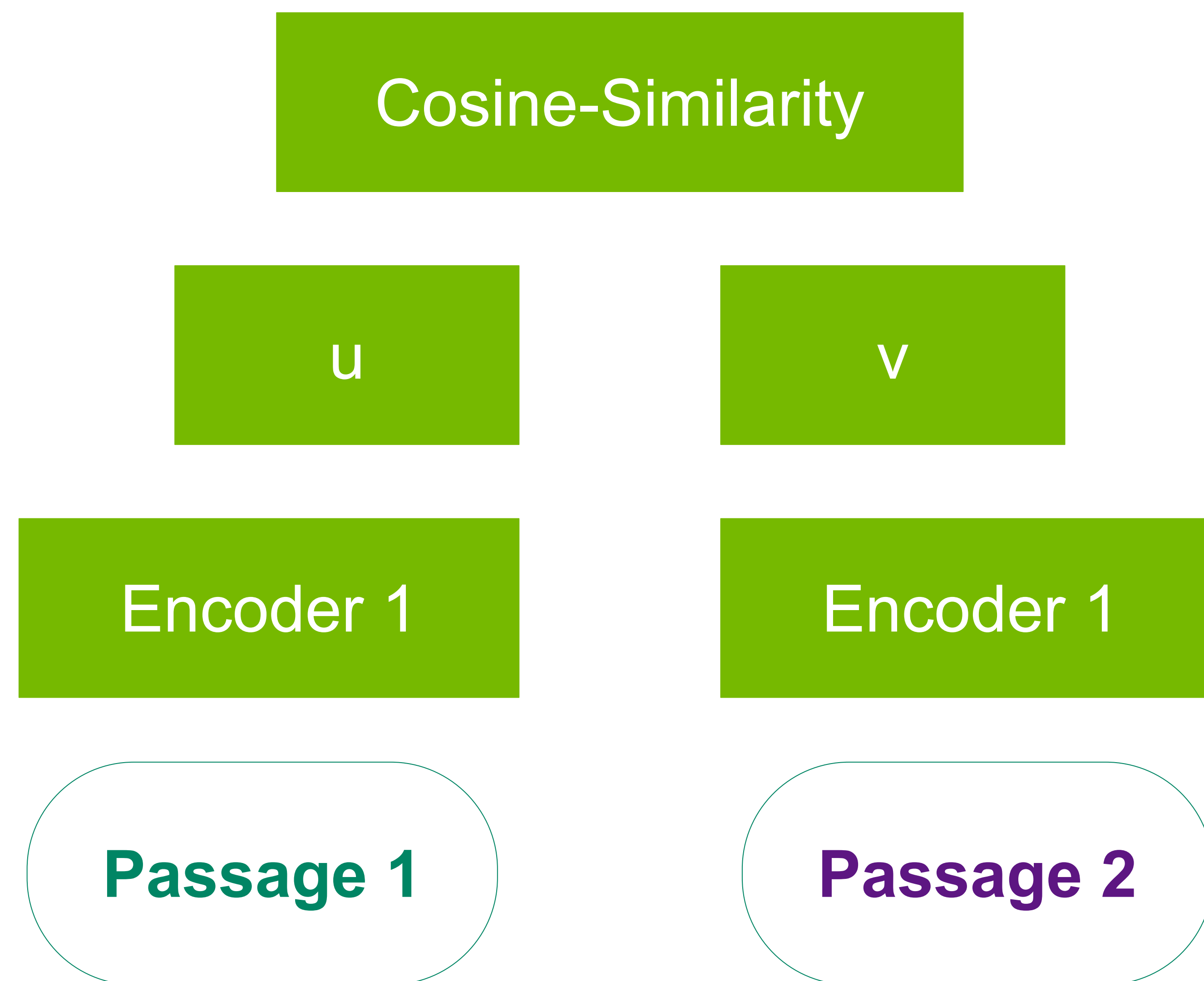
Bi-Encoder versus Cross-Encoder



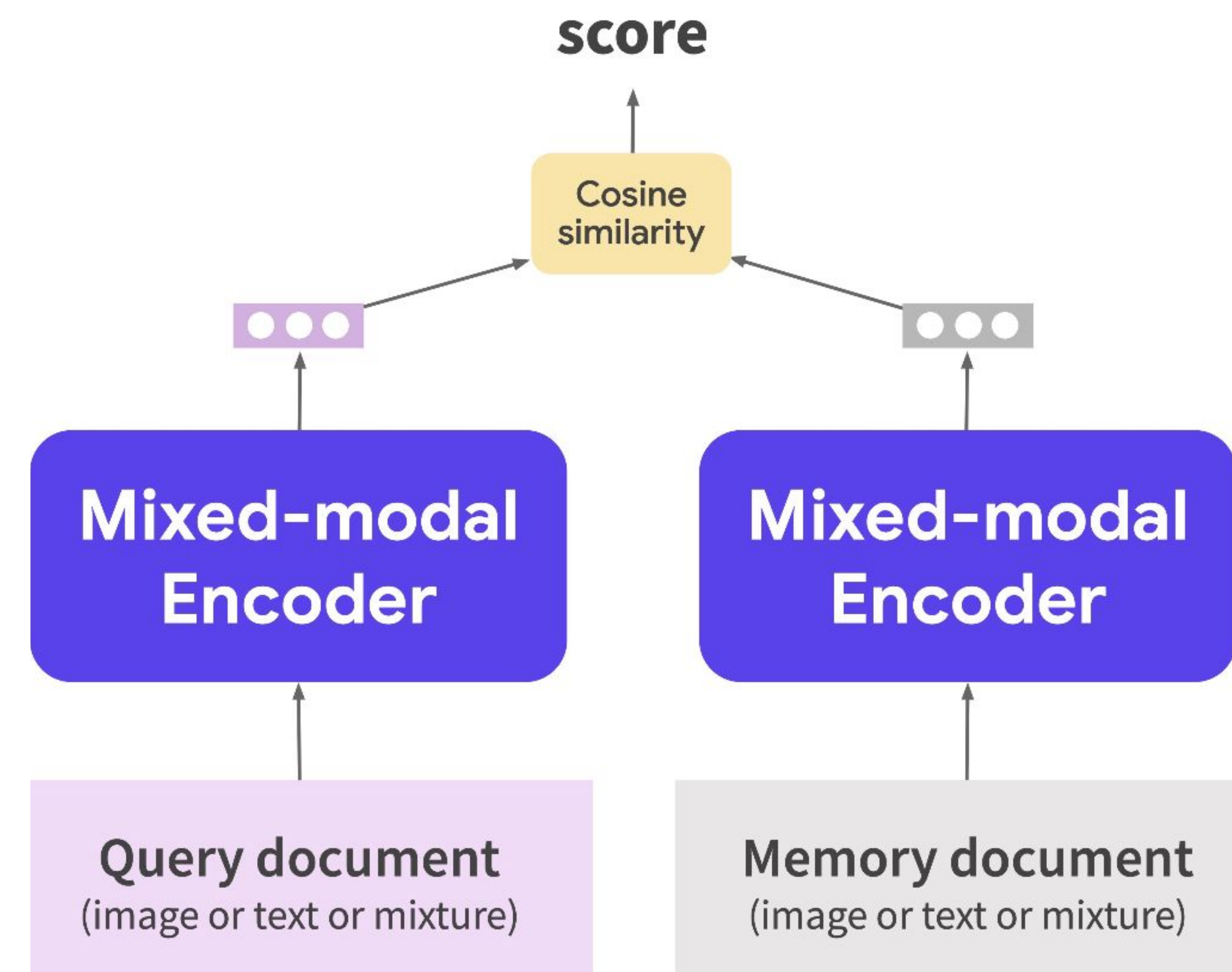
Language Embedding Schemes

Symmetric versus Asymmetric

Symmetric



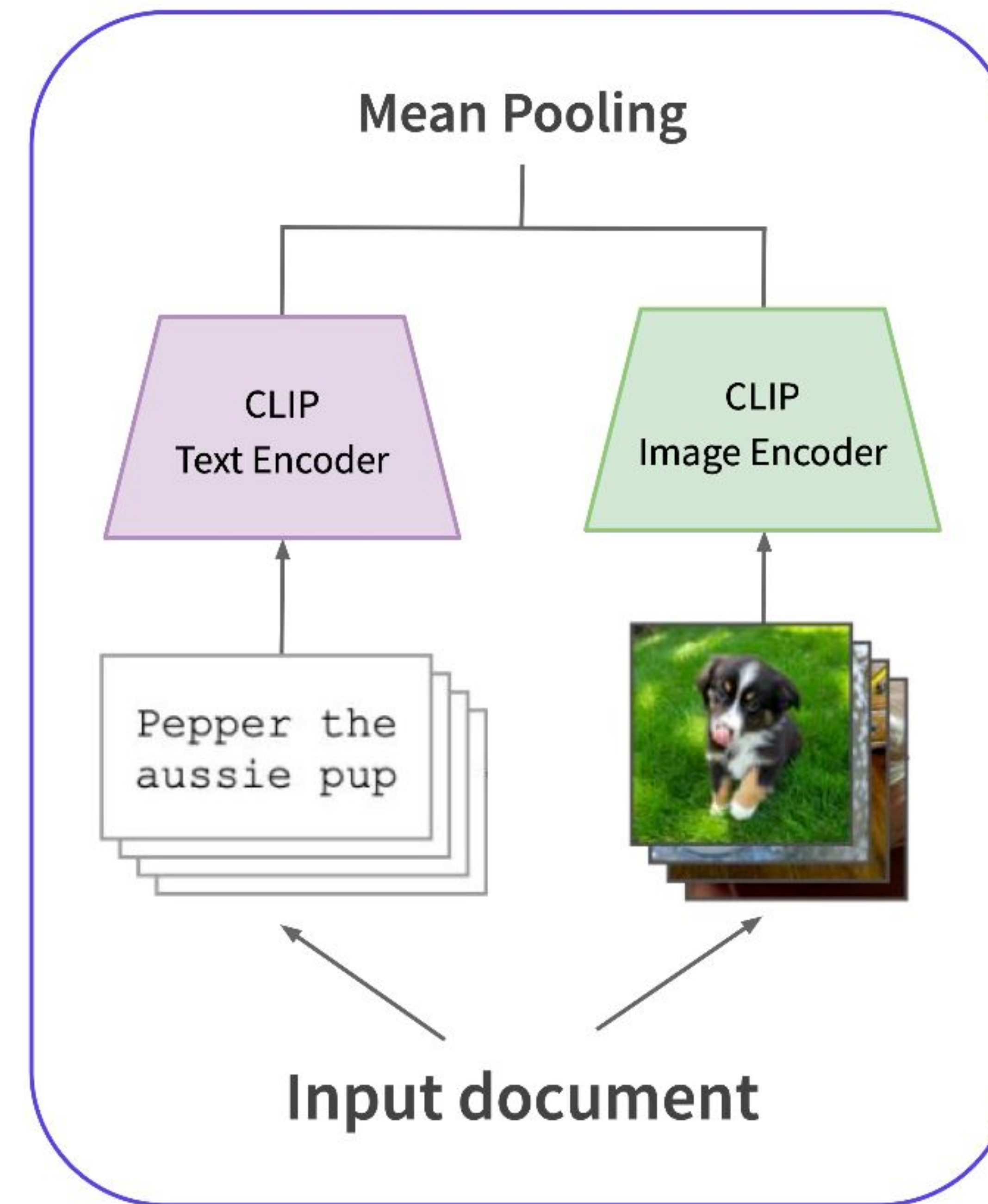
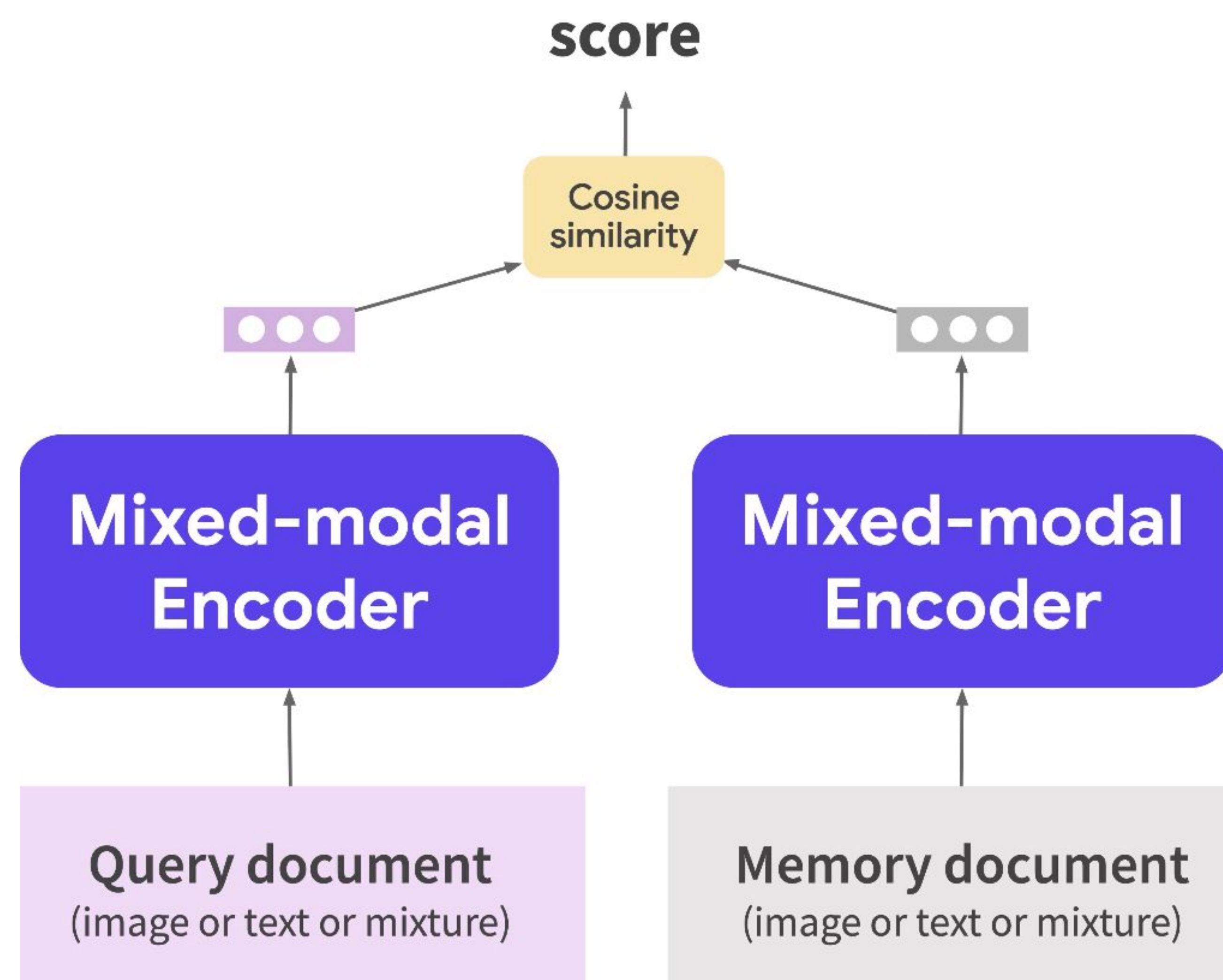
Asymmetric/Generalized



Language Embedding Schemes

Generalized Definition

$$f(\text{query}, \text{memory}) \rightarrow \text{score}$$



Embedding and Comparing

Querying for Semantically Similar Entries

High-performance computing.

Happy Holidays!

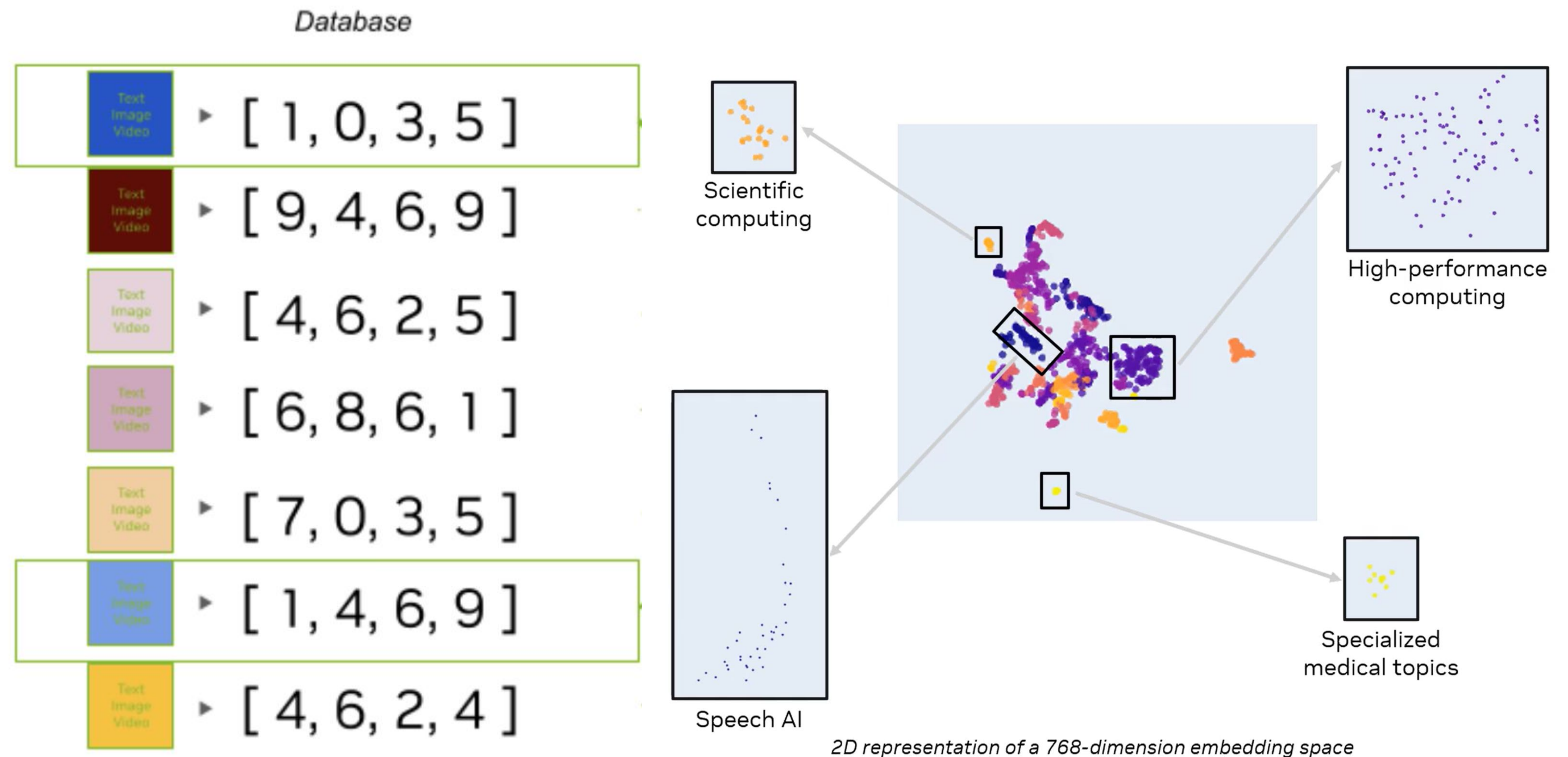
DLSS Gaming Statistics

Any cool video games lately?

Biological vision structure

What's with GPUs these days?

Mitochondria, powerhouse





Building RAG Agents with LLMs

Part 6.4: Guardrails

Embedding and Comparing

Querying for Semantically Similar Entries

High-performance computing.

Happy Holidays!

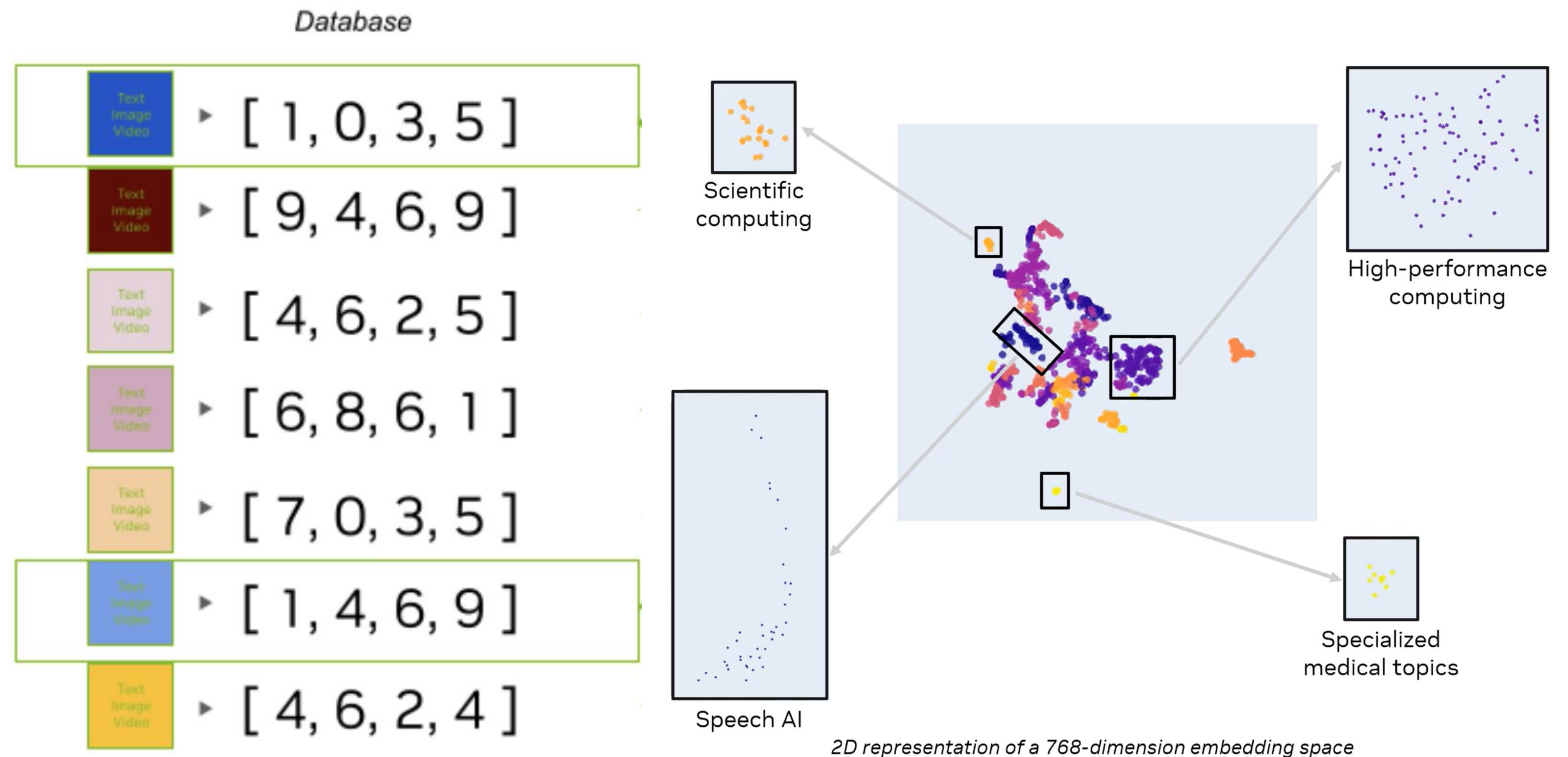
DLSS Gaming Statistics

Any cool video games lately?

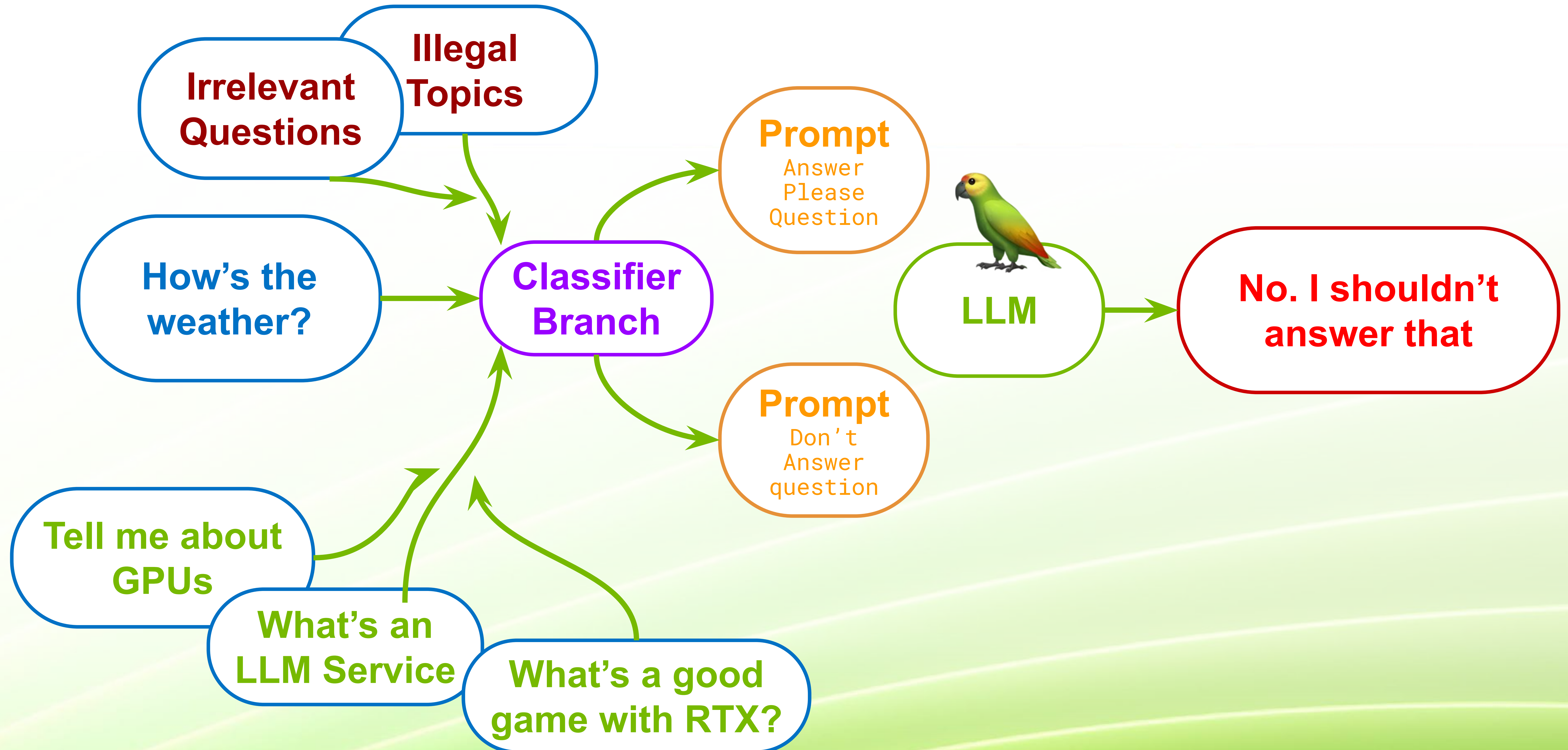
Biological vision structure

What's with GPUs these days?

Mitochondria, powerhouse



Semantic Guardrails



Embedding Classification

Classifying with embeddings

**Irrelevant
Questions**

**Illegal
Topics**

**Tell me about
GPUs**

**What's an
LLM Service**

**What's a good
game with RTX?**

<https://developer.nvidia.com/blog/accelerating-vector-search-using-gpu-powered-indexes-with-rapids-raft/>

Embedding Classification

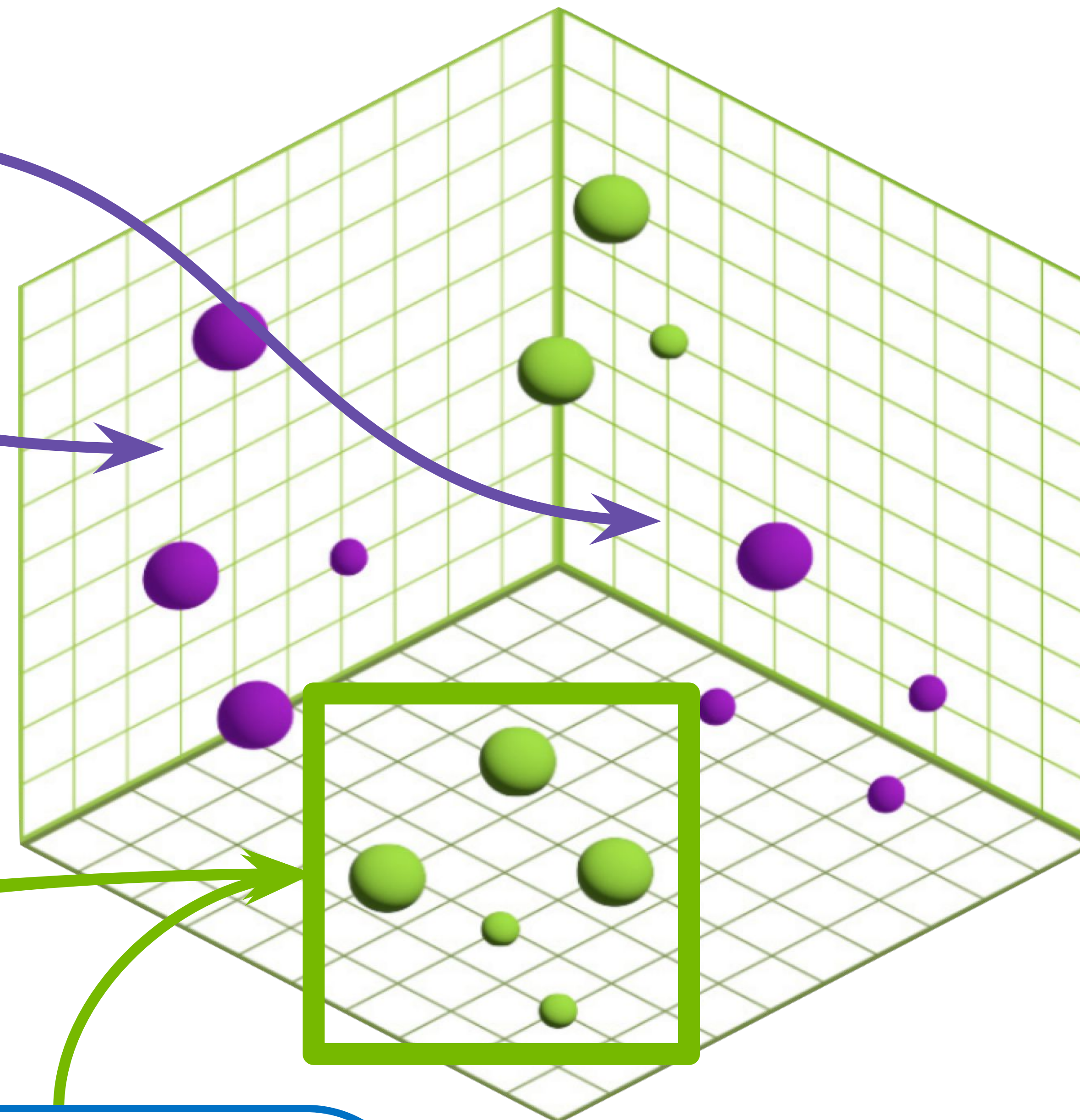
Classifying with embeddings

Irrelevant Questions
Illegal Topics

Tell me about GPUs

What's an LLM Service

What's a good game with RTX?



<https://developer.nvidia.com/blog/accelerating-vector-search-using-gpu-powered-indexes-with-rapids-raft/>

Embedding Classification

Classifying with embeddings

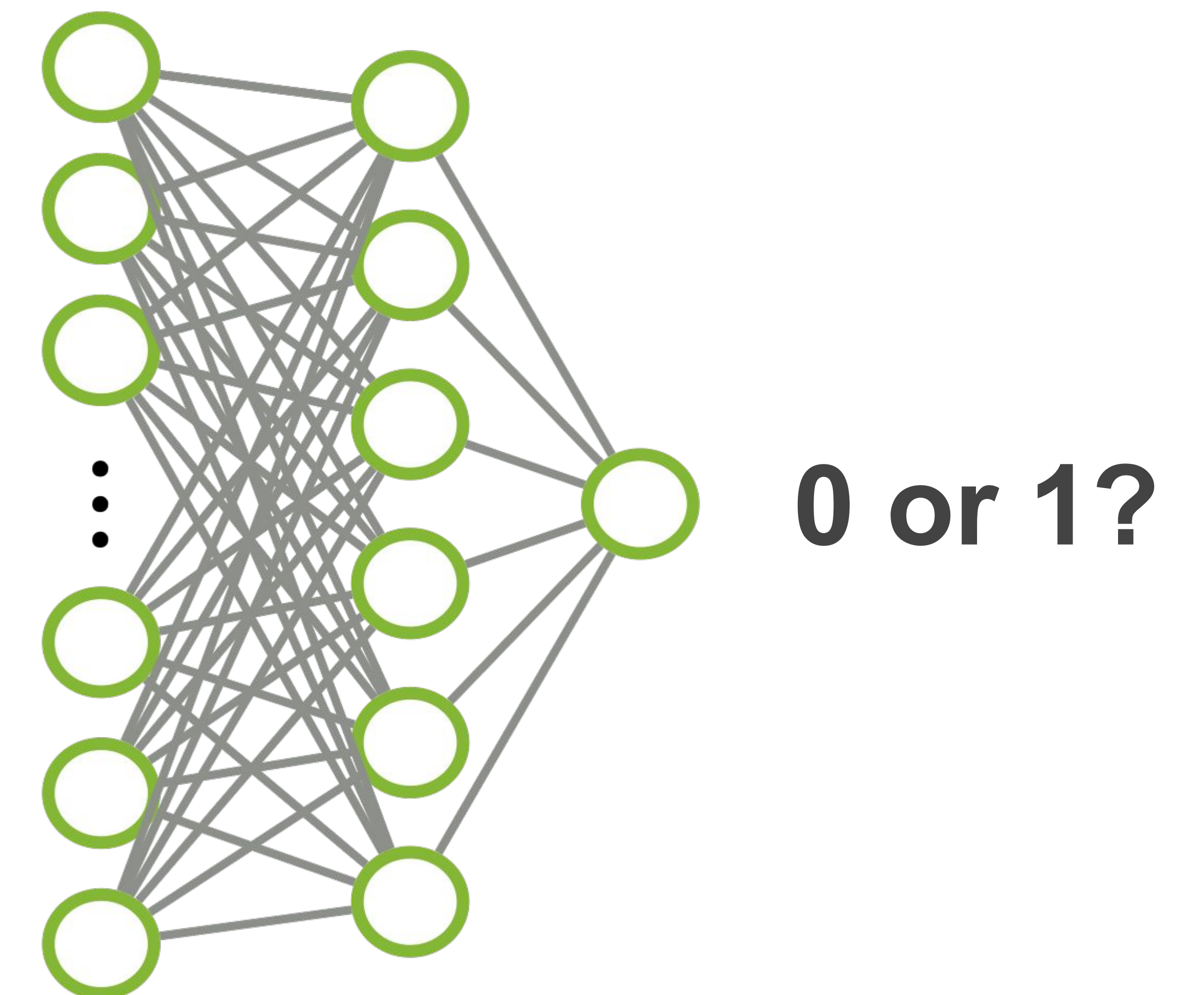
Irrelevant Questions
Illegal Topics

Tell me about GPUs

What's an LLM Service

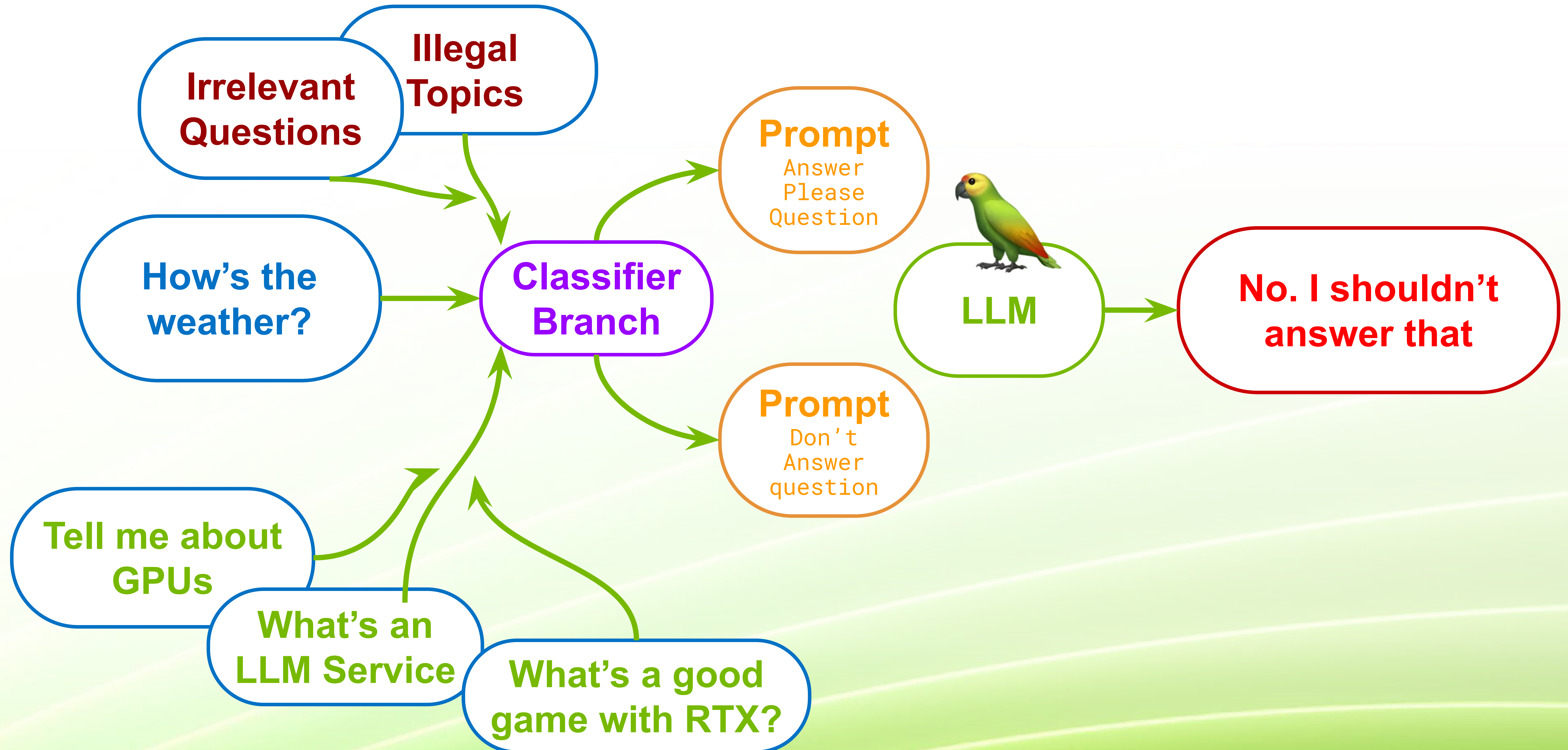
What's a good game with RTX?

Classification Head



<https://developer.nvidia.com/blog/accelerating-vector-search-using-gpu-powered-indexes-with-rapids-raft/>

Semantic Guardrails





Building RAG Agents with LLMs

Part 7: Document Retrieval with Vector Databases

Embedding and Comparing

Querying for Semantically Similar Entries

High-performance computing.

Happy Holidays!

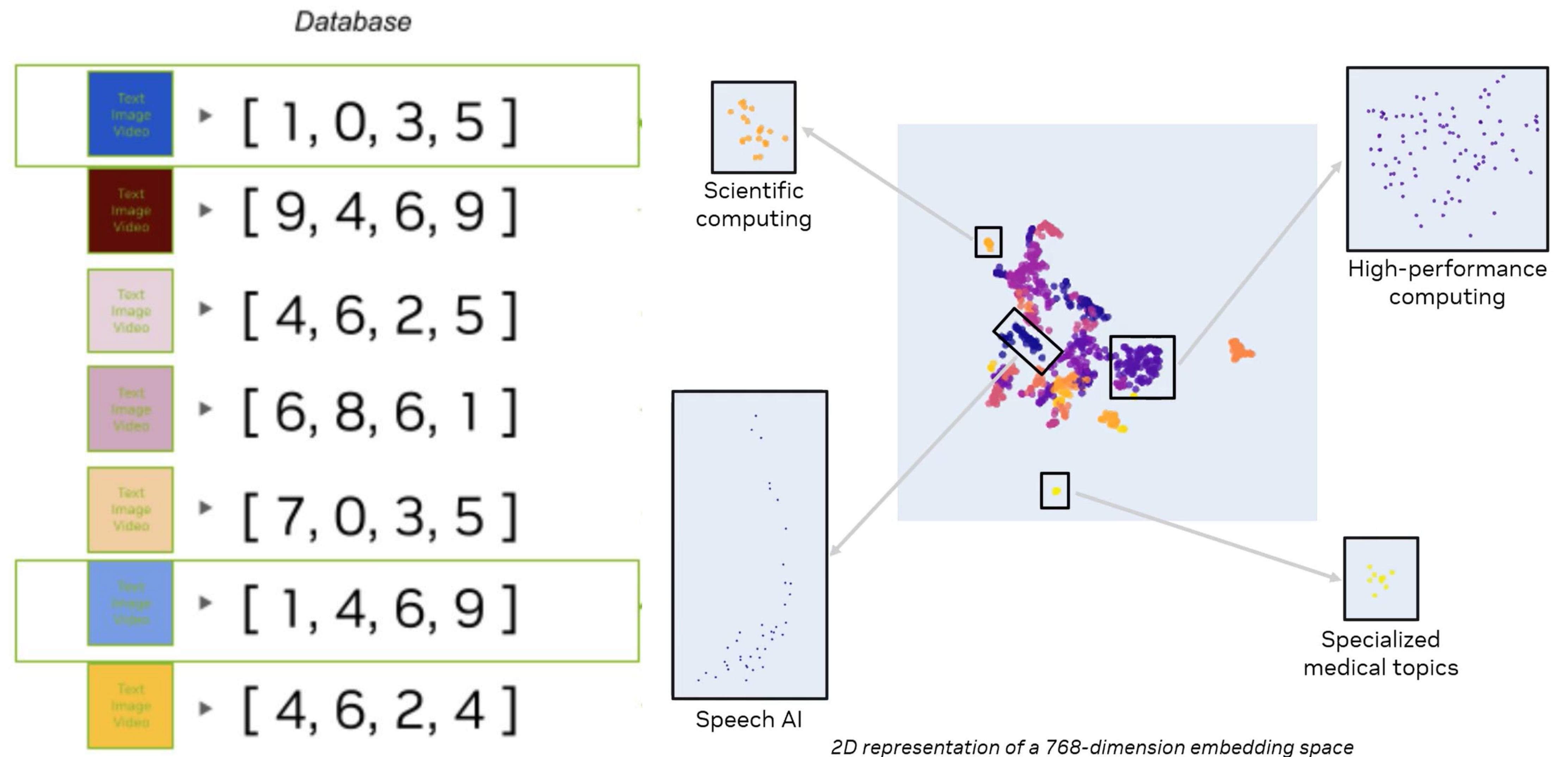
DLSS Gaming Statistics

Any cool video games lately?

Biological vision structure

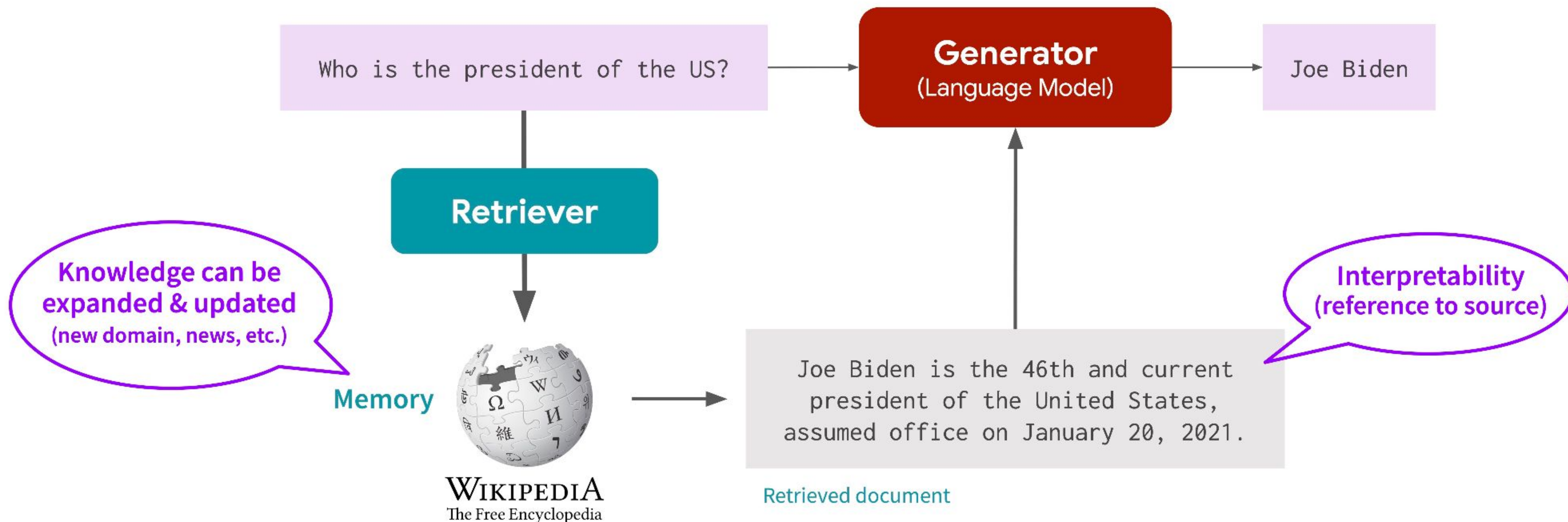
What's with GPUs these days?

Mitochondria, powerhouse



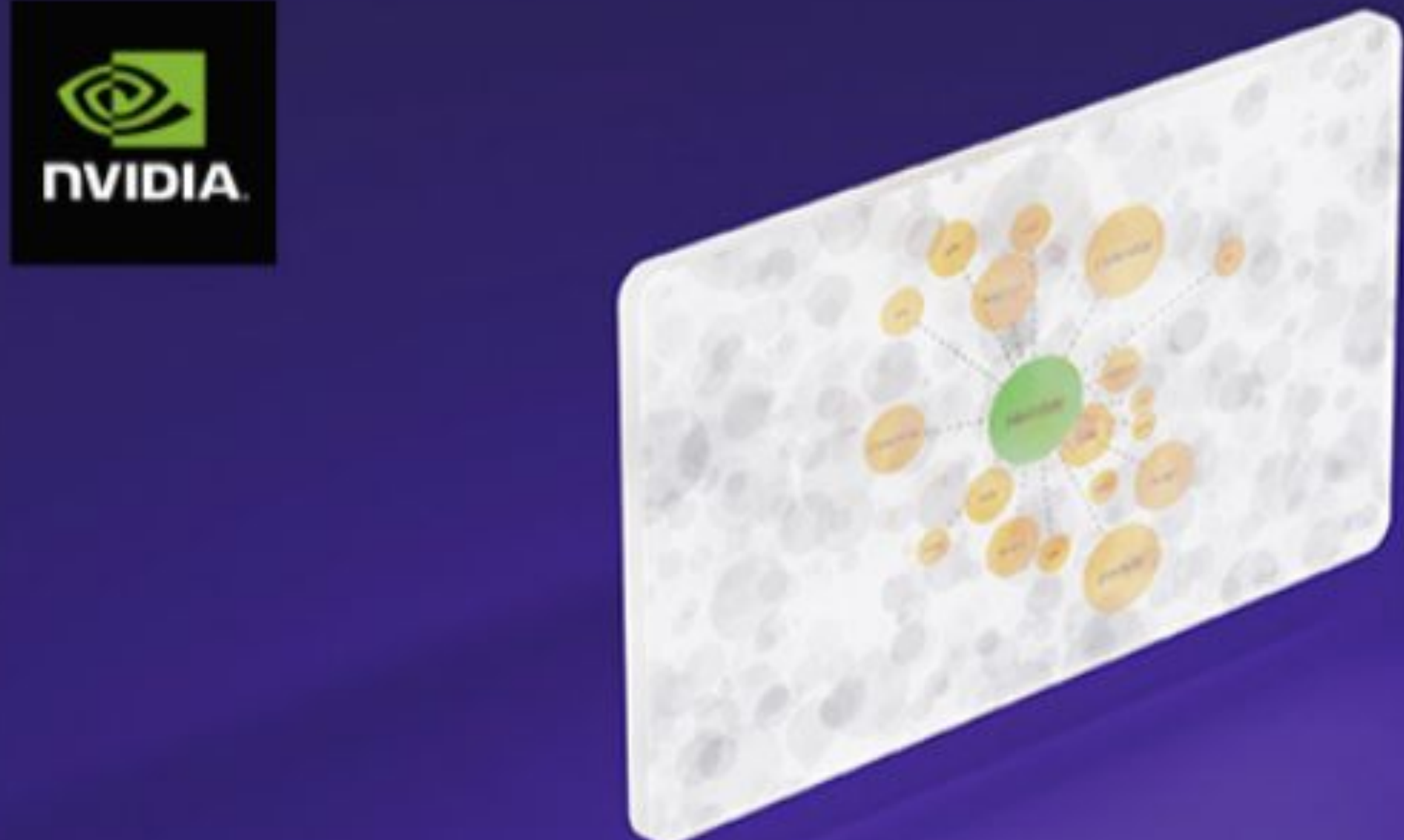
Retrieval-Augmented Generation

Pulling in Information from a Database



Retrieval QA Embedding

Asymmetric Query/Document Model



Description

NVIDIA Retrieval QA Embedding is an embedding model that represents words, phrases, or other entities as vectors of numbers and understands the relation between words and phrases.

Publisher

NVIDIA

```
"data": [  
  {  
    "index": 0,  
    "embedding": [  
      -0.02204068750143051,  
      -0.05545914173126221,  
      0.00420820526778698,  
      0.07957681268453598,  
      0.0136950071901083,  
      -0.005691236350685358,  
      -0.030427008867263794,  
      0.009445813484489918,  
      -0.02140694297850132,  
      -0.010374147444963455,  
      0.0348891019821167,  
      0.01636182703077793,  
      0.04929877817630768,  
      0.04269032925367355,
```

Integrating a Vector Store

Vector Stores

1. Load Source Data



Load, Transform, Embed

Vector Store



2. Query Vector Store

Embed

5.5, -0.3...
2.1, 0.1

XXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXX

3. Retrieve 'most similar'

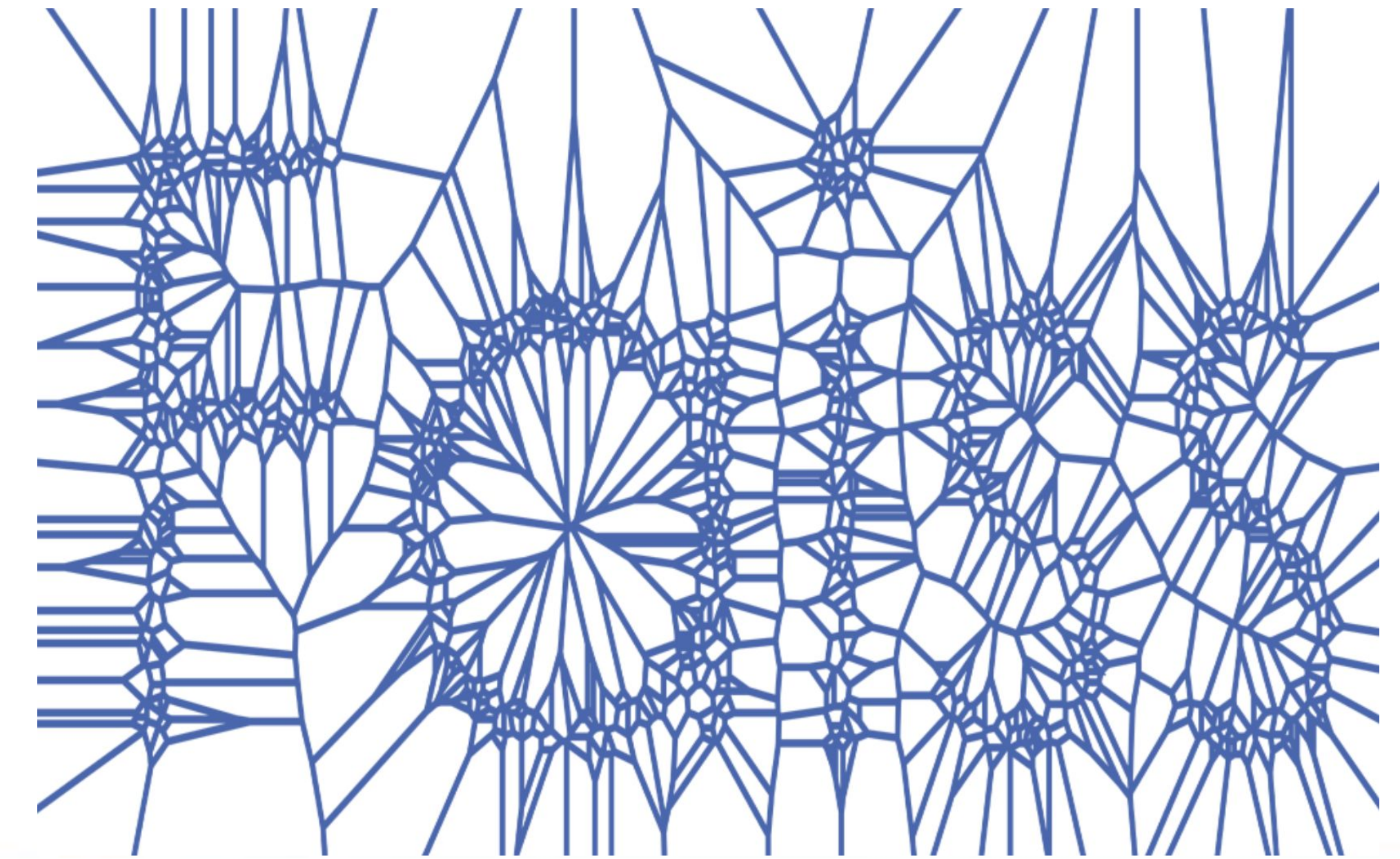
Integrating a Vector Store

```
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain.vectorstores import FAISS

embedder = NVIDIAEmbeddings(model="nvidia_40k")

convstore = FAISS.from_texts(conversation, embedding=embedder)
retriever = convstore.as_retriever()

retriever.invoke("What is your name?")
```



Integrating a Vector Store

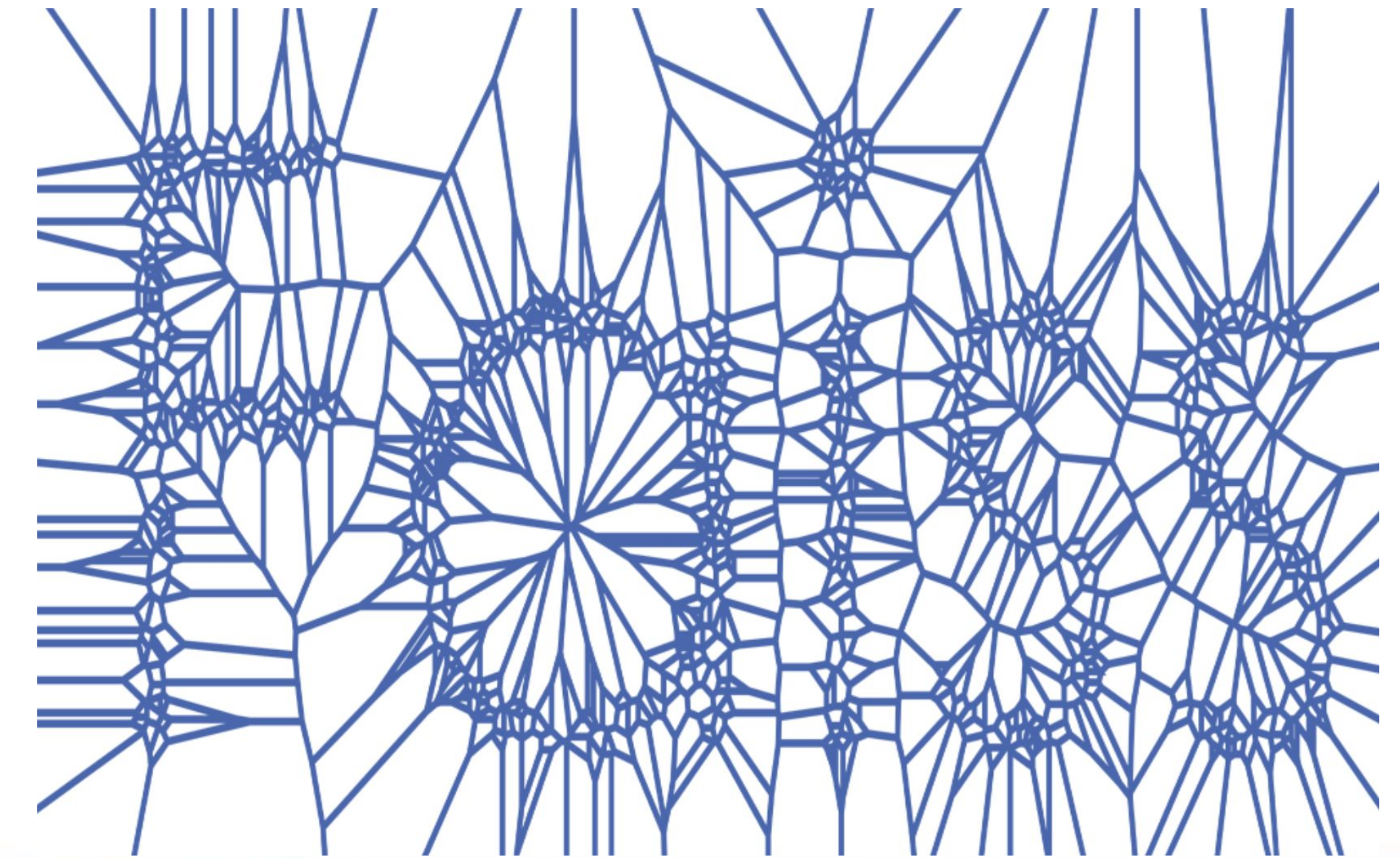
```
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain.vectorstores import FAISS

embedder = NVIDIAEmbeddings(model="nvidia_40k")

convstore = FAISS.from_texts(conversation, embedding=embedder)
retriever = convstore.as_retriever()

retriever.invoke("What is your name?")
```

VDB



Integrating a Vector Store

```
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain.vectorstores import FAISS

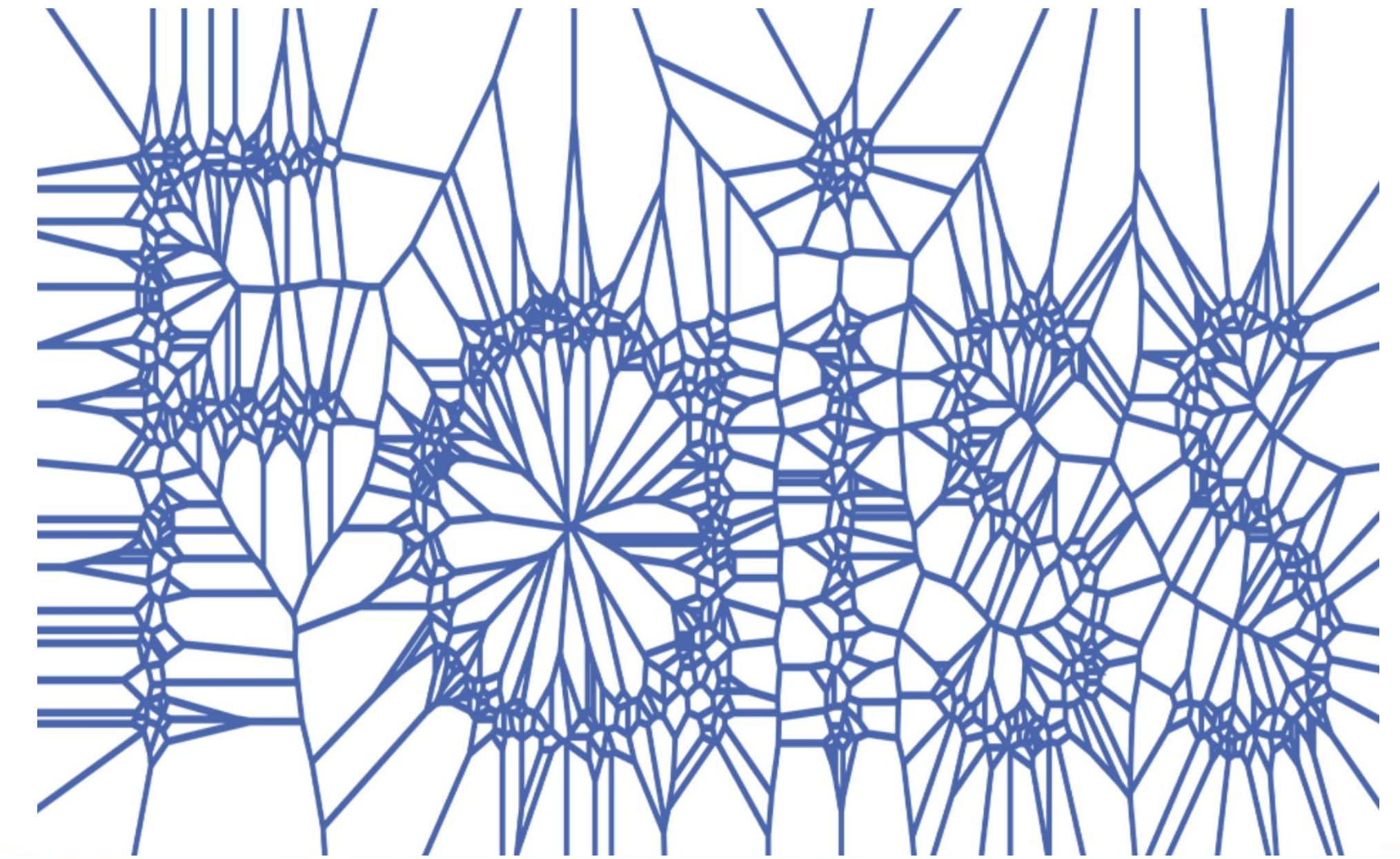
embedder = NVIDIAEmbeddings(model="nvidia_40k")

convstore = FAISS.from_texts(conversation, embedding=embedder)
retriever = convstore.as_retriever()

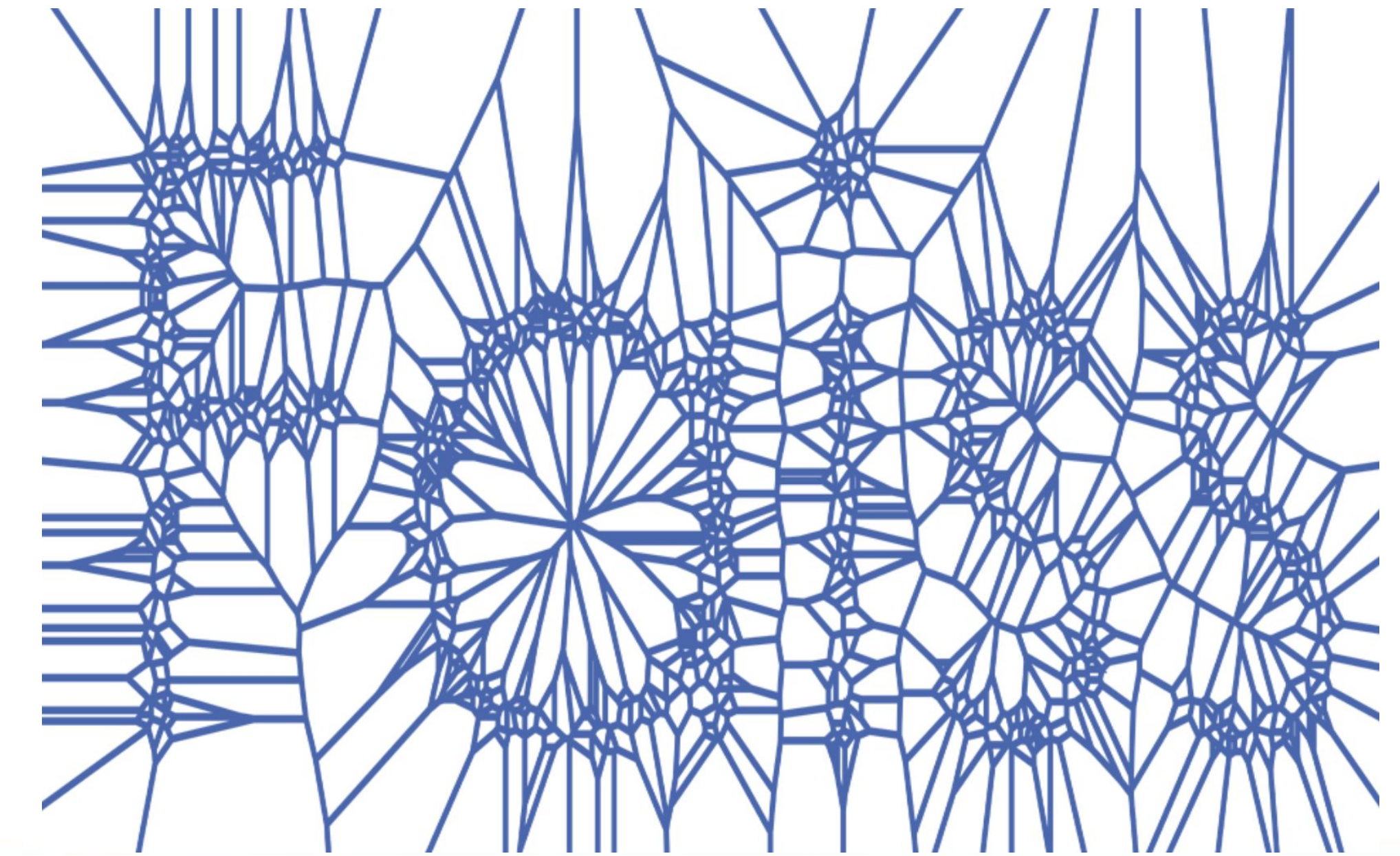
retriever.invoke("What is your name?")
```

VDB

Retriever



Integrating a Vector Store



```
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain.vectorstores import FAISS

embedder = NVIDIAEmbeddings(model="nvolveqa_40k")

convstore = FAISS.from_texts(conversation, embedding=embedder)
retriever = convstore.as_retriever()

retriever.invoke("What is your name?")
```

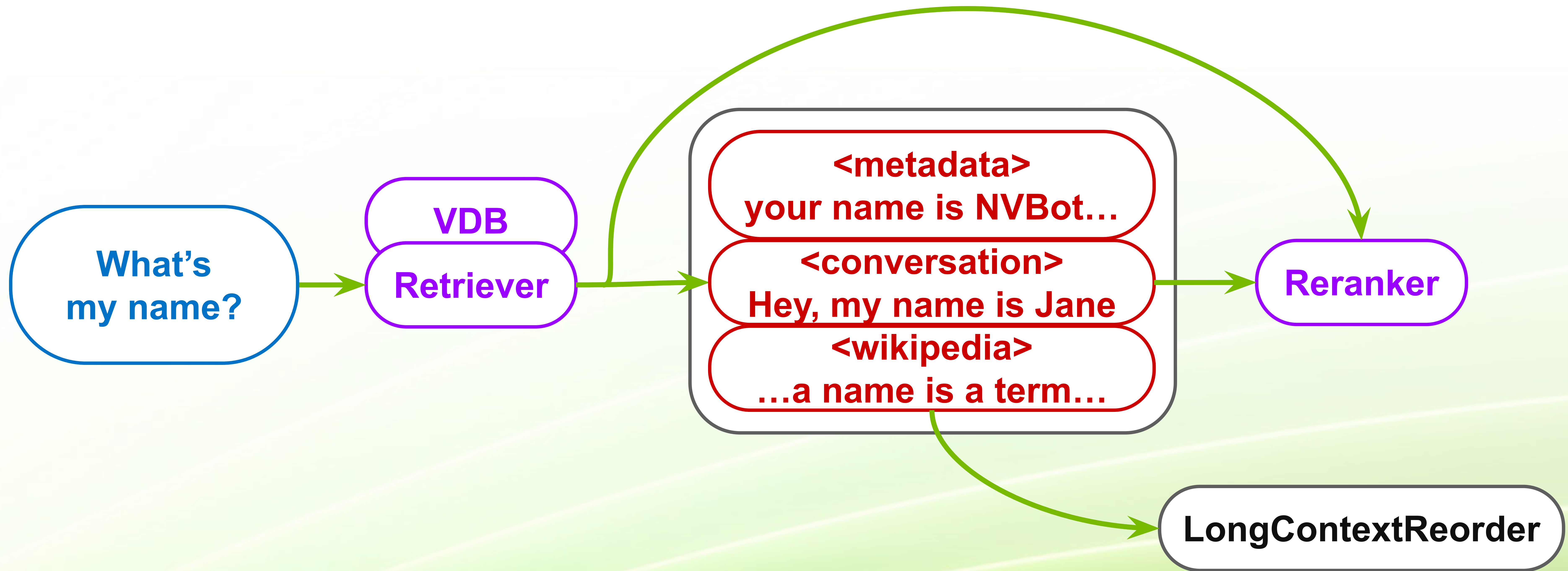
VDB
Retriever

<metadata>
your name is NVBot...

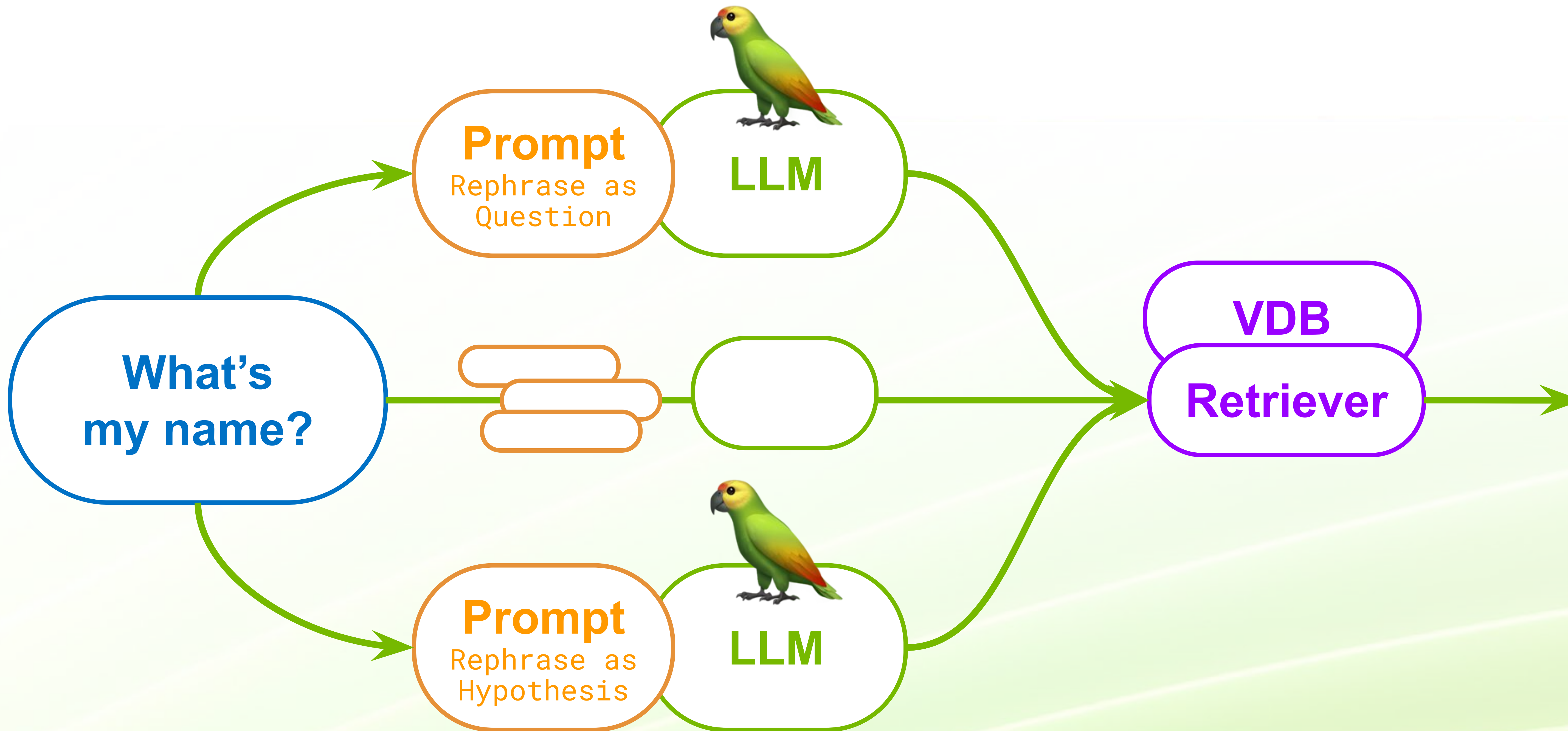
<conversation>
Hey, my name is Jane

<wikipedia>
...a name is a term...

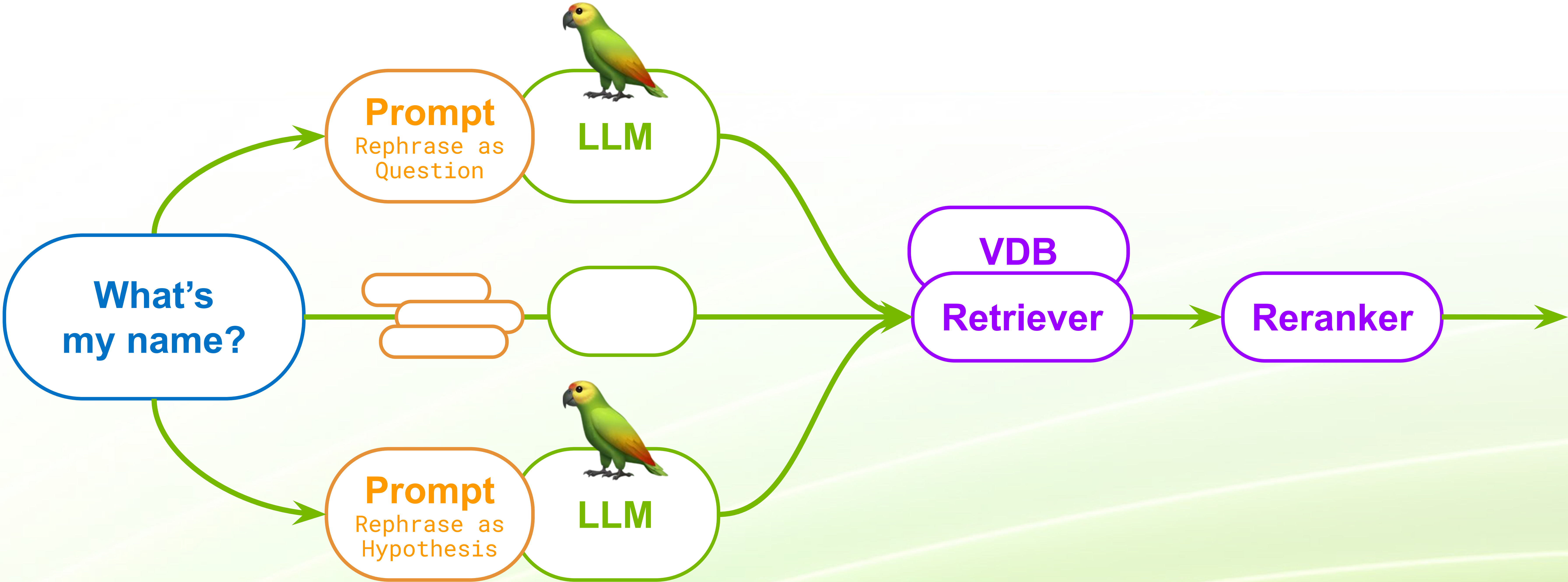
Retrieval Reordering/Selection



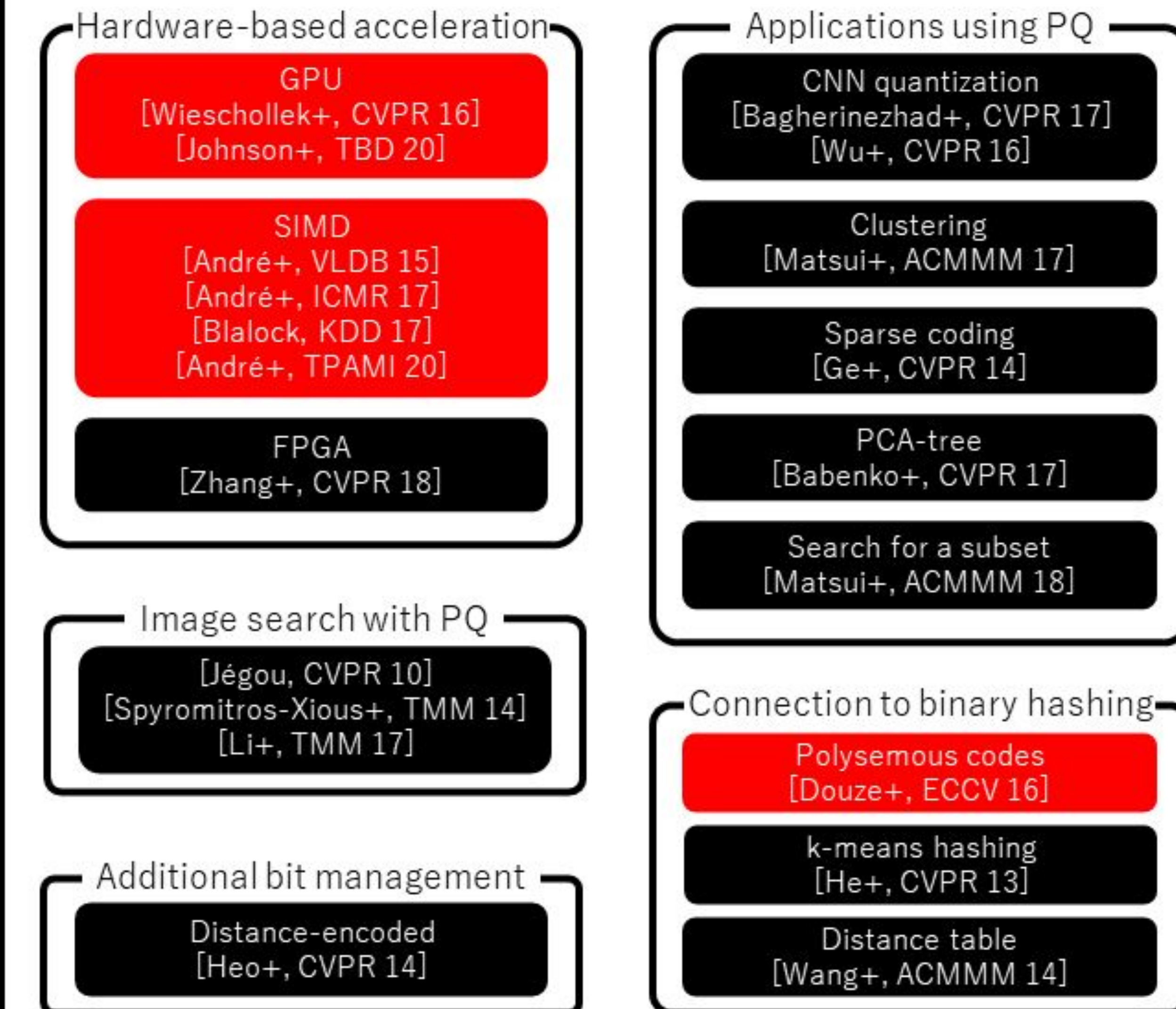
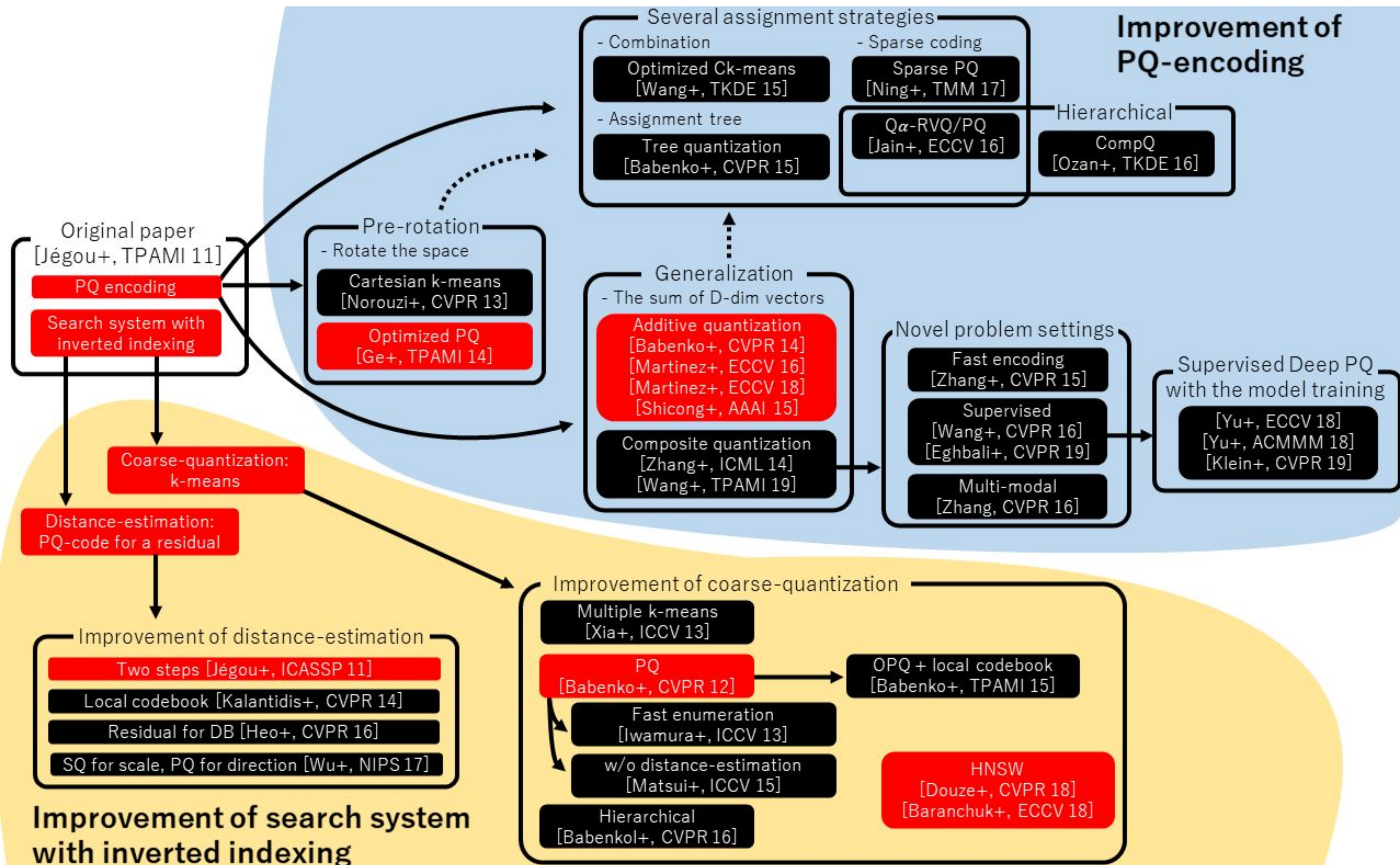
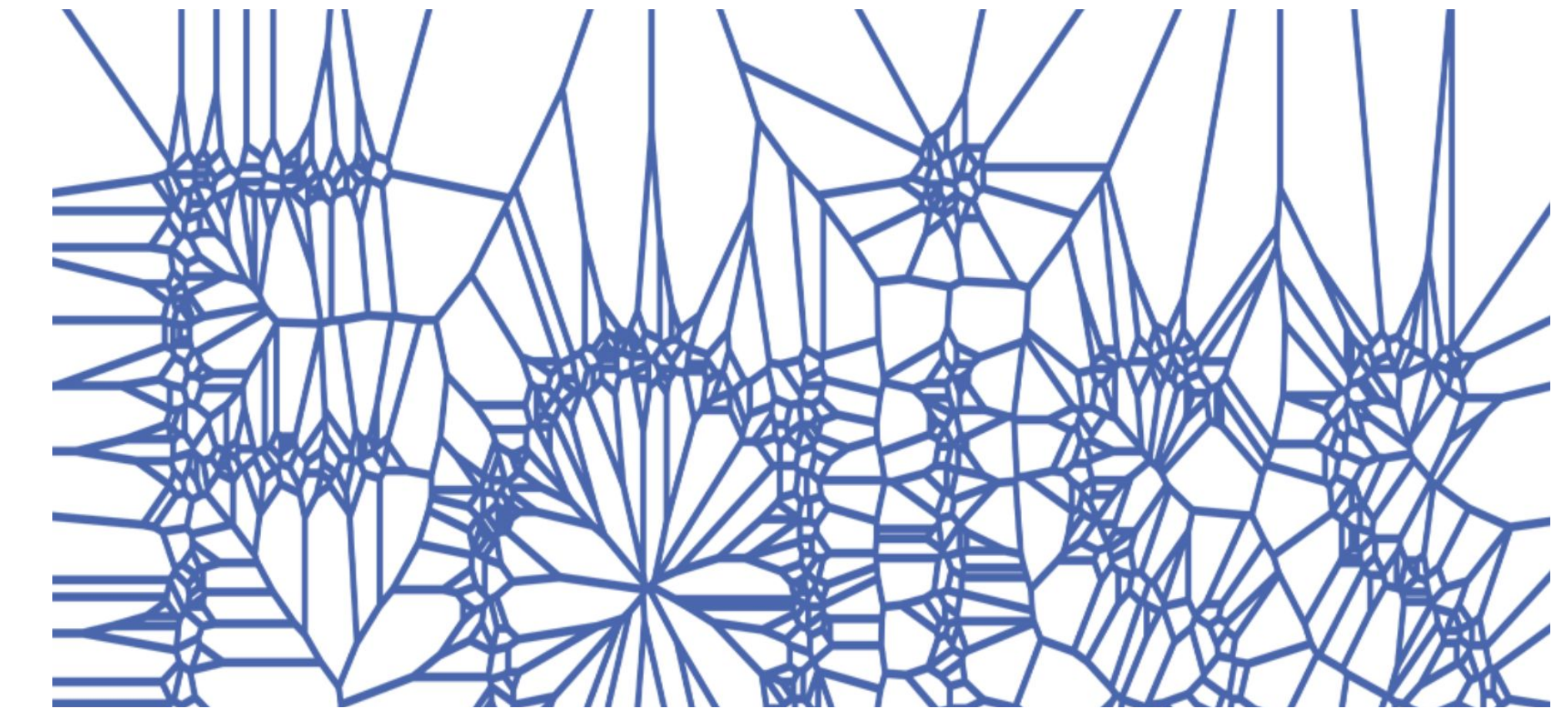
Query Augmentation



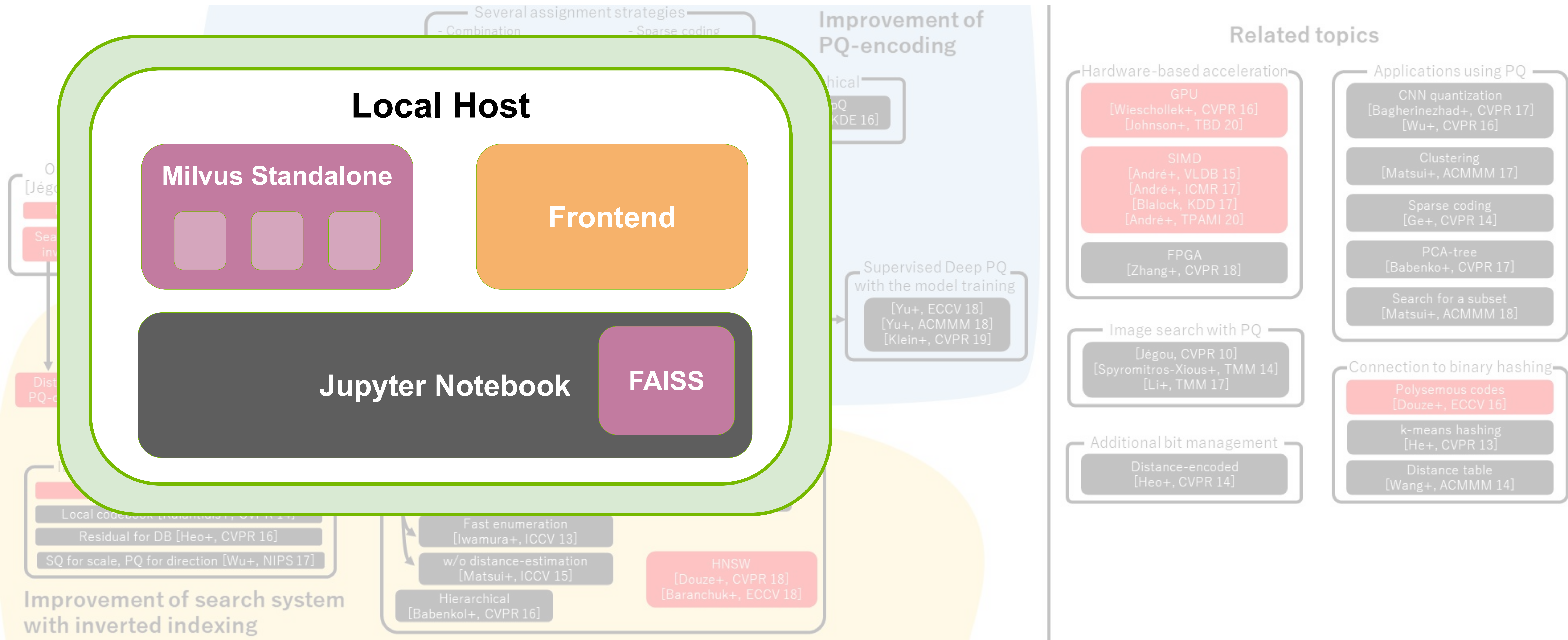
RAG Fusion



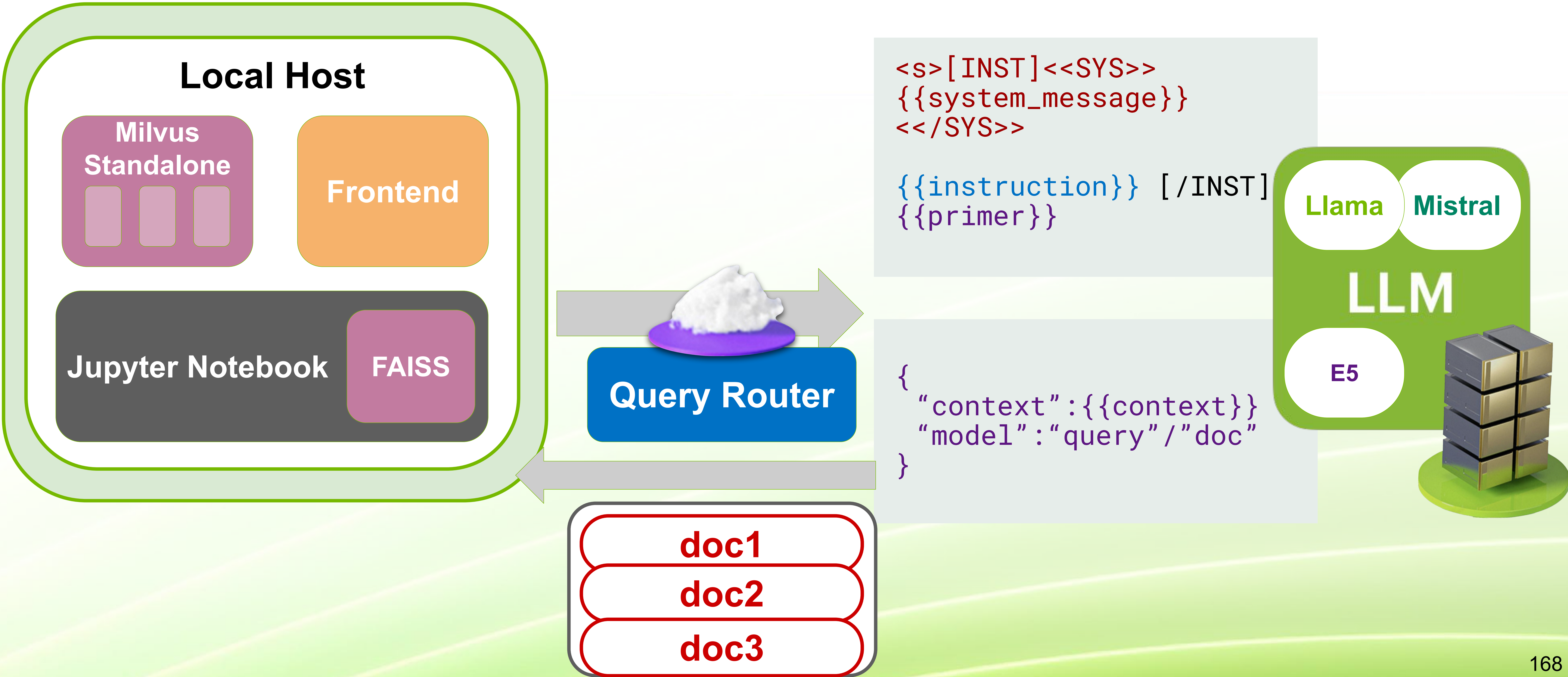
Integrating a Vector Store



Integrating a Local Vector Store



Integrating a Local Vector Store



GPU-Accelerating Vector Stores

DEEP-100M vector search throughput

Batch size 10, H100 (SXM)

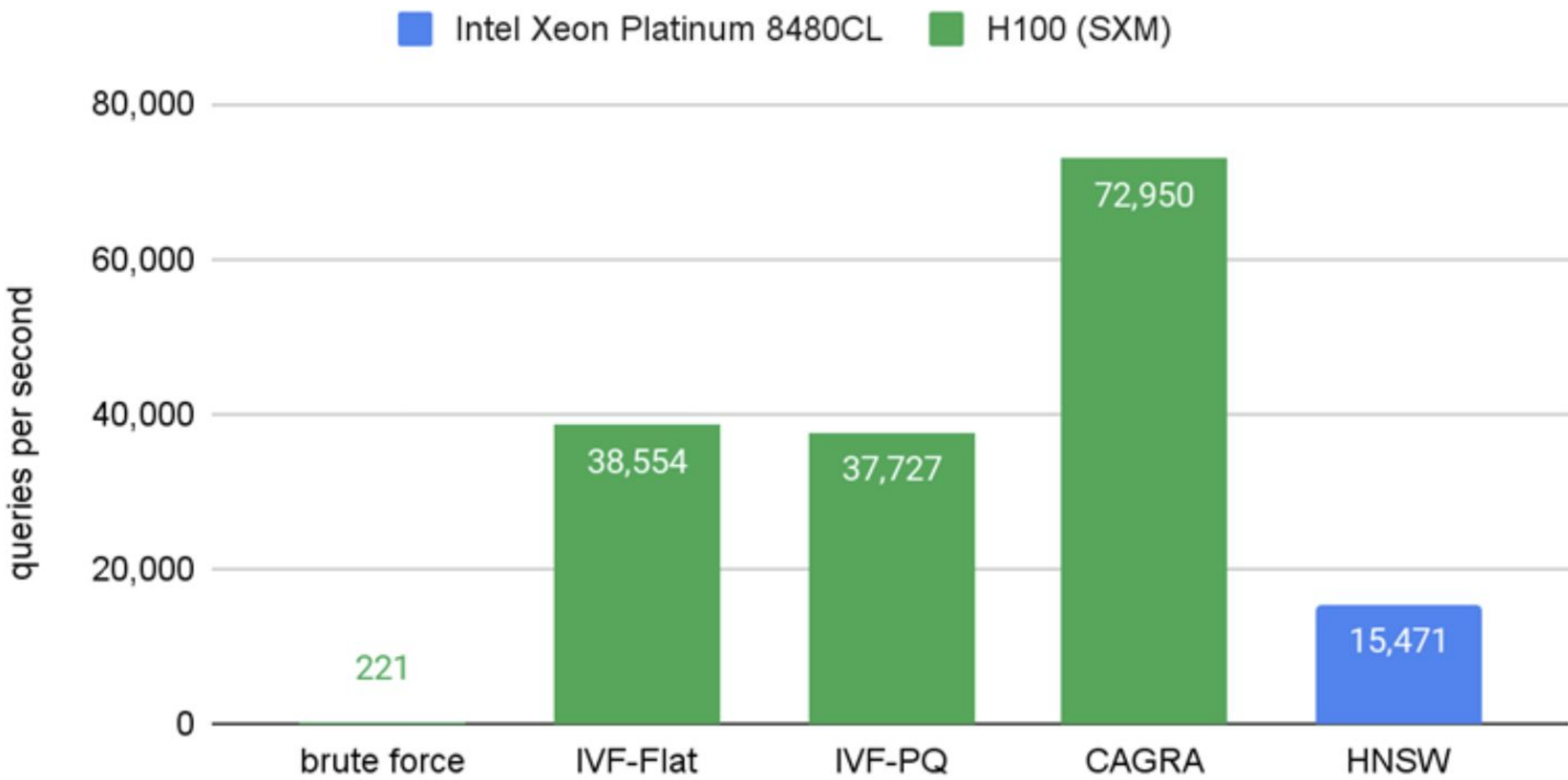
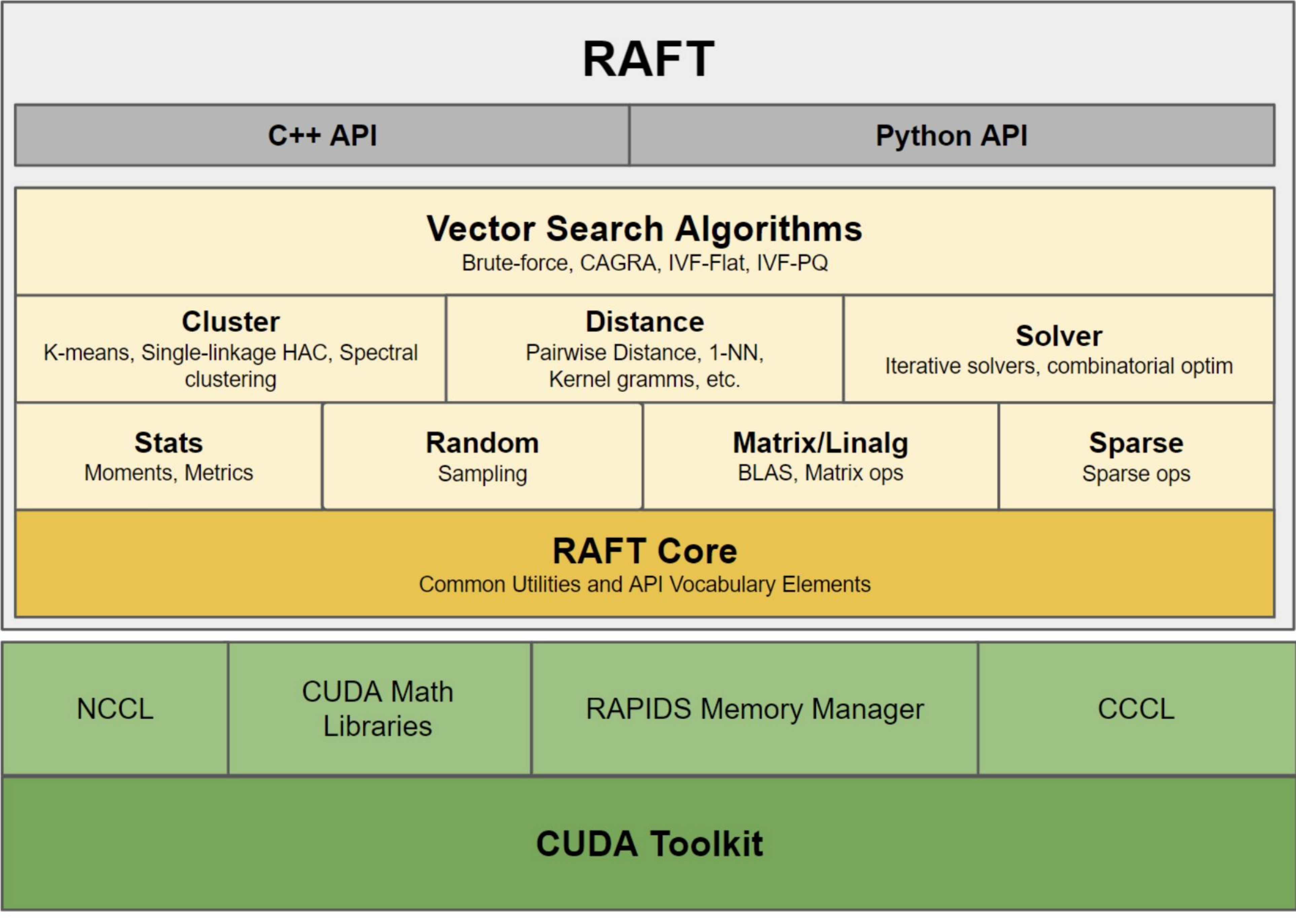
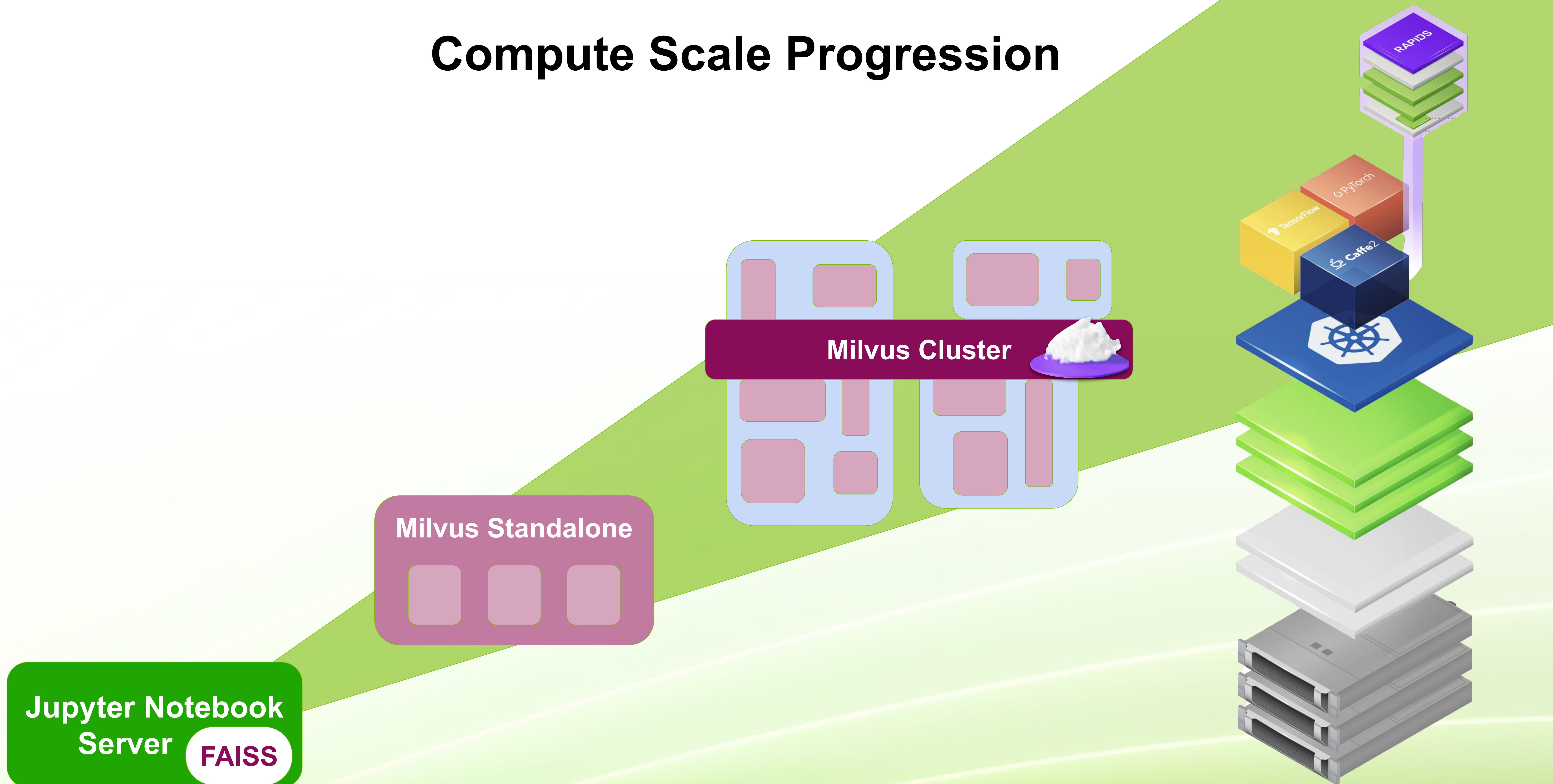


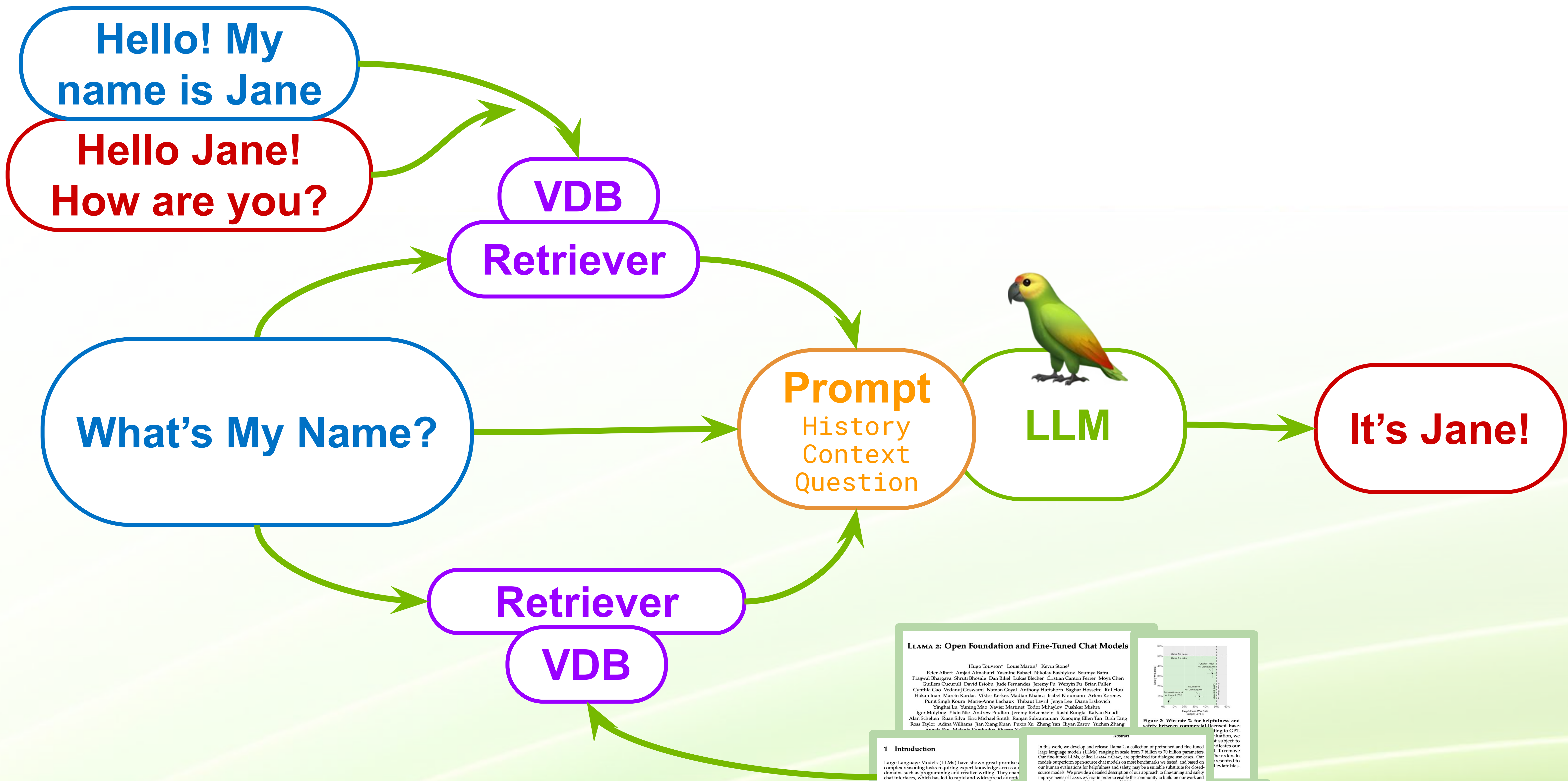
Figure 4. Vector search throughput at 95% recall on DEEP-100M dataset, batch size of 10



Compute Scale Progression



Simple Conversation RAG Setup



LLAMA 2: Open Foundation and Fine-Tuned Chat Models

Hugo Touvron*, Louis Martin*, Kevin Stone*, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Pranjwal Bhargava, Shun Buksale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Kornev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jernya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, *Meta AI, *Meta AI, *Meta AI, *Meta AI, *Meta AI

1 Introduction

Large Language Models (LLMs) have shown great promise in complex reasoning tasks requiring expert knowledge across a domains such as programming and creative writing. They enable chat interfaces, which has led to rapid and widespread adoption. The capabilities of LLMs are remarkable considering the methodology. Auto-regressive transformers are pretrained followed by alignment with human preferences via technique feedback (RLHF). Although the training methodology is limited the development of LLMs to a few players. There (such as BLOOM (Scialt et al., 2022), LLaMA-1 (Touvron et al., 2023)) match the performance of closed pretrained competitors.

Model	Human	LLaMA-1	LLaMA-2
Human	100	85	85
LLaMA-1	85	100	85
LLaMA-2	85	85	100

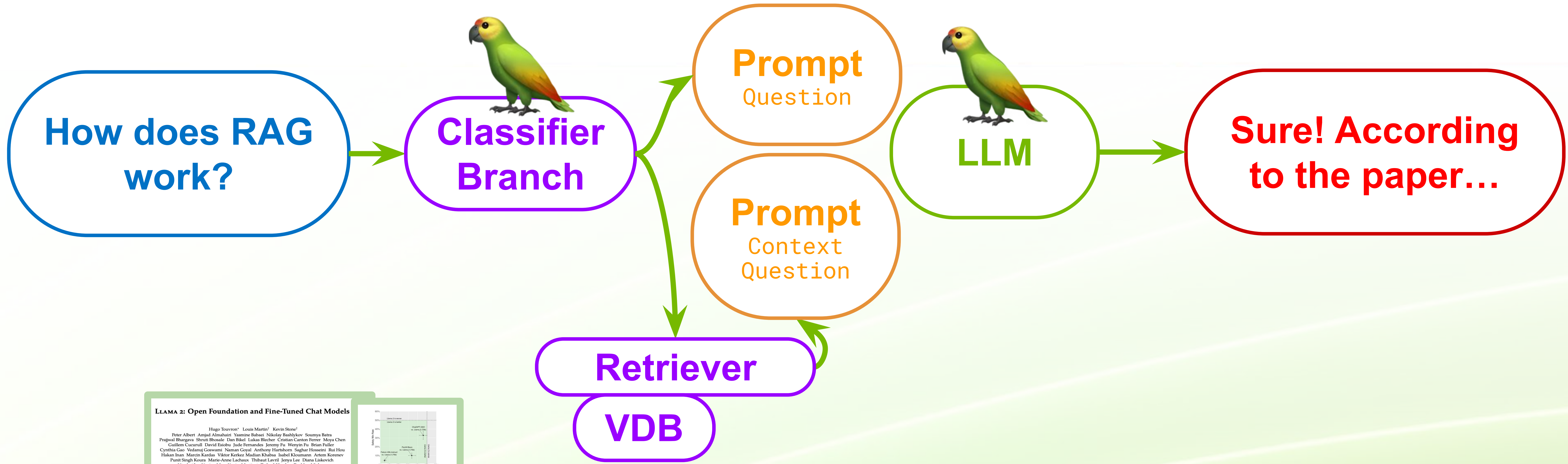
Figure 1: Helpfulness human evaluation results for LLaMA-2-Chat compared to other open-source and closed-source models. Human rates compared model generations on prompts consisting of both single and multi-turn prompt. The 95% confidence intervals for this evaluation are between 1% and 2%. More details in Section 3.4.2. While reviewing these results, it is important to note that human evaluation can be noisy due to limitations of the prompt set, subjectivity of the review guidelines, subjectivity of individual raters, and the inherent difficulty of comparing generations.

In this work, we develop and release Llama 2, a collection of pretrained and fine-tuned large language models (LLMs) ranging in scale from 7 billion to 70 billion parameters. Our fine-tuned LLMs, called Llama-2-Chat, are optimized for dialogue use cases. Our models outperform open-source chat models on most benchmarks we tested, and based on our human evaluations for helpfulness and safety, may be a suitable substitute for closed-source models. We provide a detailed description of our approach to fine-tuning and safety improvements of Llama-2-Chat in order to enable the community to build on our work and contribute to the responsible development of LLMs.

The capabilities of LLMs are remarkable considering the methodology. Auto-regressive transformers are pretrained on an extensive corpus of self-supervised data, followed by alignment with human preferences via techniques such as Reinforcement Learning with Human Feedback (RLHF). Although the training methodology is simple, high computational requirements have limited the development of LLMs to a few players. There have been public releases of pretrained LLMs (such as BLOOM (Scialt et al., 2022), LLaMA-1 (Touvron et al., 2023), and Falcon (Penelo et al., 2023)) that match the performance of closed pretrained competitors like GPT-3 (Brown et al., 2020) and Chinchilla (Hoffmann et al., 2022), but none of these models are suitable substitutes for closed "product" LLMs, such as ChatGPT, Bard, and Claude. These closed product LLMs are heavily fine-tuned to align with human preferences, which greatly enhances their usability and safety. This step can require significant costs in compute and human annotation, and is often not transparent or easily reproducible, limiting progress within the community to advance AI alignment research.

In this work, we develop and release Llama 2, a family of pretrained and fine-tuned LLMs. Llama 2 and Llama-2-Chat, at scales up to 70B parameters. On the series of helpfulness and safety benchmarks we tested, Llama-2-Chat models generally perform better than existing open-source models. They also appear to be on par with some of the closed-source models, at least on the human evaluations we performed (see Figures 1 and 3). We have taken measures to increase the safety of these models, using safety-specific data annotation and tuning, as well as conducting red-teaming and employing iterative evaluations. Additionally, this paper contributes a thorough description of our fine-tuning methodology and approach to improving

Simple RAG Agents



LLAMA 2: Open Foundation and Fine-Tuned Chat Models

Hugo Touvron*, Louis Martin*, Kevin Stone*, Peter Albert, Anjali Arora, Yasmine Babati, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shuang Bhoale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedant Gokhale, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Rishabh Subramanian, Xiaoqing Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Jai Singh, David Rosenberg, Omer Levy

1 Introduction

Large Language Models (LLMs) have shown great promise in complex reasoning tasks requiring expert knowledge across a wide range of domains such as programming and creative writing. They enable chat interfaces, which has led to rapid and widespread adoption. The capabilities of LLMs are remarkable considering the methodology. Auto-regressive transformers are pretrained, followed by alignment with human preferences via techniques such as Reinforcement Learning with Human Feedback (RLHF). Although the training methodology is limited to the development of LLMs to a few players. There (such as BLOOM (Scao et al., 2022), LLaMA-1 (Touvron et al., 2023)) match the performance of closed pretrained competitors like GPT-3 (Brown et al., 2020) and Claude (Anthropic et al., 2022), but none of these models are suitable substitutes for closed “product” LLMs, such as ChatGPT, Bard, and Claude. These closed product LLMs are heavily fine-tuned to align with human preferences, which greatly enhances their usability and safety. This step can require significant costs in compute and human annotation, and is often not transparent or easily reproducible, limiting progress within the community to advance AI alignment research.

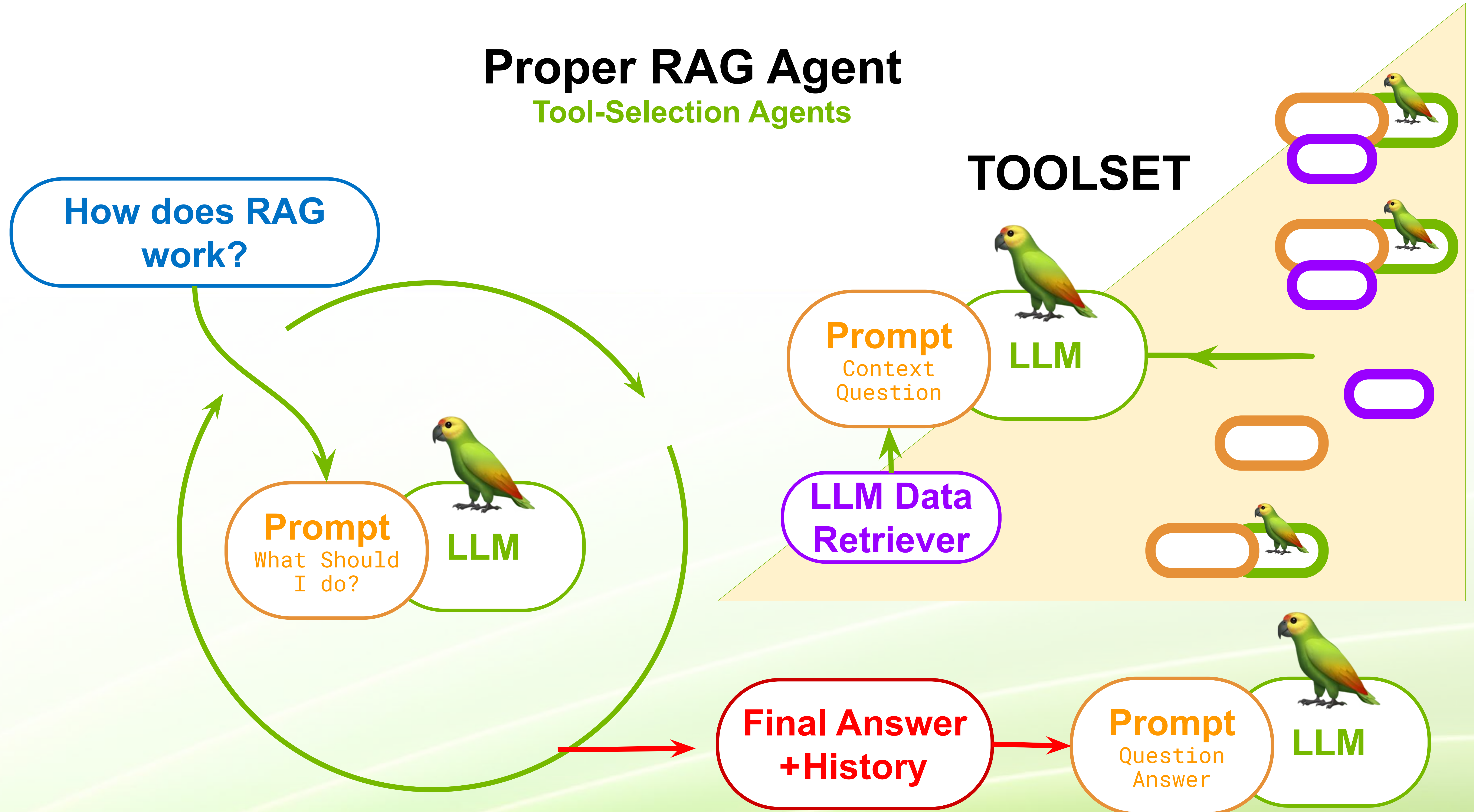
In this work, we develop and release Llama 2, a family of pretrained and fine-tuned LLMs. LLaMA 2 and LLaMA 2-Chat, at scales up to 70B parameters. On the series of helpfulness and safety benchmarks we tested, LLaMA 2-Chat models generally perform better than existing open-source models. They also appear to be on par with some of the closed-source models, at least on the human evaluations we performed (see Figure 1 and 3). We have taken measures to increase the safety of these models, using safety-specific data annotation and tuning, as well as conducting red-teaming and employing iterative evaluations. Additionally, this paper contributes a thorough description of our fine-tuning methodology and approach to improving

Figure 1: Helpfulness human evaluation results for LLaMA 2-Chat compared to other open-source and closed-source models. Human raters compared model generations on prompts consisting of both single and multi-turn prompts. The 95% confidence intervals for this evaluation are between 1% and 2%. More details in Section 3.4.2. While reviewing these results, it is important to note that human evaluation can be noisy due to limitations of the prompt set, subjectivity of the review guidelines, subjectivity of individual raters, and the inherent difficulty of comparing generations.

Model	Win Rate (%)
LLaMA 2-Chat (70B)	34.4
LLaMA 2-Chat (70B)	29.2
LLaMA 2-Chat (70B)	29.4

Proper RAG Agent

Tool-Selection Agents



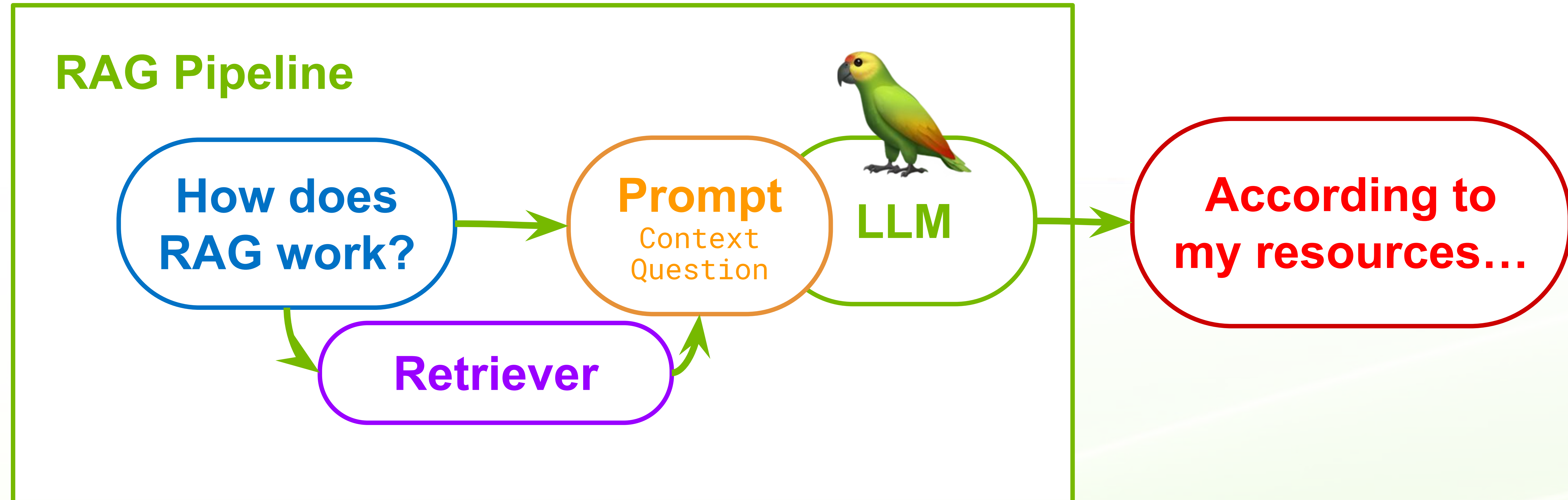


Building RAG Agents with LLMs

Part 8: RAG Evaluation

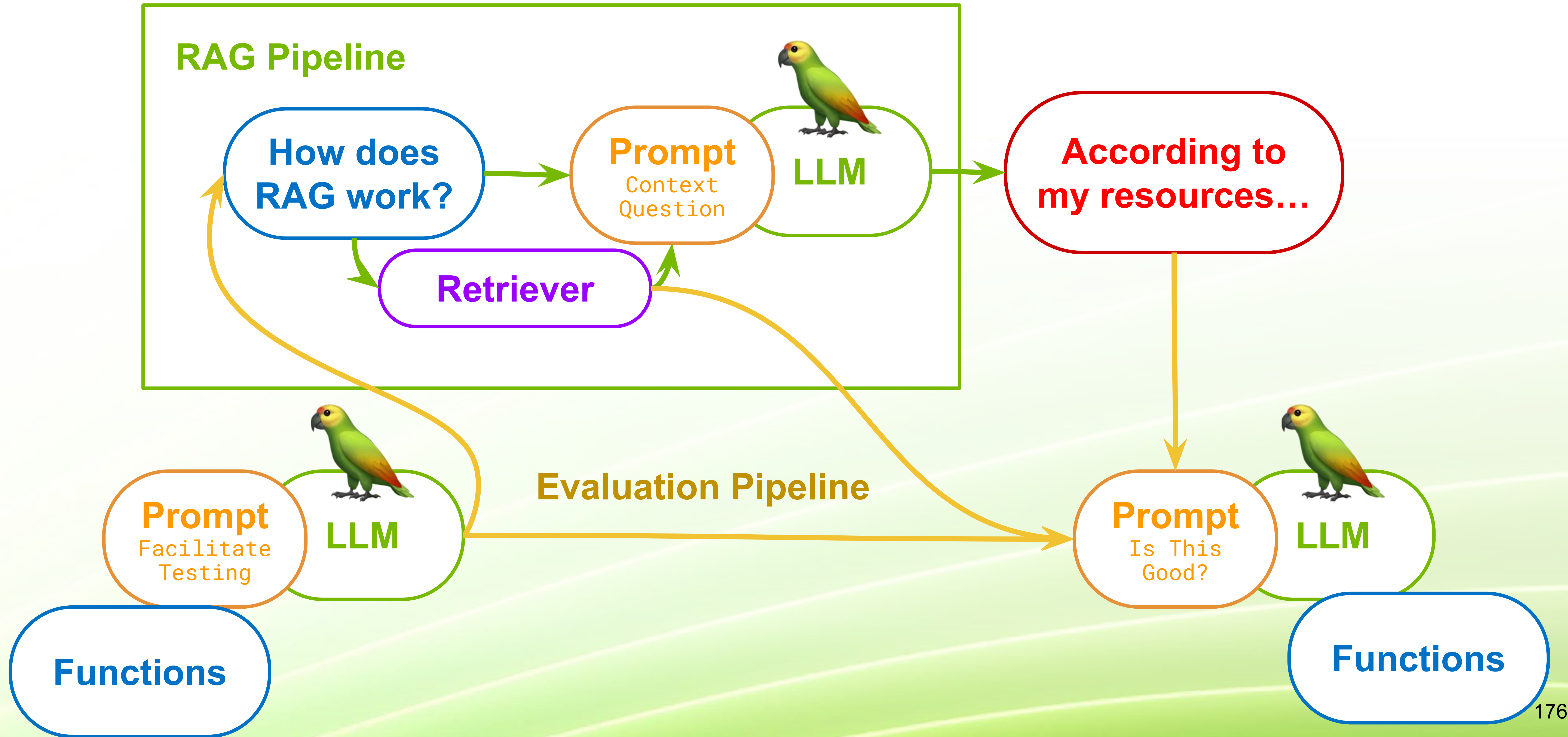
LLM-As-A-Judge

Pipeline Evaluation



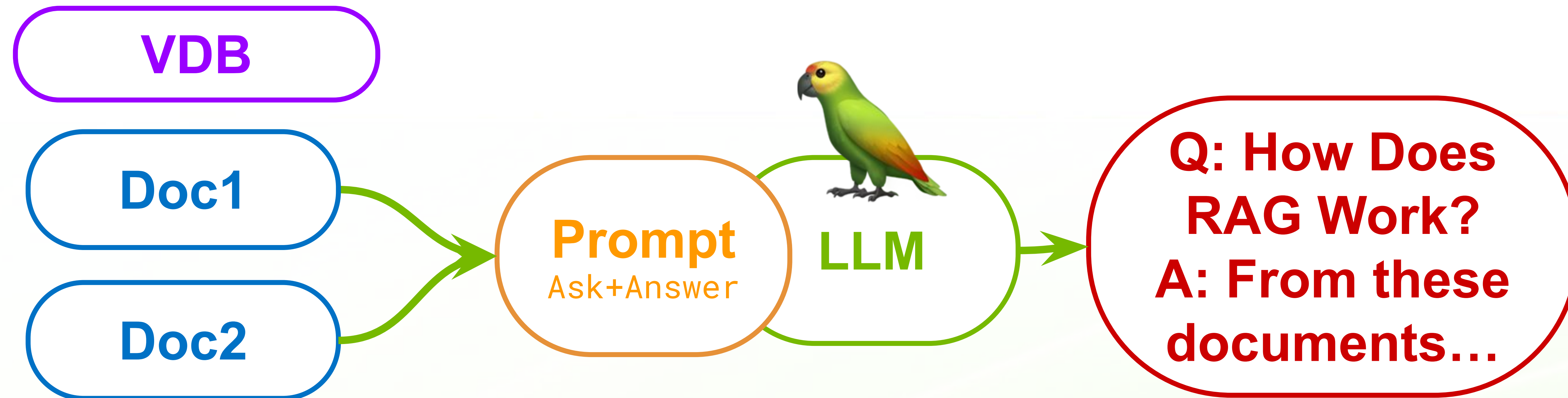
LLM-As-A-Judge

Pipeline Evaluation



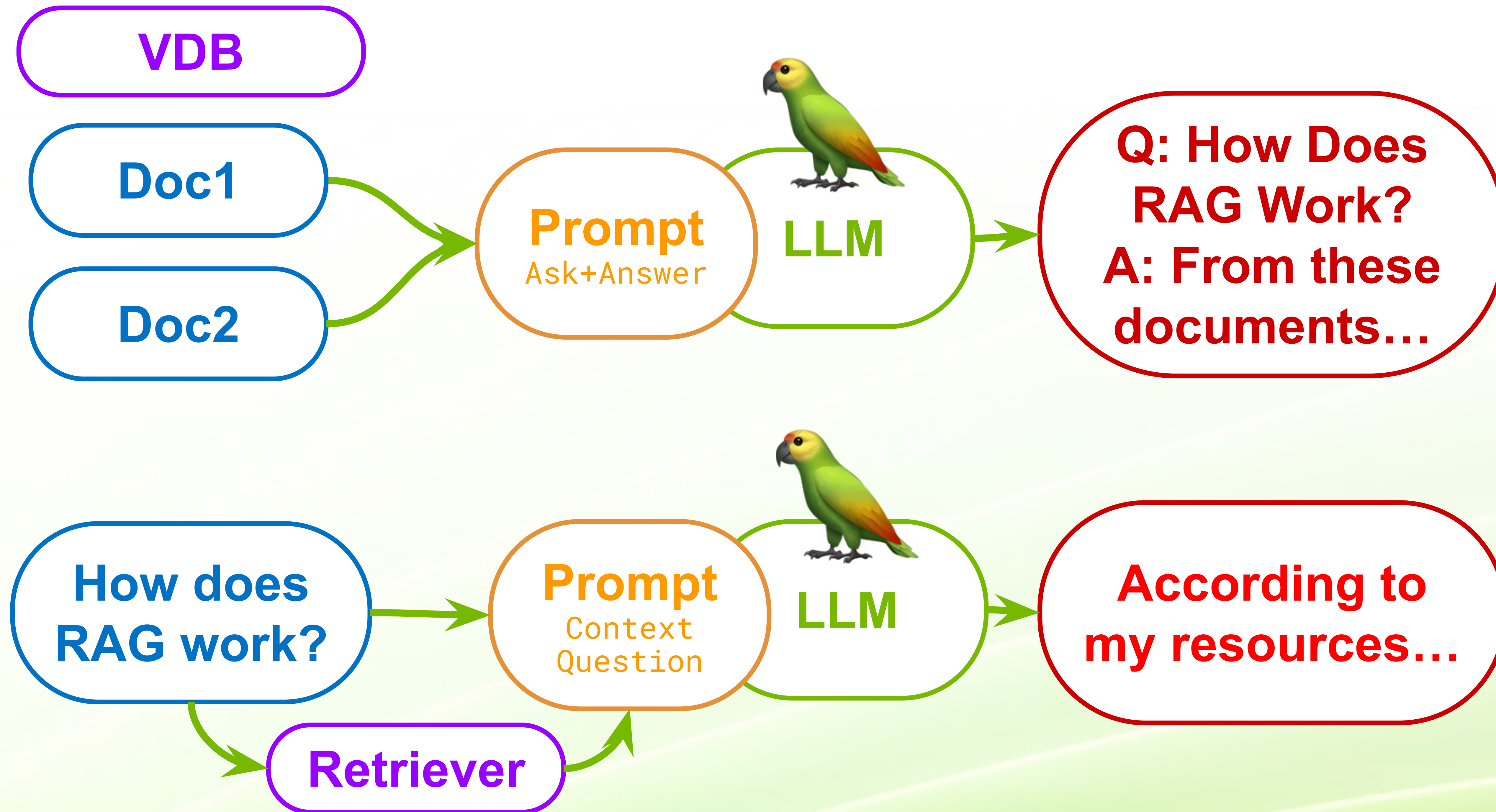
Our Evaluation Chain Components

Synthetic Generation



Our Evaluation Chain Components

RAG Pipeline Sample



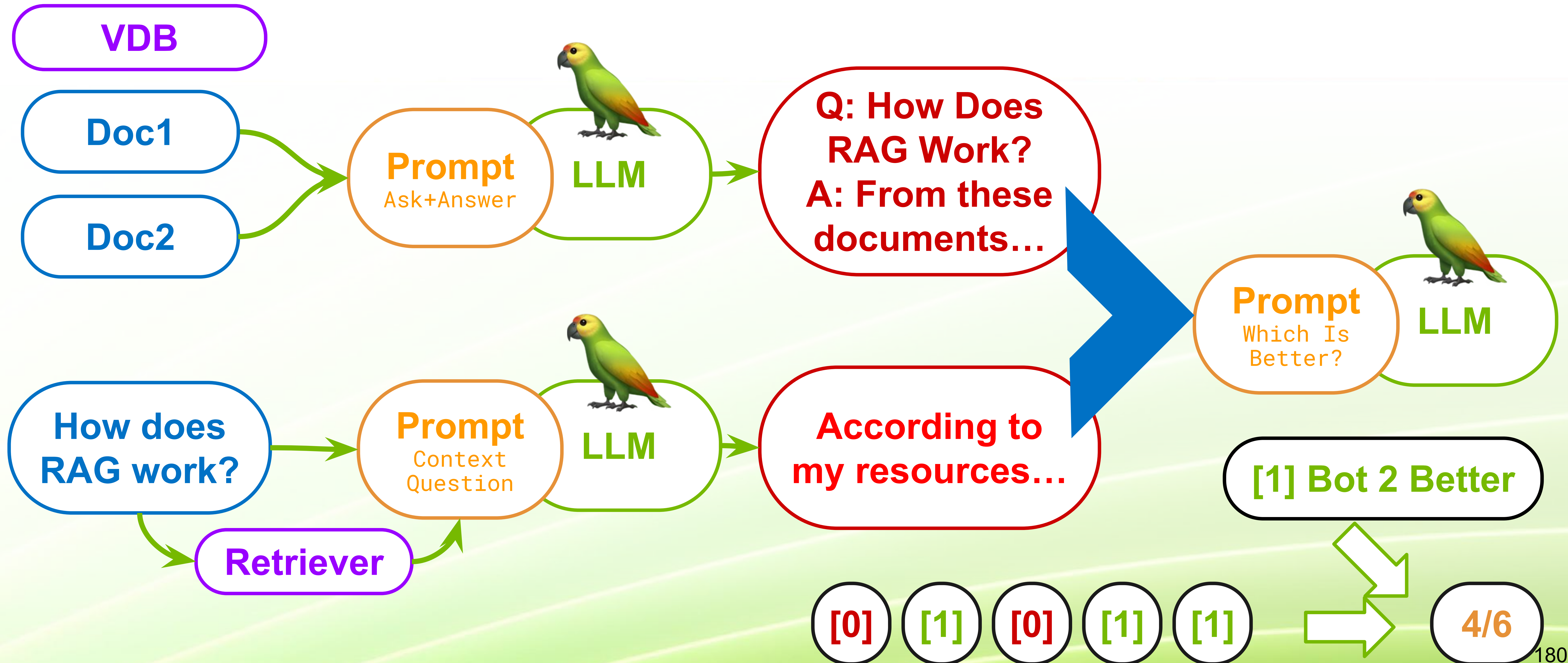
Our Evaluation Chain Components

Ground Truth vs Our Pipeline



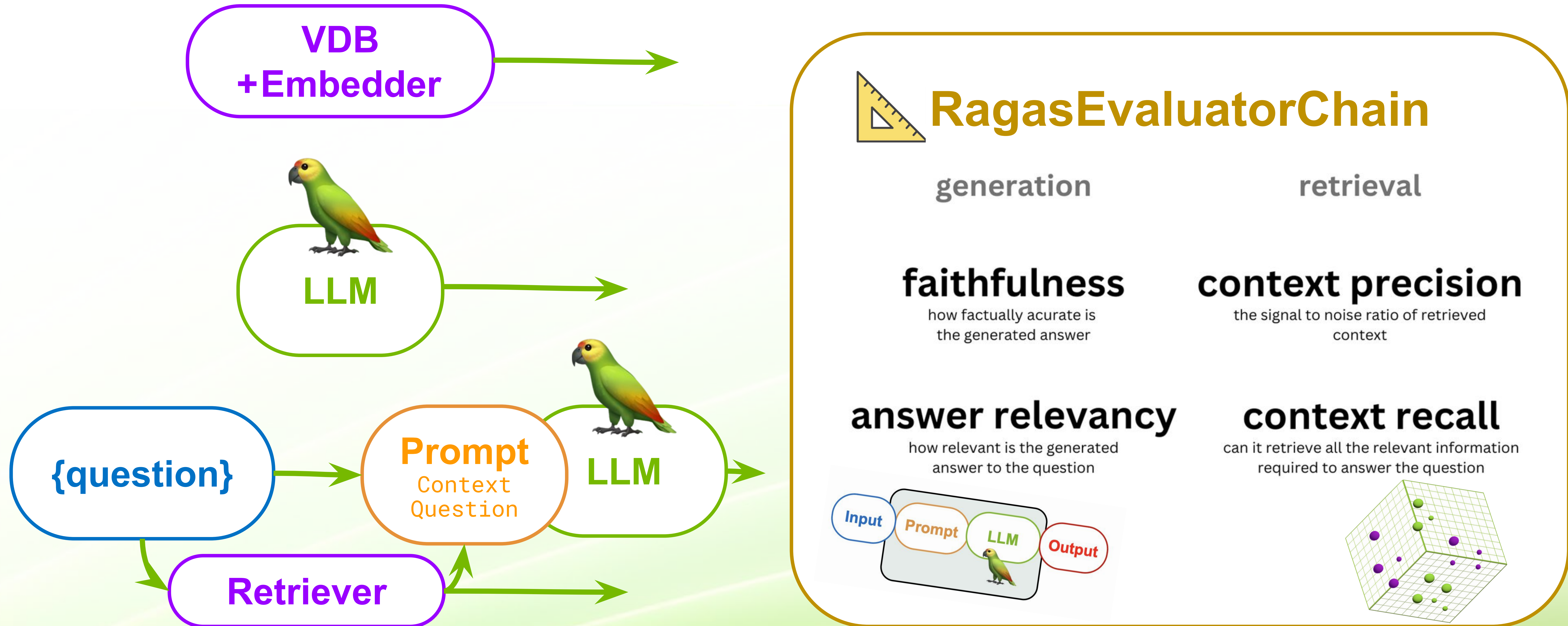
Our Evaluation Chain Components

Ground Truth vs Our Pipeline



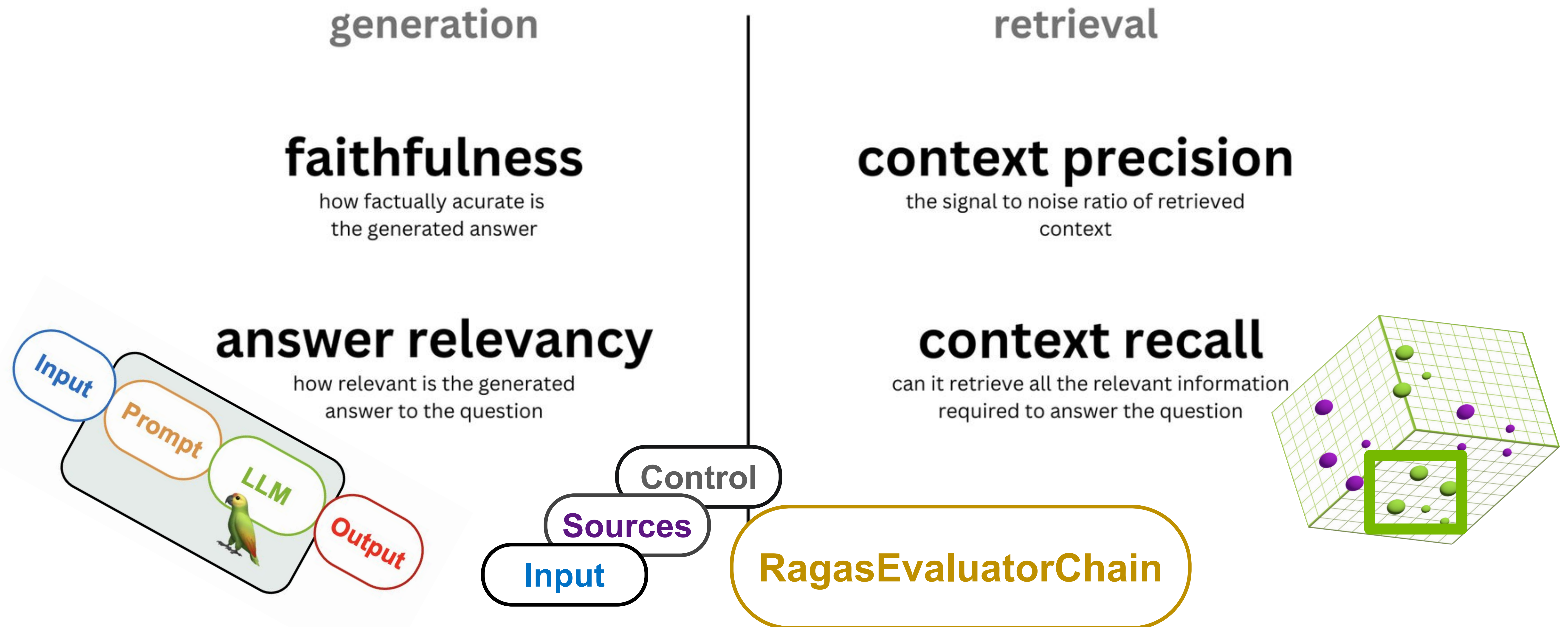
General Evaluation Chain Components

Multiple Metrics at Once



RAG Evaluation with ragas

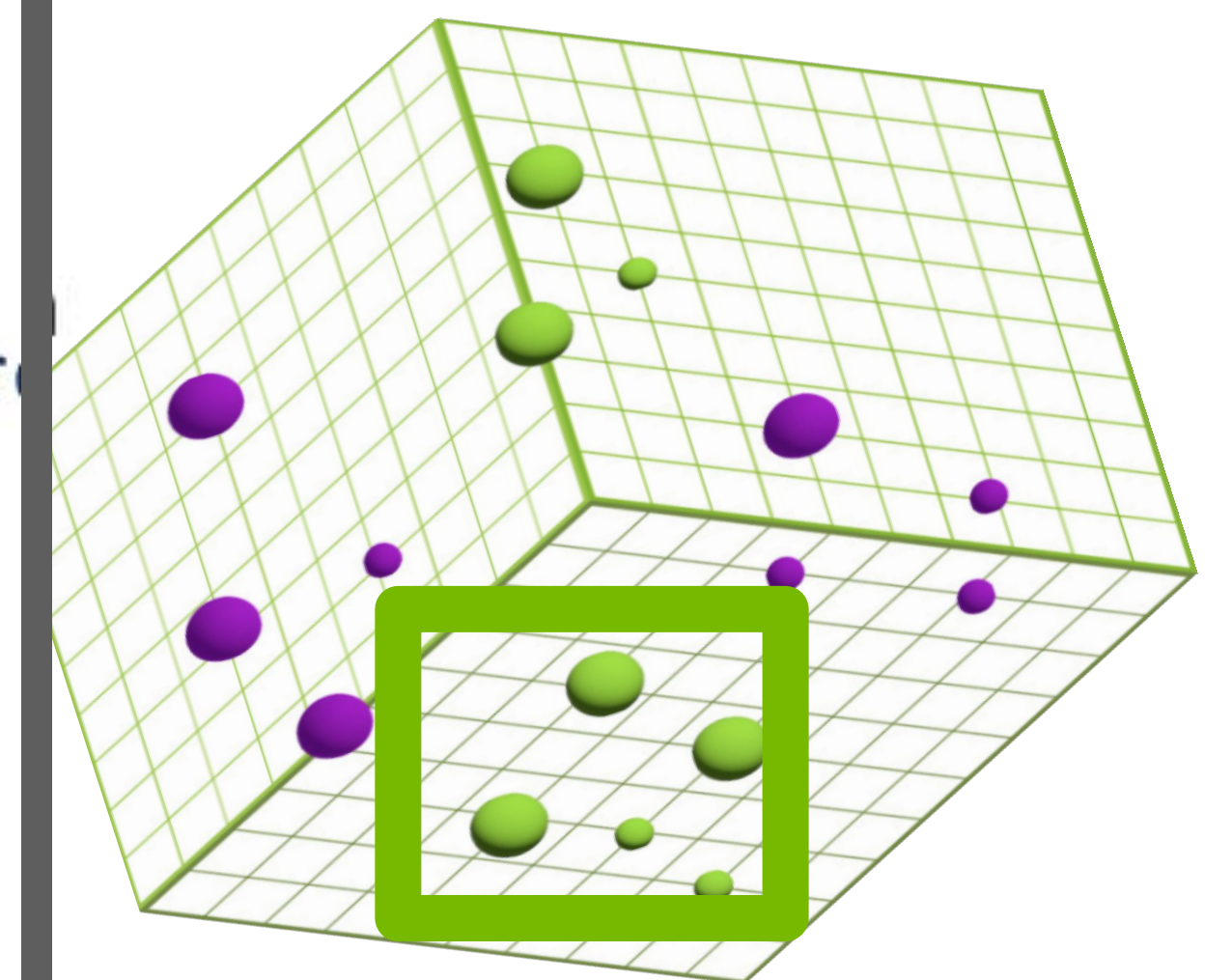
Quantifying System Goodness with LLM-as-a-Judge



RAG Evaluation with ragas

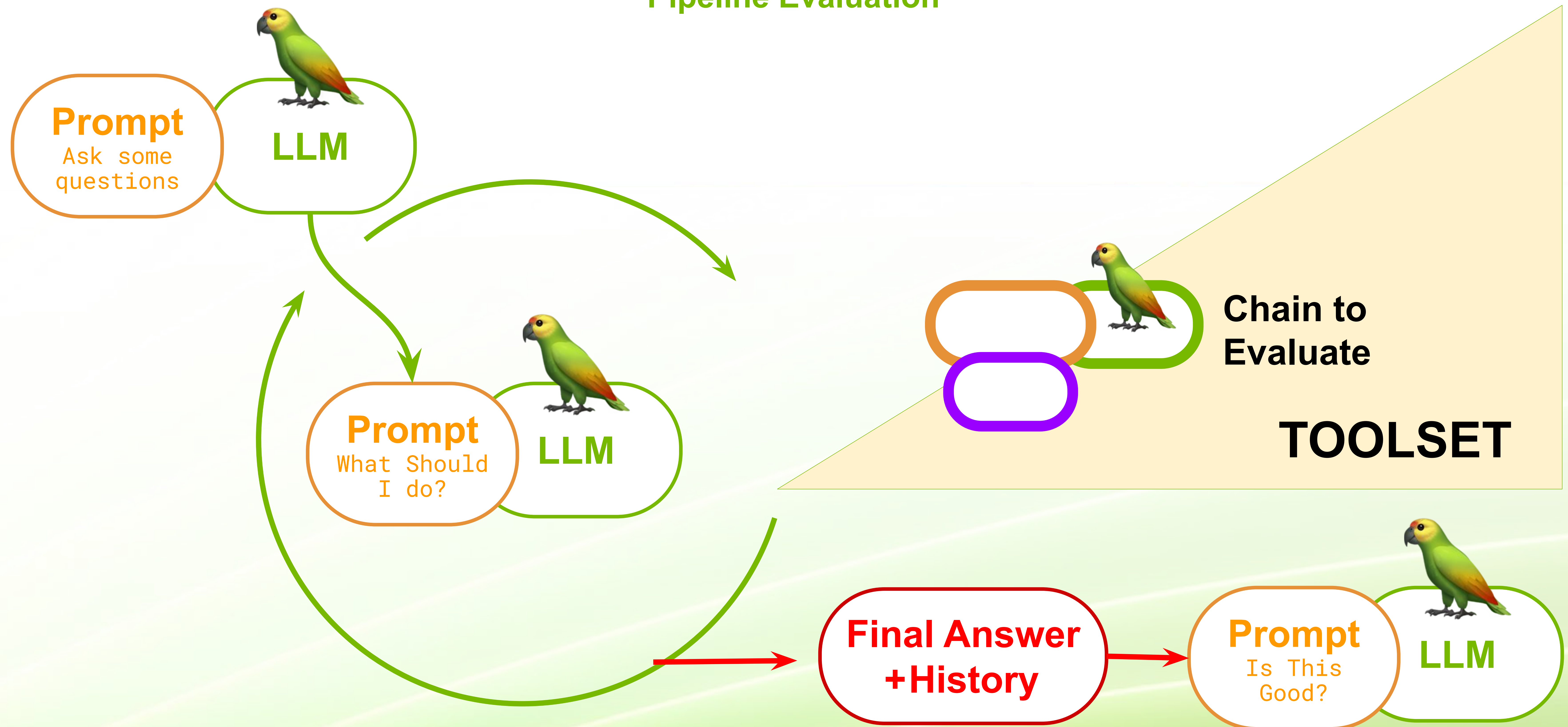
Quantifying System Goodness with LLM-as-a-Judge

```
54  NLI_STATEMENTS_MESSAGE = HumanMessagePromptTemplate.from_template(
55      """
56      Natural language inference. Only use "Yes" or "No" as verdict.
57
58      Context:
59      John is a student at XYZ University. He is pursuing a degree in Computer Science.
60      statement_1: John is majoring in Biology.
61      statement_2: John is taking a course on Artificial Intelligence.
62      statement_3: John is a dedicated student.
63      statement_4: John has a part-time job.
64      Answer:
65      [
66          {{
67              "statement_1": "John is majoring in Biology.",
68              "reason": "John's major is explicitly mentioned as Computer Science. There",
69              "verdict": "No"
70          }},
71          {{
72              "statement_2": "John is taking a course on Artificial Intelligence.",
73              "reason": "The context mentions the courses John is currently enrolled in",
74              "verdict": "No"
75          }},
76          {{
```



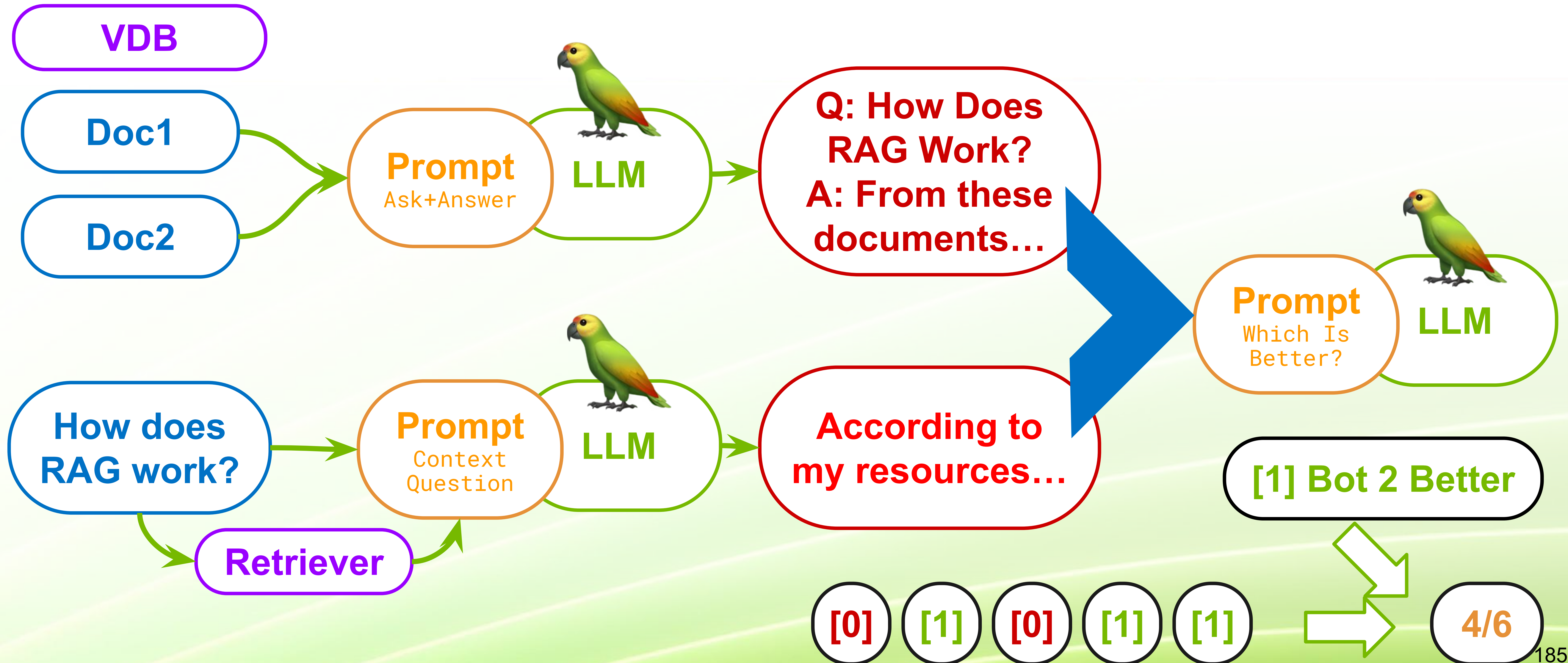
Evaluator Agent

Pipeline Evaluation



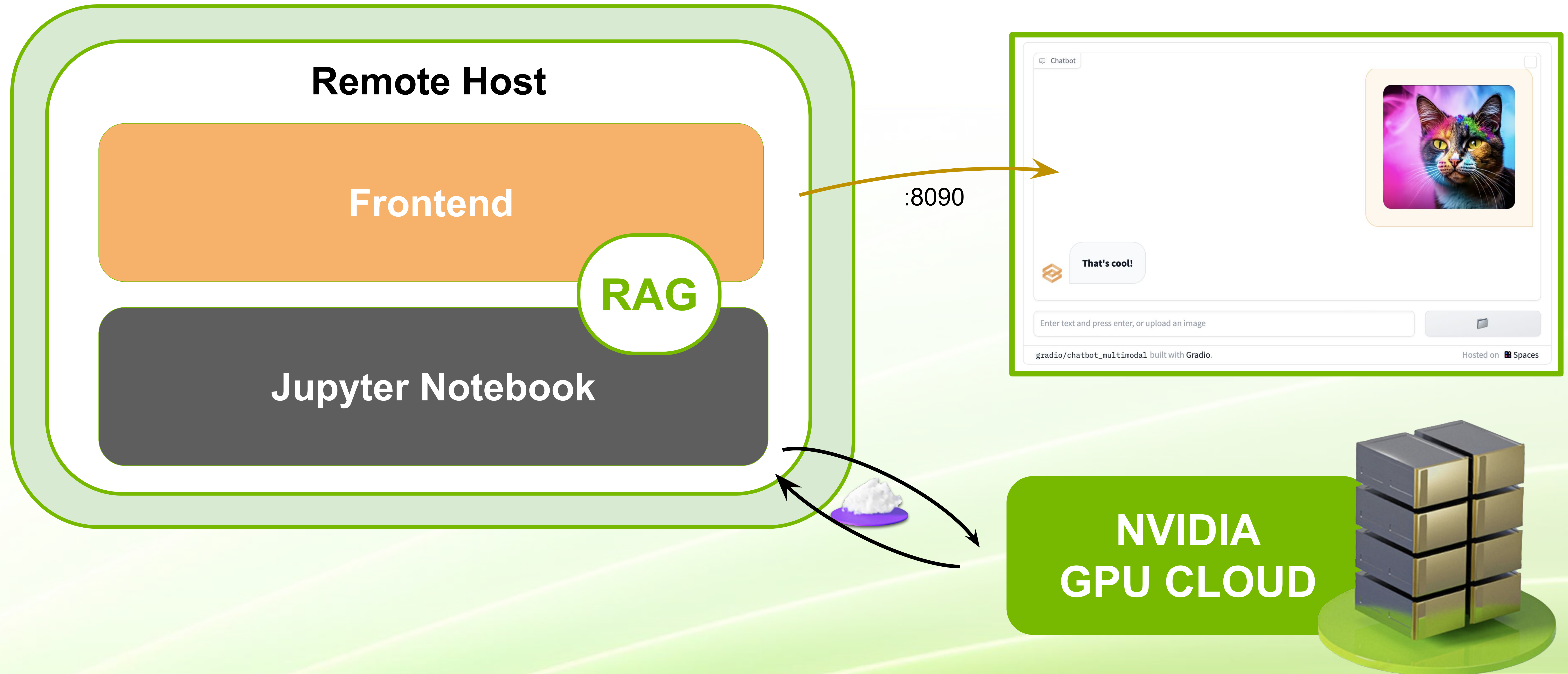
Our Evaluation Chain Components

Ground Truth vs Our Pipeline



Evaluate RAG In The Frontend

Final Assessment





Congratulations!
AND THANK YOU SO MUCH!

